

Essays, III

Paul Graham

Contents

1	The 18 Mistakes That Kill Startups	1
2	How Art Can Be Good	19
3	Learning from Founders	29
4	Is It Worth Being Wise?	33
5	Why to Not Not Start a Startup	45
6	Microsoft is Dead	63
7	Two Kinds of Judgement	67
8	The Hacker’s Guide to Investors	71
9	An Alternative Theory of Unions	91
10	The Equity Equation	93
11	Stuff	97
12	Holding a Program in One’s Head	101
13	How Not to Die	107
14	News from the Front	113
15	How to Do Philosophy	121
16	The Future of Web Startups	135
17	Why to Move to a Startup Hub	147
18	Six Principles for Making New Things	153
19	Trolls	157

20	A New Venture Animal	161
21	You Weren't Meant to Have a Boss	167
22	How to Disagree	175
23	Some Heroes	181
24	Why There Aren't More Googles	191
25	Be Good	197
26	Lies We Tell Kids	207
27	Disconnecting Distraction	223
28	Cities and Ambition	227
29	The Pooled-Risk Company Management Company	237
30	A Fundraising Survival Guide	243
31	Why to Start a Startup in a Bad Economy	257
32	The Other Half of "Artists Ship"	261
33	The High-Res Society	265
34	Could VC be a Casualty of the Recession?	271
35	After Credentials	277
36	Keep Your Identity Small	285
37	Startups in 13 Sentences	289
38	What I've Learned from Hacker News	295
39	Can You Buy a Silicon Valley? Maybe.	305
40	Why TV Lost	311
41	How to Be an Angel Investor	317
42	Relentlessly Resourceful	329
43	Five Founders	333
44	The Founder Visa	337

45	Why Twitter is a Big Deal	339
46	A Local Revolution?	341
47	Maker's Schedule, Manager's Schedule	345
48	Ramen Profitable	349
49	The Trouble with the Segway	355
50	What Kate Saw in Silicon Valley	357
51	The Anatomy of Determination	361
52	The List of N Things	367
53	Post-Medium Publishing	371
54	Persuade xor Discover	377

Chapter 1

The 18 Mistakes That Kill Startups

October 2006

In the Q & A period after a recent talk, someone asked what made startups fail. After standing there gaping for a few seconds I realized this was kind of a trick question. It's equivalent to asking how to make a startup succeed — if you avoid every cause of failure, you succeed — and that's too big a question to answer on the fly.

Afterwards I realized it could be helpful to look at the problem from this direction. If you have a list of all the things you shouldn't do, you can turn that into a recipe for succeeding just by negating. And this form of list may be more useful in practice. It's easier to catch yourself doing something you shouldn't than always to remember to do something you should.¹

In a sense there's just one mistake that kills startups: not making something users want. If you make something users want, you'll probably be fine, whatever else you do or don't do. And if you don't make something users want, then you're dead, whatever else you do or don't do. So really this is a list of 18 things that cause startups not to make something users want. Nearly all failure funnels through that.

1. Single Founder

Have you ever noticed how few successful startups were founded by just one person? Even companies you think of as having one founder,

¹This is not a complete list of the causes of failure, just those you can control. There are also several you can't, notably ineptitude and bad luck.

like Oracle, usually turn out to have more. It seems unlikely this is a coincidence.

What's wrong with having one founder? To start with, it's a vote of no confidence. It probably means the founder couldn't talk any of his friends into starting the company with him. That's pretty alarming, because his friends are the ones who know him best.

But even if the founder's friends were all wrong and the company is a good bet, he's still at a disadvantage. Starting a startup is too hard for one person. Even if you could do all the work yourself, you need colleagues to brainstorm with, to talk you out of stupid decisions, and to cheer you up when things go wrong.

The last one might be the most important. The low points in a startup are so low that few could bear them alone. When you have multiple founders, esprit de corps binds them together in a way that seems to violate conservation laws. Each thinks "I can't let my friends down." This is one of the most powerful forces in human nature, and it's missing when there's just one founder.

2. Bad Location

Startups prosper in some places and not others. Silicon Valley dominates, then Boston, then Seattle, Austin, Denver, and New York. After that there's not much. Even in New York the number of startups per capita is probably a 20th of what it is in Silicon Valley. In towns like Houston and Chicago and Detroit it's too small to measure.

Why is the falloff so sharp? Probably for the same reason it is in other industries. What's the sixth largest fashion center in the US? The sixth largest center for oil, or finance, or publishing? Whatever they are they're probably so far from the top that it would be misleading even to call them centers.

It's an interesting question why cities become startup hubs, but the reason startups prosper in them is probably the same as it is for any industry: that's where the experts are. Standards are higher; people are

more sympathetic to what you're doing; the kind of people you want to hire want to live there; supporting industries are there; the people you run into in chance meetings are in the same business. Who knows exactly how these factors combine to boost startups in Silicon Valley and squish them in Detroit, but it's clear they do from the number of startups per capita in each.

3. Marginal Niche

Most of the groups that apply to Y Combinator suffer from a common problem: choosing a small, obscure niche in the hope of avoiding competition.

If you watch little kids playing sports, you notice that below a certain age they're afraid of the ball. When the ball comes near them their instinct is to avoid it. I didn't make a lot of catches as an eight year old outfielder, because whenever a fly ball came my way, I used to close my eyes and hold my glove up more for protection than in the hope of catching it.

Choosing a marginal project is the startup equivalent of my eight year old strategy for dealing with fly balls. If you make anything good, you're going to have competitors, so you may as well face that. You can only avoid competition by avoiding good ideas.

I think this shrinking from big problems is mostly unconscious. It's not that people think of grand ideas but decide to pursue smaller ones because they seem safer. Your unconscious won't even let you think of grand ideas. So the solution may be to think about ideas without involving yourself. What would be a great idea for *someone else* to do as a startup?

4. Derivative Idea

Many of the applications we get are imitations of some existing company. That's one source of ideas, but not the best. If you look at the origins of successful startups, few were started in imitation of some other startup. Where did they get their ideas? Usually from some

specific, unsolved problem the founders identified.

Our startup made software for making online stores. When we started it, there wasn't any; the few sites you could order from were hand-made at great expense by web consultants. We knew that if online shopping ever took off, these sites would have to be generated by software, so we wrote some. Pretty straightforward.

It seems like the best problems to solve are ones that affect you personally. Apple happened because Steve Wozniak wanted a computer, Google because Larry and Sergey couldn't find stuff online, Hotmail because Sabeer Bhatia and Jack Smith couldn't exchange email at work.

So instead of copying the Facebook, with some variation that the Facebook rightly ignored, look for ideas from the other direction. Instead of starting from companies and working back to the problems they solved, look for problems and imagine the company that might solve them.² What do people complain about? What do you wish there was?

5. Obstinacy

In some fields the way to succeed is to have a vision of what you want to achieve, and to hold true to it no matter what setbacks you encounter. Starting startups is not one of them. The stick-to-your-vision approach works for something like winning an Olympic gold medal, where the problem is well-defined. Startups are more like science, where you need to follow the trail wherever it leads.

So don't get too attached to your original plan, because it's probably wrong. Most successful startups end up doing something different than they originally intended — often so different that it doesn't even seem like the same company. You have to be prepared to see the better idea when it arrives. And the hardest part of that is often discarding your old idea.

²Ironically, one variant of the Facebook that might work is a facebook exclusively for college students.

But openness to new ideas has to be tuned just right. Switching to a new idea every week will be equally fatal. Is there some kind of external test you can use? One is to ask whether the ideas represent some kind of progression. If in each new idea you're able to re-use most of what you built for the previous ones, then you're probably in a process that converges. Whereas if you keep restarting from scratch, that's a bad sign.

Fortunately there's someone you can ask for advice: your users. If you're thinking about turning in some new direction and your users seem excited about it, it's probably a good bet.

6. Hiring Bad Programmers

I forgot to include this in the early versions of the list, because nearly all the founders I know are programmers. This is not a serious problem for them. They might accidentally hire someone bad, but it's not going to kill the company. In a pinch they can do whatever's required themselves.

But when I think about what killed most of the startups in the e-commerce business back in the 90s, it was bad programmers. A lot of those companies were started by business guys who thought the way startups worked was that you had some clever idea and then hired programmers to implement it. That's actually much harder than it sounds — almost impossibly hard in fact — because business guys can't tell which are the good programmers. They don't even get a shot at the best ones, because no one really good wants a job implementing the vision of a business guy.

In practice what happens is that the business guys choose people they think are good programmers (it says here on his resume that he's a Microsoft Certified Developer) but who aren't. Then they're mystified to find that their startup lumbers along like a World War II bomber while their competitors scream past like jet fighters. This kind of startup is in the same position as a big company, but without the advantages.

So how do you pick good programmers if you're not a programmer?

I don't think there's an answer. I was about to say you'd have to find a good programmer to help you hire people. But if you can't recognize good programmers, how would you even do that?

7. Choosing the Wrong Platform

A related problem (since it tends to be done by bad programmers) is choosing the wrong platform. For example, I think a lot of startups during the Bubble killed themselves by deciding to build server-based applications on Windows. Hotmail was still running on FreeBSD for years after Microsoft bought it, presumably because Windows couldn't handle the load. If Hotmail's founders had chosen to use Windows, they would have been swamped.

PayPal only just dodged this bullet. After they merged with X.com, the new CEO wanted to switch to Windows — even after PayPal co-founder Max Levchin showed that their software scaled only 1% as well on Windows as Unix. Fortunately for PayPal they switched CEOs instead.

Platform is a vague word. It could mean an operating system, or a programming language, or a “framework” built on top of a programming language. It implies something that both supports and limits, like the foundation of a house.

The scary thing about platforms is that there are always some that seem to outsiders to be fine, responsible choices and yet, like Windows in the 90s, will destroy you if you choose them. Java applets were probably the most spectacular example. This was supposed to be the new way of delivering applications. Presumably it killed just about 100% of the startups who believed that.

How do you pick the right platforms? The usual way is to hire good programmers and let them choose. But there is a trick you could use if you're not a programmer: visit a top computer science department and see what they use in research projects.

8. Slowness in Launching

Companies of all sizes have a hard time getting software done. It's intrinsic to the medium; software is always 85% done. It takes an effort of will to push through this and get something released to users.

3

Startups make all kinds of excuses for delaying their launch. Most are equivalent to the ones people use for procrastinating in everyday life. There's something that needs to happen first. Maybe. But if the software were 100% finished and ready to launch at the push of a button, would they still be waiting?

One reason to launch quickly is that it forces you to actually *finish* some quantum of work. Nothing is truly finished till it's released; you can see that from the rush of work that's always involved in releasing anything, no matter how finished you thought it was. The other reason you need to launch is that it's only by bouncing your idea off users that you fully understand it.

Several distinct problems manifest themselves as delays in launching: working too slowly; not truly understanding the problem; fear of having to deal with users; fear of being judged; working on too many different things; excessive perfectionism. Fortunately you can combat all of them by the simple expedient of forcing yourself to launch *something* fairly quickly.

9. Launching Too Early

Launching too slowly has probably killed a hundred times more startups than launching too fast, but it is possible to launch too fast. The danger here is that you ruin your reputation. You launch something, the early adopters try it out, and if it's no good they may never come back.

So what's the minimum you need to launch? We suggest startups think

³Steve Jobs tried to motivate people by saying "Real artists ship." This is a fine sentence, but unfortunately not true. Many famous works of art are unfinished. It's true in fields that have hard deadlines, like architecture and filmmaking, but even there people tend to be tweaking stuff till it's yanked out of their hands.

about what they plan to do, identify a core that's both (a) useful on its own and (b) something that can be incrementally expanded into the whole project, and then get that done as soon as possible.

This is the same approach I (and many other programmers) use for writing software. Think about the overall goal, then start by writing the smallest subset of it that does anything useful. If it's a subset, you'll have to write it anyway, so in the worst case you won't be wasting your time. But more likely you'll find that implementing a working subset is both good for morale and helps you see more clearly what the rest should do.

The early adopters you need to impress are fairly tolerant. They don't expect a newly launched product to do everything; it just has to do *something*.

10. Having No Specific User in Mind

You can't build things users like without understanding them. I mentioned earlier that the most successful startups seem to have begun by trying to solve a problem their founders had. Perhaps there's a rule here: perhaps you create wealth in proportion to how well you understand the problem you're solving, and the problems you understand best are your own.⁴

That's just a theory. What's not a theory is the converse: if you're trying to solve problems you don't understand, you're hosed.

And yet a surprising number of founders seem willing to assume that someone, they're not sure exactly who, will want what they're building. Do the founders want it? No, they're not the target market. Who is? Teenagers. People interested in local events (that one is a perennial tarpit). Or "business" users. What business users? Gas stations? Movie studios? Defense contractors?

You can of course build something for users other than yourself. We

⁴There's probably also a second factor: startup founders tend to be at the leading edge of technology, so problems they face are probably especially valuable.

did. But you should realize you're stepping into dangerous territory. You're flying on instruments, in effect, so you should (a) consciously shift gears, instead of assuming you can rely on your intuitions as you ordinarily would, and (b) look at the instruments.

In this case the instruments are the users. When designing for other people you have to be empirical. You can no longer guess what will work; you have to find users and measure their responses. So if you're going to make something for teenagers or "business" users or some other group that doesn't include you, you have to be able to talk some specific ones into using what you're making. If you can't, you're on the wrong track.

11. Raising Too Little Money

Most successful startups take funding at some point. Like having more than one founder, it seems a good bet statistically. How much should you take, though?

Startup funding is measured in time. Every startup that isn't profitable (meaning nearly all of them, initially) has a certain amount of time left before the money runs out and they have to stop. This is sometimes referred to as runway, as in "How much runway do you have left?" It's a good metaphor because it reminds you that when the money runs out you're going to be airborne or dead.

Too little money means not enough to get airborne. What airborne means depends on the situation. Usually you have to advance to a visibly higher level: if all you have is an idea, a working prototype; if you have a prototype, launching; if you're launched, significant growth. It depends on investors, because until you're profitable that's who you have to convince.

So if you take money from investors, you have to take enough to get to the next step, whatever that is.⁵ Fortunately you have some control over both how much you spend and what the next step is. We ad-

⁵You should take more than you think you'll need, maybe 50% to 100% more, because software takes longer to write and deals longer to close than you expect.

wise startups to set both low, initially: spend practically nothing, and make your initial goal simply to build a solid prototype. This gives you maximum flexibility.

12. Spending Too Much

It's hard to distinguish spending too much from raising too little. If you run out of money, you could say either was the cause. The only way to decide which to call it is by comparison with other startups. If you raised five million and ran out of money, you probably spent too much.

Burning through too much money is not as common as it used to be. Founders seem to have learned that lesson. Plus it keeps getting cheaper to start a startup. So as of this writing few startups spend too much. None of the ones we've funded have. (And not just because we make small investments; many have gone on to raise further rounds.)

The classic way to burn through cash is by hiring a lot of people. This bites you twice: in addition to increasing your costs, it slows you down—so money that's getting consumed faster has to last longer. Most hackers understand why that happens; Fred Brooks explained it in *The Mythical Man-Month*.

We have three general suggestions about hiring: (a) don't do it if you can avoid it, (b) pay people with equity rather than salary, not just to save money, but because you want the kind of people who are committed enough to prefer that, and (c) only hire people who are either going to write code or go out and get users, because those are the only things you need at first.

13. Raising Too Much Money

It's obvious how too little money could kill you, but is there such a thing as having too much?

Yes and no. The problem is not so much the money itself as what comes with it. As one VC who spoke at Y Combinator said, "Once you take several million dollars of my money, the clock is ticking." If

VCs fund you, they're not going to let you just put the money in the bank and keep operating as two guys living on ramen. They want that money to go to work.⁶ At the very least you'll move into proper office space and hire more people. That will change the atmosphere, and not entirely for the better. Now most of your people will be employees rather than founders. They won't be as committed; they'll need to be told what to do; they'll start to engage in office politics.

When you raise a lot of money, your company moves to the suburbs and has kids.

Perhaps more dangerously, once you take a lot of money it gets harder to change direction. Suppose your initial plan was to sell something to companies. After taking VC money you hire a sales force to do that. What happens now if you realize you should be making this for consumers instead of businesses? That's a completely different kind of selling. What happens, in practice, is that you don't realize that. The more people you have, the more you stay pointed in the same direction.

Another drawback of large investments is the time they take. The time required to raise money grows with the amount.⁷ When the amount rises into the millions, investors get very cautious. VCs never quite say yes or no; they just engage you in an apparently endless conversation. Raising VC scale investments is thus a huge time sink — more work, probably, than the startup itself. And you don't want to be spending all your time talking to investors while your competitors are spending theirs building things.

⁶Since people sometimes call us VCs, I should add that we're not. VCs invest large amounts of other people's money. We invest small amounts of our own, like angel investors.

⁷Not linearly of course, or it would take forever to raise five million dollars. In practice it just feels like it takes forever.

Though if you include the cases where VCs don't invest, it would literally take forever in the median case. And maybe we should, because the danger of chasing large investments is not just that they take a long time. That's the *best* case. The real danger is that you'll expend a lot of time and get nothing.

We advise founders who go on to seek VC money to take the first reasonable deal they get. If you get an offer from a reputable firm at a reasonable valuation with no unusually onerous terms, just take it and get on with building the company.⁸ Who cares if you could get a 30% better deal elsewhere? Economically, startups are an all-or-nothing game. Bargain-hunting among investors is a waste of time.

14. Poor Investor Management

As a founder, you have to manage your investors. You shouldn't ignore them, because they may have useful insights. But neither should you let them run the company. That's supposed to be your job. If investors had sufficient vision to run the companies they fund, why didn't they start them?

Pissing off investors by ignoring them is probably less dangerous than caving in to them. In our startup, we erred on the ignoring side. A lot of our energy got drained away in disputes with investors instead of going into the product. But this was less costly than giving in, which would probably have destroyed the company. If the founders know what they're doing, it's better to have half their attention focused on the product than the full attention of investors who don't.

How hard you have to work on managing investors usually depends on how much money you've taken. When you raise VC-scale money, the investors get a great deal of control. If they have a board majority, they're literally your bosses. In the more common case, where founders and investors are equally represented and the deciding vote is cast by neutral outside directors, all the investors have to do is convince the outside directors and they control the company.

If things go well, this shouldn't matter. So long as you seem to be advancing rapidly, most investors will leave you alone. But things don't always go smoothly in startups. Investors have made trouble even for

⁸Some VCs will offer you an artificially low valuation to see if you have the balls to ask for more. It's lame that VCs play such games, but some do. If you're dealing with one of those you should push back on the valuation a bit.

the most successful companies. One of the most famous examples is Apple, whose board made a nearly fatal blunder in firing Steve Jobs. Apparently even Google got a lot of grief from their investors early on.

15. Sacrificing Users to (Supposed) Profit

When I said at the beginning that if you make something users want, you'll be fine, you may have noticed I didn't mention anything about having the right business model. That's not because making money is unimportant. I'm not suggesting that founders start companies with no chance of making money in the hope of unloading them before they tank. The reason we tell founders not to worry about the business model initially is that making something people want is so much harder.

I don't know why it's so hard to make something people want. It seems like it should be straightforward. But you can tell it must be hard by how few startups do it.

Because making something people want is so much harder than making money from it, you should leave business models for later, just as you'd leave some trivial but messy feature for version 2. In version 1, solve the core problem. And the core problem in a startup is how to create wealth (= how much people want something x the number who want it), not how to convert that wealth into money.

The companies that win are the ones that put users first. Google, for example. They made search work, then worried about how to make money from it. And yet some startup founders still think it's irresponsible not to focus on the business model from the beginning. They're often encouraged in this by investors whose experience comes from less malleable industries.

It is irresponsible not to think about business models. It's just ten times more irresponsible not to think about the product.

16. Not Wanting to Get Your Hands Dirty

Nearly all programmers would rather spend their time writing code and have someone else handle the messy business of extracting money from it. And not just the lazy ones. Larry and Sergey apparently felt this way too at first. After developing their new search algorithm, the first thing they tried was to get some other company to buy it.

Start a company? Yech. Most hackers would rather just have ideas. But as Larry and Sergey found, there's not much of a market for ideas. No one trusts an idea till you embody it in a product and use that to grow a user base. Then they'll pay big time.

Maybe this will change, but I doubt it will change much. There's nothing like users for convincing acquirers. It's not just that the risk is decreased. The acquirers are human, and they have a hard time paying a bunch of young guys millions of dollars just for being clever. When the idea is embodied in a company with a lot of users, they can tell themselves they're buying the users rather than the cleverness, and this is easier for them to swallow.⁹

If you're going to attract users, you'll probably have to get up from your computer and go find some. It's unpleasant work, but if you can make yourself do it you have a much greater chance of succeeding. In the first batch of startups we funded, in the summer of 2005, most of the founders spent all their time building their applications. But there was one who was away half the time talking to executives at cell phone companies, trying to arrange deals. Can you imagine anything more painful for a hacker?¹⁰ But it paid off, because this startup seems the most successful of that group by an order of magnitude.

⁹Suppose YouTube's founders had gone to Google in 2005 and told them "Google Video is badly designed. Give us \$10 million and we'll tell you all the mistakes you made." They would have gotten the royal raspberry. Eighteen months later Google paid \$1.6 billion for the same lesson, partly because they could then tell themselves that they were buying a phenomenon, or a community, or some vague thing like that.

I don't mean to be hard on Google. They did better than their competitors, who may have now missed the video boat entirely.

¹⁰Yes, actually: dealing with the government. But phone companies are up there.

If you want to start a startup, you have to face the fact that you can't just hack. At least one hacker will have to spend some of the time doing business stuff.

17. Fights Between Founders

Fights between founders are surprisingly common. About 20% of the startups we've funded have had a founder leave. It happens so often that we've reversed our attitude to vesting. We still don't require it, but now we advise founders to vest so there will be an orderly way for people to quit.

A founder leaving doesn't necessarily kill a startup, though. Plenty of successful startups have had that happen.¹¹ Fortunately it's usually the least committed founder who leaves. If there are three founders and one who was lukewarm leaves, big deal. If you have two and one leaves, or a guy with critical technical skills leaves, that's more of a problem. But even that is survivable. Blogger got down to one person, and they bounced back.

Most of the disputes I've seen between founders could have been avoided if they'd been more careful about who they started a company with. Most disputes are not due to the situation but the people. Which means they're inevitable. And most founders who've been burned by such disputes probably had misgivings, which they suppressed, when they started the company. Don't suppress misgivings. It's much easier to fix problems before the company is started than after. So don't include your housemate in your startup because he'd feel left out otherwise. Don't start a company with someone you dislike because they have some skill you need and you worry you won't find anyone else. The people are the most important ingredient in a startup, so don't compromise there.

18. A Half-Hearted Effort

The failed startups you hear most about are the spectacular flameouts.

¹¹Many more than most people realize, because companies don't advertise this. Did you know Apple originally had three founders?

Those are actually the elite of failures. The most common type is not the one that makes spectacular mistakes, but the one that doesn't do much of anything — the one we never even hear about, because it was some project a couple guys started on the side while working on their day jobs, but which never got anywhere and was gradually abandoned.

Statistically, if you want to avoid failure, it would seem like the most important thing is to quit your day job. Most founders of failed startups don't quit their day jobs, and most founders of successful ones do. If startup failure were a disease, the CDC would be issuing bulletins warning people to avoid day jobs.

Does that mean you should quit your day job? Not necessarily. I'm guessing here, but I'd guess that many of these would-be founders may not have the kind of determination it takes to start a company, and that in the back of their minds, they know it. The reason they don't invest more time in their startup is that they know it's a bad investment.¹²

I'd also guess there's some band of people who could have succeeded if they'd taken the leap and done it full-time, but didn't. I have no idea how wide this band is, but if the winner/borderline/hopeless progression has the sort of distribution you'd expect, the number of people who could have made it, if they'd quit their day job, is probably an order of magnitude larger than the number who do make it.¹³

If that's true, most startups that could succeed fail because the

¹²I'm not dissing these people. I don't have the determination myself. I've twice come close to starting startups since Viaweb, and both times I bailed because I realized that without the spur of poverty I just wasn't willing to endure the stress of a startup.

¹³So how do you know whether you're in the category of people who should quit their day job, or the presumably larger one who shouldn't? I got to the point of saying that this was hard to judge for yourself and that you should seek outside advice, before realizing that that's what we do. We think of ourselves as investors, but viewed from the other direction Y Combinator is a service for advising people whether or not to quit their day job. We could be mistaken, and no doubt often are, but we do at least bet money on our conclusions.

founders don't devote their whole efforts to them. That certainly accords with what I see out in the world. Most startups fail because they don't make something people want, and the reason most don't is that they don't try hard enough.

In other words, starting startups is just like everything else. The biggest mistake you can make is not to try hard enough. To the extent there's a secret to success, it's not to be in denial about that.

Thanks to Sam Altman, Jessica Livingston, Greg McAdoo, and Robert Morris for reading drafts of this.

Chapter 2

How Art Can Be Good



December 2006

I grew up believing that taste is just a matter of personal preference. Each person has things they like, but no one's preferences are any better than anyone else's. There is no such thing as *good* taste.

Like a lot of things I grew up believing, this turns out to be false, and I'm going to try to explain why.

One problem with saying there's no such thing as good taste is that it also means there's no such thing as good art. If there were good art, then people who liked it would have better taste than people who didn't. So if you discard taste, you also have to discard the idea of art being good, and artists being good at making it.

It was pulling on that thread that unravelled my childhood faith in relativism. When you're trying to make things, taste becomes a practical matter. You have to decide what to do next. Would it make the painting better if I changed that part? If there's no such thing as better, it doesn't matter what you do. In fact, it doesn't matter if you paint at all. You could just go out and buy a ready-made blank canvas. If there's no such thing as good, that would be just as great an achievement as the ceiling of the Sistine Chapel. Less laborious, certainly, but if you

can achieve the same level of performance with less effort, surely that's more impressive, not less.

Yet that doesn't seem quite right, does it?

Audience

I think the key to this puzzle is to remember that art has an audience. Art has a purpose, which is to interest its audience. Good art (like good anything) is art that achieves its purpose particularly well. The meaning of "interest" can vary. Some works of art are meant to shock, and others to please; some are meant to jump out at you, and others to sit quietly in the background. But all art has to work on an audience, and—here's the critical point—members of the audience share things in common.

For example, nearly all humans find human faces engaging. It seems to be wired into us. Babies can recognize faces practically from birth. In fact, faces seem to have co-evolved with our interest in them; the face is the body's billboard. So all other things being equal, a painting with faces in it will interest people more than one without.¹

One reason it's easy to believe that taste is merely personal preference is that, if it isn't, how do you pick out the people with better taste? There are billions of people, each with their own opinion; on what grounds can you prefer one to another?²

But if audiences have a lot in common, you're not in a position of having to choose one out of a random set of individual biases, because the set isn't random. All humans find faces engaging—practically by

¹This is not to say, of course, that good paintings must have faces in them, just that everyone's visual piano has that key on it. There are situations in which you want to avoid faces, precisely because they attract so much attention. But you can see how universally faces work by their prevalence in advertising.

²The other reason it's easy to believe is that it makes people feel good. To a kid, this idea is crack. In every other respect they're constantly being told that they have a lot to learn. But in this they're perfect. Their opinion carries the same weight as any adult's. You should probably question anything you believed as a kid that you'd want to believe this much.

definition: face recognition is in our DNA. And so having a notion of good art, in the sense of art that does its job well, doesn't require you to pick out a few individuals and label their opinions as correct. No matter who you pick, they'll find faces engaging.

Of course, space aliens probably wouldn't find human faces engaging. But there might be other things they shared in common with us. The most likely source of examples is math. I expect space aliens would agree with us most of the time about which of two proofs was better. Erdos thought so. He called a maximally elegant proof one out of God's book, and presumably God's book is universal.³

Once you start talking about audiences, you don't have to argue simply that there are or aren't standards of taste. Instead tastes are a series of concentric rings, like ripples in a pond. There are some things that will appeal to you and your friends, others that will appeal to most people your age, others that will appeal to most humans, and perhaps others that would appeal to most sentient beings (whatever that means).

The picture is slightly more complicated than that, because in the middle of the pond there are overlapping sets of ripples. For example, there might be things that appealed particularly to men, or to people from a certain culture.

If good art is art that interests its audience, then when you talk about art being good, you also have to say for what audience. So is it meaningless to talk about art simply being good or bad? No, because one audience is the set of all possible humans. I think that's the audience people are implicitly talking about when they say a work of art is good: they mean it would engage any human.⁴

³It's conceivable that the elegance of proofs is quantifiable, in the sense that there may be some formal measure that turns out to coincide with mathematicians' judgements. Perhaps it would be worth trying to make a formal language for proofs in which those considered more elegant consistently came out shorter (perhaps after being macroexpanded or compiled).

⁴Maybe it would be possible to make art that would appeal to space aliens, but

And that is a meaningful test, because although, like any everyday concept, “human” is fuzzy around the edges, there are a lot of things practically all humans have in common. In addition to our interest in faces, there’s something special about primary colors for nearly all of us, because it’s an artifact of the way our eyes work. Most humans will also find images of 3D objects engaging, because that also seems to be built into our visual perception.⁵ And beneath that there’s edge-finding, which makes images with definite shapes more engaging than mere blur.

Humans have a lot more in common than this, of course. My goal is not to compile a complete list, just to show that there’s some solid ground here. People’s preferences aren’t random. So an artist working on a painting and trying to decide whether to change some part of it doesn’t have to think “Why bother? I might as well flip a coin.” Instead he can ask “What would make the painting more interesting to people?” And the reason you can’t equal Michelangelo by going out and buying a blank canvas is that the ceiling of the Sistine Chapel is more interesting to people.

A lot of philosophers have had a hard time believing it was possible for there to be objective standards for art. It seemed obvious that beauty, for example, was something that happened in the head of the observer, not something that was a property of objects. It was thus “subjective” rather than “objective.” But in fact if you narrow the definition of beauty to something that works a certain way on humans, and you observe how much humans have in common, it turns out to be a property of objects after all. You don’t have to choose between something being a property of the subject or the object if subjects all react similarly. Being good art is thus a property of objects as much

I’m not going to get into that because (a) it’s too hard to answer, and (b) I’m satisfied if I can establish that good art is a meaningful idea for human audiences.

⁵If early abstract paintings seem more interesting than later ones, it may be because the first abstract painters were trained to paint from life, and their hands thus tended to make the kind of gestures you use in representing physical things. In effect they were saying “scaramara” instead of “uebfghbsb.”

as, say, being toxic to humans is: it's good art if it consistently affects humans in a certain way.

Error

So could we figure out what the best art is by taking a vote? After all, if appealing to humans is the test, we should be able to just ask them, right?

Well, not quite. For products of nature that might work. I'd be willing to eat the apple the world's population had voted most delicious, and I'd probably be willing to visit the beach they voted most beautiful, but having to look at the painting they voted the best would be a crapshoot.

Man-made stuff is different. For one thing, artists, unlike apple trees, often deliberately try to trick us. Some tricks are quite subtle. For example, any work of art sets expectations by its level of finish. You don't expect photographic accuracy in something that looks like a quick sketch. So one widely used trick, especially among illustrators, is to intentionally make a painting or drawing look like it was done faster than it was. The average person looks at it and thinks: how amazingly skillful. It's like saying something clever in a conversation as if you'd thought of it on the spur of the moment, when in fact you'd worked it out the day before.

Another much less subtle influence is brand. If you go to see the Mona Lisa, you'll probably be disappointed, because it's hidden behind a thick glass wall and surrounded by a frenzied crowd taking pictures of themselves in front of it. At best you can see it the way you see a friend across the room at a crowded party. The Louvre might as well replace it with copy; no one would be able to tell. And yet the Mona Lisa is a small, dark painting. If you found people who'd never seen an image of it and sent them to a museum in which it was hanging among other paintings with a tag labelling it as a portrait by an unknown fifteenth century artist, most would walk by without giving it a second look.

For the average person, brand dominates all other factors in the judge-

ment of art. Seeing a painting they recognize from reproductions is so overwhelming that their response to it as a painting is drowned out.

And then of course there are the tricks people play on themselves. Most adults looking at art worry that if they don't like what they're supposed to, they'll be thought uncultured. This doesn't just affect what they claim to like; they actually make themselves like things they're supposed to.

That's why you can't just take a vote. Though appeal to people is a meaningful test, in practice you can't measure it, just as you can't find north using a compass with a magnet sitting next to it. There are sources of error so powerful that if you take a vote, all you're measuring is the error.

We can, however, approach our goal from another direction, by using ourselves as guinea pigs. You're human. If you want to know what the basic human reaction to a piece of art would be, you can at least approach that by getting rid of the sources of error in your own judgments.

For example, while anyone's reaction to a famous painting will be warped at first by its fame, there are ways to decrease its effects. One is to come back to the painting over and over. After a few days the fame wears off, and you can start to see it as a painting. Another is to stand close. A painting familiar from reproductions looks more familiar from ten feet away; close in you see details that get lost in reproductions, and which you're therefore seeing for the first time.

There are two main kinds of error that get in the way of seeing a work of art: biases you bring from your own circumstances, and tricks played by the artist. Tricks are straightforward to correct for. Merely being aware of them usually prevents them from working. For example, when I was ten I used to be very impressed by airbrushed lettering that looked like shiny metal. But once you study how it's done, you see that it's a pretty cheesy trick—one of the sort that relies on pushing a few visual buttons really hard to temporarily overwhelm the viewer.

It's like trying to convince someone by shouting at them.

The way not to be vulnerable to tricks is to explicitly seek out and catalog them. When you notice a whiff of dishonesty coming from some kind of art, stop and figure out what's going on. When someone is obviously pandering to an audience that's easily fooled, whether it's someone making shiny stuff to impress ten year olds, or someone making conspicuously avant-garde stuff to impress would-be intellectuals, learn how they do it. Once you've seen enough examples of specific types of tricks, you start to become a connoisseur of trickery in general, just as professional magicians are.

What counts as a trick? Roughly, it's something done with contempt for the audience. For example, the guys designing Ferraris in the 1950s were probably designing cars that they themselves admired. Whereas I suspect over at General Motors the marketing people are telling the designers, "Most people who buy SUVs do it to seem manly, not to drive off-road. So don't worry about the suspension; just make that sucker as big and tough-looking as you can."⁶

I think with some effort you can make yourself nearly immune to tricks. It's harder to escape the influence of your own circumstances, but you can at least move in that direction. The way to do it is to travel widely, in both time and space. If you go and see all the different kinds of things people like in other cultures, and learn about all the different things people have liked in the past, you'll probably find it changes what you like. I doubt you could ever make yourself into a completely universal person, if only because you can only travel in one direction in time. But if you find a work of art that would appeal equally to your friends, to people in Nepal, and to the ancient Greeks, you're probably onto something.

My main point here is not how to have good taste, but that there can even be such a thing. And I think I've shown that. There is such a

⁶It's a bit more complicated, because sometimes artists unconsciously use tricks by imitating art that does.

thing as good art. It's art that interests its human audience, and since humans have a lot in common, what interests them is not random. Since there's such a thing as good art, there's also such a thing as good taste, which is the ability to recognize it.

If we were talking about the taste of apples, I'd agree that taste is just personal preference. Some people like certain kinds of apples and others like other kinds, but how can you say that one is right and the other wrong? ⁷

The thing is, art isn't apples. Art is man-made. It comes with a lot of cultural baggage, and in addition the people who make it often try to trick us. Most people's judgement of art is dominated by these extraneous factors; they're like someone trying to judge the taste of apples in a dish made of equal parts apples and jalapeno peppers. All they're tasting is the peppers. So it turns out you can pick out some people and say that they have better taste than others: they're the ones who actually taste art like apples.

Or to put it more prosaically, they're the people who (a) are hard to trick, and (b) don't just like whatever they grew up with. If you could find people who'd eliminated all such influences on their judgement, you'd probably still see variation in what they liked. But because humans have so much in common, you'd also find they agreed on a lot. They'd nearly all prefer the ceiling of the Sistine Chapel to a blank canvas.

Making It

I wrote this essay because I was tired of hearing "taste is subjective" and wanted to kill it once and for all. Anyone who makes things knows intuitively that's not true. When you're trying to make art, the temptation to be lazy is as great as in any other kind of work. Of course it matters to do a good job. And yet you can see how great a hold "taste

⁷I phrased this in terms of the taste of apples because if people can see the apples, they can be fooled. When I was a kid most apples were a variety called Red Delicious that had been bred to look appealing in stores, but which didn't taste very good.

is subjective” has even in the art world by how nervous it makes people to talk about art being good or bad. Those whose jobs require them to judge art, like curators, mostly resort to euphemisms like “significant” or “important” or (getting dangerously close) “realized.”⁸

I don’t have any illusions that being able to talk about art being good or bad will cause the people who talk about it to have anything more useful to say. Indeed, one of the reasons “taste is subjective” found such a receptive audience is that, historically, the things people have said about good taste have generally been such nonsense.

It’s not for the people who talk about art that I want to free the idea of good art, but for those who make it. Right now, ambitious kids going to art school run smack into a brick wall. They arrive hoping one day to be as good as the famous artists they’ve seen in books, and the first thing they learn is that the concept of good has been retired. Instead everyone is just supposed to explore their own personal vision.⁹

When I was in art school, we were looking one day at a slide of some great fifteenth century painting, and one of the students asked “Why don’t artists paint like that now?” The room suddenly got quiet. Though rarely asked out loud, this question lurks uncomfortably in the back of every art student’s mind. It was as if someone had brought up the topic of lung cancer in a meeting within Philip Morris.

“Well,” the professor replied, “we’re interested in different questions now.” He was a pretty nice guy, but at the time I couldn’t help wish-

⁸To be fair, curators are in a difficult position. If they’re dealing with recent art, they have to include things in shows that they think are bad. That’s because the test for what gets included in shows is basically the market price, and for recent art that is largely determined by successful businessmen and their wives. So it’s not always intellectual dishonesty that makes curators and dealers use neutral-sounding language.

⁹What happens in practice is that everyone gets really good at *talking* about art. As the art itself gets more random, the effort that would have gone into the work goes instead into the intellectual sounding theory behind it. “My work represents an exploration of gender and sexuality in an urban context,” etc. Different people win at that game.

ing I could send him back to fifteenth century Florence to explain in person to Leonardo & Co. how we had moved beyond their early, limited concept of art. Just imagine that conversation.

In fact, one of the reasons artists in fifteenth century Florence made such great things was that they believed you could make great things.¹⁰ They were intensely competitive and were always trying to outdo one another, like mathematicians or physicists today—maybe like anyone who has ever done anything really well.

The idea that you could make great things was not just a useful illusion. They were actually right. So the most important consequence of realizing there can be good art is that it frees artists to try to make it. To the ambitious kids arriving at art school this year hoping one day to make great things, I say: don't believe it when they tell you this is a naive and outdated ambition. There is such a thing as good art, and if you try to make it, there are people who will notice.

Thanks to Trevor Blackwell, Jessica Livingston, and Robert Morris for reading drafts of this, and to Paul Watson for permission to use the image at the top.

¹⁰There were several other reasons, including that Florence was then the richest and most sophisticated city in the world, and that they lived in a time before photography had (a) killed portraiture as a source of income and (b) made brand the dominant factor in the sale of art.

Incidentally, I'm not saying that good art = fifteenth century European art. I'm not saying we should make what they made, but that we should work like they worked. There are fields now in which many people work with the same energy and honesty that fifteenth century artists did, but art is not one of them.

Chapter 3

Learning from Founders

January 2007

(Foreword to Jessica Livingston's Founders at Work.)

Apparently sprinters reach their highest speed right out of the blocks, and spend the rest of the race slowing down. The winners slow down the least. It's that way with most startups too. The earliest phase is usually the most productive. That's when they have the really big ideas. Imagine what Apple was like when 100% of its employees were either Steve Jobs or Steve Wozniak.

The striking thing about this phase is that it's completely different from most people's idea of what business is like. If you looked in people's heads (or stock photo collections) for images representing "business," you'd get images of people dressed up in suits, groups sitting around conference tables looking serious, Powerpoint presentations, people producing thick reports for one another to read. Early stage startups are the exact opposite of this. And yet they're probably the most productive part of the whole economy.

Why the disconnect? I think there's a general principle at work here: the less energy people expend on performance, the more they expend on appearances to compensate. More often than not the energy they expend on seeming impressive makes their actual performance worse. A few years ago I read an article in which a car magazine modified the "sports" model of some production car to get the fastest possible standing quarter mile. You know how they did it? They cut off all the crap the manufacturer had bolted onto the car to make it *look* fast.

Business is broken the same way that car was. The effort that goes

into looking productive is not merely wasted, but actually makes organizations less productive. Suits, for example. Suits do not help people to think better. I bet most executives at big companies do their best thinking when they wake up on Sunday morning and go downstairs in their bathrobe to make a cup of coffee. That's when you have ideas. Just imagine what a company would be like if people could think that well at work. People do in startups, at least some of the time. (Half the time you're in a panic because your servers are on fire, but the other half you're thinking as deeply as most people only get to sitting alone on a Sunday morning.)

Ditto for most of the other differences between startups and what passes for productivity in big companies. And yet conventional ideas of professionalism have such an iron grip on our minds that even startup founders are affected by them. In our startup, when outsiders came to visit we tried hard to seem "professional." We'd clean up our offices, wear better clothes, try to arrange that a lot of people were there during conventional office hours. In fact, programming didn't get done by well-dressed people at clean desks during office hours. It got done by badly dressed people (I was notorious for programming wearing just a towel) in offices strewn with junk at 2 in the morning. But no visitor would understand that. Not even investors, who are supposed to be able to recognize real productivity when they see it. Even we were affected by the conventional wisdom. We thought of ourselves as impostors, succeeding despite being totally unprofessional. It was as if we'd created a Formula 1 car but felt sheepish because it didn't look like a car was supposed to look.

In the car world, there are at least some people who know that a high performance car looks like a Formula 1 racecar, not a sedan with giant rims and a fake spoiler bolted to the trunk. Why not in business? Probably because startups are so small. The really dramatic growth happens when a startup only has three or four people, so only three or four people see that, whereas tens of thousands see business as it's practiced by Boeing or Philip Morris.

This book can help fix that problem, by showing everyone what, till now, only a handful people got to see: what happens in the first year of a startup. This is what real productivity looks like. This is the Formula 1 racecar. It looks weird, but it goes fast.

Of course, big companies won't be able to do everything these startups do. In big companies there's always going to be more politics, and less scope for individual decisions. But seeing what startups are really like will at least show other organizations what to aim for. The time may soon be coming when instead of startups trying to seem more corporate, corporations will try to seem more like startups. That would be a good thing.

Chapter 4

Is It Worth Being Wise?

February 2007

A few days ago I finally figured out something I've wondered about for 25 years: the relationship between wisdom and intelligence. Anyone can see they're not the same by the number of people who are smart, but not very wise. And yet intelligence and wisdom do seem related. How?

What is wisdom? I'd say it's knowing what to do in a lot of situations. I'm not trying to make a deep point here about the true nature of wisdom, just to figure out how we use the word. A wise person is someone who usually knows the right thing to do.

And yet isn't being smart also knowing what to do in certain situations? For example, knowing what to do when the teacher tells your elementary school class to add all the numbers from 1 to 100? ¹

Some say wisdom and intelligence apply to different types of problems—wisdom to human problems and intelligence to abstract ones. But that isn't true. Some wisdom has nothing to do with people: for example, the wisdom of the engineer who knows certain structures are less prone to failure than others. And certainly smart people can find clever solutions to human problems as well as abstract ones. ²

¹Gauss was supposedly asked this when he was 10. Instead of laboriously adding together the numbers like the other students, he saw that they consisted of 50 pairs that each summed to 101 ($100 + 1$, $99 + 2$, etc), and that he could just multiply 101 by 50 to get the answer, 5050.

²A variant is that intelligence is the ability to solve problems, and wisdom the judgement to know how to use those solutions. But while this is certainly an important relationship between wisdom and intelligence, it's not the *distinction between* them.

Another popular explanation is that wisdom comes from experience while intelligence is innate. But people are not simply wise in proportion to how much experience they have. Other things must contribute to wisdom besides experience, and some may be innate: a reflective disposition, for example.

Neither of the conventional explanations of the difference between wisdom and intelligence stands up to scrutiny. So what is the difference? If we look at how people use the words “wise” and “smart,” what they seem to mean is different shapes of performance.

Curve

“Wise” and “smart” are both ways of saying someone knows what to do. The difference is that “wise” means one has a high average outcome across all situations, and “smart” means one does spectacularly well in a few. That is, if you had a graph in which the x axis represented situations and the y axis the outcome, the graph of the wise person would be high overall, and the graph of the smart person would have high peaks.

The distinction is similar to the rule that one should judge talent at its best and character at its worst. Except you judge intelligence at its best, and wisdom by its average. That’s how the two are related: they’re the two different senses in which the same curve can be high.

So a wise person knows what to do in most situations, while a smart person knows what to do in situations where few others could. We need to add one more qualification: we should ignore cases where someone knows what to do because they have inside information.³

Wisdom is useful in solving problems too, and intelligence can help in deciding what to do with the solutions.

³In judging both intelligence and wisdom we have to factor out some knowledge. People who know the combination of a safe will be better at opening it than people who don’t, but no one would say that was a test of intelligence or wisdom.

But knowledge overlaps with wisdom and probably also intelligence. A knowledge of human nature is certainly part of wisdom. So where do we draw the line?

Perhaps the solution is to discount knowledge that at some point has a sharp drop

But aside from that, I don't think we can get much more specific without starting to be mistaken.

Nor do we need to. Simple as it is, this explanation predicts, or at least accords with, both of the conventional stories about the distinction between wisdom and intelligence. Human problems are the most common type, so being good at solving those is key in achieving a high average outcome. And it seems natural that a high average outcome depends mostly on experience, but that dramatic peaks can only be achieved by people with certain rare, innate qualities; nearly anyone can learn to be a good swimmer, but to be an Olympic swimmer you need a certain body type.

This explanation also suggests why wisdom is such an elusive concept: there's no such thing. "Wise" means something—that one is on average good at making the right choice. But giving the name "wisdom" to the supposed quality that enables one to do that doesn't mean such a thing exists. To the extent "wisdom" means anything, it refers to a grab-bag of qualities as various as self-discipline, experience, and empathy.⁴

in utility. For example, understanding French will help you in a large number of situations, but its value drops sharply as soon as no one else involved knows French. Whereas the value of understanding vanity would decline more gradually.

The knowledge whose utility drops sharply is the kind that has little relation to other knowledge. This includes mere conventions, like languages and safe combinations, and also what we'd call "random" facts, like movie stars' birthdays, or how to distinguish 1956 from 1957 Studebakers.

⁴People seeking some single thing called "wisdom" have been fooled by grammar. Wisdom is just knowing the right thing to do, and there are a hundred and one different qualities that help in that. Some, like selflessness, might come from meditating in an empty room, and others, like a knowledge of human nature, might come from going to drunken parties.

Perhaps realizing this will help dispel the cloud of semi-sacred mystery that surrounds wisdom in so many people's eyes. The mystery comes mostly from looking for something that doesn't exist. And the reason there have historically been so many different schools of thought about how to achieve wisdom is that they've focused on different components of it.

When I use the word "wisdom" in this essay, I mean no more than whatever col-

Likewise, though “intelligent” means something, we’re asking for trouble if we insist on looking for a single thing called “intelligence.” And whatever its components, they’re not all innate. We use the word “intelligent” as an indication of ability: a smart person can grasp things few others could. It does seem likely there’s some inborn predisposition to intelligence (and wisdom too), but this predisposition is not itself intelligence.

One reason we tend to think of intelligence as inborn is that people trying to measure it have concentrated on the aspects of it that are most measurable. A quality that’s inborn will obviously be more convenient to work with than one that’s influenced by experience, and thus might vary in the course of a study. The problem comes when we drag the word “intelligence” over onto what they’re measuring. If they’re measuring something inborn, they can’t be measuring intelligence. Three year olds aren’t smart. When we describe one as smart, it’s shorthand for “smarter than other three year olds.”

Split

Perhaps it’s a technicality to point out that a predisposition to intelligence is not the same as intelligence. But it’s an important technicality, because it reminds us that we can become smarter, just as we can become wiser.

The alarming thing is that we may have to choose between the two.

If wisdom and intelligence are the average and peaks of the same curve, then they converge as the number of points on the curve decreases. If there’s just one point, they’re identical: the average and maximum are the same. But as the number of points increases, wisdom and intelligence diverge. And historically the number of points on the curve seems to have been increasing: our ability is tested in an ever wider range of situations.

In the time of Confucius and Socrates, people seem to have regarded

lection of qualities helps people make the right choice in a wide variety of situations.

wisdom, learning, and intelligence as more closely related than we do. Distinguishing between “wise” and “smart” is a modern habit.⁵ And the reason we do is that they’ve been diverging. As knowledge gets more specialized, there are more points on the curve, and the distinction between the spikes and the average becomes sharper, like a digital image rendered with more pixels.

One consequence is that some old recipes may have become obsolete. At the very least we have to go back and figure out if they were really recipes for wisdom or intelligence. But the really striking change, as intelligence and wisdom drift apart, is that we may have to decide which we prefer. We may not be able to optimize for both simultaneously.

Society seems to have voted for intelligence. We no longer admire the sage—not the way people did two thousand years ago. Now we admire the genius. Because in fact the distinction we began with has a rather brutal converse: just as you can be smart without being very wise, you can be wise without being very smart. That doesn’t sound especially admirable. That gets you James Bond, who knows what to do in a lot of situations, but has to rely on Q for the ones involving math.

Intelligence and wisdom are obviously not mutually exclusive. In fact, a high average may help support high peaks. But there are reasons to believe that at some point you have to choose between them. One is the example of very smart people, who are so often unwise that in popular culture this now seems to be regarded as the rule rather than the exception. Perhaps the absent-minded professor is wise in his way, or wiser than he seems, but he’s not wise in the way Confucius or Socrates wanted people to be.⁶

⁵Even in English, our sense of the word “intelligence” is surprisingly recent. Predecessors like “understanding” seem to have had a broader meaning.

⁶There is of course some uncertainty about how closely the remarks attributed to Confucius and Socrates resemble their actual opinions. I’m using these names as we use the name “Homer,” to mean the hypothetical people who said the things attributed to them.

New

For both Confucius and Socrates, wisdom, virtue, and happiness were necessarily related. The wise man was someone who knew what the right choice was and always made it; to be the right choice, it had to be morally right; he was therefore always happy, knowing he'd done the best he could. I can't think of many ancient philosophers who would have disagreed with that, so far as it goes.

"The superior man is always happy; the small man sad," said Confucius.⁷

Whereas a few years ago I read an interview with a mathematician who said that most nights he went to bed discontented, feeling he hadn't made enough progress.⁸ The Chinese and Greek words we translate as "happy" didn't mean exactly what we do by it, but there's enough overlap that this remark contradicts them.

Is the mathematician a small man because he's discontented? No; he's just doing a kind of work that wasn't very common in Confucius's day.

Human knowledge seems to grow fractally. Time after time, something that seemed a small and uninteresting area—experimental error, even—turns out, when examined up close, to have as much in it as all knowledge up to that point. Several of the fractal buds that have exploded since ancient times involve inventing and discovering new things. Math, for example, used to be something a handful of people did part-time. Now it's the career of thousands. And in work that involves making new things, some old rules don't apply.

⁷*Analects* VII:36, Fung trans.

Some translators use "calm" instead of "happy." One source of difficulty here is that present-day English speakers have a different idea of happiness from many older societies. Every language probably has a word meaning "how one feels when things are going well," but different cultures react differently when things go well. We react like children, with smiles and laughter. But in a more reserved society, or in one where life was tougher, the reaction might be a quiet contentment.

⁸It may have been Andrew Wiles, but I'm not sure. If anyone remembers such an interview, I'd appreciate hearing from you.

Recently I've spent some time advising people, and there I find the ancient rule still works: try to understand the situation as well as you can, give the best advice you can based on your experience, and then don't worry about it, knowing you did all you could. But I don't have anything like this serenity when I'm writing an essay. Then I'm worried. What if I run out of ideas? And when I'm writing, four nights out of five I go to bed discontented, feeling I didn't get enough done.

Advising people and writing are fundamentally different types of work. When people come to you with a problem and you have to figure out the right thing to do, you don't (usually) have to invent anything. You just weigh the alternatives and try to judge which is the prudent choice. But *prudence* can't tell me what sentence to write next. The search space is too big.

Someone like a judge or a military officer can in much of his work be guided by duty, but duty is no guide in making things. Makers depend on something more precarious: inspiration. And like most people who lead a precarious existence, they tend to be worried, not contented. In that respect they're more like the small man of Confucius's day, always one bad harvest (or ruler) away from starvation. Except instead of being at the mercy of weather and officials, they're at the mercy of their own imagination.

Limits

To me it was a relief just to realize it might be ok to be discontented. The idea that a successful person should be happy has thousands of years of momentum behind it. If I was any good, why didn't I have the easy confidence winners are supposed to have? But that, I now believe, is like a runner asking "If I'm such a good athlete, why do I feel so tired?" Good runners still get tired; they just get tired at higher speeds.

People whose work is to invent or discover things are in the same position as the runner. There's no way for them to do the best they can, because there's no limit to what they could do. The closest you can

come is to compare yourself to other people. But the better you do, the less this matters. An undergrad who gets something published feels like a star. But for someone at the top of the field, what's the test of doing well? Runners can at least compare themselves to others doing exactly the same thing; if you win an Olympic gold medal, you can be fairly content, even if you think you could have run a bit faster. But what is a novelist to do?

Whereas if you're doing the kind of work in which problems are presented to you and you have to choose between several alternatives, there's an upper bound on your performance: choosing the best every time. In ancient societies, nearly all work seems to have been of this type. The peasant had to decide whether a garment was worth mending, and the king whether or not to invade his neighbor, but neither was expected to invent anything. In principle they could have; the king could have invented firearms, then invaded his neighbor. But in practice innovations were so rare that they weren't expected of you, any more than goalkeepers are expected to score goals.⁹ In practice, it seemed as if there was a correct decision in every situation, and if you made it you'd done your job perfectly, just as a goalkeeper who prevents the other team from scoring is considered to have played a perfect game.

In this world, wisdom seemed paramount.¹⁰ Even now, most people do work in which problems are put before them and they have to choose the best alternative. But as knowledge has grown more specialized, there are more and more types of work in which people have

⁹Confucius claimed proudly that he had never invented anything—that he had simply passed on an accurate account of ancient traditions. [*Analects* VII:1] It's hard for us now to appreciate how important a duty it must have been in preliterate societies to remember and pass on the group's accumulated knowledge. Even in Confucius's time it still seems to have been the first duty of the scholar.

¹⁰The bias toward wisdom in ancient philosophy may be exaggerated by the fact that, in both Greece and China, many of the first philosophers (including Confucius and Plato) saw themselves as teachers of administrators, and so thought disproportionately about such matters. The few people who did invent things, like storytellers, must have seemed an outlying data point that could be ignored.

to make up new things, and in which performance is therefore unbounded. Intelligence has become increasingly important relative to wisdom because there is more room for spikes.

Recipes

Another sign we may have to choose between intelligence and wisdom is how different their recipes are. Wisdom seems to come largely from curing childish qualities, and intelligence largely from cultivating them.

Recipes for wisdom, particularly ancient ones, tend to have a remedial character. To achieve wisdom one must cut away all the debris that fills one's head on emergence from childhood, leaving only the important stuff. Both self-control and experience have this effect: to eliminate the random biases that come from your own nature and from the circumstances of your upbringing respectively. That's not all wisdom is, but it's a large part of it. Much of what's in the sage's head is also in the head of every twelve year old. The difference is that in the head of the twelve year old it's mixed together with a lot of random junk.

The path to intelligence seems to be through working on hard problems. You develop intelligence as you might develop muscles, through exercise. But there can't be too much compulsion here. No amount of discipline can replace genuine curiosity. So cultivating intelligence seems to be a matter of identifying some bias in one's character—some tendency to be interested in certain types of things—and nurturing it. Instead of obliterating your idiosyncrasies in an effort to make yourself a neutral vessel for the truth, you select one and try to grow it from a seedling into a tree.

The wise are all much alike in their wisdom, but very smart people tend to be smart in distinctive ways.

Most of our educational traditions aim at wisdom. So perhaps one reason schools work badly is that they're trying to make intelligence using recipes for wisdom. Most recipes for wisdom have an element of

subjection. At the very least, you're supposed to do what the teacher says. The more extreme recipes aim to break down your individuality the way basic training does. But that's not the route to intelligence. Whereas wisdom comes through humility, it may actually help, in cultivating intelligence, to have a mistakenly high opinion of your abilities, because that encourages you to keep working. Ideally till you realize how mistaken you were.

(The reason it's hard to learn new skills late in life is not just that one's brain is less malleable. Another probably even worse obstacle is that one has higher standards.)

I realize we're on dangerous ground here. I'm not proposing the primary goal of education should be to increase students' "self-esteem." That just breeds laziness. And in any case, it doesn't really fool the kids, not the smart ones. They can tell at a young age that a contest where everyone wins is a fraud.

A teacher has to walk a narrow path: you want to encourage kids to come up with things on their own, but you can't simply applaud everything they produce. You have to be a good audience: appreciative, but not too easily impressed. And that's a lot of work. You have to have a good enough grasp of kids' capacities at different ages to know when to be surprised.

That's the opposite of traditional recipes for education. Traditionally the student is the audience, not the teacher; the student's job is not to invent, but to absorb some prescribed body of material. (The use of the term "recitation" for sections in some colleges is a fossil of this.) The problem with these old traditions is that they're too much influenced by recipes for wisdom.

Different

I deliberately gave this essay a provocative title; of course it's worth being wise. But I think it's important to understand the relationship between intelligence and wisdom, and particularly what seems to be the growing gap between them. That way we can avoid applying rules

and standards to intelligence that are really meant for wisdom. These two senses of “knowing what to do” are more different than most people realize. The path to wisdom is through discipline, and the path to intelligence through carefully selected self-indulgence. Wisdom is universal, and intelligence idiosyncratic. And while wisdom yields calmness, intelligence much of the time leads to discontentment.

That’s particularly worth remembering. A physicist friend recently told me half his department was on Prozac. Perhaps if we acknowledge that some amount of frustration is inevitable in certain kinds of work, we can mitigate its effects. Perhaps we can box it up and put it away some of the time, instead of letting it flow together with everyday sadness to produce what seems an alarmingly large pool. At the very least, we can avoid being discontented about being discontented.

If you feel exhausted, it’s not necessarily because there’s something wrong with you. Maybe you’re just running fast.

Thanks to Trevor Blackwell, Sarah Harlin, Jessica Livingston, and Robert Morris for reading drafts of this.

Chapter 5

Why to Not Not Start a Startup

March 2007

(This essay is derived from talks at the 2007 Startup School and the Berkeley CSUA.)

We've now been doing Y Combinator long enough to have some data about success rates. Our first batch, in the summer of 2005, had eight startups in it. Of those eight, it now looks as if at least four succeeded. Three have been acquired: Reddit was a merger of two, Reddit and Infogami, and a third was acquired that we can't talk about yet. Another from that batch was Loopt, which is doing so well they could probably be acquired in about ten minutes if they wanted to.

So about half the founders from that first summer, less than two years ago, are now rich, at least by their standards. (One thing you learn when you get rich is that there are many degrees of it.)

I'm not ready to predict our success rate will stay as high as 50%. That first batch could have been an anomaly. But we should be able to do better than the oft-quoted (and probably made up) standard figure of 10%. I'd feel safe aiming at 25%.

Even the founders who fail don't seem to have such a bad time. Of those first eight startups, three are now probably dead. In two cases the founders just went on to do other things at the end of the summer. I don't think they were traumatized by the experience. The closest to a traumatic failure was Kiko, whose founders kept working on their startup for a whole year before being squashed by Google Calendar. But they ended up happy. They sold their software on eBay for a quarter of a million dollars. After they paid back their angel investors,

they had about a year's salary each.¹ Then they immediately went on to start a new and much more exciting startup, Justin.TV.

So here is an even more striking statistic: 0% of that first batch had a terrible experience. They had ups and downs, like every startup, but I don't think any would have traded it for a job in a cubicle. And that statistic is probably not an anomaly. Whatever our long-term success rate ends up being, I think the rate of people who wish they'd gotten a regular job will stay close to 0%.

The big mystery to me is: why don't more people start startups? If nearly everyone who does it prefers it to a regular job, and a significant percentage get rich, why doesn't everyone want to do this? A lot of people think we get thousands of applications for each funding cycle. In fact we usually only get several hundred. Why don't more people apply? And while it must seem to anyone watching this world that startups are popping up like crazy, the number is small compared to the number of people with the necessary skills. The great majority of programmers still go straight from college to cubicle, and stay there.

It seems like people are not acting in their own interest. What's going on? Well, I can answer that. Because of Y Combinator's position at the very start of the venture funding process, we're probably the world's leading experts on the psychology of people who aren't sure if they want to start a company.

There's nothing wrong with being unsure. If you're a hacker thinking about starting a startup and hesitating before taking the leap, you're part of a grand tradition. Larry and Sergey seem to have felt the same before they started Google, and so did Jerry and Filo before they started Yahoo. In fact, I'd guess the most successful startups are the ones started by uncertain hackers rather than gung-ho business guys.

We have some evidence to support this. Several of the most successful

¹The only people who lost were us. The angels had convertible debt, so they had first claim on the proceeds of the auction. Y Combinator only got 38 cents on the dollar.

startups we've funded told us later that they only decided to apply at the last moment. Some decided only hours before the deadline.

The way to deal with uncertainty is to analyze it into components. Most people who are reluctant to do something have about eight different reasons mixed together in their heads, and don't know themselves which are biggest. Some will be justified and some bogus, but unless you know the relative proportion of each, you don't know whether your overall uncertainty is mostly justified or mostly bogus.

So I'm going to list all the components of people's reluctance to start startups, and explain which are real. Then would-be founders can use this as a checklist to examine their own feelings.

I admit my goal is to increase your self-confidence. But there are two things different here from the usual confidence-building exercise. One is that I'm motivated to be honest. Most people in the confidence-building business have already achieved their goal when you buy the book or pay to attend the seminar where they tell you how great you are. Whereas if I encourage people to start startups who shouldn't, I make my own life worse. If I encourage too many people to apply to Y Combinator, it just means more work for me, because I have to read all the applications.

The other thing that's going to be different is my approach. Instead of being positive, I'm going to be negative. Instead of telling you "come on, you can do it" I'm going to consider all the reasons you aren't doing it, and show why most (but not all) should be ignored. We'll start with the one everyone's born with.

1. Too young

A lot of people think they're too young to start a startup. Many are right. The median age worldwide is about 27, so probably a third of the population can truthfully say they're too young.

What's too young? One of our goals with Y Combinator was to discover the lower bound on the age of startup founders. It always

seemed to us that investors were too conservative here—that they wanted to fund professors, when really they should be funding grad students or even undergrads.

The main thing we’ve discovered from pushing the edge of this envelope is not where the edge is, but how fuzzy it is. The outer limit may be as low as 16. We don’t look beyond 18 because people younger than that can’t legally enter into contracts. But the most successful founder we’ve funded so far, Sam Altman, was 19 at the time.

Sam Altman, however, is an outlying data point. When he was 19, he seemed like he had a 40 year old inside him. There are other 19 year olds who are 12 inside.

There’s a reason we have a distinct word “adult” for people over a certain age. There is a threshold you cross. It’s conventionally fixed at 21, but different people cross it at greatly varying ages. You’re old enough to start a startup if you’ve crossed this threshold, whatever your age.

How do you tell? There are a couple tests adults use. I realized these tests existed after meeting Sam Altman, actually. I noticed that I felt like I was talking to someone much older. Afterward I wondered, what am I even measuring? What made him seem older?

One test adults use is whether you still have the kid flake reflex. When you’re a little kid and you’re asked to do something hard, you can cry and say “I can’t do it” and the adults will probably let you off. As a kid there’s a magic button you can press by saying “I’m just a kid” that will get you out of most difficult situations. Whereas adults, by definition, are not allowed to flake. They still do, of course, but when they do they’re ruthlessly pruned.

The other way to tell an adult is by how they react to a challenge. Someone who’s not yet an adult will tend to respond to a challenge from an adult in a way that acknowledges their dominance. If an adult says “that’s a stupid idea,” a kid will either crawl away with his tail between his legs, or rebel. But rebelling presumes inferiority as

much as submission. The adult response to “that’s a stupid idea,” is simply to look the other person in the eye and say “Really? Why do you think so?”

There are a lot of adults who still react childishly to challenges, of course. What you don’t often find are kids who react to challenges like adults. When you do, you’ve found an adult, whatever their age.

2. Too inexperienced

I once wrote that startup founders should be at least 23, and that people should work for another company for a few years before starting their own. I no longer believe that, and what changed my mind is the example of the startups we’ve funded.

I still think 23 is a better age than 21. But the best way to get experience if you’re 21 is to start a startup. So, paradoxically, if you’re too inexperienced to start a startup, what you should do is start one. That’s a way more efficient cure for inexperience than a normal job. In fact, getting a normal job may actually make you less able to start a startup, by turning you into a tame animal who thinks he needs an office to work in and a product manager to tell him what software to write.

What really convinced me of this was the Kikos. They started a startup right out of college. Their inexperience caused them to make a lot of mistakes. But by the time we funded their second startup, a year later, they had become extremely formidable. They were certainly not tame animals. And there is no way they’d have grown so much if they’d spent that year working at Microsoft, or even Google. They’d still have been diffident junior programmers.

So now I’d advise people to go ahead and start startups right out of college. There’s no better time to take risks than when you’re young. Sure, you’ll probably fail. But even failure will get you to the ultimate goal faster than getting a job.

It worries me a bit to be saying this, because in effect we’re advising

people to educate themselves by failing at our expense, but it's the truth.

3. Not determined enough

You need a lot of determination to succeed as a startup founder. It's probably the single best predictor of success.

Some people may not be determined enough to make it. It's hard for me to say for sure, because I'm so determined that I can't imagine what's going on in the heads of people who aren't. But I know they exist.

Most hackers probably underestimate their determination. I've seen a lot become visibly more determined as they get used to running a startup. I can think of several we've funded who would have been delighted at first to be bought for \$2 million, but are now set on world domination.

How can you tell if you're determined enough, when Larry and Sergey themselves were unsure at first about starting a company? I'm guessing here, but I'd say the test is whether you're sufficiently driven to work on your own projects. Though they may have been unsure whether they wanted to start a company, it doesn't seem as if Larry and Sergey were meek little research assistants, obediently doing their advisors' bidding. They started projects of their own.

4. Not smart enough

You may need to be moderately smart to succeed as a startup founder. But if you're worried about this, you're probably mistaken. If you're smart enough to worry that you might not be smart enough to start a startup, you probably are.

And in any case, starting a startup just doesn't require that much intelligence. Some startups do. You have to be good at math to write Mathematica. But most companies do more mundane stuff where the decisive factor is effort, not brains. Silicon Valley can warp your perspective on this, because there's a cult of smartness here. People who

aren't smart at least try to act that way. But if you think it takes a lot of intelligence to get rich, try spending a couple days in some of the fancier bits of New York or LA.

If you don't think you're smart enough to start a startup doing something technically difficult, just write enterprise software. Enterprise software companies aren't technology companies, they're sales companies, and sales depends mostly on effort.

5. Know nothing about business

This is another variable whose coefficient should be zero. You don't need to know anything about business to start a startup. The initial focus should be the product. All you need to know in this phase is how to build things people want. If you succeed, you'll have to think about how to make money from it. But this is so easy you can pick it up on the fly.

I get a fair amount of flak for telling founders just to make something great and not worry too much about making money. And yet all the empirical evidence points that way: pretty much 100% of startups that make something popular manage to make money from it. And acquirers tell me privately that revenue is not what they buy startups for, but their strategic value. Which means, because they made something people want. Acquirers know the rule holds for them too: if users love you, you can always make money from that somehow, and if they don't, the cleverest business model in the world won't save you.

So why do so many people argue with me? I think one reason is that they hate the idea that a bunch of twenty year olds could get rich from building something cool that doesn't make any money. They just don't want that to be possible. But how possible it is doesn't depend on how much they want it to be.

For a while it annoyed me to hear myself described as some kind of irresponsible pied piper, leading impressionable young hackers down the road to ruin. But now I realize this kind of controversy is a sign of a good idea.

The most valuable truths are the ones most people don't believe. They're like undervalued stocks. If you start with them, you'll have the whole field to yourself. So when you find an idea you know is good but most people disagree with, you should not merely ignore their objections, but push aggressively in that direction. In this case, that means you should seek out ideas that would be popular but seem hard to make money from.

We'll bet a seed round you can't make something popular that we can't figure out how to make money from.

6. No cofounder

Not having a cofounder is a real problem. A startup is too much for one person to bear. And though we differ from other investors on a lot of questions, we all agree on this. All investors, without exception, are more likely to fund you with a cofounder than without.

We've funded two single founders, but in both cases we suggested their first priority should be to find a cofounder. Both did. But we'd have preferred them to have cofounders before they applied. It's not super hard to get a cofounder for a project that's just been funded, and we'd rather have cofounders committed enough to sign up for something super hard.

If you don't have a cofounder, what should you do? Get one. It's more important than anything else. If there's no one where you live who wants to start a startup with you, move where there are people who do. If no one wants to work with you on your current idea, switch to an idea people want to work on.

If you're still in school, you're surrounded by potential cofounders. A few years out it gets harder to find them. Not only do you have a smaller pool to draw from, but most already have jobs, and perhaps even families to support. So if you had friends in college you used to scheme about startups with, stay in touch with them as well as you can. That may help keep the dream alive.

It's possible you could meet a cofounder through something like a user's group or a conference. But I wouldn't be too optimistic. You need to work with someone to know whether you want them as a cofounder.²

The real lesson to draw from this is not how to find a cofounder, but that you should start startups when you're young and there are lots of them around.

7. No idea

In a sense, it's not a problem if you don't have a good idea, because most startups change their idea anyway. In the average Y Combinator startup, I'd guess 70% of the idea is new at the end of the first three months. Sometimes it's 100%.

In fact, we're so sure the founders are more important than the initial idea that we're going to try something new this funding cycle. We're going to let people apply with no idea at all. If you want, you can answer the question on the application form that asks what you're going to do with "We have no idea." If you seem really good we'll accept you anyway. We're confident we can sit down with you and cook up some promising project.

Really this just codifies what we do already. We put little weight on the idea. We ask mainly out of politeness. The kind of question on the application form that we really care about is the one where we ask what cool things you've made. If what you've made is version one of a promising startup, so much the better, but the main thing we care about is whether you're good at making things. Being lead developer of a popular open source project counts almost as much.

That solves the problem if you get funded by Y Combinator. What about in the general case? Because in another sense, it is a problem if you don't have an idea. If you start a startup with no idea, what do

²The best kind of organization for that might be an open source project, but those don't involve a lot of face to face meetings. Maybe it would be worth starting one that did.

you do next?

So here's the brief recipe for getting startup ideas. Find something that's missing in your own life, and supply that need—no matter how specific to you it seems. Steve Wozniak built himself a computer; who knew so many other people would want them? A need that's narrow but genuine is a better starting point than one that's broad but hypothetical. So even if the problem is simply that you don't have a date on Saturday night, if you can think of a way to fix that by writing software, you're onto something, because a lot of other people have the same problem.

8. No room for more startups

A lot of people look at the ever-increasing number of startups and think “this can't continue.” Implicit in their thinking is a fallacy: that there is some limit on the number of startups there could be. But this is false. No one claims there's any limit on the number of people who can work for salary at 1000-person companies. Why should there be any limit on the number who can work for equity at 5-person companies?³

Nearly everyone who works is satisfying some kind of need. Breaking up companies into smaller units doesn't make those needs go away. Existing needs would probably get satisfied more efficiently by a network of startups than by a few giant, hierarchical organizations, but I don't think that would mean less opportunity, because satisfying current needs would lead to more. Certainly this tends to be the case in individuals. Nor is there anything wrong with that. We take for granted things that medieval kings would have considered effeminate luxuries, like whole buildings heated to spring temperatures year round. And if things go well, our descendants will take for granted things we would consider shockingly luxurious. There is no absolute standard for material wealth. Health care is a component of it, and that alone is a

³There need to be some number of big companies to acquire the startups, so the number of big companies couldn't decrease to zero.

black hole. For the foreseeable future, people will want ever more material wealth, so there is no limit to the amount of work available for companies, and for startups in particular.

Usually the limited-room fallacy is not expressed directly. Usually it's implicit in statements like "there are only so many startups Google, Microsoft, and Yahoo can buy." Maybe, though the list of acquirers is a lot longer than that. And whatever you think of other acquirers, Google is not stupid. The reason big companies buy startups is that they've created something valuable. And why should there be any limit to the number of valuable startups companies can acquire, any more than there is a limit to the amount of wealth individual people want? Maybe there would be practical limits on the number of startups any one acquirer could assimilate, but if there is value to be had, in the form of upside that founders are willing to forgo in return for an immediate payment, acquirers will evolve to consume it. Markets are pretty smart that way.

9. Family to support

This one is real. I wouldn't advise anyone with a family to start a startup. I'm not saying it's a bad idea, just that I don't want to take responsibility for advising it. I'm willing to take responsibility for telling 22 year olds to start startups. So what if they fail? They'll learn a lot, and that job at Microsoft will still be waiting for them if they need it. But I'm not prepared to cross moms.

What you can do, if you have a family and want to start a startup, is start a consulting business you can then gradually turn into a product business. Empirically the chances of pulling that off seem very small. You're never going to produce Google this way. But at least you'll never be without an income.

Another way to decrease the risk is to join an existing startup instead of starting your own. Being one of the first employees of a startup is a lot like being a founder, in both the good ways and the bad. You'll be roughly $1/n^2$ founder, where n is your employee number.

As with the question of cofounders, the real lesson here is to start startups when you're young.

10. Independently wealthy

This is my excuse for not starting a startup. Startups are stressful. Why do it if you don't need the money? For every "serial entrepreneur," there are probably twenty sane ones who think "Start another company? Are you crazy?"

I've come close to starting new startups a couple times, but I always pull back because I don't want four years of my life to be consumed by random schleps. I know this business well enough to know you can't do it half-heartedly. What makes a good startup founder so dangerous is his willingness to endure infinite schleps.

There is a bit of a problem with retirement, though. Like a lot of people, I like to work. And one of the many weird little problems you discover when you get rich is that a lot of the interesting people you'd like to work with are not rich. They need to work at something that pays the bills. Which means if you want to have them as colleagues, you have to work at something that pays the bills too, even though you don't need to. I think this is what drives a lot of serial entrepreneurs, actually.

That's why I love working on Y Combinator so much. It's an excuse to work on something interesting with people I like.

11. Not ready for commitment

This was my reason for not starting a startup for most of my twenties. Like a lot of people that age, I valued freedom most of all. I was reluctant to do anything that required a commitment of more than a few months. Nor would I have wanted to do anything that completely took over my life the way a startup does. And that's fine. If you want to spend your time travelling around, or playing in a band, or whatever, that's a perfectly legitimate reason not to start a company.

If you start a startup that succeeds, it's going to consume at least three

or four years. (If it fails, you'll be done a lot quicker.) So you shouldn't do it if you're not ready for commitments on that scale. Be aware, though, that if you get a regular job, you'll probably end up working there for as long as a startup would take, and you'll find you have much less spare time than you might expect. So if you're ready to clip on that ID badge and go to that orientation session, you may also be ready to start that startup.

12. Need for structure

I'm told there are people who need structure in their lives. This seems to be a nice way of saying they need someone to tell them what to do. I believe such people exist. There's plenty of empirical evidence: armies, religious cults, and so on. They may even be the majority.

If you're one of these people, you probably shouldn't start a startup. In fact, you probably shouldn't even go to work for one. In a good startup, you don't get told what to do very much. There may be one person whose job title is CEO, but till the company has about twelve people no one should be telling anyone what to do. That's too inefficient. Each person should just do what they need to without anyone telling them.

If that sounds like a recipe for chaos, think about a soccer team. Eleven people manage to work together in quite complicated ways, and yet only in occasional emergencies does anyone tell anyone else what to do. A reporter once asked David Beckham if there were any language problems at Real Madrid, since the players were from about eight different countries. He said it was never an issue, because everyone was so good they never had to talk. They all just did the right thing.

How do you tell if you're independent-minded enough to start a startup? If you'd bristle at the suggestion that you aren't, then you probably are.

13. Fear of uncertainty

Perhaps some people are deterred from starting startups because they don't like the uncertainty. If you go to work for Microsoft, you can predict fairly accurately what the next few years will be like—all too accurately, in fact. If you start a startup, anything might happen.

Well, if you're troubled by uncertainty, I can solve that problem for you: if you start a startup, it will probably fail. Seriously, though, this is not a bad way to think about the whole experience. Hope for the best, but expect the worst. In the worst case, it will at least be interesting. In the best case you might get rich.

No one will blame you if the startup tanks, so long as you made a serious effort. There may once have been a time when employers would regard that as a mark against you, but they wouldn't now. I asked managers at big companies, and they all said they'd prefer to hire someone who'd tried to start a startup and failed over someone who'd spent the same time working at a big company.

Nor will investors hold it against you, as long as you didn't fail out of laziness or incurable stupidity. I'm told there's a lot of stigma attached to failing in other places—in Europe, for example. Not here. In America, companies, like practically everything else, are disposable.

14. Don't realize what you're avoiding

One reason people who've been out in the world for a year or two make better founders than people straight from college is that they know what they're avoiding. If their startup fails, they'll have to get a job, and they know how much jobs suck.

If you've had summer jobs in college, you may think you know what jobs are like, but you probably don't. Summer jobs at technology companies are not real jobs. If you get a summer job as a waiter, that's a real job. Then you have to carry your weight. But software companies don't hire students for the summer as a source of cheap labor. They do it in the hope of recruiting them when they graduate. So while they're happy if you produce, they don't expect you to.

That will change if you get a real job after you graduate. Then you'll have to earn your keep. And since most of what big companies do is boring, you're going to have to work on boring stuff. Easy, compared to college, but boring. At first it may seem cool to get paid for doing easy stuff, after paying to do hard stuff in college. But that wears off after a few months. Eventually it gets demoralizing to work on dumb stuff, even if it's easy and you get paid a lot.

And that's not the worst of it. The thing that really sucks about having a regular job is the expectation that you're supposed to be there at certain times. Even Google is afflicted with this, apparently. And what this means, as everyone who's had a regular job can tell you, is that there are going to be times when you have absolutely no desire to work on anything, and you're going to have to go to work anyway and sit in front of your screen and pretend to. To someone who likes work, as most good hackers do, this is torture.

In a startup, you skip all that. There's no concept of office hours in most startups. Work and life just get mixed together. But the good thing about that is that no one minds if you have a life at work. In a startup you can do whatever you want most of the time. If you're a founder, what you want to do most of the time is work. But you never have to pretend to.

If you took a nap in your office in a big company, it would seem unprofessional. But if you're starting a startup and you fall asleep in the middle of the day, your cofounders will just assume you were tired.

15. Parents want you to be a doctor

A significant number of would-be startup founders are probably dissuaded from doing it by their parents. I'm not going to say you shouldn't listen to them. Families are entitled to their own traditions, and who am I to argue with them? But I will give you a couple reasons why a safe career might not be what your parents really want for you.

One is that parents tend to be more conservative for their kids than they would be for themselves. This is actually a rational response to

their situation. Parents end up sharing more of their kids' ill fortune than good fortune. Most parents don't mind this; it's part of the job; but it does tend to make them excessively conservative. And erring on the side of conservatism is still erring. In almost everything, reward is proportionate to risk. So by protecting their kids from risk, parents are, without realizing it, also protecting them from rewards. If they saw that, they'd want you to take more risks.

The other reason parents may be mistaken is that, like generals, they're always fighting the last war. If they want you to be a doctor, odds are it's not just because they want you to help the sick, but also because it's a prestigious and lucrative career.⁴ But not so lucrative or prestigious as it was when their opinions were formed. When I was a kid in the seventies, a doctor was *the* thing to be. There was a sort of golden triangle involving doctors, Mercedes 450SLs, and tennis. All three vertices now seem pretty dated.

The parents who want you to be a doctor may simply not realize how much things have changed. Would they be that unhappy if you were Steve Jobs instead? So I think the way to deal with your parents' opinions about what you should do is to treat them like feature requests. Even if your only goal is to please them, the way to do that is not simply to give them what they ask for. Instead think about why they're asking for something, and see if there's a better way to give them what they need.

16. A job is the default

This leads us to the last and probably most powerful reason people get regular jobs: it's the default thing to do. Defaults are enormously powerful, precisely because they operate without any conscious choice.

To almost everyone except criminals, it seems an axiom that if you need money, you should get a job. Actually this tradition is not much more than a hundred years old. Before that, the default way to make

⁴Thought experiment: If doctors did the same work, but as impoverished outcasts, which parents would still want their kids to be doctors?

a living was by farming. It's a bad plan to treat something only a hundred years old as an axiom. By historical standards, that's something that's changing pretty rapidly.

We may be seeing another such change right now. I've read a lot of economic history, and I understand the startup world pretty well, and it now seems to me fairly likely that we're seeing the beginning of a change like the one from farming to manufacturing.

And you know what? If you'd been around when that change began (around 1000 in Europe) it would have seemed to nearly everyone that running off to the city to make your fortune was a crazy thing to do. Though serfs were in principle forbidden to leave their manors, it can't have been that hard to run away to a city. There were no guards patrolling the perimeter of the village. What prevented most serfs from leaving was that it seemed insanely risky. Leave one's plot of land? Leave the people you'd spent your whole life with, to live in a giant city of three or four thousand complete strangers? How would you live? How would you get food, if you didn't grow it?

Frightening as it seemed to them, it's now the default with us to live by our wits. So if it seems risky to you to start a startup, think how risky it once seemed to your ancestors to live as we do now. Oddly enough, the people who know this best are the very ones trying to get you to stick to the old model. How can Larry and Sergey say you should come work as their employee, when they didn't get jobs themselves?

Now we look back on medieval peasants and wonder how they stood it. How grim it must have been to till the same fields your whole life with no hope of anything better, under the thumb of lords and priests you had to give all your surplus to and acknowledge as your masters. I wouldn't be surprised if one day people look back on what we consider a normal job in the same way. How grim it would be to commute every day to a cubicle in some soulless office complex, and be told what to do by someone you had to acknowledge as a boss—someone who could call you into their office and say “take a seat,” and you'd sit! Imagine having to ask *permission* to release software to

users. Imagine being sad on Sunday afternoons because the weekend was almost over, and tomorrow you'd have to get up and go to work. How did they stand it?

It's exciting to think we may be on the cusp of another shift like the one from farming to manufacturing. That's why I care about startups. Startups aren't interesting just because they're a way to make a lot of money. I couldn't care less about other ways to do that, like speculating in securities. At most those are interesting the way puzzles are. There's more going on with startups. They may represent one of those rare, historic shifts in the way wealth is created.

That's ultimately what drives us to work on Y Combinator. We want to make money, if only so we don't have to stop doing it, but that's not the main goal. There have only been a handful of these great economic shifts in human history. It would be an amazing hack to make one happen faster.

Thanks to Trevor Blackwell, Jessica Livingston, and Robert Morris for reading drafts of this, to the founders of Zenter for letting me use their web-based PowerPoint killer even though it isn't launched yet, and to Ming-Hay Luk of the Berkeley CSUA for inviting me to speak.

Chapter 6

Microsoft is Dead

April 2007

A few days ago I suddenly realized Microsoft was dead. I was talking to a young startup founder about how Google was different from Yahoo. I said that Yahoo had been warped from the start by their fear of Microsoft. That was why they'd positioned themselves as a "media company" instead of a technology company. Then I looked at his face and realized he didn't understand. It was as if I'd told him how much girls liked Barry Manilow in the mid 80s. Barry who?

Microsoft? He didn't say anything, but I could tell he didn't quite believe anyone would be frightened of them.

Microsoft cast a shadow over the software world for almost 20 years starting in the late 80s. I can remember when it was IBM before them. I mostly ignored this shadow. I never used Microsoft software, so it only affected me indirectly—for example, in the spam I got from botnets. And because I wasn't paying attention, I didn't notice when the shadow disappeared.

But it's gone now. I can sense that. No one is even afraid of Microsoft anymore. They still make a lot of money—so does IBM, for that matter. But they're not dangerous.

When did Microsoft die, and of what? I know they seemed dangerous as late as 2001, because I wrote an essay then about how they were less dangerous than they seemed. I'd guess they were dead by 2005. I know when we started Y Combinator we didn't worry about Microsoft as competition for the startups we funded. In fact, we've never even invited them to the demo days we organize for startups to present

to investors. We invite Yahoo and Google and some other Internet companies, but we've never bothered to invite Microsoft. Nor has anyone there ever even sent us an email. They're in a different world.

What killed them? Four things, I think, all of them occurring simultaneously in the mid 2000s.

The most obvious is Google. There can only be one big man in town, and they're clearly it. Google is the most dangerous company now by far, in both the good and bad senses of the word. Microsoft can at best limp along afterward.

When did Google take the lead? There will be a tendency to push it back to their IPO in August 2004, but they weren't setting the terms of the debate then. I'd say they took the lead in 2005. Gmail was one of the things that put them over the edge. Gmail showed they could do more than search.

Gmail also showed how much you could do with web-based software, if you took advantage of what later came to be called "Ajax." And that was the second cause of Microsoft's death: everyone can see the desktop is over. It now seems inevitable that applications will live on the web—not just email, but everything, right up to Photoshop. Even Microsoft sees that now.

Ironically, Microsoft unintentionally helped create Ajax. The x in Ajax is from the XMLHttpRequest object, which lets the browser communicate with the server in the background while displaying a page. (Originally the only way to communicate with the server was to ask for a new page.) XMLHttpRequest was created by Microsoft in the late 90s because they needed it for Outlook. What they didn't realize was that it would be useful to a lot of other people too—in fact, to anyone who wanted to make web apps work like desktop ones.

The other critical component of Ajax is Javascript, the programming language that runs in the browser. Microsoft saw the danger of

Javascript and tried to keep it broken for as long as they could.¹ But eventually the open source world won, by producing Javascript libraries that grew over the brokenness of Explorer the way a tree grows over barbed wire.

The third cause of Microsoft's death was broadband Internet. Anyone who cares can have fast Internet access now. And the bigger the pipe to the server, the less you need the desktop.

The last nail in the coffin came, of all places, from Apple. Thanks to OS X, Apple has come back from the dead in a way that is extremely rare in technology.² Their victory is so complete that I'm now surprised when I come across a computer running Windows. Nearly all the people we fund at Y Combinator use Apple laptops. It was the same in the audience at startup school. All the computer people use Macs or Linux now. Windows is for grandmas, like Macs used to be in the 90s. So not only does the desktop no longer matter, no one who cares about computers uses Microsoft's anyway.

And of course Apple has Microsoft on the run in music too, with TV and phones on the way.

I'm glad Microsoft is dead. They were like Nero or Commodus—evil in the way only inherited power can make you. Because remember, the Microsoft monopoly didn't begin with Microsoft. They got it from IBM. The software business was overhung by a monopoly from about the mid-1950s to about 2005. For practically its whole existence, that is. One of the reasons "Web 2.0" has such an air of euphoria about it is the feeling, conscious or not, that this era of monopoly may finally be over.

¹It doesn't take a conscious effort to make software incompatible. All you have to do is not work too hard at fixing bugs—which, if you're a big company, you produce in copious quantities. The situation is analogous to the writing of "literary theorists." Most don't try to be obscure; they just don't make an effort to be clear. It wouldn't pay.

²In part because Steve Jobs got pushed out by John Sculley in a way that's rare among technology companies. If Apple's board hadn't made that blunder, they wouldn't have had to bounce back.

Of course, as a hacker I can't help thinking about how something broken could be fixed. Is there some way Microsoft could come back? In principle, yes. To see how, envision two things: (a) the amount of cash Microsoft now has on hand, and (b) Larry and Sergey making the rounds of all the search engines ten years ago trying to sell the idea for Google for a million dollars, and being turned down by everyone.

The surprising fact is, brilliant hackers—dangerously brilliant hackers—can be had very cheaply, by the standards of a company as rich as Microsoft. They can't hire smart people anymore, but they could buy as many as they wanted for only an order of magnitude more. So if they wanted to be a contender again, this is how they could do it:

1. Buy all the good “Web 2.0” startups. They could get substantially all of them for less than they'd have to pay for Facebook.
2. Put them all in a building in Silicon Valley, surrounded by lead shielding to protect them from any contact with Redmond.

I feel safe suggesting this, because they'd never do it. Microsoft's biggest weakness is that they still don't realize how much they suck. They still think they can write software in house. Maybe they can, by the standards of the desktop world. But that world ended a few years ago.

I already know what the reaction to this essay will be. Half the readers will say that Microsoft is still an enormously profitable company, and that I should be more careful about drawing conclusions based on what a few people think in our insular little “Web 2.0” bubble. The other half, the younger half, will complain that this is old news.

See also: Microsoft is Dead: the Cliffs Notes

Chapter 7

Two Kinds of Judgement

April 2007

There are two different ways people judge you. Sometimes judging you correctly is the end goal. But there's a second much more common type of judgement where it isn't. We tend to regard all judgements of us as the first type. We'd probably be happier if we realized which are and which aren't.

The first type of judgement, the type where judging you is the end goal, include court cases, grades in classes, and most competitions. Such judgements can of course be mistaken, but because the goal is to judge you correctly, there's usually some kind of appeals process. If you feel you've been misjudged, you can protest that you've been treated unfairly.

Nearly all the judgements made on children are of this type, so we get into the habit early in life of thinking that all judgements are.

But in fact there is a second much larger class of judgements where judging you is only a means to something else. These include college admissions, hiring and investment decisions, and of course the judgements made in dating. This kind of judgement is not really about you.

Put yourself in the position of someone selecting players for a national team. Suppose for the sake of simplicity that this is a game with no positions, and that you have to select 20 players. There will be a few stars who clearly should make the team, and many players who clearly shouldn't. The only place your judgement makes a difference is in the borderline cases. Suppose you screw up and underestimate the 20th

best player, causing him not to make the team, and his place to be taken by the 21st best. You've still picked a good team. If the players have the usual distribution of ability, the 21st best player will be only slightly worse than the 20th best. Probably the difference between them will be less than the measurement error.

The 20th best player may feel he has been misjudged. But your goal here wasn't to provide a service estimating people's ability. It was to pick a team, and if the difference between the 20th and 21st best players is less than the measurement error, you've still done that optimally.

It's a false analogy even to use the word unfair to describe this kind of misjudgement. It's not aimed at producing a correct estimate of any given individual, but at selecting a reasonably optimal set.

One thing that leads us astray here is that the selector seems to be in a position of power. That makes him seem like a judge. If you regard someone judging you as a customer instead of a judge, the expectation of fairness goes away. The author of a good novel wouldn't complain that readers were *unfair* for preferring a potboiler with a racy cover. Stupid, perhaps, but not unfair.

Our early training and our self-centeredness combine to make us believe that every judgement of us is about us. In fact most aren't. This is a rare case where being less self-centered will make people more confident. Once you realize how little most people judging you care about judging you accurately—once you realize that because of the normal distribution of most applicant pools, it matters least to judge accurately in precisely the cases where judgement has the most effect—you won't take rejection so personally.

And curiously enough, taking rejection less personally may help you to get rejected less often. If you think someone judging you will work hard to judge you correctly, you can afford to be passive. But the more you realize that most judgements are greatly influenced by random, extraneous factors—that most people judging you are more like a fickle novel buyer than a wise and perceptive magistrate—the more

you realize you can do things to influence the outcome.

One good place to apply this principle is in college applications. Most high school students applying to college do it with the usual child's mix of inferiority and self-centeredness: inferiority in that they assume that admissions committees must be all-seeing; self-centeredness in that they assume admissions committees care enough about them to dig down into their application and figure out whether they're good or not. These combine to make applicants passive in applying and hurt when they're rejected. If college applicants realized how quick and impersonal most selection processes are, they'd make more effort to sell themselves, and take the outcome less personally.

Chapter 8

The Hacker's Guide to Investors

April 2007

(This essay is derived from a keynote talk at the 2007 ASES Summit at Stanford.)

The world of investors is a foreign one to most hackers—partly because investors are so unlike hackers, and partly because they tend to operate in secret. I've been dealing with this world for many years, both as a founder and an investor, and I still don't fully understand it.

In this essay I'm going to list some of the more surprising things I've learned about investors. Some I only learned in the past year.

Teaching hackers how to deal with investors is probably the second most important thing we do at Y Combinator. The most important thing for a startup is to make something good. But everyone knows that's important. The dangerous thing about investors is that hackers don't know how little they know about this strange world.

1. The investors are what make a startup hub.

About a year ago I tried to figure out what you'd need to reproduce Silicon Valley. I decided the critical ingredients were rich people and nerds—investors and founders. People are all you need to make technology, and all the other people will move.

If I had to narrow that down, I'd say investors are the limiting factor. Not because they contribute more to the startup, but simply because they're least willing to move. They're rich. They're not going to move to Albuquerque just because there are some smart hackers there they could invest in. Whereas hackers will move to the Bay Area to find investors.

2. Angel investors are the most critical.

There are several types of investors. The two main categories are angels and VCs: VCs invest other people's money, and angels invest their own.

Though they're less well known, the angel investors are probably the more critical ingredient in creating a silicon valley. Most companies that VCs invest in would never have made it that far if angels hadn't invested first. VCs say between half and three quarters of companies that raise series A rounds have taken some outside investment already.

¹

Angels are willing to fund riskier projects than VCs. They also give valuable advice, because (unlike VCs) many have been startup founders themselves.

Google's story shows the key role angels play. A lot of people know Google raised money from Kleiner and Sequoia. What most don't realize is how late. That VC round was a series B round; the premoney valuation was \$75 million. Google was already a successful company at that point. Really, Google was funded with angel money.

It may seem odd that the canonical Silicon Valley startup was funded by angels, but this is not so surprising. Risk is always proportionate to reward. So the most successful startup of all is likely to have seemed an extremely risky bet at first, and that is exactly the kind VCs won't touch.

Where do angel investors come from? From other startups. So startup hubs like Silicon Valley benefit from something like the marketplace effect, but shifted in time: startups are there because startups were there.

3. Angels don't like publicity.

If angels are so important, why do we hear more about VCs? Because

¹I may be underestimating VCs. They may play some behind the scenes role in IPOs, which you ultimately need if you want to create a silicon valley.

VCS like publicity. They need to market themselves to the investors who are their “customers”—the endowments and pension funds and rich families whose money they invest—and also to founders who might come to them for funding.

Angels don’t need to market themselves to investors because they invest their own money. Nor do they want to market themselves to founders: they don’t want random people pestering them with business plans. Actually, neither do VCs. Both angels and VCs get deals almost exclusively through personal introductions.²

The reason VCs want a strong brand is not to draw in more business plans over the transom, but so they win deals when competing against other VCs. Whereas angels are rarely in direct competition, because (a) they do fewer deals, (b) they’re happy to split them, and (c) they invest at a point where the stream is broader.

4. Most investors, especially VCs, are not like founders.

Some angels are, or were, hackers. But most VCs are a different type of people: they’re dealmakers.

If you’re a hacker, here’s a thought experiment you can run to understand why there are basically no hacker VCs: How would you like a job where you never got to make anything, but instead spent all your time listening to other people pitch (mostly terrible) projects, deciding whether to fund them, and sitting on their boards if you did? That would not be fun for most hackers. Hackers like to make things. This would be like being an administrator.

Because most VCs are a different species of people from founders, it’s hard to know what they’re thinking. If you’re a hacker, the last time you had to deal with these guys was in high school. Maybe in college you walked past their fraternity on your way to the lab. But don’t underestimate them. They’re as expert in their world as you are

²A few VCs have an email address you can send your business plan to, but the number of startups that get funded this way is basically zero. You should always get a personal introduction—and to a partner, not an associate.

in yours. What they're good at is reading people, and making deals work to their advantage. Think twice before you try to beat them at that.

5. Most investors are momentum investors.

Because most investors are dealmakers rather than technology people, they generally don't understand what you're doing. I knew as a founder that most VCs didn't get technology. I also knew some made a lot of money. And yet it never occurred to me till recently to put those two ideas together and ask "How can VCs make money by investing in stuff they don't understand?"

The answer is that they're like momentum investors. You can (or could once) make a lot of money by noticing sudden changes in stock prices. When a stock jumps upward, you buy, and when it suddenly drops, you sell. In effect you're insider trading, without knowing what you know. You just know someone knows something, and that's making the stock move.

This is how most venture investors operate. They don't try to look at something and predict whether it will take off. They win by noticing that something *is* taking off a little sooner than everyone else. That generates almost as good returns as actually being able to pick winners. They may have to pay a little more than they would if they got in at the very beginning, but only a little.

Investors always say what they really care about is the team. Actually what they care most about is your traffic, then what other investors think, then the team. If you don't yet have any traffic, they fall back on number 2, what other investors think. And this, as you can imagine, produces wild oscillations in the "stock price" of a startup. One week everyone wants you, and they're begging not to be cut out of the deal. But all it takes is for one big investor to cool on you, and the next week no one will return your phone calls. We regularly have startups go from hot to cold or cold to hot in a matter of days, and literally nothing has changed.

There are two ways to deal with this phenomenon. If you're feeling really confident, you can try to ride it. You can start by asking a comparatively lowly VC for a small amount of money, and then after generating interest there, ask more prestigious VCs for larger amounts, stirring up a crescendo of buzz, and then "sell" at the top. This is extremely risky, and takes months even if you succeed. I wouldn't try it myself. My advice is to err on the side of safety: when someone offers you a decent deal, just take it and get on with building the company. Startups win or lose based on the quality of their product, not the quality of their funding deals.

6. Most investors are looking for big hits.

Venture investors like companies that could go public. That's where the big returns are. They know the odds of any individual startup going public are small, but they want to invest in those that at least have a *chance* of going public.

Currently the way VCs seem to operate is to invest in a bunch of companies, most of which fail, and one of which is Google. Those few big wins compensate for losses on their other investments. What this means is that most VCs will only invest in you if you're a potential Google. They don't care about companies that are a safe bet to be acquired for \$20 million. There needs to be a chance, however small, of the company becoming really big.

Angels are different in this respect. They're happy to invest in a company where the most likely outcome is a \$20 million acquisition if they can do it at a low enough valuation. But of course they like companies that could go public too. So having an ambitious long-term plan pleases everyone.

If you take VC money, you have to mean it, because the structure of VC deals prevents early acquisitions. If you take VC money, they won't let you sell early.

7. VCs want to invest large amounts.

The fact that they're running investment funds makes VCs want to invest large amounts. A typical VC fund is now hundreds of millions of dollars. If \$400 million has to be invested by 10 partners, they have to invest \$40 million each. VCs usually sit on the boards of companies they fund. If the average deal size was \$1 million, each partner would have to sit on 40 boards, which would not be fun. So they prefer bigger deals, where they can put a lot of money to work at once.

VCs don't regard you as a bargain if you don't need a lot of money. That may even make you less attractive, because it means their investment creates less of a barrier to entry for competitors.

Angels are in a different position because they're investing their own money. They're happy to invest small amounts—sometimes as little as \$20,000—as long as the potential returns look good enough. So if you're doing something inexpensive, go to angels.

8. Valuations are fiction.

VCs admit that valuations are an artifact. They decide how much money you need and how much of the company they want, and those two constraints yield a valuation.

Valuations increase as the size of the investment does. A company that an angel is willing to put \$50,000 into at a valuation of a million can't take \$6 million from VCs at that valuation. That would leave the founders less than a seventh of the company between them (since the option pool would also come out of that seventh). Most VCs wouldn't want that, which is why you never hear of deals where a VC invests \$6 million at a premoney valuation of \$1 million.

If valuations change depending on the amount invested, that shows how far they are from reflecting any kind of value of the company.

Since valuations are made up, founders shouldn't care too much about them. That's not the part to focus on. In fact, a high valuation can be a bad thing. If you take funding at a premoney valuation of \$10 million, you won't be selling the company for 20. You'll have to sell for over

50 for the VCs to get even a 5x return, which is low to them. More likely they'll want you to hold out for 100. But needing to get a high price decreases the chance of getting bought at all; many companies can buy you for \$10 million, but only a handful for 100. And since a startup is like a pass/fail course for the founders, what you want to optimize is your chance of a good outcome, not the percentage of the company you keep.

So why do founders chase high valuations? They're tricked by misplaced ambition. They feel they've achieved more if they get a higher valuation. They usually know other founders, and if they get a higher valuation they can say "mine is bigger than yours." But funding is not the real test. The real test is the final outcome for the founder, and getting too high a valuation may just make a good outcome less likely.

The one advantage of a high valuation is that you get less dilution. But there is another less sexy way to achieve that: just take less money.

9. Investors look for founders like the current stars.

Ten years ago investors were looking for the next Bill Gates. This was a mistake, because Microsoft was a very anomalous startup. They started almost as a contract programming operation, and the reason they became huge was that IBM happened to drop the PC standard in their lap.

Now all the VCs are looking for the next Larry and Sergey. This is a good trend, because Larry and Sergey are closer to the ideal startup founders.

Historically investors thought it was important for a founder to be an expert in business. So they were willing to fund teams of MBAs who planned to use the money to pay programmers to build their product for them. This is like funding Steve Ballmer in the hope that the programmer he'll hire is Bill Gates—kind of backward, as the events of the Bubble showed. Now most VCs know they should be funding technical guys. This is more pronounced among the very top funds; the lamer ones still want to fund MBAs.

If you're a hacker, it's good news that investors are looking for Larry and Sergey. The bad news is, the only investors who can do it right are the ones who knew them when they were a couple of CS grad students, not the confident media stars they are today. What investors still don't get is how clueless and tentative great founders can seem at the very beginning.

10. The contribution of investors tends to be underestimated.

Investors do more for startups than give them money. They're helpful in doing deals and arranging introductions, and some of the smarter ones, particularly angels, can give good advice about the product.

In fact, I'd say what separates the great investors from the mediocre ones is the quality of their advice. Most investors give advice, but the top ones give *good* advice.

Whatever help investors give a startup tends to be underestimated. It's to everyone's advantage to let the world think the founders thought of everything. The goal of the investors is for the company to become valuable, and the company seems more valuable if it seems like all the good ideas came from within.

This trend is compounded by the obsession that the press has with founders. In a company founded by two people, 10% of the ideas might come from the first guy they hire. Arguably they've done a bad job of hiring otherwise. And yet this guy will be almost entirely overlooked by the press.

I say this as a founder: the contribution of founders is always overestimated. The danger here is that new founders, looking at existing founders, will think that they're supermen that one couldn't possibly equal oneself. Actually they have a hundred different types of support people just offscreen making the whole show possible.³

³Several people have told us that the most valuable thing about startup school was that they got to see famous startup founders and realized they were just ordinary guys. Though we're happy to provide this service, this is not generally the way we

11. VCs are afraid of looking bad.

I've been very surprised to discover how timid most VCs are. They seem to be afraid of looking bad to their partners, and perhaps also to the limited partners—the people whose money they invest.

You can measure this fear in how much less risk VCs are willing to take. You can tell they won't make investments for their fund that they might be willing to make themselves as angels. Though it's not quite accurate to say that VCs are less willing to take risks. They're less willing to do things that might look bad. That's not the same thing.

For example, most VCs would be very reluctant to invest in a startup founded by a pair of 18 year old hackers, no matter how brilliant, because if the startup failed their partners could turn on them and say “What, you invested \$x million of our money in a pair of 18 year olds?” Whereas if a VC invested in a startup founded by three former banking executives in their 40s who planned to outsource their product development—which to my mind is actually a lot riskier than investing in a pair of really smart 18 year olds—he couldn't be faulted, if it failed, for making such an apparently prudent investment.

As a friend of mine said, “Most VCs can't do anything that would sound bad to the kind of doofuses who run pension funds.” Angels can take greater risks because they don't have to answer to anyone.

12. Being turned down by investors doesn't mean much.

Some founders are quite dejected when they get turned down by investors. They shouldn't take it so much to heart. To start with, investors are often wrong. It's hard to think of a successful startup that wasn't turned down by investors at some point. Lots of VCs rejected Google. So obviously the reaction of investors is not a very meaningful test.

Investors will often reject you for what seem to be superficial reasons. I read of one VC who turned down a startup simply because they'd

pitch startup school to potential speakers.

given away so many little bits of stock that the deal required too many signatures to close.⁴ The reason investors can get away with this is that they see so many deals. It doesn't matter if they underestimate you because of some surface imperfection, because the next best deal will be almost as good. Imagine picking out apples at a grocery store. You grab one with a little bruise. Maybe it's just a surface bruise, but why even bother checking when there are so many other unbruised apples to choose from?

Investors would be the first to admit they're often wrong. So when you get rejected by investors, don't think "we suck," but instead ask "do we suck?" Rejection is a question, not an answer.

13. Investors are emotional.

I've been surprised to discover how emotional investors can be. You'd expect them to be cold and calculating, or at least businesslike, but often they're not. I'm not sure if it's their position of power that makes them this way, or the large sums of money involved, but investment negotiations can easily turn personal. If you offend investors, they'll leave in a huff.

A while ago an eminent VC firm offered a series A round to a startup we'd seed funded. Then they heard a rival VC firm was also interested. They were so afraid that they'd be rejected in favor of this other firm that they gave the startup what's known as an "exploding termsheet." They had, I think, 24 hours to say yes or no, or the deal was off. Exploding termsheets are a somewhat dubious device, but not uncommon. What surprised me was their reaction when I called to talk about it. I asked if they'd still be interested in the startup if the rival VC didn't end up making an offer, and they said no. What rational basis could they have had for saying that? If they thought the startup was worth investing in, what difference should it make what some other VC thought? Surely it was their duty to their limited part-

⁴Actually this sounds to me like a VC who got buyer's remorse, then used a technicality to get out of the deal. But it's telling that it even seemed a plausible excuse.

ners simply to invest in the best opportunities they found; they should be delighted if the other VC said no, because it would mean they'd overlooked a good opportunity. But of course there was no rational basis for their decision. They just couldn't stand the idea of taking this rival firm's rejects.

In this case the exploding termsheet was not (or not only) a tactic to pressure the startup. It was more like the high school trick of breaking up with someone before they can break up with you. In an earlier essay I said that VCs were a lot like high school girls. A few VCs have joked about that characterization, but none have disputed it.

14. The negotiation never stops till the closing.

Most deals, for investment or acquisition, happen in two phases. There's an initial phase of negotiation about the big questions. If this succeeds you get a termsheet, so called because it outlines the key terms of a deal. A termsheet is not legally binding, but it is a definite step. It's supposed to mean that a deal is going to happen, once the lawyers work out all the details. In theory these details are minor ones; by definition all the important points are supposed to be covered in the termsheet.

Inexperience and wishful thinking combine to make founders feel that when they have a termsheet, they have a deal. They want there to be a deal; everyone acts like they have a deal; so there must be a deal. But there isn't and may not be for several months. A lot can change for a startup in several months. It's not uncommon for investors and acquirers to get buyer's remorse. So you have to keep pushing, keep selling, all the way to the close. Otherwise all the "minor" details left unspecified in the termsheet will be interpreted to your disadvantage. The other side may even break the deal; if they do that, they'll usually seize on some technicality or claim you misled them, rather than admitting they changed their minds.

It can be hard to keep the pressure on an investor or acquirer all the way to the closing, because the most effective pressure is competition

from other investors or acquirers, and these tend to drop away when you get a termsheet. You should try to stay as close friends as you can with these rivals, but the most important thing is just to keep up the momentum in your startup. The investors or acquirers chose you because you seemed hot. Keep doing whatever made you seem hot. Keep releasing new features; keep getting new users; keep getting mentioned in the press and in blogs.

15. Investors like to co-invest.

I've been surprised how willing investors are to split deals. You might think that if they found a good deal they'd want it all to themselves, but they seem positively eager to syndicate. This is understandable with angels; they invest on a smaller scale and don't like to have too much money tied up in any one deal. But VCs also share deals a lot. Why?

Partly I think this is an artifact of the rule I quoted earlier: after traffic, VCs care most what other VCs think. A deal that has multiple VCs interested in it is more likely to close, so of deals that close, more will have multiple investors.

There is one rational reason to want multiple VCs in a deal: Any investor who co-invests with you is one less investor who could fund a competitor. Apparently Kleiner and Sequoia didn't like splitting the Google deal, but it did at least have the advantage, from each one's point of view, that there probably wouldn't be a competitor funded by the other. Splitting deals thus has similar advantages to confusing paternity.

But I think the main reason VCs like splitting deals is the fear of looking bad. If another firm shares the deal, then in the event of failure it will seem to have been a prudent choice—a consensus decision, rather than just the whim of an individual partner.

16. Investors collude.

Investing is not covered by antitrust law. At least, it better not be,

because investors regularly do things that would be illegal otherwise. I know personally of cases where one investor has talked another out of making a competitive offer, using the promise of sharing future deals.

In principle investors are all competing for the same deals, but the spirit of cooperation is stronger than the spirit of competition. The reason, again, is that there are so many deals. Though a professional investor may have a closer relationship with a founder he invests in than with other investors, his relationship with the founder is only going to last a couple years, whereas his relationship with other firms will last his whole career. There isn't so much at stake in his interactions with other investors, but there will be a lot of them. Professional investors are constantly trading little favors.

Another reason investors stick together is to preserve the power of investors as a whole. So you will not, as of this writing, be able to get investors into an auction for your series A round. They'd rather lose the deal than establish a precedent of VCs competitively bidding against one another. An efficient startup funding market may be coming in the distant future; things tend to move in that direction; but it's certainly not here now.

17. Large-scale investors care about their portfolio, not any individual company.

The reason startups work so well is that everyone with power also has equity. The only way any of them can succeed is if they all do. This makes everyone naturally pull in the same direction, subject to differences of opinion about tactics.

The problem is, larger scale investors don't have exactly the same motivation. Close, but not identical. They don't need any given startup to succeed, like founders do, just their portfolio as a whole to. So in borderline cases the rational thing for them to do is to sacrifice unpromising startups.

Large-scale investors tend to put startups in three categories: successes, failures, and the "living dead"—companies that are plugging

along but don't seem likely in the immediate future to get bought or go public. To the founders, "living dead" sounds harsh. These companies may be far from failures by ordinary standards. But they might as well be from a venture investor's point of view, and they suck up just as much time and attention as the successes. So if such a company has two possible strategies, a conservative one that's slightly more likely to work in the end, or a risky one that within a short time will either yield a giant success or kill the company, VCs will push for the kill-or-cure option. To them the company is already a write-off. Better to have resolution, one way or the other, as soon as possible.

If a startup gets into real trouble, instead of trying to save it VCs may just sell it at a low price to another of their portfolio companies. Philip Greenspun said in *Founders at Work* that Ars Digita's VCs did this to them.

18. Investors have different risk profiles from founders.

Most people would rather a 100% chance of \$1 million than a 20% chance of \$10 million. Investors are rich enough to be rational and prefer the latter. So they'll always tend to encourage founders to keep rolling the dice. If a company is doing well, investors will want founders to turn down most acquisition offers. And indeed, most startups that turn down acquisition offers ultimately do better. But it's still hair-raising for the founders, because they might end up with nothing. When someone's offering to buy you for a price at which your stock is worth \$5 million, saying no is equivalent to having \$5 million and betting it all on one spin of the roulette wheel.

Investors will tell you the company is worth more. And they may be right. But that doesn't mean it's wrong to sell. Any financial advisor who put all his client's assets in the stock of a single, private company would probably lose his license for it.

More and more, investors are letting founders cash out partially. That should correct the problem. Most founders have such low standards that they'll feel rich with a sum that doesn't seem huge to investors.

But this custom is spreading too slowly, because VCs are afraid of seeming irresponsible. No one wants to be the first VC to give someone fuck-you money and then actually get told “fuck you.” But until this does start to happen, we know VCs are being too conservative.

19. Investors vary greatly.

Back when I was a founder I used to think all VCs were the same. And in fact they do all look the same. They’re all what hackers call “suits.” But since I’ve been dealing with VCs more I’ve learned that some suits are smarter than others.

They’re also in a business where winners tend to keep winning and losers to keep losing. When a VC firm has been successful in the past, everyone wants funding from them, so they get the pick of all the new deals. The self-reinforcing nature of the venture funding market means that the top ten firms live in a completely different world from, say, the hundredth. As well as being smarter, they tend to be calmer and more upstanding; they don’t need to do iffy things to get an edge, and don’t want to because they have more brand to protect.

There are only two kinds of VCs you want to take money from, if you have the luxury of choosing: the “top tier” VCs, meaning about the top 20 or so firms, plus a few new ones that are not among the top 20 only because they haven’t been around long enough.

It’s particularly important to raise money from a top firm if you’re a hacker, because they’re more confident. That means they’re less likely to stick you with a business guy as CEO, like VCs used to do in the 90s. If you seem smart and want to do it, they’ll let you run the company.

20. Investors don’t realize how much it costs to raise money from them.

Raising money is a huge time suck at just the point where startups can least afford it. It’s not unusual for it to take five or six months to close a funding round. Six weeks is fast. And raising money is not just something you can leave running as a background process. When

you're raising money, it's inevitably the main focus of the company. Which means building the product isn't.

Suppose a Y Combinator company starts talking to VCs after demo day, and is successful in raising money from them, closing the deal after a comparatively short 8 weeks. Since demo day occurs after 10 weeks, the company is now 18 weeks old. Raising money, rather than working on the product, has been the company's main focus for 44% of its existence. And mind you, this an example where things turned out *well*.

When a startup does return to working on the product after a funding round finally closes, it's as if they were returning to work after a months-long illness. They've lost most of their momentum.

Investors have no idea how much they damage the companies they invest in by taking so long to do it. But companies do. So there is a big opportunity here for a new kind of venture fund that invests smaller amounts at lower valuations, but promises to either close or say no very quickly. If there were such a firm, I'd recommend it to startups in preference to any other, no matter how prestigious. Startups live on speed and momentum.

21. Investors don't like to say no.

The reason funding deals take so long to close is mainly that investors can't make up their minds. VCs are not big companies; they can do a deal in 24 hours if they need to. But they usually let the initial meetings stretch out over a couple weeks. The reason is the selection algorithm I mentioned earlier. Most don't try to predict whether a startup will win, but to notice quickly that it already is winning. They care what the market thinks of you and what other VCs think of you, and they can't judge those just from meeting you.

Because they're investing in things that (a) change fast and (b) they don't understand, a lot of investors will reject you in a way that can later be claimed not to have been a rejection. Unless you know this world, you may not even realize you've been rejected. Here's a VC

saying no:

We're really excited about your project, and we want to keep in close touch as you develop it further.

Translated into more straightforward language, this means: We're not investing in you, but we may change our minds if it looks like you're taking off. Sometimes they're more candid and say explicitly that they need to "see some traction." They'll invest in you if you start to get lots of users. But so would any VC. So all they're saying is that you're still at square 1.

Here's a test for deciding whether a VC's response was yes or no. Look down at your hands. Are you holding a termsheet?

22. You need investors.

Some founders say "Who needs investors?" Empirically the answer seems to be: everyone who wants to succeed. Practically every successful startup takes outside investment at some point.

Why? What the people who think they don't need investors forget is that they will have competitors. The question is not whether you *need* outside investment, but whether it could help you at all. If the answer is yes, and you don't take investment, then competitors who do will have an advantage over you. And in the startup world a little advantage can expand into a lot.

Mike Moritz famously said that he invested in Yahoo because he thought they had a few weeks' lead over their competitors. That may not have mattered quite so much as he thought, because Google came along three years later and kicked Yahoo's ass. But there is something in what he said. Sometimes a small lead can grow into the yes half of a binary choice.

Maybe as it gets cheaper to start a startup, it will start to be possible to succeed in a competitive market without outside funding. There are

certainly costs to raising money. But as of this writing the empirical evidence says it's a net win.

23. Investors like it when you don't need them.

A lot of founders approach investors as if they needed their permission to start a company—as if it were like getting into college. But you don't need investors to start most companies; they just make it easier.

And in fact, investors greatly prefer it if you don't need them. What excites them, both consciously and unconsciously, is the sort of startup that approaches them saying “the train's leaving the station; are you in or out?” not the one saying “please can we have some money to start a company?”

Most investors are “bottoms” in the sense that the startups they like most are those that are rough with them. When Google stuck Kleiner and Sequoia with a \$75 million premoney valuation, their reaction was probably “Ouch! That feels so good.” And they were right, weren't they? That deal probably made them more than any other they've done.

The thing is, VCs are pretty good at reading people. So don't try to act tough with them unless you really are the next Google, or they'll see through you in a second. Instead of acting tough, what most startups should do is simply always have a backup plan. Always have some alternative plan for getting started if any given investor says no. Having one is the best insurance against needing one.

So you shouldn't start a startup that's expensive to start, because then you'll be at the mercy of investors. If you ultimately want to do something that will cost a lot, start by doing a cheaper subset of it, and expand your ambitions when and if you raise more money.

Apparently the most likely animals to be left alive after a nuclear war are cockroaches, because they're so hard to kill. That's what you want to be as a startup, initially. Instead of a beautiful but fragile flower that needs to have its stem in a plastic tube to support itself, better to be

small, ugly, and indestructible.

Thanks to Sam Altman, Paul Buchheit, Hutch Fishman, and Robert Morris for reading drafts of this, and to Kenneth King of ASES for inviting me to speak.

Chapter 9

An Alternative Theory of Unions



May 2007

People who worry about the increasing gap between rich and poor generally look back on the mid twentieth century as a golden age. In those days we had a large number of high-paying union manufacturing jobs that boosted the median income. I wouldn't quite call the high-paying union job a myth, but I think people who dwell on it are reading too much into it.

Oddly enough, it was working with startups that made me realize where the high-paying union job came from. In a rapidly growing market, you don't worry too much about efficiency. It's more important to grow fast. If there's some mundane problem getting in your way, and there's a simple solution that's somewhat expensive, just take it and get on with more important things. EBay didn't win by paying less for servers than their competitors.

Difficult though it may be to imagine now, manufacturing was a growth industry in the mid twentieth century. This was an era when small firms making everything from cars to candy were getting consolidated into a new kind of corporation with national reach and huge economies of scale. You had to grow fast or die. Workers were for these companies what servers are for an Internet startup. A reliable

supply was more important than low cost.

If you looked in the head of a 1950s auto executive, the attitude must have been: sure, give 'em whatever they ask for, so long as the new model isn't delayed.

In other words, those workers were not paid what their work was worth. Circumstances being what they were, companies would have been stupid to insist on paying them so little.

If you want a less controversial example of this phenomenon, ask anyone who worked as a consultant building web sites during the Internet Bubble. In the late nineties you could get paid huge sums of money for building the most trivial things. And yet does anyone who was there have any expectation those days will ever return? I doubt it. Surely everyone realizes that was just a temporary aberration.

The era of labor unions seems to have been the same kind of aberration, just spread over a longer period, and mixed together with a lot of ideology that prevents people from viewing it with as cold an eye as they would something like consulting during the Bubble.

Basically, unions were just Razorfish.

People who think the labor movement was the creation of heroic union organizers have a problem to explain: why are unions shrinking now? The best they can do is fall back on the default explanation of people living in fallen civilizations. Our ancestors were giants. The workers of the early twentieth century must have had a moral courage that's lacking today.

In fact there's a simpler explanation. The early twentieth century was just a fast-growing startup overpaying for infrastructure. And we in the present are not a fallen people, who have abandoned whatever mysterious high-minded principles produced the high-paying union job. We simply live in a time when the fast-growing companies overspend on different things.

Chapter 10

The Equity Equation

July 2007

An investor wants to give you money for a certain percentage of your startup. Should you take it? You're about to hire your first employee. How much stock should you give him?

These are some of the hardest questions founders face. And yet both have the same answer:

$$1/(1 - n)$$

Whenever you're trading stock in your company for anything, whether it's money or an employee or a deal with another company, the test for whether to do it is the same. You should give up $n\%$ of your company if what you trade it for improves your average outcome enough that the $(100 - n)\%$ you have left is worth more than the whole company was before.

For example, if an investor wants to buy half your company, how much does that investment have to improve your average outcome for you to break even? Obviously it has to double: if you trade half your company for something that more than doubles the company's average outcome, you're net ahead. You have half as big a share of something worth more than twice as much.

In the general case, if n is the fraction of the company you're giving up, the deal is a good one if it makes the company worth more than $1/(1 - n)$.

For example, suppose Y Combinator offers to fund you in return for 7% of your company. In this case, n is .07 and $1/(1 - n)$ is 1.075. So

you should take the deal if you believe we can improve your average outcome by more than 7.5%. If we improve your outcome by 10%, you're net ahead, because the remaining .93 you hold is worth $.93 \times 1.1 = 1.023$.¹

One of the things the equity equation shows us is that, financially at least, taking money from a top VC firm can be a really good deal. Greg Mcadoo from Sequoia recently said at a YC dinner that when Sequoia invests alone they like to take about 30% of a company. $1/.7 = 1.43$, meaning that deal is worth taking if they can improve your outcome by more than 43%. For the average startup, that would be an extraordinary bargain. It would improve the average startup's prospects by more than 43% just to be able to *say* they were funded by Sequoia, even if they never actually got the money.

The reason Sequoia is such a good deal is that the percentage of the company they take is artificially low. They don't even try to get market price for their investment; they limit their holdings to leave the founders enough stock to feel the company is still theirs.

The catch is that Sequoia gets about 6000 business plans a year and funds about 20 of them, so the odds of getting this great deal are 1 in 300. The companies that make it through are not average startups.

Of course, there are other factors to consider in a VC deal. It's never just a straight trade of money for stock. But if it were, taking money from a top firm would generally be a bargain.

You can use the same formula when giving stock to employees, but it works in the other direction. If i is the average outcome for the company with the addition of some new person, then they're worth n such that $i = 1/(1 - n)$. Which means $n = (i - 1)/i$.

For example, suppose you're just two founders and you want to hire an additional hacker who's so good you feel he'll increase the average

¹This is why we can't believe anyone would think Y Combinator was a bad deal. Does anyone really think we're so useless that in three months we can't improve a startup's prospects by 7.5%?

outcome of the whole company by 20%. $n = (1.2 - 1)/1.2 = .167$. So you'll break even if you trade 16.7% of the company for him.

That doesn't mean 16.7% is the right amount of stock to give him. Stock is not the only cost of hiring someone: there's usually salary and overhead as well. And if the company merely breaks even on the deal, there's no reason to do it.

I think to translate salary and overhead into stock you should multiply the annual rate by about 1.5. Most startups grow fast or die; if you die you don't have to pay the guy, and if you grow fast you'll be paying next year's salary out of next year's valuation, which should be 3x this year's. If your valuation grows 3x a year, the total cost in stock of a new hire's salary and overhead is 1.5 years' cost at the present valuation.²

How much of an additional margin should the company need as the "activation energy" for the deal? Since this is in effect the company's profit on a hire, the market will determine that: if you're a hot opportunity, you can charge more.

Let's run through an example. Suppose the company wants to make a "profit" of 50% on the new hire mentioned above. So subtract a third from 16.7% and we have 11.1% as his "retail" price. Suppose further that he's going to cost \$60k a year in salary and overhead, $\times 1.5 = \$90k$ total. If the company's valuation is \$2 million, $\$90k$ is 4.5%. $11.1\% - 4.5\% =$ an offer of 6.6%.

Incidentally, notice how important it is for early employees to take little salary. It comes right out of stock that could otherwise be given to them.

Obviously there is a great deal of play in these numbers. I'm not claiming that stock grants can now be reduced to a formula. Ultimately you

²The obvious choice for your present valuation is the post-money valuation of your last funding round. This probably undervalues the company, though, because (a) unless your last round just happened, the company is presumably worth more, and (b) the valuation of an early funding round usually reflects some other contribution by the investors.

always have to guess. But at least know what you're guessing. If you choose a number based on your gut feel, or a table of typical grant sizes supplied by a VC firm, understand what those are estimates of.

And more generally, when you make any decision involving equity, run it through $1/(1 - n)$ to see if it makes sense. You should always feel richer after trading equity. If the trade didn't increase the value of your remaining shares enough to put you net ahead, you wouldn't have (or shouldn't have) done it.

Thanks to Sam Altman, Trevor Blackwell, Paul Buchheit, Hutch Fishman, David Hornik, Paul Kedrosky, Jessica Livingston, Gary Sabot, and Joshua Schachter for reading drafts of this.

Chapter 11

Stuff

July 2007

I have too much stuff. Most people in America do. In fact, the poorer people are, the more stuff they seem to have. Hardly anyone is so poor that they can't afford a front yard full of old cars.

It wasn't always this way. Stuff used to be rare and valuable. You can still see evidence of that if you look for it. For example, in my house in Cambridge, which was built in 1876, the bedrooms don't have closets. In those days people's stuff fit in a chest of drawers. Even as recently as a few decades ago there was a lot less stuff. When I look back at photos from the 1970s, I'm surprised how empty houses look. As a kid I had what I thought was a huge fleet of toy cars, but they'd be dwarfed by the number of toys my nephews have. All together my Matchboxes and Corgis took up about a third of the surface of my bed. In my nephews' rooms the bed is the only clear space.

Stuff has gotten a lot cheaper, but our attitudes toward it haven't changed correspondingly. We overvalue stuff.

That was a big problem for me when I had no money. I felt poor, and stuff seemed valuable, so almost instinctively I accumulated it. Friends would leave something behind when they moved, or I'd see something as I was walking down the street on trash night (beware of anything you find yourself describing as "perfectly good"), or I'd find something in almost new condition for a tenth its retail price at a garage sale. And pow, more stuff.

In fact these free or nearly free things weren't bargains, because they were worth even less than they cost. Most of the stuff I accumulated

was worthless, because I didn't need it.

What I didn't understand was that the value of some new acquisition wasn't the difference between its retail price and what I paid for it. It was the value I derived from it. Stuff is an extremely illiquid asset. Unless you have some plan for selling that valuable thing you got so cheaply, what difference does it make what it's "worth?" The only way you're ever going to extract any value from it is to use it. And if you don't have any immediate use for it, you probably never will.

Companies that sell stuff have spent huge sums training us to think stuff is still valuable. But it would be closer to the truth to treat stuff as worthless.

In fact, worse than worthless, because once you've accumulated a certain amount of stuff, it starts to own you rather than the other way around. I know of one couple who couldn't retire to the town they preferred because they couldn't afford a place there big enough for all their stuff. Their house isn't theirs; it's their stuff's.

And unless you're extremely organized, a house full of stuff can be very depressing. A cluttered room saps one's spirits. One reason, obviously, is that there's less room for people in a room full of stuff. But there's more going on than that. I think humans constantly scan their environment to build a mental model of what's around them. And the harder a scene is to parse, the less energy you have left for conscious thoughts. A cluttered room is literally exhausting.

(This could explain why clutter doesn't seem to bother kids as much as adults. Kids are less perceptive. They build a coarser model of their surroundings, and this consumes less energy.)

I first realized the worthlessness of stuff when I lived in Italy for a year. All I took with me was one large backpack of stuff. The rest of my stuff I left in my landlady's attic back in the US. And you know what? All I missed were some of the books. By the end of the year I couldn't even remember what else I had stored in that attic.

And yet when I got back I didn't discard so much as a box of it. Throw away a perfectly good rotary telephone? I might need that one day.

The really painful thing to recall is not just that I accumulated all this useless stuff, but that I often spent money I desperately needed on stuff that I didn't.

Why would I do that? Because the people whose job is to sell you stuff are really, really good at it. The average 25 year old is no match for companies that have spent years figuring out how to get you to spend money on stuff. They make the experience of buying stuff so pleasant that "shopping" becomes a leisure activity.

How do you protect yourself from these people? It can't be easy. I'm a fairly skeptical person, and their tricks worked on me well into my thirties. But one thing that might work is to ask yourself, before buying something, "is this going to make my life noticeably better?"

A friend of mine cured herself of a clothes buying habit by asking herself before she bought anything "Am I going to wear this all the time?" If she couldn't convince herself that something she was thinking of buying would become one of those few things she wore all the time, she wouldn't buy it. I think that would work for any kind of purchase. Before you buy anything, ask yourself: will this be something I use constantly? Or is it just something nice? Or worse still, a mere bargain?

The worst stuff in this respect may be stuff you don't use much because it's too good. Nothing owns you like fragile stuff. For example, the "good china" so many households have, and whose defining quality is not so much that it's fun to use, but that one must be especially careful not to break it.

Another way to resist acquiring stuff is to think of the overall cost of owning it. The purchase price is just the beginning. You're going to have to *think* about that thing for years—perhaps for the rest of your life. Every thing you own takes energy away from you. Some give more than they take. Those are the only things worth having.

I've now stopped accumulating stuff. Except books—but books are different. Books are more like a fluid than individual objects. It's not especially inconvenient to own several thousand books, whereas if you owned several thousand random possessions you'd be a local celebrity. But except for books, I now actively avoid stuff. If I want to spend money on some kind of treat, I'll take services over goods any day.

I'm not claiming this is because I've achieved some kind of zenlike detachment from material things. I'm talking about something more mundane. A historical change has taken place, and I've now realized it. Stuff used to be valuable, and now it's not.

In industrialized countries the same thing happened with food in the middle of the twentieth century. As food got cheaper (or we got richer; they're indistinguishable), eating too much started to be a bigger danger than eating too little. We've now reached that point with stuff. For most people, rich or poor, stuff has become a burden.

The good news is, if you're carrying a burden without knowing it, your life could be better than you realize. Imagine walking around for years with five pound ankle weights, then suddenly having them removed.

Chapter 12

Holding a Program in One's Head

August 2007

A good programmer working intensively on his own code can hold it in his mind the way a mathematician holds a problem he's working on. Mathematicians don't answer questions by working them out on paper the way schoolchildren are taught to. They do more in their heads: they try to understand a problem space well enough that they can walk around it the way you can walk around the memory of the house you grew up in. At its best programming is the same. You hold the whole program in your head, and you can manipulate it at will.

That's particularly valuable at the start of a project, because initially the most important thing is to be able to change what you're doing. Not just to solve the problem in a different way, but to change the problem you're solving.

Your code is your understanding of the problem you're exploring. So it's only when you have your code in your head that you really understand the problem.

It's not easy to get a program into your head. If you leave a project for a few months, it can take days to really understand it again when you return to it. Even when you're actively working on a program it can take half an hour to load into your head when you start work each day. And that's in the best case. Ordinary programmers working in typical office conditions never enter this mode. Or to put it more dramatically, ordinary programmers working in typical office conditions never really understand the problems they're solving.

Even the best programmers don't always have the whole program

they're working on loaded into their heads. But there are things you can do to help:

1. **Avoid distractions.** Distractions are bad for many types of work, but especially bad for programming, because programmers tend to operate at the limit of the detail they can handle.

The danger of a distraction depends not on how long it is, but on how much it scrambles your brain. A programmer can leave the office and go and get a sandwich without losing the code in his head. But the wrong kind of interruption can wipe your brain in 30 seconds.

Oddly enough, scheduled distractions may be worse than unscheduled ones. If you know you have a meeting in an hour, you don't even start working on something hard.

2. **Work in long stretches.** Since there's a fixed cost each time you start working on a program, it's more efficient to work in a few long sessions than many short ones. There will of course come a point where you get stupid because you're tired. This varies from person to person. I've heard of people hacking for 36 hours straight, but the most I've ever been able to manage is about 18, and I work best in chunks of no more than 12.

The optimum is not the limit you can physically endure. There's an advantage as well as a cost of breaking up a project. Sometimes when you return to a problem after a rest, you find your unconscious mind has left an answer waiting for you.

3. **Use succinct languages.** More powerful programming languages make programs shorter. And programmers seem to think of programs at least partially in the language they're using to write them. The more succinct the language, the shorter the program, and the easier it is to load and keep in your head.

You can magnify the effect of a powerful language by using a style called bottom-up programming, where you write programs in multiple layers, the lower ones acting as programming

languages for those above. If you do this right, you only have to keep the topmost layer in your head.

4. **Keep rewriting your program.** Rewriting a program often yields a cleaner design. But it would have advantages even if it didn't: you have to understand a program completely to rewrite it, so there is no better way to get one loaded into your head.
5. **Write rereadable code.** All programmers know it's good to write readable code. But you yourself are the most important reader. Especially in the beginning; a prototype is a conversation with yourself. And when writing for yourself you have different priorities. If you're writing for other people, you may not want to make code too dense. Some parts of a program may be easiest to read if you spread things out, like an introductory textbook. Whereas if you're writing code to make it easy to reload into your head, it may be best to go for brevity.
6. **Work in small groups.** When you manipulate a program in your head, your vision tends to stop at the edge of the code you own. Other parts you don't understand as well, and more importantly, can't take liberties with. So the smaller the number of programmers, the more completely a project can mutate. If there's just one programmer, as there often is at first, you can do all-encompassing redesigns.
7. **Don't have multiple people editing the same piece of code.** You never understand other people's code as well as your own. No matter how thoroughly you've read it, you've only read it, not written it. So if a piece of code is written by multiple authors, none of them understand it as well as a single author would.

And of course you can't safely redesign something other people are working on. It's not just that you'd have to ask permission. You don't even let yourself think of such things. Redesigning code with several authors is like changing laws; redesigning

code you alone control is like seeing the other interpretation of an ambiguous image.

If you want to put several people to work on a project, divide it into components and give each to one person.

8. **Start small.** A program gets easier to hold in your head as you become familiar with it. You can start to treat parts as black boxes once you feel confident you've fully explored them. But when you first start working on a project, you're forced to see everything. If you start with too big a problem, you may never quite be able to encompass it. So if you need to write a big, complex program, the best way to begin may not be to write a spec for it, but to write a prototype that solves a subset of the problem. Whatever the advantages of planning, they're often outweighed by the advantages of being able to keep a program in your head.

It's striking how often programmers manage to hit all eight points by accident. Someone has an idea for a new project, but because it's not officially sanctioned, he has to do it in off hours—which turn out to be more productive because there are no distractions. Driven by his enthusiasm for the new project he works on it for many hours at a stretch. Because it's initially just an experiment, instead of a “production” language he uses a mere “scripting” language—which is in fact far more powerful. He completely rewrites the program several times; that wouldn't be justifiable for an official project, but this is a labor of love and he wants it to be perfect. And since no one is going to see it except him, he omits any comments except the note-to-self variety. He works in a small group perforce, because he either hasn't told anyone else about the idea yet, or it seems so unpromising that no one else is allowed to work on it. Even if there is a group, they couldn't have multiple people editing the same code, because it changes too fast for that to be possible. And the project starts small because the idea *is* small at first; he just has some cool hack he wants to try out.

Even more striking are the number of officially sanctioned projects that manage to do *all eight things wrong*. In fact, if you look at the way software gets written in most organizations, it's almost as if they were deliberately trying to do things wrong. In a sense, they are. One of the defining qualities of organizations since there have been such a thing is to treat individuals as interchangeable parts. This works well for more parallelizable tasks, like fighting wars. For most of history a well-drilled army of professional soldiers could be counted on to beat an army of individual warriors, no matter how valorous. But having ideas is not very parallelizable. And that's what programs are: ideas.

It's not merely true that organizations dislike the idea of depending on individual genius, it's a tautology. It's part of the definition of an organization not to. Of our current concept of an organization, at least.

Maybe we could define a new kind of organization that combined the efforts of individuals without requiring them to be interchangeable. Arguably a market is such a form of organization, though it may be more accurate to describe a market as a degenerate case—as what you get by default when organization isn't possible.

Probably the best we'll do is some kind of hack, like making the programming parts of an organization work differently from the rest. Perhaps the optimal solution is for big companies not even to try to develop ideas in house, but simply to buy them. But regardless of what the solution turns out to be, the first step is to realize there's a problem. There is a contradiction in the very phrase “software company.” The two words are pulling in opposite directions. Any good programmer in a large organization is going to be at odds with it, because organizations are designed to prevent what programmers strive for.

Good programmers manage to get a lot done anyway. But often it requires practically an act of rebellion against the organizations that employ them. Perhaps it will help if more people understand that the way programmers behave is driven by the demands of the work they do. It's not because they're irresponsible that they work in long

binges during which they blow off all other obligations, plunge straight into programming instead of writing specs first, and rewrite code that already works. It's not because they're unfriendly that they prefer to work alone, or growl at people who pop their head in the door to say hello. This apparently random collection of annoying habits has a single explanation: the power of holding a program in one's head.

Whether or not understanding this can help large organizations, it can certainly help their competitors. The weakest point in big companies is that they don't let individual programmers do great work. So if you're a little startup, this is the place to attack them. Take on the kind of problems that have to be solved in one big brain.

Thanks to Sam Altman, David Greenspan, Aaron Iba, Jessica Livingston, Robert Morris, Peter Norvig, Lisa Randall, Emmett Shear, Sergei Tsarev, and Stephen Wolfram for reading drafts of this.

Chapter 13

How Not to Die

August 2007

(This is a talk I gave at the last Y Combinator dinner of the summer. Usually we don't have a speaker at the last dinner; it's more of a party. But it seemed worth spoiling the atmosphere if I could save some of the startups from preventable deaths. So at the last minute I cooked up this rather grim talk. I didn't mean this as an essay; I wrote it down because I only had two hours before dinner and think fastest while writing.)

A couple days ago I told a reporter that we expected about a third of the companies we funded to succeed. Actually I was being conservative. I'm hoping it might be as much as a half. Wouldn't it be amazing if we could achieve a 50% success rate?

Another way of saying that is that half of you are going to die. Phrased that way, it doesn't sound good at all. In fact, it's kind of weird when you think about it, because our definition of success is that the founders get rich. If half the startups we fund succeed, then half of you are going to get rich and the other half are going to get nothing.

If you can just avoid dying, you get rich. That sounds like a joke, but it's actually a pretty good description of what happens in a typical startup. It certainly describes what happened in Viaweb. We avoided dying till we got rich.

It was really close, too. When we were visiting Yahoo to talk about being acquired, we had to interrupt everything and borrow one of their conference rooms to talk down an investor who was about to back out of a new funding round we needed to stay alive. So even in the middle of getting rich we were fighting off the grim reaper.

You may have heard that quote about luck consisting of opportunity meeting preparation. You've now done the preparation. The work you've done so far has, in effect, put you in a position to get lucky: you can now get rich by not letting your company die. That's more than most people have. So let's talk about how not to die.

We've done this five times now, and we've seen a bunch of startups die. About 10 of them so far. We don't know exactly what happens when they die, because they generally don't die loudly and heroically. Mostly they crawl off somewhere and die.

For us the main indication of impending doom is when we don't hear from you. When we haven't heard from, or about, a startup for a couple months, that's a bad sign. If we send them an email asking what's up, and they don't reply, that's a really bad sign. So far that is a 100% accurate predictor of death.

Whereas if a startup regularly does new deals and releases and either sends us mail or shows up at YC events, they're probably going to live.

I realize this will sound naive, but maybe the linkage works in both directions. Maybe if you can arrange that we keep hearing from you, you won't die.

That may not be so naive as it sounds. You've probably noticed that having dinners every Tuesday with us and the other founders causes you to get more done than you would otherwise, because every dinner is a mini Demo Day. Every dinner is a kind of a deadline. So the mere constraint of staying in regular contact with us will push you to make things happen, because otherwise you'll be embarrassed to tell us that you haven't done anything new since the last time we talked.

If this works, it would be an amazing hack. It would be pretty cool if merely by staying in regular contact with us you could get rich. It sounds crazy, but there's a good chance that would work.

A variant is to stay in touch with other YC-funded startups. There is now a whole neighborhood of them in San Francisco. If you move

there, the peer pressure that made you work harder all summer will continue to operate.

When startups die, the official cause of death is always either running out of money or a critical founder bailing. Often the two occur simultaneously. But I think the underlying cause is usually that they've become demoralized. You rarely hear of a startup that's working around the clock doing deals and pumping out new features, and dies because they can't pay their bills and their ISP unplugs their server.

Startups rarely die in mid keystroke. So keep typing!

If so many startups get demoralized and fail when merely by hanging on they could get rich, you have to assume that running a startup can be demoralizing. That is certainly true. I've been there, and that's why I've never done another startup. The low points in a startup are just unbelievably low. I bet even Google had moments where things seemed hopeless.

Knowing that should help. If you know it's going to feel terrible sometimes, then when it feels terrible you won't think "ouch, this feels terrible, I give up." It feels that way for everyone. And if you just hang on, things will probably get better. The metaphor people use to describe the way a startup feels is at least a roller coaster and not drowning. You don't just sink and sink; there are ups after the downs.

Another feeling that seems alarming but is in fact normal in a startup is the feeling that what you're doing isn't working. The reason you can expect to feel this is that what you do probably won't work. Startups almost never get it right the first time. Much more commonly you launch something, and no one cares. Don't assume when this happens that you've failed. That's normal for startups. But don't sit around doing nothing. Iterate.

I like Paul Buchheit's suggestion of trying to make something that at least someone really loves. As long as you've made something that a few users are ecstatic about, you're on the right track. It will be good for your morale to have even a handful of users who really love you,

and startups run on morale. But also it will tell you what to focus on. What is it about you that they love? Can you do more of that? Where can you find more people who love that sort of thing? As long as you have some core of users who love you, all you have to do is expand it. It may take a while, but as long as you keep plugging away, you'll win in the end. Both Blogger and Delicious did that. Both took years to succeed. But both began with a core of fanatically devoted users, and all Evan and Joshua had to do was grow that core incrementally. Wufoo is on the same trajectory now.

So when you release something and it seems like no one cares, look more closely. Are there zero users who really love you, or is there at least some little group that does? It's quite possible there will be zero. In that case, tweak your product and try again. Every one of you is working on a space that contains at least one winning permutation somewhere in it. If you just keep trying, you'll find it.

Let me mention some things not to do. The number one thing not to do is other things. If you find yourself saying a sentence that ends with "but we're going to keep working on the startup," you are in big trouble. Bob's going to grad school, but we're going to keep working on the startup. We're moving back to Minnesota, but we're going to keep working on the startup. We're taking on some consulting projects, but we're going to keep working on the startup. You may as well just translate these to "we're giving up on the startup, but we're not willing to admit that to ourselves," because that's what it means most of the time. A startup is so hard that working on it can't be preceded by "but."

In particular, don't go to graduate school, and don't start other projects. Distraction is fatal to startups. Going to (or back to) school is a huge predictor of death because in addition to the distraction it gives you something to say you're doing. If you're only doing a startup, then if the startup fails, you fail. If you're in grad school and your startup fails, you can say later "Oh yeah, we had this startup on the side when I was in grad school, but it didn't go anywhere."

You can't use euphemisms like "didn't go anywhere" for something that's your only occupation. People won't let you.

One of the most interesting things we've discovered from working on Y Combinator is that founders are more motivated by the fear of looking bad than by the hope of getting millions of dollars. So if you want to get millions of dollars, put yourself in a position where failure will be public and humiliating.

When we first met the founders of Octopart, they seemed very smart, but not a great bet to succeed, because they didn't seem especially committed. One of the two founders was still in grad school. It was the usual story: he'd drop out if it looked like the startup was taking off. Since then he has not only dropped out of grad school, but appeared full length in *Newsweek* with the word "Billionaire" printed across his chest. He just cannot fail now. Everyone he knows has seen that picture. Girls who dissed him in high school have seen it. His mom probably has it on the fridge. It would be unthinkable humiliating to fail now. At this point he is committed to fight to the death.

I wish every startup we funded could appear in a *Newsweek* article describing them as the next generation of billionaires, because then none of them would be able to give up. The success rate would be 90%. I'm not kidding.

When we first knew the Octoparts they were lighthearted, cheery guys. Now when we talk to them they seem grimly determined. The electronic parts distributors are trying to squash them to keep their monopoly pricing. (If it strikes you as odd that people still order electronic parts out of thick paper catalogs in 2007, there's a reason for that. The distributors want to prevent the transparency that comes from having prices online.) I feel kind of bad that we've transformed these guys from lighthearted to grimly determined. But that comes with the territory. If a startup succeeds, you get millions of dollars, and you don't get that kind of money just by asking for it. You have to assume it takes some amount of pain.

And however tough things get for the Octoparts, I predict they'll succeed. They may have to morph themselves into something totally different, but they won't just crawl off and die. They're smart; they're working in a promising field; and they just cannot give up.

All of you guys already have the first two. You're all smart and working on promising ideas. Whether you end up among the living or the dead comes down to the third ingredient, not giving up.

So I'll tell you now: bad shit is coming. It always is in a startup. The odds of getting from launch to liquidity without some kind of disaster happening are one in a thousand. So don't get demoralized. When the disaster strikes, just say to yourself, ok, this was what Paul was talking about. What did he say to do? Oh, yeah. Don't give up.

Chapter 14

News from the Front

September 2007

A few weeks ago I had a thought so heretical that it really surprised me. It may not matter all that much where you go to college.

For me, as for a lot of middle class kids, getting into a good college was more or less the meaning of life when I was growing up. What was I? A student. To do that well meant to get good grades. Why did one have to get good grades? To get into a good college. And why did one want to do that? There seemed to be several reasons: you'd learn more, get better jobs, make more money. But it didn't matter exactly what the benefits would be. College was a bottleneck through which all your future prospects passed; everything would be better if you went to a better college.

A few weeks ago I realized that somewhere along the line I had stopped believing that.

What first set me thinking about this was the new trend of worrying obsessively about what kindergarten your kids go to. It seemed to me this couldn't possibly matter. Either it won't help your kid get into Harvard, or if it does, getting into Harvard won't mean much anymore. And then I thought: how much does it mean even now?

It turns out I have a lot of data about that. My three partners and I run a seed stage investment firm called Y Combinator. We invest when the company is just a couple guys and an idea. The idea doesn't matter much; it will change anyway. Most of our decision is based on the founders. The average founder is three years out of college. Many have just graduated; a few are still in school. So we're in much

the same position as a graduate program, or a company hiring people right out of college. Except our choices are immediately and visibly tested. There are two possible outcomes for a startup: success or failure—and usually you know within a year which it will be.

The test applied to a startup is among the purest of real world tests. A startup succeeds or fails depending almost entirely on the efforts of the founders. Success is decided by the market: you only succeed if users like what you've built. And users don't care where you went to college.

As well as having precisely measurable results, we have a lot of them. Instead of doing a small number of large deals like a traditional venture capital fund, we do a large number of small ones. We currently fund about 40 companies a year, selected from about 900 applications representing a total of about 2000 people.¹

Between the volume of people we judge and the rapid, unequivocal test that's applied to our choices, Y Combinator has been an unprecedented opportunity for learning how to pick winners. One of the most surprising things we've learned is how little it matters where people went to college.

I thought I'd already been cured of caring about that. There's nothing like going to grad school at Harvard to cure you of any illusions you might have about the average Harvard undergrad. And yet Y Combinator showed us we were still overestimating people who'd been to elite colleges. We'd interview people from MIT or Harvard or Stanford and sometimes find ourselves thinking: they *must* be smarter than they seem. It took us a few iterations to learn to trust our senses.

Practically everyone thinks that someone who went to MIT or Harvard or Stanford must be smart. Even people who hate you for it

¹Is what we measure worth measuring? I think so. You can get rich simply by being energetic and unscrupulous, but getting rich from a technology startup takes some amount of brains. It is just the kind of work the upper middle class values; it has about the same intellectual component as being a doctor.

believe it.

But when you think about what it means to have gone to an elite college, how could this be true? We're talking about a decision made by admissions officers—basically, HR people—based on a cursory examination of a huge pile of depressingly similar applications submitted by seventeen year olds. And what do they have to go on? An easily gamed standardized test; a short essay telling you what the kid thinks you want to hear; an interview with a random alum; a high school record that's largely an index of obedience. Who would rely on such a test?

And yet a lot of companies do. A lot of companies are very much influenced by where applicants went to college. How could they be? I think I know the answer to that.

There used to be a saying in the corporate world: “No one ever got fired for buying IBM.” You no longer hear this about IBM specifically, but the idea is very much alive; there is a whole category of “enterprise” software companies that exist to take advantage of it. People buying technology for large organizations don't care if they pay a fortune for mediocre software. It's not their money. They just want to buy from a supplier who seems safe—a company with an established name, confident salesmen, impressive offices, and software that conforms to all the current fashions. Not necessarily a company that will deliver so much as one that, if they do let you down, will still seem to have been a prudent choice. So companies have evolved to fill that niche.

A recruiter at a big company is in much the same position as someone buying technology for one. If someone went to Stanford and is not obviously insane, they're probably a safe bet. And a safe bet is enough. No one ever measures recruiters by the later performance of people they turn down.²

²Actually, someone did, once. Mitch Kapor's wife Freada was in charge of HR at Lotus in the early years. (As he is at pains to point out, they did not become romantically involved till afterward.) At one point they worried Lotus was losing its

I'm not saying, of course, that elite colleges have evolved to prey upon the weaknesses of large organizations the way enterprise software companies have. But they work as if they had. In addition to the power of the brand name, graduates of elite colleges have two critical qualities that plug right into the way large organizations work. They're good at doing what they're asked, since that's what it takes to please the adults who judge you at seventeen. And having been to an elite college makes them more confident.

Back in the days when people might spend their whole career at one big company, these qualities must have been very valuable. Graduates of elite colleges would have been capable, yet amenable to authority. And since individual performance is so hard to measure in large organizations, their own confidence would have been the starting point for their reputation.

Things are very different in the new world of startups. We couldn't save someone from the market's judgement even if we wanted to. And being charming and confident counts for nothing with users. All users care about is whether you make something they like. If you don't, you're dead.

Knowing that test is coming makes us work a lot harder to get the right answers than anyone would if they were merely hiring people. We can't afford to have any illusions about the predictors of success. And what we've found is that the variation between schools is so much smaller than the variation between individuals that it's negligible by comparison. We can learn more about someone in the first minute of talking to them than by knowing where they went to school.

It seems obvious when you put it that way. Look at the individual, not where they went to college. But that's a weaker statement than the idea I began with, that it doesn't matter much where a given individual

startup edge and turning into a big company. So as an experiment she sent their recruiters the resumes of the first 40 employees, with identifying details changed. These were the people who had made Lotus into the star it was. Not one got an interview.

goes to college. Don't you learn things at the best schools that you wouldn't learn at lesser places?

Apparently not. Obviously you can't prove this in the case of a single individual, but you can tell from aggregate evidence: you can't, without asking them, distinguish people who went to one school from those who went to another three times as far down the *US News* list.³ Try it and see.

How can this be? Because how much you learn in college depends a lot more on you than the college. A determined party animal can get through the best school without learning anything. And someone with a real thirst for knowledge will be able to find a few smart people to learn from at a school that isn't prestigious at all.

The other students are the biggest advantage of going to an elite college; you learn more from them than the professors. But you should be able to reproduce this at most colleges if you make a conscious effort to find smart friends. At most colleges you can find at least a handful of other smart students, and most people have only a handful of close friends in college anyway.⁴ The odds of finding smart professors are even better. The curve for faculty is a lot flatter than for students, especially in math and the hard sciences; you have to go pretty far down the list of colleges before you stop finding smart professors in the math department.

³The *US News* list? Surely no one trusts that. Even if the statistics they consider are useful, how do they decide on the relative weights? The reason the *US News* list is meaningful is precisely because they are so intellectually dishonest in that respect. There is no external source they can use to calibrate the weighting of the statistics they use; if there were, we could just use that instead. What they must do is adjust the weights till the top schools are the usual suspects in about the right order. So in effect what the *US News* list tells us is what the editors think the top schools are, which is probably not far from the conventional wisdom on the matter. The amusing thing is, because some schools work hard to game the system, the editors will have to keep tweaking their algorithm to get the rankings they want.

⁴Possible doesn't mean easy, of course. A smart student at a party school will inevitably be something of an outcast, just as he or she would be in most high schools.

So it's not surprising that we've found the relative prestige of different colleges useless in judging individuals. There's a lot of randomness in how colleges select people, and what they learn there depends much more on them than the college. Between these two sources of variation, the college someone went to doesn't mean a lot. It is to some degree a predictor of ability, but so weak that we regard it mainly as a source of error and try consciously to ignore it.

I doubt what we've discovered is an anomaly specific to startups. Probably people have always overestimated the importance of where one goes to college. We're just finally able to measure it.

The unfortunate thing is not just that people are judged by such a superficial test, but that so many judge themselves by it. A lot of people, probably the majority of people in America, have some amount of insecurity about where, or whether, they went to college. The tragedy of the situation is that by far the greatest liability of not having gone to the college you'd have liked is your own feeling that you're thereby lacking something. Colleges are a bit like exclusive clubs in this respect. There is only one real advantage to being a member of most exclusive clubs: you know you wouldn't be missing much if you weren't. When you're excluded, you can only imagine the advantages of being an insider. But invariably they're larger in your imagination than in real life.

So it is with colleges. Colleges differ, but they're nothing like the stamp of destiny so many imagine them to be. People aren't what some admissions officer decides about them at seventeen. They're what they make themselves.

Indeed, the great advantage of not caring where people went to college is not just that you can stop judging them (and yourself) by superficial measures, but that you can focus instead on what really matters. What matters is what you make of yourself. I think that's what we should tell kids. Their job isn't to get good grades so they can get into a good college, but to learn and do. And not just because that's more rewarding than worldly success. That will increasingly *be* the route to

worldly success.

Thanks to Trevor Blackwell, Sarah Harlin, Jessica Livingston, Jackie McDonough, Peter Norvig, and Robert Morris for reading drafts of this.

Chapter 15

How to Do Philosophy



September 2007

In high school I decided I was going to study philosophy in college. I had several motives, some more honorable than others. One of the less honorable was to shock people. College was regarded as job training where I grew up, so studying philosophy seemed an impressively impractical thing to do. Sort of like slashing holes in your clothes or putting a safety pin through your ear, which were other forms of impressive impracticality then just coming into fashion.

But I had some more honest motives as well. I thought studying philosophy would be a shortcut straight to wisdom. All the people majoring in other things would just end up with a bunch of domain knowledge. I would be learning what was really what.

I'd tried to read a few philosophy books. Not recent ones; you wouldn't find those in our high school library. But I tried to read Plato and Aristotle. I doubt I believed I understood them, but they sounded like they were talking about something important. I assumed I'd learn what in college.

The summer before senior year I took some college classes. I learned a lot in the calculus class, but I didn't learn much in Philosophy 101. And yet my plan to study philosophy remained intact. It was my fault I hadn't learned anything. I hadn't read the books we were assigned

carefully enough. I'd give Berkeley's *Principles of Human Knowledge* another shot in college. Anything so admired and so difficult to read must have something in it, if one could only figure out what.

Twenty-six years later, I still don't understand Berkeley. I have a nice edition of his collected works. Will I ever read it? Seems unlikely.

The difference between then and now is that now I understand why Berkeley is probably not worth trying to understand. I think I see now what went wrong with philosophy, and how we might fix it.

Words

I did end up being a philosophy major for most of college. It didn't work out as I'd hoped. I didn't learn any magical truths compared to which everything else was mere domain knowledge. But I do at least know now why I didn't. Philosophy doesn't really have a subject matter in the way math or history or most other university subjects do. There is no core of knowledge one must master. The closest you come to that is a knowledge of what various individual philosophers have said about different topics over the years. Few were sufficiently correct that people have forgotten who discovered what they discovered.

Formal logic has some subject matter. I took several classes in logic. I don't know if I learned anything from them.¹ It does seem to me very important to be able to flip ideas around in one's head: to see when two ideas don't fully cover the space of possibilities, or when one idea is the same as another but with a couple things changed. But did studying logic teach me the importance of thinking this way, or make me any better at it? I don't know.

There are things I know I learned from studying philosophy. The most dramatic I learned immediately, in the first semester of freshman year,

¹In practice formal logic is not much use, because despite some progress in the last 150 years we're still only able to formalize a small percentage of statements. We may never do that much better, for the same reason 1980s-style "knowledge representation" could never have worked; many statements may have no representation more concise than a huge, analog brain state.

in a class taught by Sydney Shoemaker. I learned that I don't exist. I am (and you are) a collection of cells that lurches around driven by various forces, and calls itself *I*. But there's no central, indivisible thing that your identity goes with. You could conceivably lose half your brain and live. Which means your brain could conceivably be split into two halves and each transplanted into different bodies. Imagine waking up after such an operation. You have to imagine being two people.

The real lesson here is that the concepts we use in everyday life are fuzzy, and break down if pushed too hard. Even a concept as dear to us as *I*. It took me a while to grasp this, but when I did it was fairly sudden, like someone in the nineteenth century grasping evolution and realizing the story of creation they'd been told as a child was all wrong.² Outside of math there's a limit to how far you can push words; in fact, it would not be a bad definition of math to call it the study of terms that have precise meanings. Everyday words are inherently imprecise. They work well enough in everyday life that you don't notice. Words seem to work, just as Newtonian physics seems to. But you can always make them break if you push them far enough.

I would say that this has been, unfortunately for philosophy, the central fact of philosophy. Most philosophical debates are not merely afflicted by but driven by confusions over words. Do we have free will? Depends what you mean by "free." Do abstract ideas exist? Depends what you mean by "exist."

Wittgenstein is popularly credited with the idea that most philosoph-

²It was harder for Darwin's contemporaries to grasp this than we can easily imagine. The story of creation in the Bible is not just a Judeo-Christian concept; it's roughly what everyone must have believed since before people were people. The hard part of grasping evolution was to realize that species weren't, as they seem to be, unchanging, but had instead evolved from different, simpler organisms over unimaginably long periods of time.

Now we don't have to make that leap. No one in an industrialized country encounters the idea of evolution for the first time as an adult. Everyone's taught about it as a child, either as truth or heresy.

ical controversies are due to confusions over language. I'm not sure how much credit to give him. I suspect a lot of people realized this, but reacted simply by not studying philosophy, rather than becoming philosophy professors.

How did things get this way? Can something people have spent thousands of years studying really be a waste of time? Those are interesting questions. In fact, some of the most interesting questions you can ask about philosophy. The most valuable way to approach the current philosophical tradition may be neither to get lost in pointless speculations like Berkeley, nor to shut them down like Wittgenstein, but to study it as an example of reason gone wrong.

History

Western philosophy really begins with Socrates, Plato, and Aristotle. What we know of their predecessors comes from fragments and references in later works; their doctrines could be described as speculative cosmology that occasionally strays into analysis. Presumably they were driven by whatever makes people in every other society invent cosmologies.³

With Socrates, Plato, and particularly Aristotle, this tradition turned a corner. There started to be a lot more analysis. I suspect Plato and Aristotle were encouraged in this by progress in math. Mathematicians had by then shown that you could figure things out in a much more conclusive way than by making up fine sounding stories about them.⁴

³Greek philosophers before Plato wrote in verse. This must have affected what they said. If you try to write about the nature of the world in verse, it inevitably turns into incantation. Prose lets you be more precise, and more tentative.

⁴Philosophy is like math's ne'er-do-well brother. It was born when Plato and Aristotle looked at the works of their predecessors and said in effect "why can't you be more like your brother?" Russell was still saying the same thing 2300 years later.

Math is the precise half of the most abstract ideas, and philosophy the imprecise half. It's probably inevitable that philosophy will suffer by comparison, because there's no lower bound to its precision. Bad math is merely boring, whereas bad philosophy is nonsense. And yet there are *some* good ideas in the imprecise half.

People talk so much about abstractions now that we don't realize what a leap it must have been when they first started to. It was presumably many thousands of years between when people first started describing things as hot or cold and when someone asked "what is heat?" No doubt it was a very gradual process. We don't know if Plato or Aristotle were the first to ask any of the questions they did. But their works are the oldest we have that do this on a large scale, and there is a freshness (not to say naivete) about them that suggests some of the questions they asked were new to them, at least.

Aristotle in particular reminds me of the phenomenon that happens when people discover something new, and are so excited by it that they race through a huge percentage of the newly discovered territory in one lifetime. If so, that's evidence of how new this kind of thinking was.⁵

This is all to explain how Plato and Aristotle can be very impressive and yet naive and mistaken. It was impressive even to ask the questions they did. That doesn't mean they always came up with good answers. It's not considered insulting to say that ancient Greek mathematicians were naive in some respects, or at least lacked some concepts that would have made their lives easier. So I hope people will not be too offended if I propose that ancient philosophers were similarly naive. In particular, they don't seem to have fully grasped what I earlier called the central fact of philosophy: that words break if you push them too far.

"Much to the surprise of the builders of the first digital computers," Rod Brooks wrote, "programs written for them usually did not work."

⁶ Something similar happened when people first started trying to talk about abstractions. Much to their surprise, they didn't arrive at answers they agreed upon. In fact, they rarely seemed to arrive at an-

⁵Aristotle's best work was in logic and zoology, both of which he can be said to have invented. But the most dramatic departure from his predecessors was a new, much more analytical style of thinking. He was arguably the first scientist.

⁶Brooks, Rodney, *Programming in Common Lisp*, Wiley, 1985, p. 94.

swers at all.

They were in effect arguing about artifacts induced by sampling at too low a resolution.

The proof of how useless some of their answers turned out to be is how little effect they have. No one after reading Aristotle's *Metaphysics* does anything differently as a result.⁷

Surely I'm not claiming that ideas have to have practical applications to be interesting? No, they may not have to. Hardy's boast that number theory had no use whatsoever wouldn't disqualify it. But he turned out to be mistaken. In fact, it's suspiciously hard to find a field of math that truly has no practical use. And Aristotle's explanation of the ultimate goal of philosophy in Book A of the *Metaphysics* implies that philosophy should be useful too.

Theoretical Knowledge

Aristotle's goal was to find the most general of general principles. The examples he gives are convincing: an ordinary worker builds things a certain way out of habit; a master craftsman can do more because he grasps the underlying principles. The trend is clear: the more general the knowledge, the more admirable it is. But then he makes a mistake—possibly the most important mistake in the history of philosophy. He has noticed that theoretical knowledge is often acquired for its own sake, out of curiosity, rather than for any practical need. So he proposes there are two kinds of theoretical knowledge: some that's useful in practical matters and some that isn't. Since people interested in the latter are interested in it for its own sake, it must be more noble.

⁷Some would say we depend on Aristotle more than we realize, because his ideas were one of the ingredients in our common culture. Certainly a lot of the words we use have a connection with Aristotle, but it seems a bit much to suggest that we wouldn't have the concept of the essence of something or the distinction between matter and form if Aristotle hadn't written about them.

One way to see how much we really depend on Aristotle would be to diff European culture with Chinese: what ideas did European culture have in 1800 that Chinese culture didn't, in virtue of Aristotle's contribution?

So he sets as his goal in the *Metaphysics* the exploration of knowledge that has no practical use. Which means no alarms go off when he takes on grand but vaguely understood questions and ends up getting lost in a sea of words.

His mistake was to confuse motive and result. Certainly, people who want a deep understanding of something are often driven by curiosity rather than any practical need. But that doesn't mean what they end up learning is useless. It's very valuable in practice to have a deep understanding of what you're doing; even if you're never called on to solve advanced problems, you can see shortcuts in the solution of simple ones, and your knowledge won't break down in edge cases, as it would if you were relying on formulas you didn't understand. Knowledge is power. That's what makes theoretical knowledge prestigious. It's also what causes smart people to be curious about certain things and not others; our DNA is not so disinterested as we might think.

So while ideas don't have to have immediate practical applications to be interesting, the kinds of things we find interesting will surprisingly often turn out to have practical applications.

The reason Aristotle didn't get anywhere in the *Metaphysics* was partly that he set off with contradictory aims: to explore the most abstract ideas, guided by the assumption that they were useless. He was like an explorer looking for a territory to the north of him, starting with the assumption that it was located to the south.

And since his work became the map used by generations of future explorers, he sent them off in the wrong direction as well.⁸ Perhaps

⁸The meaning of the word "philosophy" has changed over time. In ancient times it covered a broad range of topics, comparable in scope to our "scholarship" (though without the methodological implications). Even as late as Newton's time it included what we now call "science." But core of the subject today is still what seemed to Aristotle the core: the attempt to discover the most general truths.

Aristotle didn't call this "metaphysics." That name got assigned to it because the books we now call the *Metaphysics* came after (meta = after) the *Physics* in the standard edition of Aristotle's works compiled by Andronicus of Rhodes three centuries later. What we call "metaphysics" Aristotle called "first philosophy."

worst of all, he protected them from both the criticism of outsiders and the promptings of their own inner compass by establishing the principle that the most noble sort of theoretical knowledge had to be useless.

The *Metaphysics* is mostly a failed experiment. A few ideas from it turned out to be worth keeping; the bulk of it has had no effect at all. The *Metaphysics* is among the least read of all famous books. It's not hard to understand the way Newton's *Principia* is, but the way a garbled message is.

Arguably it's an interesting failed experiment. But unfortunately that was not the conclusion Aristotle's successors derived from works like the *Metaphysics*.⁹ Soon after, the western world fell on intellectual hard times. Instead of version 1s to be superseded, the works of Plato and Aristotle became revered texts to be mastered and discussed. And so things remained for a shockingly long time. It was not till around 1600 (in Europe, where the center of gravity had shifted by then) that one found people confident enough to treat Aristotle's work as a catalog of mistakes. And even then they rarely said so outright.

If it seems surprising that the gap was so long, consider how little progress there was in math between Hellenistic times and the Renaissance.

In the intervening years an unfortunate idea took hold: that it was not only acceptable to produce works like the *Metaphysics*, but that it was a particularly prestigious line of work, done by a class of people called philosophers. No one thought to go back and debug Aristotle's motivating argument. And so instead of correcting the problem Aristotle discovered by falling into it—that you can easily get lost if you talk too loosely about very abstract ideas—they continued to fall into it.

The Singularity

Curiously, however, the works they produced continued to attract new

⁹Some of Aristotle's immediate successors may have realized this, but it's hard to say because most of their works are lost.

readers. Traditional philosophy occupies a kind of singularity in this respect. If you write in an unclear way about big ideas, you produce something that seems tantalizingly attractive to inexperienced but intellectually ambitious students. Till one knows better, it's hard to distinguish something that's hard to understand because the writer was unclear in his own mind from something like a mathematical proof that's hard to understand because the ideas it represents are hard to understand. To someone who hasn't learned the difference, traditional philosophy seems extremely attractive: as hard (and therefore impressive) as math, yet broader in scope. That was what lured me in as a high school student.

This singularity is even more singular in having its own defense built in. When things are hard to understand, people who suspect they're nonsense generally keep quiet. There's no way to prove a text is meaningless. The closest you can get is to show that the official judges of some class of texts can't distinguish them from placebos.¹⁰

And so instead of denouncing philosophy, most people who suspected it was a waste of time just studied other things. That alone is fairly damning evidence, considering philosophy's claims. It's supposed to be about the ultimate truths. Surely all smart people would be interested in it, if it delivered on that promise.

Because philosophy's flaws turned away the sort of people who might have corrected them, they tended to be self-perpetuating. Bertrand Russell wrote in a letter in 1912:

Hitherto the people attracted to philosophy have been mostly those who loved the big generalizations, which were all wrong, so that few people with exact minds have

¹⁰Sokal, Alan, "Transgressing the Boundaries: Toward a Transformative Hermeneutics of Quantum Gravity," *Social Text* 46/47, pp. 217-252.

Abstract-sounding nonsense seems to be most attractive when it's aligned with some axe the audience already has to grind. If this is so we should find it's most popular with groups that are (or feel) weak. The powerful don't need its reassurance.

taken up the subject.¹¹

His response was to launch Wittgenstein at it, with dramatic results.

I think Wittgenstein deserves to be famous not for the discovery that most previous philosophy was a waste of time, which judging from the circumstantial evidence must have been made by every smart person who studied a little philosophy and declined to pursue it further, but for how he acted in response.¹² Instead of quietly switching to another field, he made a fuss, from inside. He was Gorbachev.

The field of philosophy is still shaken from the fright Wittgenstein gave it.¹³ Later in life he spent a lot of time talking about how words worked. Since that seems to be allowed, that's what a lot of philosophers do now. Meanwhile, sensing a vacuum in the metaphysical speculation department, the people who used to do literary criticism have been edging Kantward, under new names like "literary theory," "critical theory," and when they're feeling ambitious, plain "theory." The writing is the familiar word salad:

Gender is not like some of the other grammatical modes which express precisely a mode of conception without any reality that corresponds to the conceptual mode, and consequently do not express precisely something in reality by which the intellect could be moved to conceive a

¹¹Letter to Ottoline Morrell, December 1912. Quoted in:

Monk, Ray, *Ludwig Wittgenstein: The Duty of Genius*, Penguin, 1991, p. 75.

¹²A preliminary result, that all metaphysics between Aristotle and 1783 had been a waste of time, is due to I. Kant.

¹³Wittgenstein asserted a sort of mastery to which the inhabitants of early 20th century Cambridge seem to have been peculiarly vulnerable—perhaps partly because so many had been raised religious and then stopped believing, so had a vacant space in their heads for someone to tell them what to do (others chose Marx or Cardinal Newman), and partly because a quiet, earnest place like Cambridge in that era had no natural immunity to messianic figures, just as European politics then had no natural immunity to dictators.

thing the way it does, even where that motive is not something in the thing as such.¹⁴

The singularity I've described is not going away. There's a market for writing that sounds impressive and can't be disproven. There will always be both supply and demand. So if one group abandons this territory, there will always be others ready to occupy it.

A Proposal

We may be able to do better. Here's an intriguing possibility. Perhaps we should do what Aristotle meant to do, instead of what he did. The goal he announces in the *Metaphysics* seems one worth pursuing: to discover the most general truths. That sounds good. But instead of trying to discover them because they're useless, let's try to discover them because they're useful.

I propose we try again, but that we use that heretofore despised criterion, applicability, as a guide to keep us from wondering off into a swamp of abstractions. Instead of trying to answer the question:

What are the most general truths?

let's try to answer the question

Of all the useful things we can say, which are the most general?

The test of utility I propose is whether we cause people who read what we've written to do anything differently afterward. Knowing we have to give definite (if implicit) advice will keep us from straying beyond the resolution of the words we're using.

¹⁴This is actually from the *Ordinatio* of Duns Scotus (ca. 1300), with "number" replaced by "gender." Plus ca change.

Wolter, Allan (trans), *Duns Scotus: Philosophical Writings*, Nelson, 1963, p. 92.

The goal is the same as Aristotle's; we just approach it from a different direction.

As an example of a useful, general idea, consider that of the controlled experiment. There's an idea that has turned out to be widely applicable. Some might say it's part of science, but it's not part of any specific science; it's literally meta-physics (in our sense of "meta"). The idea of evolution is another. It turns out to have quite broad applications—for example, in genetic algorithms and even product design. Frankfurt's distinction between lying and bullshitting seems a promising recent example.¹⁵

These seem to me what philosophy should look like: quite general observations that would cause someone who understood them to do something differently.

Such observations will necessarily be about things that are imprecisely defined. Once you start using words with precise meanings, you're doing math. So starting from utility won't entirely solve the problem I described above—it won't flush out the metaphysical singularity. But it should help. It gives people with good intentions a new roadmap into abstraction. And they may thereby produce things that make the writing of the people with bad intentions look bad by comparison.

One drawback of this approach is that it won't produce the sort of writing that gets you tenure. And not just because it's not currently the fashion. In order to get tenure in any field you must not arrive at conclusions that members of tenure committees can disagree with. In practice there are two kinds of solutions to this problem. In math and the sciences, you can prove what you're saying, or at any rate adjust your conclusions so you're not claiming anything false ("6 of 8 subjects had lower blood pressure after the treatment"). In the humanities you can either avoid drawing any definite conclusions (e.g. conclude that an issue is a complex one), or draw conclusions so narrow that no one cares enough to disagree with you.

¹⁵Frankfurt, Harry, *On Bullshit*, Princeton University Press, 2005.

The kind of philosophy I'm advocating won't be able to take either of these routes. At best you'll be able to achieve the essayist's standard of proof, not the mathematician's or the experimentalist's. And yet you won't be able to meet the usefulness test without implying definite and fairly broadly applicable conclusions. Worse still, the usefulness test will tend to produce results that annoy people: there's no use in telling people things they already believe, and people are often upset to be told things they don't.

Here's the exciting thing, though. Anyone can do this. Getting to general plus useful by starting with useful and cranking up the generality may be unsuitable for junior professors trying to get tenure, but it's better for everyone else, including professors who already have it. This side of the mountain is a nice gradual slope. You can start by writing things that are useful but very specific, and then gradually make them more general. Joe's has good burritos. What makes a good burrito? What makes good food? What makes anything good? You can take as long as you want. You don't have to get all the way to the top of the mountain. You don't have to tell anyone you're doing philosophy.

If it seems like a daunting task to do philosophy, here's an encouraging thought. The field is a lot younger than it seems. Though the first philosophers in the western tradition lived about 2500 years ago, it would be misleading to say the field is 2500 years old, because for most of that time the leading practitioners weren't doing much more than writing commentaries on Plato or Aristotle while watching over their shoulders for the next invading army. In the times when they weren't, philosophy was hopelessly intermingled with religion. It didn't shake itself free till a couple hundred years ago, and even then was afflicted by the structural problems I've described above. If I say this, some will say it's a ridiculously overbroad and uncharitable generalization, and others will say it's old news, but here goes: judging from their works, most philosophers up to the present have been wasting their time. So in a sense the field is still at the first step.¹⁶

¹⁶Some introductions to philosophy now take the line that philosophy is worth

That sounds a preposterous claim to make. It won't seem so preposterous in 10,000 years. Civilization always seems old, because it's always the oldest it's ever been. The only way to say whether something is really old or not is by looking at structural evidence, and structurally philosophy is young; it's still reeling from the unexpected breakdown of words.

Philosophy is as young now as math was in 1500. There is a lot more to discover.

Thanks to Trevor Blackwell, Paul Buchheit, Jessica Livingston, Robert Morris, Mark Nitzberg, and Peter Norvig for reading drafts of this.

studying as a process rather than for any particular truths you'll learn. The philosophers whose works they cover would be rolling in their graves at that. They hoped they were doing more than serving as examples of how to argue: they hoped they were getting results. Most were wrong, but it doesn't seem an impossible hope.

This argument seems to me like someone in 1500 looking at the lack of results achieved by alchemy and saying its value was as a process. No, they were going about it wrong. It turns out it is possible to transmute lead into gold (though not economically at current energy prices), but the route to that knowledge was to back-track and try another approach.

Chapter 16

The Future of Web Startups

October 2007

(This essay is derived from a keynote at FOWA in October 2007.)

There's something interesting happening right now. Startups are undergoing the same transformation that technology does when it becomes cheaper.

It's a pattern we see over and over in technology. Initially there's some device that's very expensive and made in small quantities. Then someone discovers how to make them cheaply; many more get built; and as a result they can be used in new ways.

Computers are a familiar example. When I was a kid, computers were big, expensive machines built one at a time. Now they're a commodity. Now we can stick computers in everything.

This pattern is very old. Most of the turning points in economic history are instances of it. It happened to steel in the 1850s, and to power in the 1780s. It happened to cloth manufacture in the thirteenth century, generating the wealth that later brought about the Renaissance. Agriculture itself was an instance of this pattern.

Now as well as being produced by startups, this pattern is happening *to* startups. It's so cheap to start web startups that orders of magnitudes more will be started. If the pattern holds true, that should cause dramatic changes.

1. Lots of Startups

So my first prediction about the future of web startups is pretty

straightforward: there will be a lot of them. When starting a startup was expensive, you had to get the permission of investors to do it. Now the only threshold is courage.

Even that threshold is getting lower, as people watch others take the plunge and survive. In the last batch of startups we funded, we had several founders who said they'd thought of applying before, but weren't sure and got jobs instead. It was only after hearing reports of friends who'd done it that they decided to try it themselves.

Starting a startup is hard, but having a 9 to 5 job is hard too, and in some ways a worse kind of hard. In a startup you have lots of worries, but you don't have that feeling that your life is flying by like you do in a big company. Plus in a startup you could make much more money.

As word spreads that startups work, the number may grow to a point that would now seem surprising.

We now think of it as normal to have a job at a company, but this is the thinnest of historical veneers. Just two or three lifetimes ago, most people in what are now called industrialized countries lived by farming. So while it may seem surprising to propose that large numbers of people will change the way they make a living, it would be more surprising if they didn't.

2. Standardization

When technology makes something dramatically cheaper, standardization always follows. When you make things in large volumes you tend to standardize everything that doesn't need to change.

At Y Combinator we still only have four people, so we try to standardize everything. We could hire employees, but we want to be forced to figure out how to scale investing.

We often tell startups to release a minimal version one quickly, then let the needs of the users determine what to do next. In essence, let the market design the product. We've done the same thing ourselves. We think of the techniques we're developing for dealing with large

numbers of startups as like software. Sometimes it literally is software, like Hacker News and our application system.

One of the most important things we've been working on standardizing are investment terms. Till now investment terms have been individually negotiated. This is a problem for founders, because it makes raising money take longer and cost more in legal fees. So as well as using the same paperwork for every deal we do, we've commissioned generic angel paperwork that all the startups we fund can use for future rounds.

Some investors will still want to cook up their own deal terms. Series A rounds, where you raise a million dollars or more, will be custom deals for the foreseeable future. But I think angel rounds will start to be done mostly with standardized agreements. An angel who wants to insert a bunch of complicated terms into the agreement is probably not one you want anyway.

3. New Attitude to Acquisition

Another thing I see starting to get standardized is acquisitions. As the volume of startups increases, big companies will start to develop standardized procedures that make acquisitions little more work than hiring someone.

Google is the leader here, as in so many areas of technology. They buy a lot of startups— more than most people realize, because they only announce a fraction of them. And being Google, they're figuring out how to do it efficiently.

One problem they've solved is how to think about acquisitions. For most companies, acquisitions still carry some stigma of inadequacy. Companies do them because they have to, but there's usually some feeling they shouldn't have to—that their own programmers should be able to build everything they need.

Google's example should cure the rest of the world of this idea. Google has by far the best programmers of any public technology company. If

they don't have a problem doing acquisitions, the others should have even less problem. However many Google does, Microsoft should do ten times as many.

One reason Google doesn't have a problem with acquisitions is that they know first-hand the quality of the people they can get that way. Larry and Sergey only started Google after making the rounds of the search engines trying to sell their idea and finding no takers. They've *been* the guys coming in to visit the big company, so they know who might be sitting across that conference table from them.

4. Riskier Strategies are Possible

Risk is always proportionate to reward. The way to get really big returns is to do things that seem crazy, like starting a new search engine in 1998, or turning down a billion dollar acquisition offer.

This has traditionally been a problem in venture funding. Founders and investors have different attitudes to risk. Knowing that risk is on average proportionate to reward, investors like risky strategies, while founders, who don't have a big enough sample size to care what's true on average, tend to be more conservative.

If startups are easy to start, this conflict goes away, because founders can start them younger, when it's rational to take more risk, and can start more startups total in their careers. When founders can do lots of startups, they can start to look at the world in the same portfolio-optimizing way as investors. And that means the overall amount of wealth created can be greater, because strategies can be riskier.

5. Younger, Nerdier Founders

If startups become a cheap commodity, more people will be able to have them, just as more people could have computers once micro-processors made them cheap. And in particular, younger and more technical founders will be able to start startups than could before.

Back when it cost a lot to start a startup, you had to convince investors to let you do it. And that required very different skills from actually do-

ing the startup. If investors were perfect judges, the two would require exactly the same skills. But unfortunately most investors are terrible judges. I know because I see behind the scenes what an enormous amount of work it takes to raise money, and the amount of selling required in an industry is always inversely proportional to the judgment of the buyers.

Fortunately, if startups get cheaper to start, there's another way to convince investors. Instead of going to venture capitalists with a business plan and trying to convince them to fund it, you can get a product launched on a few tens of thousands of dollars of seed money from us or your uncle, and approach them with a working company instead of a plan for one. Then instead of having to seem smooth and confident, you can just point them to Alexa.

This way of convincing investors is better suited to hackers, who often went into technology in part because they felt uncomfortable with the amount of fakeness required in other fields.

6. Startup Hubs Will Persist

It might seem that if startups get cheap to start, it will mean the end of startup hubs like Silicon Valley. If all you need to start a startup is rent money, you should be able to do it anywhere.

This is kind of true and kind of false. It's true that you can now *start* a startup anywhere. But you have to do more with a startup than just start it. You have to make it succeed. And that is more likely to happen in a startup hub.

I've thought a lot about this question, and it seems to me the increasing cheapness of web startups will if anything increase the importance of startup hubs. The value of startup hubs, like centers for any kind of business, lies in something very old-fashioned: face to face meetings. No technology in the immediate future will replace walking down University Ave and running into a friend who tells you how to fix a bug that's been bothering you all weekend, or visiting a friend's startup down the street and ending up in a conversation with one of their

investors.

The question of whether to be in a startup hub is like the question of whether to take outside investment. The question is not whether you *need* it, but whether it brings any advantage at all. Because anything that brings an advantage will give your competitors an advantage over you if they do it and you don't. So if you hear someone saying "we don't need to be in Silicon Valley," that use of the word "need" is a sign they're not even thinking about the question right.

And while startup hubs are as powerful magnets as ever, the increasing cheapness of starting a startup means the particles they're attracting are getting lighter. A startup now can be just a pair of 22 year old guys. A company like that can move much more easily than one with 10 people, half of whom have kids.

We know because we make people move for Y Combinator, and it doesn't seem to be a problem. The advantage of being able to work together face to face for three months outweighs the inconvenience of moving. Ask anyone who's done it.

The mobility of seed-stage startups means that seed funding is a national business. One of the most common emails we get is from people asking if we can help them set up a local clone of Y Combinator. But this just wouldn't work. Seed funding isn't regional, just as big research universities aren't.

Is seed funding not merely national, but international? Interesting question. There are signs it may be. We've had an ongoing stream of founders from outside the US, and they tend to do particularly well, because they're all people who were so determined to succeed that they were willing to move to another country to do it.

The more mobile startups get, the harder it would be to start new silicon valleys. If startups are mobile, the best local talent will go to the real Silicon Valley, and all they'll get at the local one will be the people who didn't have the energy to move.

This is not a nationalistic idea, incidentally. It's cities that compete, not countries. Atlanta is just as hosed as Munich.

7. Better Judgement Needed

If the number of startups increases dramatically, then the people whose job is to judge them are going to have to get better at it. I'm thinking particularly of investors and acquirers. We now get on the order of 1000 applications a year. What are we going to do if we get 10,000?

That's actually an alarming idea. But we'll figure out some kind of answer. We'll have to. It will probably involve writing some software, but fortunately we can do that.

Acquirers will also have to get better at picking winners. They generally do better than investors, because they pick later, when there's more performance to measure. But even at the most advanced acquirers, identifying companies to buy is extremely ad hoc, and completing the acquisition often involves a great deal of unnecessary friction.

I think acquirers may eventually have chief acquisition officers who will both identify good acquisitions and make the deals happen. At the moment those two functions are separate. Promising new startups are often discovered by developers. If someone powerful enough wants to buy them, the deal is handed over to corp dev guys to negotiate. It would be better if both were combined in one group, headed by someone with a technical background and some vision of what they wanted to accomplish. Maybe in the future big companies will have both a VP of Engineering responsible for technology developed in-house, and a CAO responsible for bringing technology in from outside.

At the moment, there is no one within big companies who gets in trouble when they buy a startup for \$200 million that they could have bought earlier for \$20 million. There should start to be someone who gets in trouble for that.

8. College Will Change

If the best hackers start their own companies after college instead of getting jobs, that will change what happens in college. Most of these changes will be for the better. I think the experience of college is warped in a bad way by the expectation that afterward you'll be judged by potential employers.

One change will be in the meaning of "after college," which will switch from when one graduates from college to when one leaves it. If you're starting your own company, why do you need a degree? We don't encourage people to start startups during college, but the best founders are certainly capable of it. Some of the most successful companies we've funded were started by undergrads.

I grew up in a time where college degrees seemed really important, so I'm alarmed to be saying things like this, but there's nothing magical about a degree. There's nothing that magically changes after you take that last exam. The importance of degrees is due solely to the administrative needs of large organizations. These can certainly affect your life—it's hard to get into grad school, or to get a work visa in the US, without an undergraduate degree—but tests like this will matter less and less.

As well as mattering less whether students get degrees, it will also start to matter less where they go to college. In a startup you're judged by users, and they don't care where you went to college. So in a world of startups, elite universities will play less of a role as gatekeepers. In the US it's a national scandal how easily children of rich parents game college admissions. But the way this problem ultimately gets solved may not be by reforming the universities but by going around them. We in the technology world are used to that sort of solution: you don't beat the incumbents; you redefine the problem to make them irrelevant.

The greatest value of universities is not the brand name or perhaps even the classes so much as the people you meet. If it becomes common to start a startup after college, students may start trying to max-

imize this. Instead of focusing on getting internships at companies they want to work for, they may start to focus on working with other students they want as cofounders.

What students do in their classes will change too. Instead of trying to get good grades to impress future employers, students will try to learn things. We're talking about some pretty dramatic changes here.

9. Lots of Competitors

If it gets easier to start a startup, it's easier for competitors too. That doesn't erase the advantage of increased cheapness, however. You're not all playing a zero-sum game. There's not some fixed number of startups that can succeed, regardless of how many are started.

In fact, I don't think there's any limit to the number of startups that could succeed. Startups succeed by creating wealth, which is the satisfaction of people's desires. And people's desires seem to be effectively infinite, at least in the short term.

What the increasing number of startups does mean is that you won't be able to sit on a good idea. Other people have your idea, and they'll be increasingly likely to do something about it.

10. Faster Advances

There's a good side to that, at least for consumers of technology. If people get right to work implementing ideas instead of sitting on them, technology will evolve faster.

Some kinds of innovations happen a company at a time, like the punctuated equilibrium model of evolution. There are some kinds of ideas that are so threatening that it's hard for big companies even to think of them. Look at what a hard time Microsoft is having discovering web apps. They're like a character in a movie that everyone in the audience can see something bad is about to happen to, but who can't see it himself. The big innovations that happen a company at a time will obviously happen faster if the rate of new companies increases.

But in fact there will be a double speed increase. People won't wait as long to act on new ideas, but also those ideas will increasingly be developed within startups rather than big companies. Which means technology will evolve faster per company as well.

Big companies are just not a good place to make things happen fast. I talked recently to a founder whose startup had been acquired by a big company. He was a precise sort of guy, so he'd measured their productivity before and after. He counted lines of code, which can be a dubious measure, but in this case was meaningful because it was the same group of programmers. He found they were one thirteenth as productive after the acquisition.

The company that bought them was not a particularly stupid one. I think what he was measuring was mostly the cost of bigness. I experienced this myself, and his number sounds about right. There's something about big companies that just sucks the energy out of you.

Imagine what all that energy could do if it were put to use. There is an enormous latent capacity in the world's hackers that most people don't even realize is there. That's the main reason we do Y Combinator: to let loose all this energy by making it easy for hackers to start their own startups.

A Series of Tubes

The process of starting startups is currently like the plumbing in an old house. The pipes are narrow and twisty, and there are leaks in every joint. In the future this mess will gradually be replaced by a single, huge pipe. The water will still have to get from A to B, but it will get there faster and without the risk of spraying out through some random leak.

This will change a lot of things for the better. In a big, straight pipe like that, the force of being measured by one's performance will propagate back through the whole system. Performance is always the ultimate test, but there are so many kinks in the plumbing now that most people are insulated from it most of the time. So you end up with a world

in which high school students think they need to get good grades to get into elite colleges, and college students think they need to get good grades to impress employers, within which the employees waste most of their time in political battles, and from which consumers have to buy anyway because there are so few choices. Imagine if that sequence became a big, straight pipe. Then the effects of being measured by performance would propagate all the way back to high school, flushing out all the arbitrary stuff people are measured by now. That is the future of web startups.

Thanks to Brian Oberkirch and Simon Willison for inviting me to speak, and the crew at Carson Systems for making everything run smoothly.

Chapter 17

Why to Move to a Startup Hub



October 2007

After the last talk I gave, one of the organizers got up on the stage to deliver an impromptu rebuttal. That never happened before. I only heard the first few sentences, but that was enough to tell what I said that upset him: that startups would do better if they moved to Silicon Valley.

This conference was in London, and most of the audience seemed to be from the UK. So saying startups should move to Silicon Valley seemed like a nationalistic remark: an obnoxious American telling them that if they wanted to do things right they should all just move to America.

Actually I'm less American than I seem. I didn't say so, but I'm British by birth. And just as Jews are ex officio allowed to tell Jewish jokes, I don't feel like I have to bother being diplomatic with a British audience.

The idea that startups would do better to move to Silicon Valley is not even a nationalistic one.¹ It's the same thing I say to startups

¹The nationalistic idea is the converse: that startups should stay in a certain city

in the US. Y Combinator alternates between coasts every 6 months. Every other funding cycle is in Boston. And even though Boston is the second biggest startup hub in the US (and the world), we tell the startups from those cycles that their best bet is to move to Silicon Valley. If that's true of Boston, it's even more true of every other city.

This is about cities, not countries.

And I think I can prove I'm right. You can easily reduce the opposing argument ad what most people would agree was absurdum. Few would be willing to claim that it doesn't matter at all where a startup is—that a startup operating out of a small agricultural town wouldn't benefit from moving to a startup hub. Most people could see how it might be helpful to be in a place where there was infrastructure for startups, accumulated knowledge about how to make them work, and other people trying to do it. And yet whatever argument you use to prove that startups don't need to move from London to Silicon Valley could equally well be used to prove startups don't need to move from smaller towns to London.

The difference between cities is a matter of degree. And if, as nearly everyone who knows agrees, startups are better off in Silicon Valley than Boston, then they're better off in Silicon Valley than everywhere else too.

I realize I might seem to have a vested interest in this conclusion, because startups that move to the US might do it through Y Combinator. But the American startups we've funded will attest that I say the same thing to them.

I'm not claiming of course that every startup has to go to Silicon Valley to succeed. Just that all other things being equal, the more of a startup hub a place is, the better startups will do there. But other considerations can outweigh the advantages of moving. I'm not saying

because of the country it's in. If you really have a "one world" viewpoint, deciding to move from London to Silicon Valley is no different from deciding to move from Chicago to Silicon Valley.

founders with families should uproot them to move halfway around the world; that might be too much of a distraction.

Immigration difficulties might be another reason to stay put. Dealing with immigration problems is like raising money: for some reason it seems to consume all your attention. A startup can't afford much of that. One Canadian startup we funded spent about 6 months working on moving to the US. Eventually they just gave up, because they couldn't afford to take so much time away from working on their software.

(If another country wanted to establish a rival to Silicon Valley, the single best thing they could do might be to create a special visa for startup founders. US immigration policy is one of Silicon Valley's biggest weaknesses.)

If your startup is connected to a specific industry, you may be better off in one of its centers. A startup doing something related to entertainment might want to be in New York or LA.

And finally, if a good investor has committed to fund you if you stay where you are, you should probably stay. Finding investors is hard. You generally shouldn't pass up a definite funding offer to move.²

In fact, the quality of the investors may be the main advantage of startup hubs. Silicon Valley investors are noticeably more aggressive than Boston ones. Over and over, I've seen startups we've funded snatched by west coast investors out from under the noses of Boston investors who saw them first but acted too slowly. At this year's Boston Demo Day, I told the audience that this happened every year, so if they saw a startup they liked, they should make them an offer. And yet within a month it had happened again: an aggressive west coast VC who had met the founder of a YC-funded startup a week before beat out a Boston VC who had known him for years. By the time the Boston VC grasped what was happening, the deal was already gone.

²An investor who merely seems like he will fund you, however, you can ignore. Seeming like they will fund you one day is the way investors say No.

Boston investors will admit they're more conservative. Some want to believe this comes from the city's prudent Yankee character. But Occam's razor suggests the truth is less flattering. Boston investors are probably more conservative than Silicon Valley investors for the same reason Chicago investors are more conservative than Boston ones. They don't understand startups as well.

West coast investors aren't bolder because they're irresponsible cowboys, or because the good weather makes them optimistic. They're bolder because they know what they're doing. They're the skiers who ski on the diamond slopes. Boldness is the essence of venture investing. The way you get big returns is not by trying to avoid losses, but by trying to ensure you get some of the big hits. And the big hits often look risky at first.

Like Facebook. Facebook was started in Boston. Boston VCs had the first shot at them. But they said no, so Facebook moved to Silicon Valley and raised money there. The partner who turned them down now says that "may turn out to have been a mistake."

Empirically, boldness wins. If the aggressive ways of west coast investors are going to come back to bite them, it has been a long time coming. Silicon Valley has been pulling ahead of Boston since the 1970s. If there was going to be a comeuppance for the west coast investors, the bursting of the Bubble would have been it. But since then the west coast has just pulled further ahead.

West coast investors are confident enough of their judgement to act boldly; east coast investors, not so much; but anyone who thinks east coast investors act that way out of prudence should see the frantic reactions of an east coast VC in the process of losing a deal to a west coast one.

In addition to the concentration that comes from specialization, startup hubs are also markets. And markets are usually centralized. Even now, when traders could be anywhere, they cluster in a few cities. It's hard to say exactly what it is about face to face contact that makes

deals happen, but whatever it is, it hasn't yet been duplicated by technology.

Walk down University Ave at the right time, and you might overhear five different people talking on the phone about deals. In fact, this is part of the reason Y Combinator is in Boston half the time: it's hard to stand that year round. But though it can sometimes be annoying to be surrounded by people who only think about one thing, it's the place to be if that one thing is what you're trying to do.

I was talking recently to someone who works on search at Google. He knew a lot of people at Yahoo, so he was in a good position to compare the two companies. I asked him why Google was better at search. He said it wasn't anything specific Google did, but simply that they understood search so much better.

And that's why startups thrive in startup hubs like Silicon Valley. Startups are a very specialized business, as specialized as diamond cutting. And in startup hubs they understand it.

Thanks to Sam Altman, Jessica Livingston, Harjeet Taggar, and Kulveer Taggar for reading drafts of this.

Chapter 18

Six Principles for Making New Things



February 2008

The fiery reaction to the release of Arc had an unexpected consequence: it made me realize I had a design philosophy. The main complaint of the more articulate critics was that Arc seemed so flimsy. After years of working on it, all I had to show for myself were a few thousand lines of macros? Why hadn't I worked on more substantial problems?

As I was mulling over these remarks it struck me how familiar they seemed. This was exactly the kind of thing people said at first about Viaweb, and Y Combinator, and most of my essays.

When we launched Viaweb, it seemed laughable to VCs and e-commerce "experts." We were just a couple guys in an apartment, which did not seem cool in 1995 the way it does now. And the thing we'd built, as far as they could tell, wasn't even software. Software, to them, equalled big, honking Windows apps. Since Viaweb was the first web-based app they'd seen, it seemed to be nothing more than a website. They were even more contemptuous when they discov-

ered that Viaweb didn't process credit card transactions (we didn't for the whole first year). Transaction processing seemed to them what e-commerce was all about. It sounded serious and difficult.

And yet, mysteriously, Viaweb ended up crushing all its competitors.

The initial reaction to Y Combinator was almost identical. It seemed laughably lightweight. Startup funding meant series A rounds: millions of dollars given to a small number of startups founded by people with established credentials after months of serious, businesslike meetings, on terms described in a document a foot thick. Y Combinator seemed inconsequential. It's too early to say yet whether Y Combinator will turn out like Viaweb, but judging from the number of imitations, a lot of people seem to think we're on to something.

I can't measure whether my essays are successful, except in page views, but the reaction to them is at least different from when I started. At first the default reaction of the Slashdot trolls was (translated into articulate terms): "Who is this guy and what authority does he have to write about these topics? I haven't read the essay, but there's no way anything so short and written in such an informal style could have anything useful to say about such and such topic, when people with degrees in the subject have already written many thick books about it." Now there's a new generation of trolls on a new generation of sites, but they have at least started to omit the initial "Who is this guy?"

Now people are saying the same things about Arc that they said at first about Viaweb and Y Combinator and most of my essays. Why the pattern? The answer, I realized, is that my m.o. for all four has been the same.

Here it is: I like to find (a) simple solutions (b) to overlooked problems (c) that actually need to be solved, and (d) deliver them as informally as possible, (e) starting with a very crude version 1, then (f) iterating rapidly.

When I first laid out these principles explicitly, I noticed something striking: this is practically a recipe for generating a contemptuous ini-

tial reaction. Though simple solutions are better, they don't seem as impressive as complex ones. Overlooked problems are by definition problems that most people think don't matter. Delivering solutions in an informal way means that instead of judging something by the way it's presented, people have to actually understand it, which is more work. And starting with a crude version 1 means your initial effort is always small and incomplete.

I'd noticed, of course, that people never seemed to grasp new ideas at first. I thought it was just because most people were stupid. Now I see there's more to it than that. Like a contrarian investment fund, someone following this strategy will almost always be doing things that seem wrong to the average person.

As with contrarian investment strategies, that's exactly the point. This technique is successful (in the long term) because it gives you all the advantages other people forgo by trying to seem legit. If you work on overlooked problems, you're more likely to discover new things, because you have less competition. If you deliver solutions informally, you (a) save all the effort you would have had to expend to make them look impressive, and (b) avoid the danger of fooling yourself as well as your audience. And if you release a crude version 1 then iterate, your solution can benefit from the imagination of nature, which, as Feynman pointed out, is more powerful than your own.

In the case of Viaweb, the simple solution was to make the software run on the server. The overlooked problem was to generate web sites automatically; in 1995, online stores were all made by hand by human designers, but we knew this wouldn't scale. The part that actually mattered was graphic design, not transaction processing. The informal delivery mechanism was me, showing up in jeans and a t-shirt at some retailer's office. And the crude version 1 was, if I remember correctly, less than 10,000 lines of code when we launched.

The power of this technique extends beyond startups and programming languages and essays. It probably extends to any kind of creative work. Certainly it can be used in painting: this is exactly what

Cezanne and Klee did.

At Y Combinator we bet money on it, in the sense that we encourage the startups we fund to work this way. There are always new ideas right under your nose. So look for simple things that other people have overlooked—things people will later claim were “obvious”—especially when they’ve been led astray by obsolete conventions, or by trying to do things that are superficially impressive. Figure out what the real problem is, and make sure you solve that. Don’t worry about trying to look corporate; the product is what wins in the long term. And launch as soon as you can, so you start learning from users what you should have been making.

Reddit is a classic example of this approach. When Reddit first launched, it seemed like there was nothing to it. To the graphically unsophisticated its deliberately minimal design seemed like no design at all. But Reddit solved the real problem, which was to tell people what was new and otherwise stay out of the way. As a result it became massively successful. Now that conventional ideas have caught up with it, it seems obvious. People look at Reddit and think the founders were lucky. Like all such things, it was harder than it looked. The Reddits pushed so hard against the current that they reversed it; now it looks like they’re merely floating downstream.

So when you look at something like Reddit and think “I wish I could think of an idea like that,” remember: ideas like that are all around you. But you ignore them because they look wrong.

Chapter 19

Trolls

February 2008

A user on Hacker News recently posted a comment that set me thinking:

Something about hacker culture that never really set well with me was this — the nastiness. ... I just don't understand why people troll like they do.

I've thought a lot over the last couple years about the problem of trolls. It's an old one, as old as forums, but we're still just learning what the causes are and how to address them.

There are two senses of the word “troll.” In the original sense it meant someone, usually an outsider, who deliberately stirred up fights in a forum by saying controversial things.¹ For example, someone who didn't use a certain programming language might go to a forum for users of that language and make disparaging remarks about it, then sit back and watch as people rose to the bait. This sort of trolling was in the nature of a practical joke, like letting a bat loose in a room full of people.

The definition then spread to people who behaved like assholes in forums, whether intentionally or not. Now when people talk about trolls they usually mean this broader sense of the word. Though in a sense this is historically inaccurate, it is in other ways more accurate,

¹I mean forum in the general sense of a place to exchange views. The original Internet forums were not web sites but Usenet newsgroups.

because when someone is being an asshole it's usually uncertain even in their own mind how much is deliberate. That is arguably one of the defining qualities of an asshole.

I think trolling in the broader sense has four causes. The most important is distance. People will say things in anonymous forums that they'd never dare say to someone's face, just as they'll do things in cars that they'd never do as pedestrians — like tailgate people, or honk at them, or cut them off.

Trolling tends to be particularly bad in forums related to computers, and I think that's due to the kind of people you find there. Most of them (myself included) are more comfortable dealing with abstract ideas than with people. Hackers can be abrupt even in person. Put them on an anonymous forum, and the problem gets worse.

The third cause of trolling is incompetence. If you disagree with something, it's easier to say “you suck” than to figure out and explain exactly what you disagree with. You're also safe that way from refutation. In this respect trolling is a lot like graffiti. Graffiti happens at the intersection of ambition and incompetence: people want to make their mark on the world, but have no other way to do it than literally making a mark on the world.²

The final contributing factor is the culture of the forum. Trolls are like children (many *are* children) in that they're capable of a wide range of behavior depending on what they think will be tolerated. In a place where rudeness isn't tolerated, most can be polite. But vice versa as well.

There's a sort of Gresham's Law of trolls: trolls are willing to use a forum with a lot of thoughtful people in it, but thoughtful people aren't willing to use a forum with a lot of trolls in it. Which means that once trolling takes hold, it tends to become the dominant culture. That had already happened to Slashdot and Digg by the time I paid attention

²I'm talking here about everyday tagging. Some graffiti is quite impressive (anything becomes art if you do it well enough) but the median tag is just visual spam.

to comment threads there, but I watched it happen to Reddit.

News.YC is, among other things, an experiment to see if this fate can be avoided. The site's guidelines explicitly ask people not to say things they wouldn't say face to face. If someone starts being rude, other users will step in and tell them to stop. And when people seem to be deliberately trolling, we ban them ruthlessly.

Technical tweaks may also help. On Reddit, votes on your comments don't affect your karma score, but they do on News.YC. And it does seem to influence people when they can see their reputation in the eyes of their peers drain away after making an asshole remark. Often users have second thoughts and delete such comments.

One might worry this would prevent people from expressing controversial ideas, but empirically that doesn't seem to be what happens. When people say something substantial that gets modded down, they stubbornly leave it up. What people delete are wisecracks, because they have less invested in them.

So far the experiment seems to be working. The level of conversation on News.YC is as high as on any forum I've seen. But we still only have about 8,000 uniques a day. The conversations on Reddit were good when it was that small. The challenge is whether we can keep things this way.

I'm optimistic we will. We're not depending just on technical tricks. The core users of News.YC are mostly refugees from other sites that were overrun by trolls. They feel about trolls roughly the way refugees from Cuba or Eastern Europe feel about dictatorships. So there are a lot of people working to keep this from happening again.

Chapter 20

A New Venture Animal



March 2008, rev May 2013

(This essay grew out of something I wrote for myself to figure out what we do. Even though Y Combinator is now 3 years old, we're still trying to understand its implications.)

I was annoyed recently to read a description of Y Combinator that said “Y Combinator does seed funding for startups.” What was especially annoying about it was that I wrote it. This doesn’t really convey what we do. And the reason it’s inaccurate is that, paradoxically, funding very early stage startups is not mainly about funding.

Saying YC does seed funding for startups is a description in terms of earlier models. It’s like calling a car a horseless carriage.

When you scale animals you can’t just keep everything in proportion. For example, volume grows as the cube of linear dimension, but surface area only as the square. So as animals get bigger they have trouble radiating heat. That’s why mice and rabbits are furry and elephants and hippos aren’t. You can’t make a mouse by scaling down an elephant.

YC represents a new, smaller kind of animal—so much smaller that all the rules are different.

Before us, most companies in the startup funding business were venture capital funds. VCs generally fund later stage companies than we do. And they supply so much money that, even though the other things they do may be very valuable, it's not that inaccurate to regard VCs as sources of money. Good VCs are “smart money,” but they're still money.

All good investors supply a combination of money and help. But these scale differently, just as volume and surface area do. Late stage investors supply huge amounts of money and comparatively little help: when a company about to go public gets a mezzanine round of \$50 million, the deal tends to be almost entirely about money. As you move earlier in the venture funding process, the ratio of help to money increases, because earlier stage companies have different needs. Early stage companies need less money because they're smaller and cheaper to run, but they need more help because life is so precarious for them. So when VCs do a series A round for, say, \$2 million, they generally expect to offer a significant amount of help along with the money.

Y Combinator occupies the earliest end of the spectrum. We're at least one and generally two steps before VC funding. (Though some startups go straight from YC to VC, the most common trajectory is to do an angel round first.) And what happens at Y Combinator is as different from what happens in a series A round as a series A round is from a mezzanine financing.

At our end, money is almost a negligible factor. The startup usually consists of just the founders. Their living expenses are the company's main expense, and since most founders are under 30, their living expenses are low. But at this early stage companies need a lot of help. Practically every question is still unanswered. Some companies we've funded have been working on their software for a year or more, but others haven't decided what to work on, or even who the founders should be.

When PR people and journalists recount the histories of startups after they've become big, they always underestimate how uncertain things were at first. They're not being deliberately misleading. When you look at a company like Google, it's hard to imagine they could once have been small and helpless. Sure, at one point they were a just a couple guys in a garage—but even then their greatness was assured, and all they had to do was roll forward along the railroad tracks of destiny.

Far from it. A lot of startups with just as promising beginnings end up failing. Google has such momentum now that it would be hard for anyone to stop them. But all it would have taken in the beginning would have been for two Google employees to focus on the wrong things for six months, and the company could have died.

We know, because we've been there, just how vulnerable startups are in the earliest phases. Curiously enough, that's why founders tend to get so rich from them. Reward is always proportionate to risk, and very early stage startups are insanely risky.

What we really do at Y Combinator is get startups launched straight. One of many metaphors you could use for YC is a steam catapult on an aircraft carrier. We get startups airborne. Barely airborne, but enough that they can accelerate fast.

When you're launching planes they have to be set up properly or you're just launching projectiles. They have to be pointed straight down the deck; the wings have to be trimmed properly; the engines have to be at full power; the pilot has to be ready. These are the kind of problems we deal with. After we fund startups we work closely with them for three months—so closely in fact that we insist they move to where we are. And what we do in those three months is make sure everything is set up for launch. If there are tensions between cofounders we help sort them out. We get all the paperwork set up properly so there are no nasty surprises later. If the founders aren't sure what to focus on first, we try to figure that out. If there is some obstacle right in front of them, we either try to remove it, or shift the startup sideways.

The goal is to get every distraction out of the way so the founders can use that time to build (or finish building) something impressive. And then near the end of the three months we push the button on the steam catapult in the form of Demo Day, where the current group of startups present to pretty much every investor in Silicon Valley.

Launching companies isn't identical with launching products. Though we do spend a lot of time on launch strategies for products, there are some things that take too long to build for a startup to launch them before raising their next round of funding. Several of the most promising startups we've funded haven't launched their products yet, but are definitely launched as companies.

In the earliest stage, startups not only have more questions to answer, but they tend to be different kinds of questions. In later stage startups the questions are about deals, or hiring, or organization. In the earliest phase they tend to be about technology and design. What do you make? That's the first problem to solve. That's why our motto is "Make something people want." This is always a good thing for companies to do, but it's even more important early on, because it sets the bounds for every other question. Who you hire, how much money you raise, how you market yourself—they all depend on what you're making.

Because the early problems are so much about technology and design, you probably need to be hackers to do what we do. While some VCs have technical backgrounds, I don't know any who still write code. Their expertise is mostly in business—as it should be, because that's the kind of expertise you need in the phase between series A and (if you're lucky) IPO.

We're so different from VCs that we're really a different kind of animal. Can we claim founders are better off as a result of this new type of venture firm? I'm pretty sure the answer is yes, because YC is an improved version of what happened to our startup, and our case was not atypical. We started Viaweb with \$10,000 in seed money from our friend Julian. He was a lawyer and arranged all our paperwork,

so we could just code. We spent three months building a version 1, which we then presented to investors to raise more money. Sounds familiar, doesn't it? But YC improves on that significantly. Julian knew a lot about law and business, but his advice ended there; he was not a startup guy. So we made some basic mistakes early on. And when we presented to investors, we presented to only 2, because that was all we knew. If we'd had our later selves to encourage and advise us, and Demo Day to present at, we would have been in much better shape. We probably could have raised money at 3 to 5 times the valuation we did.

If we take 7% of a company we fund, the founders only have to do 7.5% better in their next round of funding to end up net ahead. We certainly manage that.

So who is our 7% coming out of? If the founders end up net ahead it's not coming out of them. So is it coming out of later stage investors? Well, they do end up paying more. But I think they pay more because the company is actually more valuable. And later stage investors have no problem with that. The returns of a VC fund depend on the quality of the companies they invest in, not how cheaply they can buy stock in them.

If what we do is useful, why wasn't anyone doing it before? There are two answers to that. One is that people were doing it before, just haphazardly on a smaller scale. Before us, seed funding came primarily from individual angel investors. Larry and Sergey, for example, got their seed funding from Andy Bechtolsheim, one of the founders of Sun. And because he was a startup guy he probably gave them useful advice. But raising money from angel investors is a hit or miss thing. It's a sideline for most of them, so they only do a handful of deals a year and they don't spend a lot of time on the startups they invest in. And they're hard to reach, because they don't want random startups pestering them with business plans. The Google guys were lucky because they knew someone who knew Bechtolsheim. It generally takes a personal introduction with angels.

The other reason no one was doing quite what we do is that till recently it was a lot more expensive to start a startup. You'll notice we haven't funded any biotech startups. That's still expensive. But advancing technology has made web startups so cheap that you really can get a company airborne for \$15,000. If you understand how to operate a steam catapult, at least.

So in effect what's happened is that a new ecological niche has opened up, and Y Combinator is the new kind of animal that has moved into it. We're not a replacement for venture capital funds. We occupy a new, adjacent niche. And conditions in our niche are really quite different. It's not just that the problems we face are different; the whole structure of the business is different. VCs are playing a zero-sum game. They're all competing for a slice of a fixed amount of "deal flow," and that explains a lot of their behavior. Whereas our m.o. is to create new deal flow, by encouraging hackers who would have gotten jobs to start their own startups instead. We compete more with employers than VCs.

It's not surprising something like this would happen. Most fields become more specialized—more articulated—as they develop, and startups are certainly an area in which there has been a lot of development over the past couple decades. The venture business in its present form is only about forty years old. It stands to reason it would evolve.

And it's natural that the new niche would at first be described, even by its inhabitants, in terms of the old one. But really Y Combinator is not in the startup funding business. Really we're more of a small, furry steam catapult.

Thanks to Trevor Blackwell, Jessica Livingston, and Robert Morris for reading drafts of this.

Chapter 21

You Weren't Meant to Have a Boss



March 2008, rev. June 2008

Technology tends to separate normal from natural. Our bodies weren't designed to eat the foods that people in rich countries eat, or to get so little exercise. There may be a similar problem with the way we work: a normal job may be as bad for us intellectually as white flour or sugar is for us physically.

I began to suspect this after spending several years working with startup founders. I've now worked with over 200 of them, and I've noticed a definite difference between programmers working on their own startups and those working for large organizations. I wouldn't say founders seem happier, necessarily; starting a startup can be very stressful. Maybe the best way to put it is to say that they're happier in the sense that your body is happier during a long run than sitting on a sofa eating doughnuts.

Though they're statistically abnormal, startup founders seem to be working in a way that's more natural for humans.

I was in Africa last year and saw a lot of animals in the wild that I'd only seen in zoos before. It was remarkable how different they seemed. Particularly lions. Lions in the wild seem about ten times more alive.

They're like different animals. I suspect that working for oneself feels better to humans in much the same way that living in the wild must feel better to a wide-ranging predator like a lion. Life in a zoo is easier, but it isn't the life they were designed for.

Trees

What's so unnatural about working for a big company? The root of the problem is that humans weren't meant to work in such large groups.

Another thing you notice when you see animals in the wild is that each species thrives in groups of a certain size. A herd of impalas might have 100 adults; baboons maybe 20; lions rarely 10. Humans also seem designed to work in groups, and what I've read about hunter-gatherers accords with research on organizations and my own experience to suggest roughly what the ideal size is: groups of 8 work well; by 20 they're getting hard to manage; and a group of 50 is really unwieldy.¹

Whatever the upper limit is, we are clearly not meant to work in groups of several hundred. And yet—for reasons having more to do with technology than human nature—a great many people work for companies with hundreds or thousands of employees.

Companies know groups that large wouldn't work, so they divide themselves into units small enough to work together. But to coordinate these they have to introduce something new: bosses.

These smaller groups are always arranged in a tree structure. Your boss is the point where your group attaches to the tree. But when you use this trick for dividing a large group into smaller ones, something strange happens that I've never heard anyone mention explicitly. In the group one level up from yours, your boss represents your entire group. A group of 10 managers is not merely a group of 10 people working together in the usual way. It's really a group of groups. Which

¹When I talk about humans being meant or designed to live a certain way, I mean by evolution.

means for a group of 10 managers to work together as if they were simply a group of 10 individuals, the group working for each manager would have to work as if they were a single person—the workers and manager would each share only one person’s worth of freedom between them.

In practice a group of people are never able to act as if they were one person. But in a large organization divided into groups in this way, the pressure is always in that direction. Each group tries its best to work as if it were the small group of individuals that humans were designed to work in. That was the point of creating it. And when you propagate that constraint, the result is that each person gets freedom of action in inverse proportion to the size of the entire tree.²

Anyone who’s worked for a large organization has felt this. You can feel the difference between working for a company with 100 employees and one with 10,000, even if your group has only 10 people.

Corn Syrup

A group of 10 people within a large organization is a kind of fake tribe. The number of people you interact with is about right. But something is missing: individual initiative. Tribes of hunter-gatherers have much more freedom. The leaders have a little more power than other members of the tribe, but they don’t generally tell them what to do and when the way a boss can.

It’s not your boss’s fault. The real problem is that in the group above you in the hierarchy, your entire group is one virtual person. Your boss is just the way that constraint is imparted to you.

So working in a group of 10 people within a large organization feels both right and wrong at the same time. On the surface it feels like the kind of group you’re meant to work in, but something major is missing. A job at a big company is like high fructose corn syrup: it has some of

²It’s not only the leaves who suffer. The constraint propagates up as well as down. So managers are constrained too; instead of just doing things, they have to act through subordinates.

the qualities of things you're meant to like, but is disastrously lacking in others.

Indeed, food is an excellent metaphor to explain what's wrong with the usual sort of job.

For example, working for a big company is the default thing to do, at least for programmers. How bad could it be? Well, food shows that pretty clearly. If you were dropped at a random point in America today, nearly all the food around you would be bad for you. Humans were not designed to eat white flour, refined sugar, high fructose corn syrup, and hydrogenated vegetable oil. And yet if you analyzed the contents of the average grocery store you'd probably find these four ingredients accounted for most of the calories. "Normal" food is terribly bad for you. The only people who eat what humans were actually designed to eat are a few Birkenstock-wearing weirdos in Berkeley.

If "normal" food is so bad for us, why is it so common? There are two main reasons. One is that it has more immediate appeal. You may feel lousy an hour after eating that pizza, but eating the first couple bites feels great. The other is economies of scale. Producing junk food scales; producing fresh vegetables doesn't. Which means (a) junk food can be very cheap, and (b) it's worth spending a lot to market it.

If people have to choose between something that's cheap, heavily marketed, and appealing in the short term, and something that's expensive, obscure, and appealing in the long term, which do you think most will choose?

It's the same with work. The average MIT graduate wants to work at Google or Microsoft, because it's a recognized brand, it's safe, and they'll get paid a good salary right away. It's the job equivalent of the pizza they had for lunch. The drawbacks will only become apparent later, and then only in a vague sense of malaise.

And founders and early employees of startups, meanwhile, are like the Birkenstock-wearing weirdos of Berkeley: though a tiny minority of the population, they're the ones living as humans are meant to. In

an artificial world, only extremists live naturally.

Programmers

The restrictiveness of big company jobs is particularly hard on programmers, because the essence of programming is to build new things. Sales people make much the same pitches every day; support people answer much the same questions; but once you've written a piece of code you don't need to write it again. So a programmer working as programmers are meant to is always making new things. And when you're part of an organization whose structure gives each person freedom in inverse proportion to the size of the tree, you're going to face resistance when you do something new.

This seems an inevitable consequence of bigness. It's true even in the smartest companies. I was talking recently to a founder who considered starting a startup right out of college, but went to work for Google instead because he thought he'd learn more there. He didn't learn as much as he expected. Programmers learn by doing, and most of the things he wanted to do, he couldn't—sometimes because the company wouldn't let him, but often because the company's code wouldn't let him. Between the drag of legacy code, the overhead of doing development in such a large organization, and the restrictions imposed by interfaces owned by other groups, he could only try a fraction of the things he would have liked to. He said he has learned much more in his own startup, despite the fact that he has to do all the company's errands as well as programming, because at least when he's programming he can do whatever he wants.

An obstacle downstream propagates upstream. If you're not allowed to implement new ideas, you stop having them. And vice versa: when you can do whatever you want, you have more ideas about what to do. So working for yourself makes your brain more powerful in the same way a low-restriction exhaust system makes an engine more powerful.

Working for yourself doesn't have to mean starting a startup, of course. But a programmer deciding between a regular job at a big company

and their own startup is probably going to learn more doing the startup.

You can adjust the amount of freedom you get by scaling the size of company you work for. If you start the company, you'll have the most freedom. If you become one of the first 10 employees you'll have almost as much freedom as the founders. Even a company with 100 people will feel different from one with 1000.

Working for a small company doesn't ensure freedom. The tree structure of large organizations sets an upper bound on freedom, not a lower bound. The head of a small company may still choose to be a tyrant. The point is that a large organization is compelled by its structure to be one.

Consequences

That has real consequences for both organizations and individuals. One is that companies will inevitably slow down as they grow larger, no matter how hard they try to keep their startup mojo. It's a consequence of the tree structure that every large organization is forced to adopt.

Or rather, a large organization could only avoid slowing down if they avoided tree structure. And since human nature limits the size of group that can work together, the only way I can imagine for larger groups to avoid tree structure would be to have no structure: to have each group actually be independent, and to work together the way components of a market economy do.

That might be worth exploring. I suspect there are already some highly partitionable businesses that lean this way. But I don't know any technology companies that have done it.

There is one thing companies can do short of structuring themselves as sponges: they can stay small. If I'm right, then it really pays to keep a company as small as it can be at every stage. Particularly a technology company. Which means it's doubly important to hire the

best people. Mediocre hires hurt you twice: they get less done, but they also make you big, because you need more of them to solve a given problem.

For individuals the upshot is the same: aim small. It will always suck to work for large organizations, and the larger the organization, the more it will suck.

In an essay I wrote a couple years ago I advised graduating seniors to work for a couple years for another company before starting their own. I'd modify that now. Work for another company if you want to, but only for a small one, and if you want to start your own startup, go ahead.

The reason I suggested college graduates not start startups immediately was that I felt most would fail. And they will. But ambitious programmers are better off doing their own thing and failing than going to work at a big company. Certainly they'll learn more. They might even be better off financially. A lot of people in their early twenties get into debt, because their expenses grow even faster than the salary that seemed so high when they left school. At least if you start a startup and fail your net worth will be zero rather than negative.³

We've now funded so many different types of founders that we have enough data to see patterns, and there seems to be no benefit from working for a big company. The people who've worked for a few years do seem better than the ones straight out of college, but only because they're that much older.

The people who come to us from big companies often seem kind of conservative. It's hard to say how much is because big companies made them that way, and how much is the natural conservatism that made them work for the big companies in the first place. But certainly a large part of it is learned. I know because I've seen it burn off.

³Do not finance your startup with credit cards. Financing a startup with debt is usually a stupid move, and credit card debt stupidest of all. Credit card debt is a bad idea, period. It is a trap set by evil companies for the desperate and the foolish.

Having seen that happen so many times is one of the things that convinces me that working for oneself, or at least for a small group, is the natural way for programmers to live. Founders arriving at Y Combinator often have the downtrodden air of refugees. Three months later they're transformed: they have so much more confidence that they seem as if they've grown several inches taller.⁴ Strange as this sounds, they seem both more worried and happier at the same time. Which is exactly how I'd describe the way lions seem in the wild.

Watching employees get transformed into founders makes it clear that the difference between the two is due mostly to environment—and in particular that the environment in big companies is toxic to programmers. In the first couple weeks of working on their own startup they seem to come to life, because finally they're working the way people are meant to.

Thanks to Trevor Blackwell, Ross Boucher, Aaron Iba, Abby Kirigin, Ivan Kirigin, Jessica Livingston, and Robert Morris for reading drafts of this.

⁴The founders we fund used to be younger (initially we encouraged undergrads to apply), and the first couple times I saw this I used to wonder if they were actually getting physically taller.

Chapter 22

How to Disagree

March 2008

The web is turning writing into a conversation. Twenty years ago, writers wrote and readers read. The web lets readers respond, and increasingly they do—in comment threads, on forums, and in their own blog posts.

Many who respond to something disagree with it. That's to be expected. Agreeing tends to motivate people less than disagreeing. And when you agree there's less to say. You could expand on something the author said, but he has probably already explored the most interesting implications. When you disagree you're entering territory he may not have explored.

The result is there's a lot more disagreeing going on, especially measured by the word. That doesn't mean people are getting angrier. The structural change in the way we communicate is enough to account for it. But though it's not anger that's driving the increase in disagreement, there's a danger that the increase in disagreement will make people angrier. Particularly online, where it's easy to say things you'd never say face to face.

If we're all going to be disagreeing more, we should be careful to do it well. What does it mean to disagree well? Most readers can tell the difference between mere name-calling and a carefully reasoned refutation, but I think it would help to put names on the intermediate stages. So here's an attempt at a disagreement hierarchy:

DH0. Name-calling.

This is the lowest form of disagreement, and probably also the most common. We've all seen comments like this:

u r a fag!!!!!!!!!!!!

But it's important to realize that more articulate name-calling has just as little weight. A comment like

The author is a self-important dilettante.

is really nothing more than a pretentious version of "u r a fag."

DH1. Ad Hominem.

An ad hominem attack is not quite as weak as mere name-calling. It might actually carry some weight. For example, if a senator wrote an article saying senators' salaries should be increased, one could respond:

Of course he would say that. He's a senator.

This wouldn't refute the author's argument, but it may at least be relevant to the case. It's still a very weak form of disagreement, though. If there's something wrong with the senator's argument, you should say what it is; and if there isn't, what difference does it make that he's a senator?

Saying that an author lacks the authority to write about a topic is a variant of ad hominem—and a particularly useless sort, because good ideas often come from outsiders. The question is whether the author is correct or not. If his lack of authority caused him to make mistakes, point those out. And if it didn't, it's not a problem.

DH2. Responding to Tone.

The next level up we start to see responses to the writing, rather than the writer. The lowest form of these is to disagree with the author's tone. E.g.

I can't believe the author dismisses intelligent design in such a cavalier fashion.

Though better than attacking the author, this is still a weak form of disagreement. It matters much more whether the author is wrong or right than what his tone is. Especially since tone is so hard to judge. Someone who has a chip on their shoulder about some topic might be offended by a tone that to other readers seemed neutral.

So if the worst thing you can say about something is to criticize its tone, you're not saying much. Is the author flippant, but correct? Better that than grave and wrong. And if the author is incorrect somewhere, say where.

DH3. Contradiction.

In this stage we finally get responses to what was said, rather than how or by whom. The lowest form of response to an argument is simply to state the opposing case, with little or no supporting evidence.

This is often combined with DH2 statements, as in:

I can't believe the author dismisses intelligent design in such a cavalier fashion. Intelligent design is a legitimate scientific theory.

Contradiction can sometimes have some weight. Sometimes merely seeing the opposing case stated explicitly is enough to see that it's right. But usually evidence will help.

DH4. Counterargument.

At level 4 we reach the first form of convincing disagreement: counterargument. Forms up to this point can usually be ignored as proving nothing. Counterargument might prove something. The problem is, it's hard to say exactly what.

Counterargument is contradiction plus reasoning and/or evidence. When aimed squarely at the original argument, it can be convincing.

But unfortunately it's common for counterarguments to be aimed at something slightly different. More often than not, two people arguing passionately about something are actually arguing about two different things. Sometimes they even agree with one another, but are so caught up in their squabble they don't realize it.

There could be a legitimate reason for arguing against something slightly different from what the original author said: when you feel they missed the heart of the matter. But when you do that, you should say explicitly you're doing it.

DH5. Refutation.

The most convincing form of disagreement is refutation. It's also the rarest, because it's the most work. Indeed, the disagreement hierarchy forms a kind of pyramid, in the sense that the higher you go the fewer instances you find.

To refute someone you probably have to quote them. You have to find a "smoking gun," a passage in whatever you disagree with that you feel is mistaken, and then explain why it's mistaken. If you can't find an actual quote to disagree with, you may be arguing with a straw man.

While refutation generally entails quoting, quoting doesn't necessarily imply refutation. Some writers quote parts of things they disagree with to give the appearance of legitimate refutation, then follow with a response as low as DH3 or even DH0.

DH6. Refuting the Central Point.

The force of a refutation depends on what you refute. The most powerful form of disagreement is to refute someone's central point.

Even as high as DH5 we still sometimes see deliberate dishonesty, as when someone picks out minor points of an argument and refutes those. Sometimes the spirit in which this is done makes it more of a sophisticated form of *ad hominem* than actual refutation. For example, correcting someone's grammar, or harping on minor mistakes in

names or numbers. Unless the opposing argument actually depends on such things, the only purpose of correcting them is to discredit one's opponent.

Truly refuting something requires one to refute its central point, or at least one of them. And that means one has to commit explicitly to what the central point is. So a truly effective refutation would look like:

The author's main point seems to be x. As he says:

But this is wrong for the following reasons...

The quotation you point out as mistaken need not be the actual statement of the author's main point. It's enough to refute something it depends upon.

What It Means

Now we have a way of classifying forms of disagreement. What good is it? One thing the disagreement hierarchy *doesn't* give us is a way of picking a winner. DH levels merely describe the form of a statement, not whether it's correct. A DH6 response could still be completely mistaken.

But while DH levels don't set a lower bound on the convincingness of a reply, they do set an upper bound. A DH6 response might be unconvincing, but a DH2 or lower response is always unconvincing.

The most obvious advantage of classifying the forms of disagreement is that it will help people to evaluate what they read. In particular, it will help them to see through intellectually dishonest arguments. An eloquent speaker or writer can give the impression of vanquishing an opponent merely by using forceful words. In fact that is probably the defining quality of a demagogue. By giving names to the different

forms of disagreement, we give critical readers a pin for popping such balloons.

Such labels may help writers too. Most intellectual dishonesty is unintentional. Someone arguing against the tone of something he disagrees with may believe he's really saying something. Zooming out and seeing his current position on the disagreement hierarchy may inspire him to try moving up to counterargument or refutation.

But the greatest benefit of disagreeing well is not just that it will make conversations better, but that it will make the people who have them happier. If you study conversations, you find there is a lot more meanness down in DH1 than up in DH6. You don't have to be mean when you have a real point to make. In fact, you don't want to. If you have something real to say, being mean just gets in the way.

If moving up the disagreement hierarchy makes people less mean, that will make most of them happier. Most people don't really enjoy being mean; they do it because they can't help it.

Thanks to Trevor Blackwell and Jessica Livingston for reading drafts of this.

Related:

Chapter 23

Some Heroes

April 2008

There are some topics I save up because they'll be so much fun to write about. This is one of them: a list of my heroes.

I'm not claiming this is a list of the n most admirable people. Who could make such a list, even if they wanted to?

Einstein isn't on the list, for example, even though he probably deserves to be on any shortlist of admirable people. I once asked a physicist friend if Einstein was really as smart as his fame implies, and she said that yes, he was. So why isn't he on the list? Because I had to ask. This is a list of people who've influenced me, not people who would have if I understood their work.

My test was to think of someone and ask "is this person my hero?" It often returned surprising answers. For example, it returned false for Montaigne, who was arguably the inventor of the essay. Why? When I thought about what it meant to call someone a hero, it meant I'd decide what to do by asking what they'd do in the same situation. That's a stricter standard than admiration.

After I made the list, I looked to see if there was a pattern, and there was, a very clear one. Everyone on the list had two qualities: they cared almost excessively about their work, and they were absolutely honest. By honest I don't mean trustworthy so much as that they never pander: they never say or do something because that's what the audience wants. They are all fundamentally subversive for this reason, though they conceal it to varying degrees.

Jack Lambert

I grew up in Pittsburgh in the 1970s. Unless you were there it's hard to imagine how that town felt about the Steelers. Locally, all the news was bad. The steel industry was dying. But the Steelers were the best team in football — and moreover, in a way that seemed to reflect the personality of the city. They didn't do anything fancy. They just got the job done.

Other players were more famous: Terry Bradshaw, Franco Harris, Lynn Swann. But they played offense, and you always get more attention for that. It seemed to me as a twelve year old football expert that the best of them all was Jack Lambert. And what made him so good was that he was utterly relentless. He didn't just care about playing well; he cared almost too much. He seemed to regard it as a personal insult when someone from the other team had possession of the ball on his side of the line of scrimmage.

The suburbs of Pittsburgh in the 1970s were a pretty dull place. School was boring. All the adults around were bored with their jobs working for big companies. Everything that came to us through the mass media was (a) blandly uniform and (b) produced elsewhere. Jack Lambert was the exception. He was like nothing else I'd seen.

Kenneth Clark

Kenneth Clark is the best nonfiction writer I know of, on any subject. Most people who write about art history don't really like art; you can tell from a thousand little signs. But Clark did, and not just intellectually, but the way one anticipates a delicious dinner.

What really makes him stand out, though, is the quality of his ideas. His style is deceptively casual, but there is more in his books than in a library of art monographs. Reading *The Nude* is like a ride in a Ferrari. Just as you're getting settled, you're slammed back in your seat by the acceleration. Before you can adjust, you're thrown sideways as the car screeches into the first turn. His brain throws off ideas almost too fast to grasp them. Finally at the end of the chapter you come to a halt,

with your eyes wide and a big smile on your face.

Kenneth Clark was a star in his day, thanks to the documentary series *Civilisation*. And if you read only one book about art history, *Civilisation* is the one I'd recommend. It's much better than the drab Sears Catalogs of art that undergraduates are forced to buy for Art History 101.

Larry Mihalko

A lot of people have a great teacher at some point in their childhood. Larry Mihalko was mine. When I look back it's like there's a line drawn between third and fourth grade. After Mr. Mihalko, everything was different.

Why? First of all, he was intellectually curious. I had a few other teachers who were smart, but I wouldn't describe them as intellectually curious. In retrospect, he was out of place as an elementary school teacher, and I think he knew it. That must have been hard for him, but it was wonderful for us, his students. His class was a constant adventure. I used to like going to school every day.

The other thing that made him different was that he liked us. Kids are good at telling that. The other teachers were at best benevolently indifferent. But Mr. Mihalko seemed like he actually wanted to be our friend. On the last day of fourth grade, he got out one of the heavy school record players and played James Taylor's "You've Got a Friend" to us. Just call out my name, and you know wherever I am, I'll come running. He died at 59 of lung cancer. I've never cried like I cried at his funeral.

Leonardo

One of the things I've learned about making things that I didn't realize when I was a kid is that much of the best stuff isn't made for audiences, but for oneself. You see paintings and drawings in museums and imagine they were made for you to look at. Actually a lot of the best ones were made as a way of exploring the world, not as a way

to please other people. The best of these explorations are sometimes more pleasing than stuff made explicitly to please.

Leonardo did a lot of things. One of his most admirable qualities was that he did so many different things that were admirable. What people know of him now is his paintings and his more flamboyant inventions, like flying machines. That makes him seem like some kind of dreamer who sketched artists' conceptions of rocket ships on the side. In fact he made a large number of far more practical technical discoveries. He was as good an engineer as a painter.

His most impressive work, to me, is his drawings. They're clearly made more as a way of studying the world than producing something beautiful. And yet they can hold their own with any work of art ever made. No one else, before or since, was that good when no one was looking.

Robert Morris

Robert Morris has a very unusual quality: he's never wrong. It might seem this would require you to be omniscient, but actually it's surprisingly easy. Don't say anything unless you're fairly sure of it. If you're not omniscient, you just don't end up saying much.

More precisely, the trick is to pay careful attention to how you qualify what you say. By using this trick, Robert has, as far as I know, managed to be mistaken only once, and that was when he was an undergrad. When the Mac came out, he said that little desktop computers would never be suitable for real hacking.

It's wrong to call it a trick in his case, though. If it were a conscious trick, he would have slipped in a moment of excitement. With Robert this quality is wired-in. He has an almost superhuman integrity. He's not just generally correct, but also correct about how correct he is.

You'd think it would be such a great thing never to be wrong that everyone would do this. It doesn't seem like that much extra work to pay as much attention to the error on an idea as to the idea itself.

And yet practically no one does. I know how hard it is, because since meeting Robert I've tried to do in software what he seems to do in hardware.

P. G. Wodehouse

People are finally starting to admit that Wodehouse was a great writer. If you want to be thought a great novelist in your own time, you have to sound intellectual. If what you write is popular, or entertaining, or funny, you're ipso facto suspect. That makes Wodehouse doubly impressive, because it meant that to write as he wanted to, he had to commit to being despised in his own lifetime.

Evelyn Waugh called him a great writer, but to most people at the time that would have read as a chivalrous or deliberately perverse gesture. At the time any random autobiographical novel by a recent college grad could count on more respectful treatment from the literary establishment.

Wodehouse may have begun with simple atoms, but the way he composed them into molecules was near faultless. His rhythm in particular. It makes me self-conscious to write about it. I can think of only two other writers who came near him for style: Evelyn Waugh and Nancy Mitford. Those three used the English language like they owned it.

But Wodehouse has something neither of them did. He's at ease. Evelyn Waugh and Nancy Mitford cared what other people thought of them: he wanted to seem aristocratic; she was afraid she wasn't smart enough. But Wodehouse didn't give a damn what anyone thought of him. He wrote exactly what he wanted.

Alexander Calder

Calder's on this list because he makes me happy. Can his work stand up to Leonardo's? Probably not. There might not be anything from the 20th Century that can. But what was good about Modernism, Calder had, and had in a way that he made seem effortless.

What was good about Modernism was its freshness. Art became stuffy in the nineteenth century. The paintings that were popular at the time were mostly the art equivalent of McMansions—big, pretentious, and fake. Modernism meant starting over, making things with the same earnest motives that children might. The artists who benefited most from this were the ones who had preserved a child's confidence, like Klee and Calder.

Klee was impressive because he could work in so many different styles. But between the two I like Calder better, because his work seemed happier. Ultimately the point of art is to engage the viewer. It's hard to predict what will; often something that seems interesting at first will bore you after a month. Calder's sculptures never get boring. They just sit there quietly radiating optimism, like a battery that never runs out. As far as I can tell from books and photographs, the happiness of Calder's work is his own happiness showing through.

Jane Austen

Everyone admires Jane Austen. Add my name to the list. To me she seems the best novelist of all time.

I'm interested in how things work. When I read most novels, I pay as much attention to the author's choices as to the story. But in her novels I can't see the gears at work. Though I'd really like to know how she does what she does, I can't figure it out, because she's so good that her stories don't seem made up. I feel like I'm reading a description of something that actually happened.

I used to read a lot of novels when I was younger. I can't read most anymore, because they don't have enough information in them. Novels seem so impoverished compared to history and biography. But reading Austen is like reading nonfiction. She writes so well you don't even notice her.

John McCarthy

John McCarthy invented Lisp, the field of (or at least the term) arti-

ficial intelligence, and was an early member of both of the top two computer science departments, MIT and Stanford. No one would dispute that he's one of the greats, but he's an especial hero to me because of Lisp.

It's hard for us now to understand what a conceptual leap that was at the time. Paradoxically, one of the reasons his achievement is hard to appreciate is that it was so successful. Practically every programming language invented in the last 20 years includes ideas from Lisp, and each year the median language gets more Lisplike.

In 1958 these ideas were anything but obvious. In 1958 there seem to have been two ways of thinking about programming. Some people thought of it as math, and proved things about Turing Machines. Others thought of it as a way to get things done, and designed languages all too influenced by the technology of the day. McCarthy alone bridged the gap. He designed a language that was math. But designed is not really the word; discovered is more like it.

The Spitfire

As I was making this list I found myself thinking of people like Douglas Bader and R.J. Mitchell and Jeffrey Quill and I realized that though all of them had done many things in their lives, there was one factor above all that connected them: the Spitfire.

This is supposed to be a list of heroes. How can a machine be on it? Because that machine was not just a machine. It was a lens of heroes. Extraordinary devotion went into it, and extraordinary courage came out.

It's a cliché to call World War II a contest between good and evil, but between fighter designs, it really was. The Spitfire's original nemesis, the ME 109, was a brutally practical plane. It was a killing machine. The Spitfire was optimism embodied. And not just in its beautiful lines: it was at the edge of what could be manufactured. But taking the high road worked. In the air, beauty had the edge, just.

Steve Jobs

People alive when Kennedy was killed usually remember exactly where they were when they heard about it. I remember exactly where I was when a friend asked if I'd heard Steve Jobs had cancer. It was like the floor dropped out. A few seconds later she told me that it was a rare operable type, and that he'd be ok. But those seconds seemed long.

I wasn't sure whether to include Jobs on this list. A lot of people at Apple seem to be afraid of him, which is a bad sign. But he compels admiration.

There's no name for what Steve Jobs is, because there hasn't been anyone quite like him before. He doesn't design Apple's products himself. Historically the closest analogy to what he does are the great Renaissance patrons of the arts. As the CEO of a company, that makes him unique.

Most CEOs delegate taste to a subordinate. The design paradox means they're choosing more or less at random. But Steve Jobs actually has taste himself — such good taste that he's shown the world how much more important taste is than they realized.

Isaac Newton

Newton has a strange role in my pantheon of heroes: he's the one I reproach myself with. He worked on big things, at least for part of his life. It's so easy to get distracted working on small stuff. The questions you're answering are pleasantly familiar. You get immediate rewards — in fact, you get bigger rewards in your time if you work on matters of passing importance. But I'm uncomfortably aware that this is the route to well-deserved obscurity.

To do really great things, you have to seek out questions people didn't even realize were questions. There have probably been other people who did this as well as Newton, for their time, but Newton is my model of this kind of thought. I can just begin to understand what it must

have felt like for him.

You only get one life. Why not do something huge? The phrase “paradigm shift” is overused now, but Kuhn was onto something. And you know more are out there, separated from us by what will later seem a surprisingly thin wall of laziness and stupidity. If we work like Newton.

Thanks to Trevor Blackwell, Jessica Livingston, and Jackie McDonough for reading drafts of this.

Chapter 24

Why There Aren't More Googles

April 2008

Umair Haque wrote recently that the reason there aren't more Googles is that most startups get bought before they can change the world.

Google, despite serious interest from Microsoft and Yahoo—what must have seemed like lucrative interest at the time—didn't sell out. Google might simply have been nothing but Yahoo's or MSN's search box.

Why isn't it? Because Google had a deeply felt sense of purpose: a conviction to change the world for the better.

This has a nice sound to it, but it isn't true. Google's founders were willing to sell early on. They just wanted more than acquirers were willing to pay.

It was the same with Facebook. They would have sold, but Yahoo blew it by offering too little.

Tip for acquirers: when a startup turns you down, consider raising your offer, because there's a good chance the outrageous price they want will later seem a bargain.¹

From the evidence I've seen so far, startups that turn down acquisition

¹Another tip: If you want to get all that value, don't destroy the startup after you buy it. Give the founders enough autonomy that they can grow the acquisition into what it would have become.

offers usually end up doing better. Not always, but usually there's a bigger offer coming, or perhaps even an IPO.

Of course, the reason startups do better when they turn down acquisition offers is not necessarily that all such offers undervalue startups. More likely the reason is that the kind of founders who have the balls to turn down a big offer also tend to be very successful. That spirit is exactly what you want in a startup.

While I'm sure Larry and Sergey do want to change the world, at least now, the reason Google survived to become a big, independent company is the same reason Facebook has so far remained independent: acquirers underestimated them.

Corporate M&A is a strange business in that respect. They consistently lose the best deals, because turning down reasonable offers is the most reliable test you could invent for whether a startup will make it big.

VCs

So what's the real reason there aren't more Googles? Curiously enough, it's the same reason Google and Facebook have remained independent: money guys undervalue the most innovative startups.

The reason there aren't more Googles is not that investors encourage innovative startups to sell out, but that they won't even fund them. I've learned a lot about VCs during the 3 years we've been doing Y Combinator, because we often have to work quite closely with them. The most surprising thing I've learned is how conservative they are. VC firms present an image of boldly encouraging innovation. Only a handful actually do, and even they are more conservative in reality than you'd guess from reading their sites.

I used to think of VCs as piratical: bold but unscrupulous. On closer acquaintance they turn out to be more like bureaucrats. They're more upstanding than I used to think (the good ones, at least), but less bold. Maybe the VC industry has changed. Maybe they used to be bolder.

But I suspect it's the startup world that has changed, not them. The low cost of starting a startup means the average good bet is a riskier one, but most existing VC firms still operate as if they were investing in hardware startups in 1985.

Howard Aiken said "Don't worry about people stealing your ideas. If your ideas are any good, you'll have to ram them down people's throats." I have a similar feeling when I'm trying to convince VCs to invest in startups Y Combinator has funded. They're terrified of really novel ideas, unless the founders are good enough salesmen to compensate.

But it's the bold ideas that generate the biggest returns. Any really good new idea will seem bad to most people; otherwise someone would already be doing it. And yet most VCs are driven by consensus, not just within their firms, but within the VC community. The biggest factor determining how a VC will feel about your startup is how other VCs feel about it. I doubt they realize it, but this algorithm guarantees they'll miss all the very best ideas. The more people who have to like a new idea, the more outliers you lose.

Whoever the next Google is, they're probably being told right now by VCs to come back when they have more "traction."

Why are VCs so conservative? It's probably a combination of factors. The large size of their investments makes them conservative. Plus they're investing other people's money, which makes them worry they'll get in trouble if they do something risky and it fails. Plus most of them are money guys rather than technical guys, so they don't understand what the startups they're investing in do.

What's Next

The exciting thing about market economies is that stupidity equals opportunity. And so it is in this case. There is a huge, unexploited opportunity in startup investing. Y Combinator funds startups at the very beginning. VCs will fund them once they're already starting to succeed. But between the two there is a substantial gap.

There are companies that will give \$20k to a startup that has nothing more than the founders, and there are companies that will give \$2 million to a startup that's already taking off, but there aren't enough investors who will give \$200k to a startup that seems very promising but still has some things to figure out. This territory is occupied mostly by individual angel investors—people like Andy Bechtolsheim, who gave Google \$100k when they seemed promising but still had some things to figure out. I like angels, but there just aren't enough of them, and investing is for most of them a part time job.

And yet as it gets cheaper to start startups, this sparsely occupied territory is becoming more and more valuable. Nowadays a lot of startups don't want to raise multi-million dollar series A rounds. They don't need that much money, and they don't want the hassles that come with it. The median startup coming out of Y Combinator wants to raise \$250-500k. When they go to VC firms they have to ask for more because they know VCs aren't interested in such small deals.

VCs are money managers. They're looking for ways to put large sums to work. But the startup world is evolving away from their current model.

Startups have gotten cheaper. That means they want less money, but also that there are more of them. So you can still get large returns on large amounts of money; you just have to spread it more broadly.

I've tried to explain this to VC firms. Instead of making one \$2 million investment, make five \$400k investments. Would that mean sitting on too many boards? Don't sit on their boards. Would that mean too much due diligence? Do less. If you're investing at a tenth the valuation, you only have to be a tenth as sure.

It seems obvious. But I've proposed to several VC firms that they set aside some money and designate one partner to make more, smaller bets, and they react as if I'd proposed the partners all get nose rings. It's remarkable how wedded they are to their standard m.o.

But there is a big opportunity here, and one way or the other it's going

to get filled. Either VCs will evolve down into this gap or, more likely, new investors will appear to fill it. That will be a good thing when it happens, because these new investors will be compelled by the structure of the investments they make to be ten times bolder than present day VCs. And that will get us a lot more Googles. At least, as long as acquirers remain stupid.

Thanks to Sam Altman, Paul Buchheit, David Hornik, Jessica Livingston, Robert Morris, and Fred Wilson for reading drafts of this.

Chapter 25

Be Good



April 2008

(This essay is derived from a talk at the 2008 Startup School.)

About a month after we started Y Combinator we came up with the phrase that became our motto: Make something people want. We've learned a lot since then, but if I were choosing now that's still the one I'd pick.

Another thing we tell founders is not to worry too much about the business model, at least at first. Not because making money is unimportant, but because it's so much easier than building something great.

A couple weeks ago I realized that if you put those two ideas together, you get something surprising. Make something people want. Don't worry too much about making money. What you've got is a description of a charity.

When you get an unexpected result like this, it could either be a bug or a new discovery. Either businesses aren't supposed to be like charities, and we've proven by *reductio ad absurdum* that one or both of the principles we began with is false. Or we have a new idea.

I suspect it's the latter, because as soon as this thought occurred to me, a whole bunch of other things fell into place.

Examples

For example, Craigslist. It's not a charity, but they run it like one. And they're astoundingly successful. When you scan down the list of most popular web sites, the number of employees at Craigslist looks like a misprint. Their revenues aren't as high as they could be, but most startups would be happy to trade places with them.

In Patrick O'Brian's novels, his captains always try to get upwind of their opponents. If you're upwind, you decide when and if to engage the other ship. Craigslist is effectively upwind of enormous revenues. They'd face some challenges if they wanted to make more, but not the sort you face when you're tacking upwind, trying to force a crappy product on ambivalent users by spending ten times as much on sales as on development.¹

I'm not saying startups should aim to end up like Craigslist. They're a product of unusual circumstances. But they're a good model for the early phases.

Google looked a lot like a charity in the beginning. They didn't have ads for over a year. At year 1, Google was indistinguishable from a nonprofit. If a nonprofit or government organization had started a project to index the web, Google at year 1 is the limit of what they'd have produced.

¹Fifty years ago it would have seemed shocking for a public company not to pay dividends. Now many tech companies don't. The markets seem to have figured out how to value potential dividends. Maybe that isn't the last step in this evolution. Maybe markets will eventually get comfortable with potential earnings. (VCs already are, and at least some of them consistently make money.)

I realize this sounds like the stuff one used to hear about the "new economy" during the Bubble. Believe me, I was not drinking that kool-aid at the time. But I'm convinced there were some good ideas buried in Bubble thinking. For example, it's ok to focus on growth instead of profits—but only if the growth is genuine. You can't be buying users; that's a pyramid scheme. But a company with rapid, genuine growth is valuable, and eventually markets learn how to value valuable things.

Back when I was working on spam filters I thought it would be a good idea to have a web-based email service with good spam filtering. I wasn't thinking of it as a company. I just wanted to keep people from getting spammed. But as I thought more about this project, I realized it would probably have to be a company. It would cost something to run, and it would be a pain to fund with grants and donations.

That was a surprising realization. Companies often claim to be benevolent, but it was surprising to realize there were purely benevolent projects that had to be embodied as companies to work.

I didn't want to start another company, so I didn't do it. But if someone had, they'd probably be quite rich now. There was a window of about two years when spam was increasing rapidly but all the big email services had terrible filters. If someone had launched a new, spam-free mail service, users would have flocked to it.

Notice the pattern here? From either direction we get to the same spot. If you start from successful startups, you find they often behaved like nonprofits. And if you start from ideas for nonprofits, you find they'd often make good startups.

Power

How wide is this territory? Would all good nonprofits be good companies? Possibly not. What makes Google so valuable is that their users have money. If you make people with money love you, you can probably get some of it. But could you also base a successful startup on behaving like a nonprofit to people who don't have money? Could you, for example, grow a successful startup out of curing an unfashionable but deadly disease like malaria?

I'm not sure, but I suspect that if you pushed this idea, you'd be surprised how far it would go. For example, people who apply to Y Combinator don't generally have much money, and yet we can profit by helping them, because with our help they could make money. Maybe the situation is similar with malaria. Maybe an organization that helped lift its weight off a country could benefit from the resulting

growth.

I'm not proposing this is a serious idea. I don't know anything about malaria. But I've been kicking ideas around long enough to know when I come across a powerful one.

One way to guess how far an idea extends is to ask yourself at what point you'd bet against it. The thought of betting against benevolence is alarming in the same way as saying that something is technically impossible. You're just asking to be made a fool of, because these are such powerful forces.²

For example, initially I thought maybe this principle only applied to Internet startups. Obviously it worked for Google, but what about Microsoft? Surely Microsoft isn't benevolent? But when I think back to the beginning, they were. Compared to IBM they were like Robin Hood. When IBM introduced the PC, they thought they were going to make money selling hardware at high prices. But by gaining control of the PC standard, Microsoft opened up the market to any manufacturer. Hardware prices plummeted, and lots of people got to have computers who couldn't otherwise have afforded them. It's the sort of thing you'd expect Google to do.

Microsoft isn't so benevolent now. Now when one thinks of what Microsoft does to users, all the verbs that come to mind begin with F.³ And yet it doesn't seem to pay. Their stock price has been flat for

²The idea of starting a company with benevolent aims is currently undervalued, because the kind of people who currently make that their explicit goal don't usually do a very good job.

It's one of the standard career paths of trustafarians to start some vaguely benevolent business. The problem with most of them is that they either have a bogus political agenda or are feebly executed. The trustafarians' ancestors didn't get rich by preserving their traditional culture; maybe people in Bolivia don't want to either. And starting an organic farm, though it's at least straightforwardly benevolent, doesn't help people on the scale that Google does.

Most explicitly benevolent projects don't hold themselves sufficiently accountable. They act as if having good intentions were enough to guarantee good effects.

³Users dislike their new operating system so much that they're starting petitions

years. Back when they were Robin Hood, their stock price rose like Google's. Could there be a connection?

You can see how there would be. When you're small, you can't bully customers, so you have to charm them. Whereas when you're big you can maltreat them at will, and you tend to, because it's easier than satisfying them. You grow big by being nice, but you can stay big by being mean.

You get away with it till the underlying conditions change, and then all your victims escape. So "Don't be evil" may be the most valuable thing Paul Buchheit made for Google, because it may turn out to be an elixir of corporate youth. I'm sure they find it constraining, but think how valuable it will be if it saves them from lapsing into the fatal laziness that afflicted Microsoft and IBM.

The curious thing is, this elixir is freely available to any other company. Anyone can adopt "Don't be evil." The catch is that people will hold you to it. So I don't think you're going to see record labels or tobacco companies using this discovery.

Morale

There's a lot of external evidence that benevolence works. But how does it work? One advantage of investing in a large number of startups is that you get a lot of data about how they work. From what we've seen, being good seems to help startups in three ways: it improves their morale, it makes other people want to help them, and above all, it helps them be decisive.

Morale is tremendously important to a startup—so important that morale alone is almost enough to determine success. Startups are often described as emotional roller-coasters. One minute you're going to take over the world, and the next you're doomed. The problem with feeling you're doomed is not just that it makes you unhappy, but

to save the old one. And the old one was nothing special. The hackers within Microsoft must know in their hearts that if the company really cared about users they'd just advise them to switch to OSX.

that it makes you *stop working*. So the downhill of the roller-coaster are more of a self fulfilling prophecy than the uphill. If feeling you're going to succeed makes you work harder, that probably improves your chances of succeeding, but if feeling you're going to fail makes you stop working, that practically guarantees you'll fail.

Here's where benevolence comes in. If you feel you're really helping people, you'll keep working even when it seems like your startup is doomed. Most of us have some amount of natural benevolence. The mere fact that someone needs you makes you want to help them. So if you start the kind of startup where users come back each day, you've basically built yourself a giant tamagotchi. You've made something you need to take care of.

Blogger is a famous example of a startup that went through really low lows and survived. At one point they ran out of money and everyone left. Evan Williams came in to work the next day, and there was no one but him. What kept him going? Partly that users needed him. He was hosting thousands of people's blogs. He couldn't just let the site die.

There are many advantages of launching quickly, but the most important may be that once you have users, the tamagotchi effect kicks in. Once you have users to take care of, you're forced to figure out what will make them happy, and that's actually very valuable information.

The added confidence that comes from trying to help people can also help you with investors. One of the founders of Chatterous told me recently that he and his cofounder had decided that this service was something the world needed, so they were going to keep working on it no matter what, even if they had to move back to Canada and live in their parents' basements.

Once they realized this, they stopped caring so much what investors thought about them. They still met with them, but they weren't going to die if they didn't get their money. And you know what? The investors got a lot more interested. They could sense that the Chat-

terouses were going to do this startup with or without them.

If you're really committed and your startup is cheap to run, you become very hard to kill. And practically all startups, even the most successful, come close to death at some point. So if doing good for people gives you a sense of mission that makes you harder to kill, that alone more than compensates for whatever you lose by not choosing a more selfish project.

Help

Another advantage of being good is that it makes other people want to help you. This too seems to be an inborn trait in humans.

One of the startups we've funded, Octopart, is currently locked in a classic battle of good versus evil. They're a search site for industrial components. A lot of people need to search for components, and before Octopart there was no good way to do it. That, it turned out, was no coincidence.

Octopart built the right way to search for components. Users like it and they've been growing rapidly. And yet for most of Octopart's life, the biggest distributor, Digi-Key, has been trying to force them take their prices off the site. Octopart is sending them customers for free, and yet Digi-Key is trying to make that traffic stop. Why? Because their current business model depends on overcharging people who have incomplete information about prices. They don't want search to work.

The Octoparts are the nicest guys in the world. They dropped out of the PhD program in physics at Berkeley to do this. They just wanted to fix a problem they encountered in their research. Imagine how much time you could save the world's engineers if they could do searches online. So when I hear that a big, evil company is trying to stop them in order to keep search broken, it makes me really want to help them. It makes me spend more time on the Octoparts than I do with most of the other startups we've funded. It just made me spend several minutes telling you how great they are. Why? Because they're good

guys and they're trying to help the world.

If you're benevolent, people will rally around you: investors, customers, other companies, and potential employees. In the long term the most important may be the potential employees. I think everyone knows now that good hackers are much better than mediocre ones. If you can attract the best hackers to work for you, as Google has, you have a big advantage. And the very best hackers tend to be idealistic. They're not desperate for a job. They can work wherever they want. So most want to work on things that will make the world better.

Compass

But the most important advantage of being good is that it acts as a compass. One of the hardest parts of doing a startup is that you have so many choices. There are just two or three of you, and a thousand things you could do. How do you decide?

Here's the answer: Do whatever's best for your users. You can hold onto this like a rope in a hurricane, and it will save you if anything can. Follow it and it will take you through everything you need to do.

It's even the answer to questions that seem unrelated, like how to convince investors to give you money. If you're a good salesman, you could try to just talk them into it. But the more reliable route is to convince them through your users: if you make something users love enough to tell their friends, you grow exponentially, and that will convince any investor.

Being good is a particularly useful strategy for making decisions in complex situations because it's stateless. It's like telling the truth. The trouble with lying is that you have to remember everything you've said in the past to make sure you don't contradict yourself. If you tell the truth you don't have to remember anything, and that's a really useful property in domains where things happen fast.

For example, Y Combinator has now invested in 80 startups, 57 of which are still alive. (The rest have died or merged or been acquired.)

When you're trying to advise 57 startups, it turns out you have to have a stateless algorithm. You can't have ulterior motives when you have 57 things going on at once, because you can't remember them. So our rule is just to do whatever's best for the founders. Not because we're particularly benevolent, but because it's the only algorithm that works on that scale.

When you write something telling people to be good, you seem to be claiming to be good yourself. So I want to say explicitly that I am not a particularly good person. When I was a kid I was firmly in the camp of bad. The way adults used the word good, it seemed to be synonymous with quiet, so I grew up very suspicious of it.

You know how there are some people whose names come up in conversation and everyone says "He's *such* a great guy?" People never say that about me. The best I get is "he means well." I am not claiming to be good. At best I speak good as a second language.

So I'm not suggesting you be good in the usual sanctimonious way. I'm suggesting it because it works. It will work not just as a statement of "values," but as a guide to strategy, and even a design spec for software. Don't just not be evil. Be good.

Thanks to Trevor Blackwell, Paul Buchheit, Jessica Livingston, and Robert Morris for reading drafts of this.

Chapter 26

Lies We Tell Kids

May 2008

Adults lie constantly to kids. I'm not saying we should stop, but I think we should at least examine which lies we tell and why.

There may also be a benefit to us. We were all lied to as kids, and some of the lies we were told still affect us. So by studying the ways adults lie to kids, we may be able to clear our heads of lies we were told.

I'm using the word "lie" in a very general sense: not just overt falsehoods, but also all the more subtle ways we mislead kids. Though "lie" has negative connotations, I don't mean to suggest we should never do this—just that we should pay attention when we do.¹

One of the most remarkable things about the way we lie to kids is how broad the conspiracy is. All adults know what their culture lies

¹One reason I stuck with such a brutally simple word is that the lies we tell kids are probably not quite as harmless as we think. If you look at what adults told children in the past, it's shocking how much they lied to them. Like us, they did it with the best intentions. So if we think we're as open as one could reasonably be with children, we're probably fooling ourselves. Odds are people in 100 years will be as shocked at some of the lies we tell as we are at some of the lies people told 100 years ago.

I can't predict which these will be, and I don't want to write an essay that will seem dumb in 100 years. So instead of using special euphemisms for lies that seem excusable according to present fashions, I'm just going to call all our lies lies.

(I have omitted one type: lies told to play games with kids' credulity. These range from "make-believe," which is not really a lie because it's told with a wink, to the frightening lies told by older siblings. There's not much to say about these: I wouldn't want the first type to go away, and wouldn't expect the second type to.)

to kids about: they're the questions you answer "Ask your parents." If a kid asked who won the World Series in 1982 or what the atomic weight of carbon was, you could just tell him. But if a kid asks you "Is there a God?" or "What's a prostitute?" you'll probably say "Ask your parents."

Since we all agree, kids see few cracks in the view of the world presented to them. The biggest disagreements are between parents and schools, but even those are small. Schools are careful what they say about controversial topics, and if they do contradict what parents want their kids to believe, parents either pressure the school into keeping quiet or move their kids to a new school.

The conspiracy is so thorough that most kids who discover it do so only by discovering internal contradictions in what they're told. It can be traumatic for the ones who wake up during the operation. Here's what happened to Einstein:

Through the reading of popular scientific books I soon reached the conviction that much in the stories of the Bible could not be true. The consequence was a positively fanatic freethinking coupled with the impression that youth is intentionally being deceived by the state through lies: it was a crushing impression.²

I remember that feeling. By 15 I was convinced the world was corrupt from end to end. That's why movies like *The Matrix* have such resonance. Every kid grows up in a fake world. In a way it would be easier if the forces behind it were as clearly differentiated as a bunch of evil machines, and one could make a clean break just by taking a pill.

Protection

If you ask adults why they lie to kids, the most common reason they give is to protect them. And kids do need protecting. The environ-

²Calaprice, Alice (ed.), *The Quotable Einstein*, Princeton University Press, 1996.

ment you want to create for a newborn child will be quite unlike the streets of a big city.

That seems so obvious it seems wrong to call it a lie. It's certainly not a bad lie to tell, to give a baby the impression the world is quiet and warm and safe. But this harmless type of lie can turn sour if left unexamined.

Imagine if you tried to keep someone in as protected an environment as a newborn till age 18. To mislead someone so grossly about the world would seem not protection but abuse. That's an extreme example, of course; when parents do that sort of thing it becomes national news. But you see the same problem on a smaller scale in the malaise teenagers feel in suburbia.

The main purpose of suburbia is to provide a protected environment for children to grow up in. And it seems great for 10 year olds. I liked living in suburbia when I was 10. I didn't notice how sterile it was. My whole world was no bigger than a few friends' houses I bicycled to and some woods I ran around in. On a log scale I was midway between crib and globe. A suburban street was just the right size. But as I grew older, suburbia started to feel suffocatingly fake.

Life can be pretty good at 10 or 20, but it's often frustrating at 15. This is too big a problem to solve here, but certainly one reason life sucks at 15 is that kids are trapped in a world designed for 10 year olds.

What do parents hope to protect their children from by raising them in suburbia? A friend who moved out of Manhattan said merely that her 3 year old daughter "saw too much." Off the top of my head, that might include: people who are high or drunk, poverty, madness, gruesome medical conditions, sexual behavior of various degrees of oddness, and violent anger.

I think it's the anger that would worry me most if I had a 3 year old. I was 29 when I moved to New York and I was surprised even then. I wouldn't want a 3 year old to see some of the disputes I saw. It would

be too frightening. A lot of the things adults conceal from smaller children, they conceal because they'd be frightening, not because they want to conceal the existence of such things. Misleading the child is just a byproduct.

This seems one of the most justifiable types of lying adults do to kids. But because the lies are indirect we don't keep a very strict accounting of them. Parents know they've concealed the facts about sex, and many at some point sit their kids down and explain more. But few tell their kids about the differences between the real world and the cocoon they grew up in. Combine this with the confidence parents try to instill in their kids, and every year you get a new crop of 18 year olds who think they know how to run the world.

Don't all 18 year olds think they know how to run the world? Actually this seems to be a recent innovation, no more than about 100 years old. In preindustrial times teenage kids were junior members of the adult world and comparatively well aware of their shortcomings. They could see they weren't as strong or skillful as the village smith. In past times people lied to kids about some things more than we do now, but the lies implicit in an artificial, protected environment are a recent invention. Like a lot of new inventions, the rich got this first. Children of kings and great magnates were the first to grow up out of touch with the world. Suburbia means half the population can live like kings in that respect.

Sex (and Drugs)

I'd have different worries about raising teenage kids in New York. I'd worry less about what they'd see, and more about what they'd do. I went to college with a lot of kids who grew up in Manhattan, and as a rule they seemed pretty jaded. They seemed to have lost their virginity at an average of about 14 and by college had tried more drugs than I'd even heard of.

The reasons parents don't want their teenage kids having sex are complex. There are some obvious dangers: pregnancy and sexually trans-

mitted diseases. But those aren't the only reasons parents don't want their kids having sex. The average parents of a 14 year old girl would hate the idea of her having sex even if there were zero risk of pregnancy or sexually transmitted diseases.

Kids can probably sense they aren't being told the whole story. After all, pregnancy and sexually transmitted diseases are just as much a problem for adults, and they have sex.

What really bothers parents about their teenage kids having sex? Their dislike of the idea is so visceral it's probably inborn. But if it's inborn it should be universal, and there are plenty of societies where parents don't mind if their teenage kids have sex—indeed, where it's normal for 14 year olds to become mothers. So what's going on? There does seem to be a universal taboo against sex with prepubescent children. One can imagine evolutionary reasons for that. And I think this is the main reason parents in industrialized societies dislike teenage kids having sex. They still think of them as children, even though biologically they're not, so the taboo against child sex still has force.

One thing adults conceal about sex they also conceal about drugs: that it can cause great pleasure. That's what makes sex and drugs so dangerous. The desire for them can cloud one's judgement—which is especially frightening when the judgement being clouded is the already wretched judgement of a teenage kid.

Here parents' desires conflict. Older societies told kids they had bad judgement, but modern parents want their children to be confident. This may well be a better plan than the old one of putting them in their place, but it has the side effect that after having implicitly lied to kids about how good their judgement is, we then have to lie again about all the things they might get into trouble with if they believed us.

If parents told their kids the truth about sex and drugs, it would be: the reason you should avoid these things is that you have lousy judgement. People with twice your experience still get burned by them. But this

may be one of those cases where the truth wouldn't be convincing, because one of the symptoms of bad judgement is believing you have good judgement. When you're too weak to lift something, you can tell, but when you're making a decision impetuously, you're all the more sure of it.

Innocence

Another reason parents don't want their kids having sex is that they want to keep them innocent. Adults have a certain model of how kids are supposed to behave, and it's different from what they expect of other adults.

One of the most obvious differences is the words kids are allowed to use. Most parents use words when talking to other adults that they wouldn't want their kids using. They try to hide even the existence of these words for as long as they can. And this is another of those conspiracies everyone participates in: everyone knows you're not supposed to swear in front of kids.

I've never heard more different explanations for anything parents tell kids than why they shouldn't swear. Every parent I know forbids their children to swear, and yet no two of them have the same justification. It's clear most start with not wanting kids to swear, then make up the reason afterward.

So my theory about what's going on is that the *function* of swearwords is to mark the speaker as an adult. There's no difference in the meaning of "shit" and "poopoo." So why should one be ok for kids to say and one forbidden? The only explanation is: by definition.³

Why does it bother adults so much when kids do things reserved for adults? The idea of a foul-mouthed, cynical 10 year old leaning against a lamppost with a cigarette hanging out of the corner of his

³If you ask parents why kids shouldn't swear, the less educated ones usually reply with some question-begging answer like "it's inappropriate," while the more educated ones come up with elaborate rationalizations. In fact the less educated parents seem closer to the truth.

mouth is very disconcerting. But why?

One reason we want kids to be innocent is that we're programmed to like certain kinds of helplessness. I've several times heard mothers say they deliberately refrained from correcting their young children's mispronunciations because they were so cute. And if you think about it, cuteness is helplessness. Toys and cartoon characters meant to be cute always have clueless expressions and stubby, ineffectual limbs.

It's not surprising we'd have an inborn desire to love and protect helpless creatures, considering human offspring are so helpless for so long. Without the helplessness that makes kids cute, they'd be very annoying. They'd merely seem like incompetent adults. But there's more to it than that. The reason our hypothetical jaded 10 year old bothers me so much is not just that he'd be annoying, but that he'd have cut off his prospects for growth so early. To be jaded you have to think you know how the world works, and any theory a 10 year old had about that would probably be a pretty narrow one.

Innocence is also open-mindedness. We want kids to be innocent so they can continue to learn. Paradoxical as it sounds, there are some kinds of knowledge that get in the way of other kinds of knowledge. If you're going to learn that the world is a brutal place full of people trying to take advantage of one another, you're better off learning it last. Otherwise you won't bother learning much more.

Very smart adults often seem unusually innocent, and I don't think this is a coincidence. I think they've deliberately avoided learning about certain things. Certainly I do. I used to think I wanted to know everything. Now I know I don't.

Death

After sex, death is the topic adults lie most conspicuously about to kids. Sex I believe they conceal because of deep taboos. But why do we conceal death from kids? Probably because small children are particularly horrified by it. They want to feel safe, and death is the ultimate threat.

One of the most spectacular lies our parents told us was about the death of our first cat. Over the years, as we asked for more details, they were compelled to invent more, so the story grew quite elaborate. The cat had died at the vet's office. Of what? Of the anaesthesia itself. Why was the cat at the vet's office? To be fixed. And why had such a routine operation killed it? It wasn't the vet's fault; the cat had a congenitally weak heart; the anaesthesia was too much for it; but there was no way anyone could have known this in advance. It was not till we were in our twenties that the truth came out: my sister, then about three, had accidentally stepped on the cat and broken its back.

They didn't feel the need to tell us the cat was now happily in cat heaven. My parents never claimed that people or animals who died had "gone to a better place," or that we'd meet them again. It didn't seem to harm us.

My grandmother told us an edited version of the death of my grandfather. She said they'd been sitting reading one day, and when she said something to him, he didn't answer. He seemed to be asleep, but when she tried to rouse him, she couldn't. "He was gone." Having a heart attack sounded like falling asleep. Later I learned it hadn't been so neat, and the heart attack had taken most of a day to kill him.

Along with such outright lies, there must have been a lot of changing the subject when death came up. I can't remember that, of course, but I can infer it from the fact that I didn't really grasp I was going to die till I was about 19. How could I have missed something so obvious for so long? Now that I've seen parents managing the subject, I can see how: questions about death are gently but firmly turned aside.

On this topic, especially, they're met half-way by kids. Kids often want to be lied to. They want to believe they're living in a comfortable, safe world as much as their parents want them to believe it. ⁴

⁴As a friend with small children pointed out, it's easy for small children to consider themselves immortal, because time seems to pass so slowly for them. To a 3 year old, a day feels like a month might to an adult. So 80 years sounds to him like 2400 years would to us.

Identity

Some parents feel a strong adherence to an ethnic or religious group and want their kids to feel it too. This usually requires two different kinds of lying: the first is to tell the child that he or she is an X, and the second is whatever specific lies Xes differentiate themselves by believing.⁵

Telling a child they have a particular ethnic or religious identity is one of the stickiest things you can tell them. Almost anything else you tell a kid, they can change their mind about later when they start to think for themselves. But if you tell a kid they're a member of a certain group, that seems nearly impossible to shake.

This despite the fact that it can be one of the most premeditated lies parents tell. When parents are of different religions, they'll often agree between themselves that their children will be "raised as Xes." And it works. The kids obligingly grow up considering themselves as Xes, despite the fact that if their parents had chosen the other way, they'd have grown up considering themselves as Ys.

One reason this works so well is the second kind of lie involved. The truth is common property. You can't distinguish your group by doing things that are rational, and believing things that are true. If you want to set yourself apart from other people, you have to do things that are arbitrary, and believe things that are false. And after having spent their whole lives doing things that are arbitrary and believing things that are false, and being regarded as odd by "outsiders" on that account, the cognitive dissonance pushing children to regard themselves as Xes

⁵I realize I'm going to get endless grief for classifying religion as a type of lie. Usually people skirt that issue with some equivocation implying that lies believed for a sufficiently long time by sufficiently large numbers of people are immune to the usual standards for truth. But because I can't predict which lies future generations will consider inexcusable, I can't safely omit any type we tell. Yes, it seems unlikely that religion will be out of fashion in 100 years, but no more unlikely than it would have seemed to someone in 1880 that schoolchildren in 1980 would be taught that masturbation was perfectly normal and not to feel guilty about it.

must be enormous. If they aren't an X, why are they attached to all these arbitrary beliefs and customs? If they aren't an X, why do all the non-Xes call them one?

This form of lie is not without its uses. You can use it to carry a payload of beneficial beliefs, and they will also become part of the child's identity. You can tell the child that in addition to never wearing the color yellow, believing the world was created by a giant rabbit, and always snapping their fingers before eating fish, Xes are also particularly honest and industrious. Then X children will grow up feeling it's part of their identity to be honest and industrious.

This probably accounts for a lot of the spread of modern religions, and explains why their doctrines are a combination of the useful and the bizarre. The bizarre half is what makes the religion stick, and the useful half is the payload.⁶

Authority

One of the least excusable reasons adults lie to kids is to maintain power over them. Sometimes these lies are truly sinister, like a child molester telling his victims they'll get in trouble if they tell anyone what happened to them. Others seem more innocent; it depends how badly adults lie to maintain their power, and what they use it for.

Most adults make some effort to conceal their flaws from children. Usually their motives are mixed. For example, a father who has an affair generally conceals it from his children. His motive is partly that it would worry them, partly that this would introduce the topic of sex, and partly (a larger part than he would admit) that he doesn't want to tarnish himself in their eyes.

⁶Unfortunately the payload can consist of bad customs as well as good ones. For example, there are certain qualities that some groups in America consider "acting white." In fact most of them could as accurately be called "acting Japanese." There's nothing specifically white about such customs. They're common to all cultures with long traditions of living in cities. So it is probably a losing bet for a group to consider behaving the opposite way as part of its identity.

If you want to learn what lies are told to kids, read almost any book written to teach them about “issues.”⁷ Peter Mayle wrote one called *Why Are We Getting a Divorce?* It begins with the three most important things to remember about divorce, one of which is:

You shouldn’t put the blame on one parent, because divorce is never only one person’s fault.⁸

Really? When a man runs off with his secretary, is it always partly his wife’s fault? But I can see why Mayle might have said this. Maybe it’s more important for kids to respect their parents than to know the truth about them.

But because adults conceal their flaws, and at the same time insist on high standards of behavior for kids, a lot of kids grow up feeling they fall hopelessly short. They walk around feeling horribly evil for having used a swearword, while in fact most of the adults around them are doing much worse things.

This happens in intellectual as well as moral questions. The more confident people are, the more willing they seem to be to answer a question “I don’t know.” Less confident people feel they have to have an answer or they’ll look bad. My parents were pretty good about admitting when they didn’t know things, but I must have been told a lot of lies of this type by teachers, because I rarely heard a teacher say “I don’t know” till I got to college. I remember because it was so surprising to hear someone say that in front of a class.

The first hint I had that teachers weren’t omniscient came in sixth grade, after my father contradicted something I’d learned in school. When I protested that the teacher had said the opposite, my father replied that the guy had no idea what he was talking about—that he was just an elementary school teacher, after all.

⁷In this context, “issues” basically means “things we’re going to lie to them about.” That’s why there’s a special name for these topics.

⁸Mayle, Peter, *Why Are We Getting a Divorce?*, Harmony, 1988.

Just a teacher? The phrase seemed almost grammatically ill-formed. Didn't teachers know everything about the subjects they taught? And if not, why were they the ones teaching us?

The sad fact is, US public school teachers don't generally understand the stuff they're teaching very well. There are some sterling exceptions, but as a rule people planning to go into teaching rank academically near the bottom of the college population. So the fact that I still thought at age 11 that teachers were infallible shows what a job the system must have done on my brain.

School

What kids get taught in school is a complex mix of lies. The most excusable are those told to simplify ideas to make them easy to learn. The problem is, a lot of propaganda gets slipped into the curriculum in the name of simplification.

Public school textbooks represent a compromise between what various powerful groups want kids to be told. The lies are rarely overt. Usually they consist either of omissions or of over-emphasizing certain topics at the expense of others. The view of history we got in elementary school was a crude hagiography, with at least one representative of each powerful group.

The famous scientists I remember were Einstein, Marie Curie, and George Washington Carver. Einstein was a big deal because his work led to the atom bomb. Marie Curie was involved with X-rays. But I was mystified about Carver. He seemed to have done stuff with peanuts.

It's obvious now that he was on the list because he was black (and for that matter that Marie Curie was on it because she was a woman), but as a kid I was confused for years about him. I wonder if it wouldn't have been better just to tell us the truth: that there weren't any famous black scientists. Ranking George Washington Carver with Einstein misled us not only about science, but about the obstacles blacks faced in his time.

As subjects got softer, the lies got more frequent. By the time you got to politics and recent history, what we were taught was pretty much pure propaganda. For example, we were taught to regard political leaders as saints—especially the recently martyred Kennedy and King. It was astonishing to learn later that they’d both been serial womanizers, and that Kennedy was a speed freak to boot. (By the time King’s plagiarism emerged, I’d lost the ability to be surprised by the misdeeds of famous people.)

I doubt you could teach kids recent history without teaching them lies, because practically everyone who has anything to say about it has some kind of spin to put on it. Much recent history *consists* of spin. It would probably be better just to teach them metafacts like that.

Probably the biggest lie told in schools, though, is that the way to succeed is through following “the rules.” In fact most such rules are just hacks to manage large groups efficiently.

Peace

Of all the reasons we lie to kids, the most powerful is probably the same mundane reason they lie to us.

Often when we lie to people it’s not part of any conscious strategy, but because they’d react violently to the truth. Kids, almost by definition, lack self-control. They react violently to things—and so they get lied to a lot.⁹

A few Thanksgivings ago, a friend of mine found himself in a situation that perfectly illustrates the complex motives we have when we lie to kids. As the roast turkey appeared on the table, his alarmingly perceptive 5 year old son suddenly asked if the turkey had wanted to die. Foreseeing disaster, my friend and his wife rapidly improvised:

⁹The ironic thing is, this is also the main reason kids lie to adults. If you freak out when people tell you alarming things, they won’t tell you them. Teenagers don’t tell their parents what happened that night they were supposed to be staying at a friend’s house for the same reason parents don’t tell 5 year olds the truth about the Thanksgiving turkey. They’d freak if they knew.

yes, the turkey had wanted to die, and in fact had lived its whole life with the aim of being their Thanksgiving dinner. And that (phew) was the end of that.

Whenever we lie to kids to protect them, we're usually also lying to keep the peace.

One consequence of this sort of calming lie is that we grow up thinking horrible things are normal. It's hard for us to feel a sense of urgency as adults over something we've literally been trained not to worry about. When I was about 10 I saw a documentary on pollution that put me into a panic. It seemed the planet was being irretrievably ruined. I went to my mother afterward to ask if this was so. I don't remember what she said, but she made me feel better, so I stopped worrying about it.

That was probably the best way to handle a frightened 10 year old. But we should understand the price. This sort of lie is one of the main reasons bad things persist: we're all trained to ignore them.

Detox

A sprinter in a race almost immediately enters a state called "oxygen debt." His body switches to an emergency source of energy that's faster than regular aerobic respiration. But this process builds up waste products that ultimately require extra oxygen to break down, so at the end of the race he has to stop and pant for a while to recover.

We arrive at adulthood with a kind of truth debt. We were told a lot of lies to get us (and our parents) through our childhood. Some may have been necessary. Some probably weren't. But we all arrive at adulthood with heads full of lies.

There's never a point where the adults sit you down and explain all the lies they told you. They've forgotten most of them. So if you're going to clear these lies out of your head, you're going to have to do it yourself.

Few do. Most people go through life with bits of packing material

adhering to their minds and never know it. You probably never can completely undo the effects of lies you were told as a kid, but it's worth trying. I've found that whenever I've been able to undo a lie I was told, a lot of other things fell into place.

Fortunately, once you arrive at adulthood you get a valuable new resource you can use to figure out what lies you were told. You're now one of the liars. You get to watch behind the scenes as adults spin the world for the next generation of kids.

The first step in clearing your head is to realize how far you are from a neutral observer. When I left high school I was, I thought, a complete skeptic. I'd realized high school was crap. I thought I was ready to question everything I knew. But among the many other things I was ignorant of was how much debris there already was in my head. It's not enough to consider your mind a blank slate. You have to consciously erase it.

Thanks to Sam Altman, Marc Andreessen, Trevor Blackwell, Patrick Collison, Jessica Livingston, Jackie McDonough, Robert Morris, and David Sloo for reading drafts of this. And since there are some controversial ideas here, I should add that none of them agreed with everything in it.

Chapter 27

Disconnecting Distraction

Note: The strategy described at the end of this essay didn't work. It would work for a while, and then I'd gradually find myself using the Internet on my work computer. I'm trying other strategies now, but I think this time I'll wait till I'm sure they work before writing about them.

May 2008

Procrastination feeds on distractions. Most people find it uncomfortable just to sit and do nothing; you avoid work by doing something else.

So one way to beat procrastination is to starve it of distractions. But that's not as straightforward as it sounds, because there are people working hard to distract you. Distraction is not a static obstacle that you avoid like you might avoid a rock in the road. Distraction seeks you out.

Chesterfield described dirt as matter out of place. Distracting is, similarly, desirable at the wrong time. And technology is continually being refined to produce more and more desirable things. Which means that as we learn to avoid one class of distractions, new ones constantly appear, like drug-resistant bacteria.

Television, for example, has after 50 years of refinement reached the point where it's like visual crack. I realized when I was 13 that TV was addictive, so I stopped watching it. But I read recently that the average American watches 4 hours of TV a day. A quarter of their life.

TV is in decline now, but only because people have found even more

addictive ways of wasting time. And what's especially dangerous is that many happen at your computer. This is no accident. An ever larger percentage of office workers sit in front of computers connected to the Internet, and distractions always evolve toward the procrastinators.

I remember when computers were, for me at least, exclusively for work. I might occasionally dial up a server to get mail or ftp files, but most of the time I was offline. All I could do was write and program. Now I feel as if someone snuck a television onto my desk. Terribly addictive things are just a click away. Run into an obstacle in what you're working on? Hmm, I wonder what's new online. Better check.

After years of carefully avoiding classic time sinks like TV, games, and Usenet, I still managed to fall prey to distraction, because I didn't realize that it evolves. Something that used to be safe, using the Internet, gradually became more and more dangerous. Some days I'd wake up, get a cup of tea and check the news, then check email, then check the news again, then answer a few emails, then suddenly notice it was almost lunchtime and I hadn't gotten any real work done. And this started to happen more and more often.

It took me surprisingly long to realize how distracting the Internet had become, because the problem was intermittent. I ignored it the way you let yourself ignore a bug that only appears intermittently. When I was in the middle of a project, distractions weren't really a problem. It was when I'd finished one project and was deciding what to do next that they always bit me.

Another reason it was hard to notice the danger of this new type of distraction was that social customs hadn't yet caught up with it. If I'd spent a whole morning sitting on a sofa watching TV, I'd have noticed very quickly. That's a known danger sign, like drinking alone. But using the Internet still looked and felt a lot like work.

Eventually, though, it became clear that the Internet had become so much more distracting that I had to start treating it differently. Basi-

cally, I had to add a new application to my list of known time sinks: Firefox.

* * *

The problem is a hard one to solve because most people still need the Internet for some things. If you drink too much, you can solve that problem by stopping entirely. But you can't solve the problem of overeating by stopping eating. I couldn't simply avoid the Internet entirely, as I'd done with previous time sinks.

At first I tried rules. For example, I'd tell myself I was only going to use the Internet twice a day. But these schemes never worked for long. Eventually something would come up that required me to use it more than that. And then I'd gradually slip back into my old ways.

Addictive things have to be treated as if they were sentient adversaries—as if there were a little man in your head always cooking up the most plausible arguments for doing whatever you're trying to stop doing. If you leave a path to it, he'll find it.

The key seems to be visibility. The biggest ingredient in most bad habits is denial. So you have to make it so that you can't merely *slip* into doing the thing you're trying to avoid. It has to set off alarms.

Maybe in the long term the right answer for dealing with Internet distractions will be software that watches and controls them. But in the meantime I've found a more drastic solution that definitely works: to set up a separate computer for using the Internet.

I now leave wifi turned off on my main computer except when I need to transfer a file or edit a web page, and I have a separate laptop on the other side of the room that I use to check mail or browse the web. (Irony of ironies, it's the computer Steve Huffman wrote Reddit on. When Steve and Alexis auctioned off their old laptops for charity, I bought them for the Y Combinator museum.)

My rule is that I can spend as much time online as I want, as long as I do it on that computer. And this turns out to be enough. When I

have to sit on the other side of the room to check email or browse the web, I become much more aware of it. Sufficiently aware, in my case at least, that it's hard to spend more than about an hour a day online.

And my main computer is now freed for work. If you try this trick, you'll probably be struck by how different it feels when your computer is disconnected from the Internet. It was alarming to me how foreign it felt to sit in front of a computer that could only be used for work, because that showed how much time I must have been wasting.

Wow. All I can do at this computer is work. Ok, I better work then.

That's the good part. Your old bad habits now help you to work. You're used to sitting in front of that computer for hours at a time. But you can't browse the web or check email now. What are you going to do? You can't just sit there. So you start working.

Chapter 28

Cities and Ambition

May 2008

Great cities attract ambitious people. You can sense it when you walk around one. In a hundred subtle ways, the city sends you a message: you could do more; you should try harder.

The surprising thing is how different these messages can be. New York tells you, above all: you should make more money. There are other messages too, of course. You should be hipper. You should be better looking. But the clearest message is that you should be richer.

What I like about Boston (or rather Cambridge) is that the message there is: you should be smarter. You really should get around to reading all those books you've been meaning to.

When you ask what message a city sends, you sometimes get surprising answers. As much as they respect brains in Silicon Valley, the message the Valley sends is: you should be more powerful.

That's not quite the same message New York sends. Power matters in New York too of course, but New York is pretty impressed by a billion dollars even if you merely inherited it. In Silicon Valley no one would care except a few real estate agents. What matters in Silicon Valley is how much effect you have on the world. The reason people there care about Larry and Sergey is not their wealth but the fact that they control Google, which affects practically everyone.

How much does it matter what message a city sends? Empirically, the answer seems to be: a lot. You might think that if you had enough

strength of mind to do great things, you'd be able to transcend your environment. Where you live should make at most a couple percent difference. But if you look at the historical evidence, it seems to matter more than that. Most people who did great things were clumped together in a few places where that sort of thing was done at the time.

You can see how powerful cities are from something I wrote about earlier: the case of the Milanese Leonardo. Practically every fifteenth century Italian painter you've heard of was from Florence, even though Milan was just as big. People in Florence weren't genetically different, so you have to assume there was someone born in Milan with as much natural ability as Leonardo. What happened to him?

If even someone with the same natural ability as Leonardo couldn't beat the force of environment, do you suppose you can?

I don't. I'm fairly stubborn, but I wouldn't try to fight this force. I'd rather use it. So I've thought a lot about where to live.

I'd always imagined Berkeley would be the ideal place — that it would basically be Cambridge with good weather. But when I finally tried living there a couple years ago, it turned out not to be. The message Berkeley sends is: you should live better. Life in Berkeley is very civilized. It's probably the place in America where someone from Northern Europe would feel most at home. But it's not humming with ambition.

In retrospect it shouldn't have been surprising that a place so pleasant would attract people interested above all in quality of life. Cambridge with good weather, it turns out, is not Cambridge. The people you find in Cambridge are not there by accident. You have to make sacrifices to live there. It's expensive and somewhat grubby, and the weather's often bad. So the kind of people you find in Cambridge are the kind of people who want to live where the smartest people are, even if that means living in an expensive, grubby place with bad weather.

As of this writing, Cambridge seems to be the intellectual capital of the

world. I realize that seems a preposterous claim. What makes it true is that it's more preposterous to claim about anywhere else. American universities currently seem to be the best, judging from the flow of ambitious students. And what US city has a stronger claim? New York? A fair number of smart people, but diluted by a much larger number of neanderthals in suits. The Bay Area has a lot of smart people too, but again, diluted; there are two great universities, but they're far apart. Harvard and MIT are practically adjacent by West Coast standards, and they're surrounded by about 20 other colleges and universities.¹

Cambridge as a result feels like a town whose main industry is ideas, while New York's is finance and Silicon Valley's is startups.

When you talk about cities in the sense we are, what you're really talking about is collections of people. For a long time cities were the only large collections of people, so you could use the two ideas interchangeably. But we can see how much things are changing from the examples I've mentioned. New York is a classic great city. But Cambridge is just part of a city, and Silicon Valley is not even that. (San Jose is not, as it sometimes claims, the capital of Silicon Valley. It's just 178 square miles at one end of it.)

Maybe the Internet will change things further. Maybe one day the most important community you belong to will be a virtual one, and it won't matter where you live physically. But I wouldn't bet on it. The physical world is very high bandwidth, and some of the ways cities send you messages are quite subtle.

¹This is one of the advantages of not having the universities in your country controlled by the government. When governments decide how to allocate resources, political deal-making causes things to be spread out geographically. No central government would put its two best universities in the same town, unless it was the capital (which would cause other problems). But scholars seem to like to cluster together as much as people in any other field, and when given the freedom to they derive the same advantages from it.

One of the exhilarating things about coming back to Cambridge every spring is walking through the streets at dusk, when you can see into the houses. When you walk through Palo Alto in the evening, you see nothing but the blue glow of TVs. In Cambridge you see shelves full of promising-looking books. Palo Alto was probably much like Cambridge in 1960, but you'd never guess now that there was a university nearby. Now it's just one of the richer neighborhoods in Silicon Valley.²

A city speaks to you mostly by accident — in things you see through windows, in conversations you overhear. It's not something you have to seek out, but something you can't turn off. One of the occupational hazards of living in Cambridge is overhearing the conversations of people who use interrogative intonation in declarative sentences. But on average I'll take Cambridge conversations over New York or Silicon Valley ones.

A friend who moved to Silicon Valley in the late 90s said the worst thing about living there was the low quality of the eavesdropping. At the time I thought she was being deliberately eccentric. Sure, it can be interesting to eavesdrop on people, but is good quality eavesdropping so important that it would affect where you chose to live? Now I understand what she meant. The conversations you overhear tell you what sort of people you're among.

No matter how determined you are, it's hard not to be influenced by the people around you. It's not so much that you do whatever a city expects of you, but that you get discouraged when no one around you cares about the same things you do.

There's an imbalance between encouragement and discouragement like that between gaining and losing money. Most people overvalue

²There are still a few old professors in Palo Alto, but one by one they die and their houses are transformed by developers into McMansions and sold to VPs of Bus Dev.

negative amounts of money: they'll work much harder to avoid losing a dollar than to gain one. Similarly, although there are plenty of people strong enough to resist doing something just because that's what one is supposed to do where they happen to be, there are few strong enough to keep working on something no one around them cares about.

Because ambitions are to some extent incompatible and admiration is a zero-sum game, each city tends to focus on one type of ambition. The reason Cambridge is the intellectual capital is not just that there's a concentration of smart people there, but that there's nothing *else* people there care about more. Professors in New York and the Bay area are second class citizens — till they start hedge funds or startups respectively.

This suggests an answer to a question people in New York have wondered about since the Bubble: whether New York could grow into a startup hub to rival Silicon Valley. One reason that's unlikely is that someone starting a startup in New York would feel like a second class citizen.³ There's already something else people in New York admire more.

In the long term, that could be a bad thing for New York. The power of an important new technology does eventually convert to money. So

³How many times have you read about startup founders who continued to live inexpensively as their companies took off? Who continued to dress in jeans and t-shirts, to drive the old car they had in grad school, and so on? If you did that in New York, people would treat you like shit. If you walk into a fancy restaurant in San Francisco wearing a jeans and a t-shirt, they're nice to you; who knows who you might be? Not in New York.

One sign of a city's potential as a technology center is the number of restaurants that still require jackets for men. According to Zagat's there are none in San Francisco, LA, Boston, or Seattle, 4 in DC, 6 in Chicago, 8 in London, 13 in New York, and 20 in Paris.

(Zagat's lists the Ritz Carlton Dining Room in SF as requiring jackets but I couldn't believe it, so I called to check and in fact they don't. Apparently there's only one restaurant left on the entire West Coast that still requires jackets: The French Laundry in Napa Valley.)

by caring more about money and less about power than Silicon Valley, New York is recognizing the same thing, but slower.⁴ And in fact it has been losing to Silicon Valley at its own game: the ratio of New York to California residents in the Forbes 400 has decreased from 1.45 (81:56) when the list was first published in 1982 to .83 (73:88) in 2007.

Not all cities send a message. Only those that are centers for some type of ambition do. And it can be hard to tell exactly what message a city sends without living there. I understand the messages of New York, Cambridge, and Silicon Valley because I've lived for several years in each of them. DC and LA seem to send messages too, but I haven't spent long enough in either to say for sure what they are.

The big thing in LA seems to be fame. There's an A List of people who are most in demand right now, and what's most admired is to be on it, or friends with those who are. Beneath that, the message is much like New York's, though perhaps with more emphasis on physical attractiveness.

In DC the message seems to be that the most important thing is who you know. You want to be an insider. In practice this seems to work much as in LA. There's an A List and you want to be on it or close to those who are. The only difference is how the A List is selected. And even that is not that different.

At the moment, San Francisco's message seems to be the same as Berkeley's: you should live better. But this will change if enough star-

⁴Ideas are one step upstream from economic power, so it's conceivable that intellectual centers like Cambridge will one day have an edge over Silicon Valley like the one the Valley has over New York.

This seems unlikely at the moment; if anything Boston is falling further and further behind. The only reason I even mention the possibility is that the path from ideas to startups has recently been getting smoother. It's a lot easier now for a couple of hackers with no business experience to start a startup than it was 10 years ago. If you extrapolate another 20 years, maybe the balance of power will start to shift back. I wouldn't bet on it, but I wouldn't bet against it either.

tups choose SF over the Valley. During the Bubble that was a predictor of failure — a self-indulgent choice, like buying expensive office furniture. Even now I'm suspicious when startups choose SF. But if enough good ones do, it stops being a self-indulgent choice, because the center of gravity of Silicon Valley will shift there.

I haven't found anything like Cambridge for intellectual ambition. Oxford and Cambridge (England) feel like Ithaca or Hanover: the message is there, but not as strong.

Paris was once a great intellectual center. If you went there in 1300, it might have sent the message Cambridge does now. But I tried living there for a bit last year, and the ambitions of the inhabitants are not intellectual ones. The message Paris sends now is: do things with style. I liked that, actually. Paris is the only city I've lived in where people genuinely cared about art. In America only a few rich people buy original art, and even the more sophisticated ones rarely get past judging it by the brand name of the artist. But looking through windows at dusk in Paris you can see that people there actually care what paintings look like. Visually, Paris has the best eavesdropping I know.

5

There's one more message I've heard from cities: in London you can still (barely) hear the message that one should be more aristocratic. If you listen for it you can also hear it in Paris, New York, and Boston. But this message is everywhere very faint. It would have been strong 100 years ago, but now I probably wouldn't have picked it up at all if I hadn't deliberately tuned in to that wavelength to see if there was any signal left.

So far the complete list of messages I've picked up from cities is:

⁵If Paris is where people care most about art, why is New York the center of gravity of the art business? Because in the twentieth century, art as brand split apart from art as stuff. New York is where the richest buyers are, but all they demand from art is brand, and since you can base brand on anything with a sufficiently identifiable style, you may as well use the local stuff.

wealth, style, hipness, physical attractiveness, fame, political power, economic power, intelligence, social class, and quality of life.

My immediate reaction to this list is that it makes me slightly queasy. I'd always considered ambition a good thing, but I realize now that was because I'd always implicitly understood it to mean ambition in the areas I cared about. When you list everything ambitious people are ambitious about, it's not so pretty.

On closer examination I see a couple things on the list that are surprising in the light of history. For example, physical attractiveness wouldn't have been there 100 years ago (though it might have been 2400 years ago). It has always mattered for women, but in the late twentieth century it seems to have started to matter for men as well. I'm not sure why — probably some combination of the increasing power of women, the increasing influence of actors as models, and the fact that so many people work in offices now: you can't show off by wearing clothes too fancy to wear in a factory, so you have to show off with your body instead.

Hipness is another thing you wouldn't have seen on the list 100 years ago. Or wouldn't you? What it means is to know what's what. So maybe it has simply replaced the component of social class that consisted of being "au fait." That could explain why hipness seems particularly admired in London: it's version 2 of the traditional English delight in obscure codes that only insiders understand.

Economic power would have been on the list 100 years ago, but what we mean by it is changing. It used to mean the control of vast human and material resources. But increasingly it means the ability to direct the course of technology, and some of the people in a position to do that are not even rich — leaders of important open source projects, for example. The Captains of Industry of times past had laboratories full of clever people cooking up new technologies for them. The new breed are themselves those people.

As this force gets more attention, another is dropping off the list: social

class. I think the two changes are related. Economic power, wealth, and social class are just names for the same thing at different stages in its life: economic power converts to wealth, and wealth to social class. So the focus of admiration is simply shifting upstream.

Does anyone who wants to do great work have to live in a great city? No; all great cities inspire some sort of ambition, but they aren't the only places that do. For some kinds of work, all you need is a handful of talented colleagues.

What cities provide is an audience, and a funnel for peers. These aren't so critical in something like math or physics, where no audience matters except your peers, and judging ability is sufficiently straightforward that hiring and admissions committees can do it reliably. In a field like math or physics all you need is a department with the right colleagues in it. It could be anywhere — in Los Alamos, New Mexico, for example.

It's in fields like the arts or writing or technology that the larger environment matters. In these the best practitioners aren't conveniently collected in a few top university departments and research labs — partly because talent is harder to judge, and partly because people pay for these things, so one doesn't need to rely on teaching or research funding to support oneself. It's in these more chaotic fields that it helps most to be in a great city: you need the encouragement of feeling that people around you care about the kind of work you do, and since you have to find peers for yourself, you need the much larger intake mechanism of a great city.

You don't have to live in a great city your whole life to benefit from it. The critical years seem to be the early and middle ones of your career. Clearly you don't have to grow up in a great city. Nor does it seem to matter if you go to college in one. To most college students a world of a few thousand people seems big enough. Plus in college you don't yet have to face the hardest kind of work — discovering new problems

to solve.

It's when you move on to the next and much harder step that it helps most to be in a place where you can find peers and encouragement. You seem to be able to leave, if you want, once you've found both. The Impressionists show the typical pattern: they were born all over France (Pissarro was born in the Carriibbean) and died all over France, but what defined them were the years they spent together in Paris.

Unless you're sure what you want to do and where the leading center for it is, your best bet is probably to try living in several places when you're young. You can never tell what message a city sends till you live there, or even whether it still sends one. Often your information will be wrong: I tried living in Florence when I was 25, thinking it would be an art center, but it turned out I was 450 years too late.

Even when a city is still a live center of ambition, you won't know for sure whether its message will resonate with you till you hear it. When I moved to New York, I was very excited at first. It's an exciting place. So it took me quite a while to realize I just wasn't like the people there. I kept searching for the Cambridge of New York. It turned out it was way, way uptown: an hour uptown by air.

Some people know at 16 what sort of work they're going to do, but in most ambitious kids, ambition seems to precede anything specific to be ambitious about. They know they want to do something great. They just haven't decided yet whether they're going to be a rock star or a brain surgeon. There's nothing wrong with that. But it means if you have this most common type of ambition, you'll probably have to figure out where to live by trial and error. You'll probably have to find the city where you feel at home to know what sort of ambition you have.

Thanks to Trevor Blackwell, Sarah Harlin, Jessica Livingston, Jackie McDonough, Robert Morris, and David Sloo for reading drafts of this.

Chapter 29

The Pooled-Risk Company Management Company

July 2008

At this year's startup school, David Heinemeier Hansson gave a talk in which he suggested that startup founders should do things the old fashioned way. Instead of hoping to get rich by building a valuable company and then selling stock in a "liquidity event," founders should start companies that make money and live off the revenues.

Sounds like a good plan. Let's think about the optimal way to do this.

One disadvantage of living off the revenues of your company is that you have to keep running it. And as anyone who runs their own business can tell you, that requires your complete attention. You can't just start a business and check out once things are going well, or they stop going well surprisingly fast.

The main economic motives of startup founders seem to be freedom and security. They want enough money that (a) they don't have to worry about running out of money and (b) they can spend their time how they want. Running your own business offers neither. You certainly don't have freedom: no boss is so demanding. Nor do you have security, because if you stop paying attention to the company, its revenues go away, and with them your income.

The best case, for most people, would be if you could hire someone to manage the company for you once you'd grown it to a certain size. Suppose you could find a really good manager. Then you would have both freedom and security. You could pay as little attention to the busi-

ness as you wanted, knowing that your manager would keep things running smoothly. And that being so, revenues would continue to flow in, so you'd have security as well.

There will of course be some founders who wouldn't like that idea: the ones who like running their company so much that there's nothing else they'd rather do. But this group must be small. The way you succeed in most businesses is to be fanatically attentive to customers' needs. What are the odds that your own desires would coincide exactly with the demands of this powerful, external force?

Sure, running your own company can be fairly interesting. Viaweb was more interesting than any job I'd had before. And since I made much more money from it, it offered the highest ratio of income to boringness of anything I'd done, by orders of magnitude. But was it *the* most interesting work I could imagine doing? No.

Whether the number of founders in the same position is asymptotic or merely large, there are certainly a lot of them. For them the right approach would be to hand the company over to a professional manager eventually, if they could find one who was good enough.

So far so good. But what if your manager was hit by a bus? What you really want is a management company to run your company for you. Then you don't depend on any one person.

If you own rental property, there are companies you can hire to manage it for you. Some will do everything, from finding tenants to fixing leaks. Of course, running companies is a lot more complicated than managing rental property, but let's suppose there were management companies that could do it for you. They'd charge a lot, but wouldn't it be worth it? I'd sacrifice a large percentage of the income for the extra peace of mind.

I realize what I'm describing already sounds too good to be true, but I can think of a way to make it even more attractive. If company man-

agement companies existed, there would be an additional service they could offer clients: they could let them insure their returns by pooling their risk. After all, even a perfect manager can't save a company when, as sometimes happens, its whole market dies, just as property managers can't save you from the building burning down. But a company that managed a large enough number of companies could say to all its clients: we'll combine the revenues from all your companies, and pay you your proportionate share.

If such management companies existed, they'd offer the maximum of freedom and security. Someone would run your company for you, and you'd be protected even if it happened to die.

Let's think about how such a management company might be organized. The simplest way would be to have a new kind of stock representing the total pool of companies they were managing. When you signed up, you'd trade your company's stock for shares of this pool, in proportion to an estimate of your company's value that you'd both agreed upon. Then you'd automatically get your share of the returns of the whole pool.

The catch is that because this kind of trade would be hard to undo, you couldn't switch management companies. But there's a way they could fix that: suppose all the company management companies got together and agreed to allow their clients to exchange shares in all their pools. Then you could, in effect, simultaneously choose all the management companies to run yours for you, in whatever proportion you wanted, and change your mind later as often as you wanted.

If such pooled-risk company management companies existed, signing up with one would seem the ideal plan for most people following the route David advocated.

Good news: they do exist. What I've just described is an acquisition by a public company.

Unfortunately, though public acquirers are structurally identical to pooled-risk company management companies, they don't think of themselves that way. With a property management company, you can just walk in whenever you want and say "manage my rental property for me" and they'll do it. Whereas acquirers are, as of this writing, extremely fickle. Sometimes they're in a buying mood and they'll overpay enormously; other times they're not interested. They're like property management companies run by madmen. Or more precisely, by Benjamin Graham's Mr. Market.

So while on average public acquirers behave like pooled-risk company managers, you need a window of several years to get average case performance. If you wait long enough (five years, say) you're likely to hit an up cycle where some acquirer is hot to buy you. But you can't choose when it happens.

You can't assume investors will carry you for as long as you might have to wait. Your company has to make money. Opinions are divided about how early to focus on that. Joe Kraus says you should try charging customers right away. And yet some of the most successful startups, including Google, ignored revenue at first and concentrated exclusively on development. The answer probably depends on the type of company you're starting. I can imagine some where trying to make sales would be a good heuristic for product design, and others where it would just be a distraction. The test is probably whether it helps you to understand your users.

You can choose whichever revenue strategy you think is best for the type of company you're starting, so long as you're profitable. Being profitable ensures you'll get at least the average of the acquisition market—in which public companies do behave as pooled-risk company management companies.

David isn't mistaken in saying you should start a company to live off its revenues. The mistake is thinking this is somehow opposed to starting a company and selling it. In fact, for most people the latter is merely the optimal case of the former.

Thanks to Trevor Blackwell, Jessica Livingston, Michael Mandel, Robert Morris, and Fred Wilson for reading drafts of this.

Chapter 30

A Fundraising Survival Guide

August 2008

Raising money is the second hardest part of starting a startup. The hardest part is making something people want: most startups that die, die because they didn't do that. But the second biggest cause of death is probably the difficulty of raising money. Fundraising is brutal.

One reason it's so brutal is simply the brutality of markets. People who've spent most of their lives in schools or big companies may not have been exposed to that. Professors and bosses usually feel some sense of responsibility toward you; if you make a valiant effort and fail, they'll cut you a break. Markets are less forgiving. Customers don't care how hard you worked, only whether you solved their problems.

Investors evaluate startups the way customers evaluate products, not the way bosses evaluate employees. If you're making a valiant effort and failing, maybe they'll invest in your next startup, but not this one.

But raising money from investors is harder than selling to customers, because there are so few of them. There's nothing like an efficient market. You're unlikely to have more than 10 who are interested; it's difficult to talk to more. So the randomness of any one investor's behavior can really affect you.

Problem number 3: investors are very random. All investors, including us, are by ordinary standards incompetent. We constantly have to make decisions about things we don't understand, and more often than not we're wrong.

And yet a lot is at stake. The amounts invested by different types of in-

vestors vary from five thousand dollars to fifty million, but the amount usually seems large for whatever type of investor it is. Investment decisions are big decisions.

That combination—making big decisions about things they don’t understand—tends to make investors very skittish. VCs are notorious for leading founders on. Some of the more unscrupulous do it deliberately. But even the most well-intentioned investors can behave in a way that would seem crazy in everyday life. One day they’re full of enthusiasm and seem ready to write you a check on the spot; the next they won’t return your phone calls. They’re not playing games with you. They just can’t make up their minds.¹

If that weren’t bad enough, these wildly fluctuating nodes are all linked together. Startup investors all know one another, and (though they hate to admit it) the biggest factor in their opinion of you is the opinion of other investors.² Talk about a recipe for an unstable system. You get the opposite of the damping that the fear/greed balance usually produces in markets. No one is interested in a startup that’s a “bargain” because everyone else hates it.

So the inefficient market you get because there are so few players is exacerbated by the fact that they act less than independently. The result is a system like some kind of primitive, multi-celled sea creature, where you irritate one extremity and the whole thing contracts

¹When investors can’t make up their minds, they sometimes describe it as if it were a property of the startup. “You’re too early for us,” they sometimes say. But which of them, if they were taken back in a time machine to the hour Google was founded, wouldn’t offer to invest at any valuation the founders chose? An hour old is not too early if it’s the right startup. What “you’re too early” really means is “we can’t figure out yet whether you’ll succeed.”

²Investors influence one another both directly and indirectly. They influence one another directly through the “buzz” that surrounds a hot startup. But they also influence one another indirectly *through the founders*. When a lot of investors are interested in you, it increases your confidence in a way that makes you much more attractive to investors.

No VC will admit they’re influenced by buzz. Some genuinely aren’t. But there are few who can say they’re not influenced by confidence.

violently.

Y Combinator is working to fix this. We're trying to increase the number of investors just as we're increasing the number of startups. We hope that as the number of both increases we'll get something more like an efficient market. As t approaches infinity, Demo Day approaches an auction.

Unfortunately, t is still very far from infinity. What does a startup do now, in the imperfect world we currently inhabit? The most important thing is not to let fundraising get you down. Startups live or die on morale. If you let the difficulty of raising money destroy your morale, it will become a self-fulfilling prophecy.

Bootstrapping (= Consulting)

Some would-be founders may by now be thinking, why deal with investors at all? If raising money is so painful, why do it?

One answer to that is obvious: because you need money to live on. It's a fine idea in principle to finance your startup with its own revenues, but you can't create instant customers. Whatever you make, you have to sell a certain amount to break even. It will take time to grow your sales to that point, and it's hard to predict, till you try, how long it will take.

We could not have bootstrapped Viaweb, for example. We charged quite a lot for our software—about \$140 per user per month—but it was at least a year before our revenues would have covered even our paltry costs. We didn't have enough saved to live on for a year.

If you factor out the “bootstrapped” companies that were actually funded by their founders through savings or a day job, the remainder either (a) got really lucky, which is hard to do on demand, or (b) began life as consulting companies and gradually transformed themselves into product companies.

Consulting is the only option you can count on. But consulting is far from free money. It's not as painful as raising money from investors,

perhaps, but the pain is spread over a longer period. Years, probably. And for many types of startup, that delay could be fatal. If you're working on something so unusual that no one else is likely to think of it, you can take your time. Joshua Schachter gradually built Delicious on the side while working on Wall Street. He got away with it because no one else realized it was a good idea. But if you were building something as obviously necessary as online store software at about the same time as Viaweb, and you were working on it on the side while spending most of your time on client work, you were not in a good position.

Bootstrapping sounds great in principle, but this apparently verdant territory is one from which few startups emerge alive. The mere fact that bootstrapped startups tend to be famous on that account should set off alarm bells. If it worked so well, it would be the norm.³

Bootstrapping may get easier, because starting a company is getting cheaper. But I don't think we'll ever reach the point where most startups can do without outside funding. Technology tends to get dramatically cheaper, but living expenses don't.

The upshot is, you can choose your pain: either the short, sharp pain of raising money, or the chronic ache of consulting. For a given total amount of pain, raising money is the better choice, because new technology is usually more valuable now than later.

But although for most startups raising money will be the lesser evil, it's still a pretty big evil—so big that it can easily kill you. Not merely in the obvious sense that if you fail to raise money you might have to shut the company down, but because the *process* of raising money itself can kill you.

To survive it you need a set of techniques mostly orthogonal to the ones used in convincing investors, just as mountain climbers need to

³One VC who read this essay wrote:

“We try to avoid companies that got bootstrapped with consulting. It creates very bad behaviors/instincts that are hard to erase from a company's culture.”

know survival techniques that are mostly orthogonal to those used in physically getting up and down mountains.

1. Have low expectations.

The reason raising money destroys so many startups' morale is not simply that it's hard, but that it's so much harder than they expected. What kills you is the disappointment. And the lower your expectations, the harder it is to be disappointed.

Startup founders tend to be optimistic. This can work well in technology, at least some of the time, but it's the wrong way to approach raising money. Better to assume investors will always let you down. Acquirers too, while we're at it. At YC one of our secondary mantras is "Deals fall through." No matter what deal you have going on, assume it will fall through. The predictive power of this simple rule is amazing.

There will be a tendency, as a deal progresses, to start to believe it will happen, and then to depend on it happening. You must resist this. Tie yourself to the mast. This is what kills you. Deals do not have a trajectory like most other human interactions, where shared plans solidify linearly over time. Deals often fall through at the last moment. Often the other party doesn't really think about what they want till the last moment. So you can't use your everyday intuitions about shared plans as a guide. When it comes to deals, you have to consciously turn them off and become pathologically cynical.

This is harder to do than it sounds. It's very flattering when eminent investors seem interested in funding you. It's easy to start to believe that raising money will be quick and straightforward. But it hardly ever is.

2. Keep working on your startup.

It sounds obvious to say that you should keep working on your startup while raising money. Actually this is hard to do. Most startups don't manage to.

Raising money has a mysterious capacity to suck up all your attention. Even if you only have one meeting a day with investors, somehow that one meeting will burn up your whole day. It costs not just the time of the actual meeting, but the time getting there and back, and the time preparing for it beforehand and thinking about it afterward.

The best way to survive the distraction of meeting with investors is probably to partition the company: to pick one founder to deal with investors while the others keep the company going. This works better when a startup has 3 founders than 2, and better when the leader of the company is not also the lead developer. In the best case, the company keeps moving forward at about half speed.

That's the best case, though. More often than not the company comes to a standstill while raising money. And that is dangerous for so many reasons. Raising money always takes longer than you expect. What seems like it's going to be a 2 week interruption turns into a 4 month interruption. That can be very demoralizing. And worse still, it can make you less attractive to investors. They want to invest in companies that are dynamic. A company that hasn't done anything new in 4 months doesn't seem dynamic, so they start to lose interest. Investors rarely grasp this, but much of what they're responding to when they lose interest in a startup is the damage done by their own indecision.

The solution: put the startup first. Fit meetings with investors into the spare moments in your development schedule, rather than doing development in the spare moments between meetings with investors. If you keep the company moving forward—releasing new features, increasing traffic, doing deals, getting written about—those investor meetings are more likely to be productive. Not just because your startup will seem more alive, but also because it will be better for your own morale, which is one of the main ways investors judge you.

3. Be conservative.

As conditions get worse, the optimal strategy becomes more conservative. When things go well you can take risks; when things are bad

you want to play it safe.

I advise approaching fundraising as if it were always going badly. The reason is that between your ability to delude yourself and the wildly unstable nature of the system you're dealing with, things probably either already are or could easily become much worse than they seem.

What I tell most startups we fund is that if someone reputable offers you funding on reasonable terms, take it. There have been startups that ignored this advice and got away with it—startups that ignored a good offer in the hope of getting a better one, and actually did. But in the same position I'd give the same advice again. Who knows how many bullets were in the gun they were playing Russian roulette with?

Corollary: if an investor seems interested, don't just let them sit. You can't assume someone interested in investing will stay interested. In fact, you can't even tell (*they* can't even tell) if they're really interested till you try to convert that interest into money. So if you have hot prospect, either close them now or write them off. And unless you already have enough funding, that reduces to: close them now.

Startups don't win by getting great funding rounds, but by making great products. So finish raising money and get back to work.

4. Be flexible.

There are two questions VCs ask that you shouldn't answer: "Who else are you talking to?" and "How much are you trying to raise?"

Vcs don't expect you to answer the first question. They ask it just in case.⁴ They do seem to expect an answer to the second. But I don't think you should just tell them a number. Not as a way to play games with them, but because you shouldn't *have* a fixed amount you need to raise.

⁴The optimal way to answer the first question is to say that it would be improper to name names, while simultaneously implying that you're talking to a bunch of other VCs who are all about to give you term sheets. If you're the sort of person who understands how to do that, go ahead. If not, don't even try. Nothing annoys VCs more than clumsy efforts to manipulate them.

The custom of a startup needing a fixed amount of funding is an obsolete one left over from the days when startups were more expensive. A company that needed to build a factory or hire 50 people obviously needed to raise a certain minimum amount. But few technology startups are in that position today.

We advise startups to tell investors there are several different routes they could take depending on how much they raised. As little as \$50k could pay for food and rent for the founders for a year. A couple hundred thousand would let them get office space and hire some smart people they know from school. A couple million would let them really blow this thing out. The message (and not just the message, but the fact) should be: we're going to succeed no matter what. Raising more money just lets us do it faster.

If you're raising an angel round, the size of the round can even change on the fly. In fact, it's just as well to make the round small initially, then expand as needed, rather than trying to raise a large round and risk losing the investors you already have if you can't raise the full amount. You may even want to do a "rolling close," where the round has no predetermined size, but instead you sell stock to investors one at a time as they say yes. That helps break deadlocks, because you can start as soon as the first one is ready to buy.⁵

5. Be independent.

A startup with a couple founders in their early twenties can have expenses so low that they could be profitable on as little as \$2000 per month. That's negligible as corporate revenues go, but the effect on your morale and your bargaining position is anything but. At YC we use the phrase "ramen profitable" to describe the situation where you're making just enough to pay your living expenses. Once you cross into ramen profitable, everything changes. You may still need

⁵The disadvantage of expanding a round on the fly is that the valuation is fixed at the start, so if you get a sudden rush of interest, you may have to decide between turning some investors away and selling more of the company than you meant to. That's a good problem to have, however.

investment to make it big, but you don't need it this month.

You can't plan when you start a startup how long it will take to become profitable. But if you find yourself in a position where a little more effort expended on sales would carry you over the threshold of ramen profitable, do it.

Investors like it when you're ramen profitable. It shows you've thought about making money, instead of just working on amusing technical problems; it shows you have the discipline to keep your expenses low; but above all, it means you don't need them.

There is nothing investors like more than a startup that seems like it's going to succeed even without them. Investors like it when they can help a startup, but they don't like startups that would die without that help.

At YC we spend a lot of time trying to predict how the startups we've funded will do, because we're trying to learn how to pick winners. We've now watched the trajectories of so many startups that we're getting better at predicting them. And when we're talking about startups we think are likely to succeed, what we find ourselves saying is things like "Oh, those guys can take care of themselves. They'll be fine." Not "those guys are really smart" or "those guys are working on a great idea."⁶ When we predict good outcomes for startups, the qualities that come up in the supporting arguments are toughness, adaptability, determination. Which means to the extent we're correct, those are the qualities you need to win.

Investors know this, at least unconsciously. The reason they like it when you don't need them is not simply that they like what they can't have, but because that quality is what makes founders succeed.

Sam Altman has it. You could parachute him into an island full of cannibals and come back in 5 years and he'd be the king. If you're Sam Altman, you don't have to be profitable to convey to investors

⁶I wouldn't say that intelligence doesn't matter in startups. We're only comparing YC startups, who've already made it over a certain threshold.

that you'll succeed with or without them. (He wasn't, and he did.) Not everyone has Sam's deal-making ability. I myself don't. But if you don't, you can let the numbers speak for you.

6. Don't take rejection personally.

Getting rejected by investors can make you start to doubt yourself. After all, they're more experienced than you. If they think your startup is lame, aren't they probably right?

Maybe, maybe not. The way to handle rejection is with precision. You shouldn't simply ignore rejection. It might mean something. But you shouldn't automatically get demoralized either.

To understand what rejection means, you have to understand first of all how common it is. Statistically, the average VC is a rejection machine. David Hornik, a partner at August, told me:

The numbers for me ended up being something like 500 to 800 plans received and read, somewhere between 50 and 100 initial 1 hour meetings held, about 20 companies that I got interested in, about 5 that I got serious about and did a bunch of work, 1 to 2 deals done in a year. So the odds are against you. You may be a great entrepreneur, working on interesting stuff, etc. but it is still incredibly unlikely that you get funded.

This is less true with angels, but VCs reject practically everyone. The structure of their business means a partner does at most 2 new investments a year, no matter how many good startups approach him.

In addition to the odds being terrible, the average investor is, as I mentioned, a pretty bad judge of startups. It's harder to judge startups than most other things, because great startup ideas tend to seem wrong. A good startup idea has to be not just good but novel. And to be both good and novel, an idea probably has to seem bad to most people, or someone would already be doing it and it wouldn't be novel.

That makes judging startups harder than most other things one judges. You have to be an intellectual contrarian to be a good startup investor. That's a problem for VCs, most of whom are not particularly imaginative. VCs are mostly money guys, not people who make things.⁷ Angels are better at appreciating novel ideas, because most were founders themselves.

So when you get a rejection, use the data that's in it, and not what's not. If an investor gives you specific reasons for not investing, look at your startup and ask if they're right. If they're real problems, fix them. But don't just take their word for it. You're supposed to be the domain expert; you have to decide.

Though a rejection doesn't necessarily tell you anything about your startup, it does suggest your pitch could be improved. Figure out what's not working and change it. Don't just think "investors are stupid." Often they are, but figure out precisely where you lose them.

Don't let rejections pile up as a depressing, undifferentiated heap. Sort them and analyze them, and then instead of thinking "no one likes us," you'll know precisely how big a problem you have, and what to do about it.

7. Be able to downshift into consulting (if appropriate).

Consulting, as I mentioned, is a dangerous way to finance a startup. But it's better than dying. It's a bit like anaerobic respiration: not the optimum solution for the long term, but it can save you from an immediate threat. If you're having trouble raising money from investors at all, it could save you to be able to shift toward consulting.

This works better for some startups than others. It wouldn't have been a natural fit for, say, Google, but if your company was making software for building web sites, you could degrade fairly gracefully into consulting by building sites for clients with it.

⁷But not all are. Though most VCs are suits at heart, the most successful ones tend not to be. Oddly enough, the best VCs tend to be the least VC-like.

So long as you were careful not to get sucked permanently into consulting, this could even have advantages. You'd understand your users well if you were using the software for them. Plus as a consulting company you might be able to get big-name users using your software that you wouldn't have gotten as a product company.

At Viaweb we were forced to operate like a consulting company initially, because we were so desperate for users that we'd offer to build merchants' sites for them if they'd sign up. But we never charged for such work, because we didn't want them to start treating us like actual consultants, and calling us every time they wanted something changed on their site. We knew we had to stay a product company, because only that scales.

8. Avoid inexperienced investors.

Though novice investors seem unthreatening they can be the most dangerous sort, because they're so nervous. Especially in proportion to the amount they invest. Raising \$20,000 from a first-time angel investor can be as much work as raising \$2 million from a VC fund.

Their lawyers are generally inexperienced too. But while the investors can admit they don't know what they're doing, their lawyers can't. One YC startup negotiated terms for a tiny round with an angel, only to receive a 70-page agreement from his lawyer. And since the lawyer could never admit, in front of his client, that he'd screwed up, he instead had to insist on retaining all the draconian terms in it, so the deal fell through.

Of course, someone has to take money from novice investors, or there would never be any experienced ones. But if you do, either (a) drive the process yourself, including supplying the paperwork, or (b) use them only to fill up a larger round led by someone else.

9. Know where you stand.

The most dangerous thing about investors is their indecisiveness. The worst case scenario is the long no, the no that comes after months of

meetings. Rejections from investors are like design flaws: inevitable, but much less costly if you discover them early.

So while you're talking to investors, constantly look for signs of where you stand. How likely are they to offer you a term sheet? What do they have to be convinced of first? You shouldn't necessarily always be asking these questions outright—that could get annoying—but you should always be collecting data about them.

Investors tend to resist committing except to the extent you push them to. It's in their interest to collect the maximum amount of information while making the minimum number of decisions. The best way to force them to act is, of course, competing investors. But you can also apply some force by focusing the discussion: by asking what specific questions they need answered to make up their minds, and then answering them. If you get through several obstacles and they keep raising new ones, assume that ultimately they're going to flake.

You have to be disciplined when collecting data about investors' intentions. Otherwise their desire to lead you on will combine with your own desire to be led on to produce completely inaccurate impressions.

Use the data to weight your strategy. You'll probably be talking to several investors. Focus on the ones that are most likely to say yes. The value of a potential investor is a combination of how good it would be if they said yes, and how likely they are to say it. Put the most weight on the second factor. Partly because the most important quality in an investor is simply investing. But also because, as I mentioned, the biggest factor in investors' opinion of you is other investors' opinion of you. If you're talking to several investors and you manage to get one over the threshold of saying yes, it will make the others much more interested. So you're not sacrificing the lukewarm investors if you focus on the hot ones; convincing the hot investors is the best way to convince the lukewarm ones.

Future

I'm hopeful things won't always be so awkward. I hope that as startups

get cheaper and the number of investors increases, raising money will become, if not easy, at least straightforward.

In the meantime, the brokenness of the funding process offers a big opportunity. Most investors have no idea how dangerous they are. They'd be surprised to hear that raising money from them is something that has to be treated as a threat to a company's survival. They just think they need a little more information to make up their minds. They don't get that there are 10 other investors who also want a little more information, and that the process of talking to them all can bring a startup to a standstill for months.

Because investors don't understand the cost of dealing with them, they don't realize how much room there is for a potential competitor to undercut them. I know from my own experience how much faster investors could decide, because we've brought our own time down to 20 minutes (5 minutes of reading an application plus a 10 minute interview plus 5 minutes of discussion). If you were investing more money you'd want to take longer, of course. But if we can decide in 20 minutes, should it take anyone longer than a couple days?

Opportunities like this don't sit unexploited forever, even in an industry as conservative as venture capital. So either existing investors will start to make up their minds faster, or new investors will emerge who do.

In the meantime founders have to treat raising money as a dangerous process. Fortunately, I can fix the biggest danger right here. The biggest danger is surprise. It's that startups will underestimate the difficulty of raising money—that they'll cruise through all the initial steps, but when they turn to raising money they'll find it surprisingly hard, get demoralized, and give up. So I'm telling you in advance: raising money is hard.

Thanks to Trevor Blackwell, David Hornik, Jessica Livingston, Robert Morris, and Fred Wilson for reading drafts of this.

Chapter 31

Why to Start a Startup in a Bad Economy

October 2008

The economic situation is apparently so grim that some experts fear we may be in for a stretch as bad as the mid seventies.

When Microsoft and Apple were founded.

As those examples suggest, a recession may not be such a bad time to start a startup. I'm not claiming it's a particularly good time either. The truth is more boring: the state of the economy doesn't matter much either way.

If we've learned one thing from funding so many startups, it's that they succeed or fail based on the qualities of the founders. The economy has some effect, certainly, but as a predictor of success it's rounding error compared to the founders.

Which means that what matters is who you are, not when you do it. If you're the right sort of person, you'll win even in a bad economy. And if you're not, a good economy won't save you. Someone who thinks "I better not start a startup now, because the economy is so bad" is making the same mistake as the people who thought during the Bubble "all I have to do is start a startup, and I'll be rich."

So if you want to improve your chances, you should think far more about who you can recruit as a cofounder than the state of the economy. And if you're worried about threats to the survival of your company, don't look for them in the news. Look in the mirror.

But for any given team of founders, would it not pay to wait till the economy is better before taking the leap? If you're starting a restaurant, maybe, but not if you're working on technology. Technology progresses more or less independently of the stock market. So for any given idea, the payoff for acting fast in a bad economy will be higher than for waiting. Microsoft's first product was a Basic interpreter for the Altair. That was exactly what the world needed in 1975, but if Gates and Allen had decided to wait a few years, it would have been too late.

Of course, the idea you have now won't be the last you have. There are always new ideas. But if you have a specific idea you want to act on, act now.

That doesn't mean you can ignore the economy. Both customers and investors will be feeling pinched. It's not necessarily a problem if customers feel pinched: you may even be able to benefit from it, by making things that save money. Startups often make things cheaper, so in that respect they're better positioned to prosper in a recession than big companies.

Investors are more of a problem. Startups generally need to raise some amount of external funding, and investors tend to be less willing to invest in bad times. They shouldn't be. Everyone knows you're supposed to buy when times are bad and sell when times are good. But of course what makes investing so counterintuitive is that in equity markets, good times are defined as everyone thinking it's time to buy. You have to be a contrarian to be correct, and by definition only a minority of investors can be.

So just as investors in 1999 were tripping over one another trying to buy into lousy startups, investors in 2009 will presumably be reluctant to invest even in good ones.

You'll have to adapt to this. But that's nothing new: startups always have to adapt to the whims of investors. Ask any founder in any economy if they'd describe investors as fickle, and watch the face they make.

Last year you had to be prepared to explain how your startup was viral. Next year you'll have to explain how it's recession-proof.

(Those are both good things to be. The mistake investors make is not the criteria they use but that they always tend to focus on one to the exclusion of the rest.)

Fortunately the way to make a startup recession-proof is to do exactly what you should do anyway: run it as cheaply as possible. For years I've been telling founders that the surest route to success is to be the cockroaches of the corporate world. The immediate cause of death in a startup is always running out of money. So the cheaper your company is to operate, the harder it is to kill. And fortunately it has gotten very cheap to run a startup. A recession will if anything make it cheaper still.

If nuclear winter really is here, it may be safer to be a cockroach even than to keep your job. Customers may drop off individually if they can no longer afford you, but you're not going to lose them all at once; markets don't "reduce headcount."

What if you quit your job to start a startup that fails, and you can't find another? That could be a problem if you work in sales or marketing. In those fields it can take months to find a new job in a bad economy. But hackers seem to be more liquid. Good hackers can always get some kind of job. It might not be your dream job, but you're not going to starve.

Another advantage of bad times is that there's less competition. Technology trains leave the station at regular intervals. If everyone else is cowering in a corner, you may have a whole car to yourself.

You're an investor too. As a founder, you're buying stock with work: the reason Larry and Sergey are so rich is not so much that they've done work worth tens of billions of dollars, but that they were the first investors in Google. And like any investor you should buy when times are bad.

Were you nodding in agreement, thinking “stupid investors” a few paragraphs ago when I was talking about how investors are reluctant to put money into startups in bad markets, even though that’s the time they should rationally be most willing to buy? Well, founders aren’t much better. When times get bad, hackers go to grad school. And no doubt that will happen this time too. In fact, what makes the preceding paragraph true is that most readers won’t believe it—at least to the extent of acting on it.

So maybe a recession is a good time to start a startup. It’s hard to say whether advantages like lack of competition outweigh disadvantages like reluctant investors. But it doesn’t matter much either way. It’s the people that matter. And for a given set of people working on a given technology, the time to act is always now.

Chapter 32

The Other Half of "Artists Ship"

November 2008

One of the differences between big companies and startups is that big companies tend to have developed procedures to protect themselves against mistakes. A startup walks like a toddler, bashing into things and falling over all the time. A big company is more deliberate.

The gradual accumulation of checks in an organization is a kind of learning, based on disasters that have happened to it or others like it. After giving a contract to a supplier who goes bankrupt and fails to deliver, for example, a company might require all suppliers to prove they're solvent before submitting bids.

As companies grow they invariably get more such checks, either in response to disasters they've suffered, or (probably more often) by hiring people from bigger companies who bring with them customs for protecting against new types of disasters.

It's natural for organizations to learn from mistakes. The problem is, people who propose new checks almost never consider that the check itself has a cost.

Every check has a cost. For example, consider the case of making suppliers verify their solvency. Surely that's mere prudence? But in fact it could have substantial costs. There's obviously the direct cost in time of the people on both sides who supply and check proofs of the supplier's solvency. But the real costs are the ones you never hear about: the company that would be the best supplier, but doesn't bid because they can't spare the effort to get verified. Or the company that would be the best supplier, but falls just short of the threshold for solvency—

which will of course have been set on the high side, since there is no apparent cost of increasing it.

Whenever someone in an organization proposes to add a new check, they should have to explain not just the benefit but the cost. No matter how bad a job they did of analyzing it, this meta-check would at least remind everyone there had to *be* a cost, and send them looking for it.

If companies started doing that, they'd find some surprises. Joel Spolsky recently spoke at Y Combinator about selling software to corporate customers. He said that in most companies software costing up to about \$1000 could be bought by individual managers without any additional approvals. Above that threshold, software purchases generally had to be approved by a committee. But babysitting this process was so expensive for software vendors that it didn't make sense to charge less than \$50,000. Which means if you're making something you might otherwise have charged \$5000 for, you have to sell it for \$50,000 instead.

The purpose of the committee is presumably to ensure that the company doesn't waste money. And yet the result is that the company pays 10 times as much.

Checks on purchases will always be expensive, because the harder it is to sell something to you, the more it has to cost. And not merely linearly, either. If you're hard enough to sell to, the people who are best at making things don't want to bother. The only people who will sell to you are companies that specialize in selling to you. Then you've sunk to a whole new level of inefficiency. Market mechanisms no longer protect you, because the good suppliers are no longer in the market.

Such things happen constantly to the biggest organizations of all, governments. But checks instituted by governments can cause much worse problems than merely overpaying. Checks instituted by governments can cripple a country's whole economy. Up till about 1400, China was richer and more technologically advanced than Europe.

One reason Europe pulled ahead was that the Chinese government restricted long trading voyages. So it was left to the Europeans to explore and eventually to dominate the rest of the world, including China.

In more recent times, Sarbanes-Oxley has practically destroyed the US IPO market. That wasn't the intention of the legislators who wrote it. They just wanted to add a few more checks on public companies. But they forgot to consider the cost. They forgot that companies about to go public are usually rather stretched, and that the weight of a few extra checks that might be easy for General Electric to bear are enough to prevent younger companies from being public at all.

Once you start to think about the cost of checks, you can start to ask other interesting questions. Is the cost increasing or decreasing? Is it higher in some areas than others? Where does it increase discontinuously? If large organizations started to ask questions like that, they'd learn some frightening things.

I think the cost of checks may actually be increasing. The reason is that software plays an increasingly important role in companies, and the people who write software are particularly harmed by checks.

Programmers are unlike many types of workers in that the best ones actually prefer to work hard. This doesn't seem to be the case in most types of work. When I worked in fast food, we didn't prefer the busy times. And when I used to mow lawns, I definitely didn't prefer it when the grass was long after a week of rain.

Programmers, though, like it better when they write more code. Or more precisely, when they release more code. Programmers like to make a difference. Good ones, anyway.

For good programmers, one of the best things about working for a startup is that there are few checks on releases. In true startups, there are no external checks at all. If you have an idea for a new feature in the morning, you can write it and push it to the production servers before lunch. And when you can do that, you have more ideas.

At big companies, software has to go through various approvals before it can be launched. And the cost of doing this can be enormous—in fact, discontinuous. I was talking recently to a group of three programmers whose startup had been acquired a few years before by a big company. When they'd been independent, they could release changes instantly. Now, they said, the absolute fastest they could get code released on the production servers was two weeks.

This didn't merely make them less productive. It made them hate working for the acquirer.

Here's a sign of how much programmers like to be able to work hard: these guys would have *paid* to be able to release code immediately, the way they used to. I asked them if they'd trade 10% of the acquisition price for the ability to release code immediately, and all three instantly said yes. Then I asked what was the maximum percentage of the acquisition price they'd trade for it. They said they didn't want to think about it, because they didn't want to know how high they'd go, but I got the impression it might be as much as half.

They'd have sacrificed hundreds of thousands of dollars, perhaps millions, just to be able to deliver more software to users. And you know what? It would have been perfectly safe to let them. In fact, the acquirer would have been better off; not only wouldn't these guys have broken anything, they'd have gotten a lot more done. So the acquirer is in fact getting worse performance at greater cost. Just like the committee approving software purchases.

And just as the greatest danger of being hard to sell to is not that you overpay but that the best suppliers won't even sell to you, the greatest danger of applying too many checks to your programmers is not that you'll make them unproductive, but that good programmers won't even want to work for you.

Steve Jobs's famous maxim "artists ship" works both ways. Artists aren't merely capable of shipping. They insist on it. So if you don't let people ship, you won't have any artists.

Chapter 33

The High-Res Society

December 2008

For nearly all of history the success of a society was proportionate to its ability to assemble large and disciplined organizations. Those who bet on economies of scale generally won, which meant the largest organizations were the most successful ones.

Things have already changed so much that this is hard for us to believe, but till just a few decades ago the largest organizations tended to be the most progressive. An ambitious kid graduating from college in 1960 wanted to work in the huge, gleaming offices of Ford, or General Electric, or NASA. Small meant small-time. Small in 1960 didn't mean a cool little startup. It meant uncle Sid's shoe store.

When I grew up in the 1970s, the idea of the “corporate ladder” was still very much alive. The standard plan was to try to get into a good college, from which one would be drafted into some organization and then rise to positions of gradually increasing responsibility. The more ambitious merely hoped to climb the same ladder faster.¹

But in the late twentieth century something changed. It turned out that economies of scale were not the only force at work. Particularly in technology, the increase in speed one could get from smaller groups started to trump the advantages of size.

The future turned out to be different from the one we were expecting in 1970. The domed cities and flying cars we expected have failed to

¹One of the bizarre consequences of this model was that the usual way to make more money was to become a manager. This is one of the things startups fix.

materialize. But fortunately so have the jumpsuits with badges indicating our specialty and rank. Instead of being dominated by a few, giant tree-structured organizations, it's now looking like the economy of the future will be a fluid network of smaller, independent units.

It's not so much that large organizations stopped working. There's no evidence that famously successful organizations like the Roman army or the British East India Company were any less afflicted by protocol and politics than organizations of the same size today. But they were competing against opponents who couldn't change the rules on the fly by discovering new technology. Now it turns out the rule "large and disciplined organizations win" needs to have a qualification appended: "at games that change slowly." No one knew till change reached a sufficient speed.

Large organizations *will* start to do worse now, though, because for the first time in history they're no longer getting the best people. An ambitious kid graduating from college now doesn't want to work for a big company. They want to work for the hot startup that's rapidly growing into one. If they're really ambitious, they want to start it.²

This doesn't mean big companies will disappear. To say that startups will succeed implies that big companies will exist, because startups that succeed either become big companies or are acquired by them.³ But large organizations will probably never again play the leading role they did up till the last quarter of the twentieth century.

It's kind of surprising that a trend that lasted so long would ever run out. How often does it happen that a rule works for thousands of years, then switches polarity?

The millennia-long run of bigger-is-better left us with a lot of traditions that are now obsolete, but extremely deeply rooted. Which

²There are a lot of reasons American car companies have been doing so much worse than Japanese car companies, but at least one of them is a cause for optimism: American graduates have more options.

³It's possible that companies will one day be able to grow big in revenues without growing big in people, but we are not very far along that trend yet.

means the ambitious can now do arbitrage on them. It will be very valuable to understand precisely which ideas to keep and which can now be discarded.

The place to look is where the spread of smallness began: in the world of startups.

There have always been occasional cases, particularly in the US, of ambitious people who grew the ladder under them instead of climbing it. But till recently this was an anomalous route that tended to be followed only by outsiders. It was no coincidence that the great industrialists of the nineteenth century had so little formal education. As huge as their companies eventually became, they were all essentially mechanics and shopkeepers at first. That was a social step no one with a college education would take if they could avoid it. Till the rise of technology startups, and in particular, Internet startups, it was very unusual for educated people to start their own businesses.

The eight men who left Shockley Semiconductor to found Fairchild Semiconductor, the original Silicon Valley startup, weren't even trying to start a company at first. They were just looking for a company willing to hire them as a group. Then one of their parents introduced them to a small investment bank that offered to find funding for them to start their own, so they did. But starting a company was an alien idea to them; it was something they backed into.⁴

Now I would guess that practically every Stanford or Berkeley undergrad who knows how to program has at least considered the idea of starting a startup. East Coast universities are not far behind, and British universities only a little behind them. This pattern suggests that attitudes at Stanford and Berkeley are not an anomaly, but a leading indicator. This is the way the world is going.

Of course, Internet startups are still only a fraction of the world's economy. Could a trend based on them be that powerful?

⁴Lecuyer, Christophe, *Making Silicon Valley*, MIT Press, 2006.

I think so. There's no reason to suppose there's any limit to the amount of work that could be done in this area. Like science, wealth seems to expand fractally. Steam power was a sliver of the British economy when Watt started working on it. But his work led to more work till that sliver had expanded into something bigger than the whole economy of which it had initially been a part.

The same thing could happen with the Internet. If Internet startups offer the best opportunity for ambitious people, then a lot of ambitious people will start them, and this bit of the economy will balloon in the usual fractal way.

Even if Internet-related applications only become a tenth of the world's economy, this component will set the tone for the rest. The most dynamic part of the economy always does, in everything from salaries to standards of dress. Not just because of its prestige, but because the principles underlying the most dynamic part of the economy tend to be ones that work.

For the future, the trend to bet on seems to be networks of small, autonomous groups whose performance is measured individually. And the societies that win will be the ones with the least impedance.

As with the original industrial revolution, some societies are going to be better at this than others. Within a generation of its birth in England, the Industrial Revolution had spread to continental Europe and North America. But it didn't spread everywhere. This new way of doing things could only take root in places that were prepared for it. It could only spread to places that already had a vigorous middle class.

There is a similar social component to the transformation that began in Silicon Valley in the 1960s. Two new kinds of techniques were developed there: techniques for building integrated circuits, and techniques for building a new type of company designed to grow fast by creating new technology. The techniques for building integrated circuits spread rapidly to other countries. But the techniques for building startups didn't. Fifty years later, startups are ubiquitous in Silicon Val-

ley and common in a handful of other US cities, but they're still an anomaly in most of the world.

Part of the reason—possibly the main reason—that startups have not spread as broadly as the Industrial Revolution did is their social disruptiveness. Though it brought many social changes, the Industrial Revolution was not fighting the principle that bigger is better. Quite the opposite: the two dovetailed beautifully. The new industrial companies adapted the customs of existing large organizations like the military and the civil service, and the resulting hybrid worked well. “Captains of industry” issued orders to “armies of workers,” and everyone knew what they were supposed to do.

Startups seem to go more against the grain, socially. It's hard for them to flourish in societies that value hierarchy and stability, just as it was hard for industrialization to flourish in societies ruled by people who stole at will from the merchant class. But there were already a handful of countries past that stage when the Industrial Revolution happened. There do not seem to be that many ready this time.

Thanks to Trevor Blackwell, Paul Buchheit, Jessica Livingston, and Robert Morris for reading drafts of this.

Chapter 34

Could VC be a Casualty of the Recession?

December 2008

(I originally wrote this at the request of a company producing a report about entrepreneurship. Unfortunately after reading it they decided it was too controversial to include.)

VC funding will probably dry up somewhat during the present recession, like it usually does in bad times. But this time the result may be different. This time the number of new startups may not decrease. And that could be dangerous for VCs.

When VC funding dried up after the Internet Bubble, startups dried up too. There were not a lot of new startups being founded in 2003. But startups aren't tied to VC the way they were 10 years ago. It's now possible for VCs and startups to diverge. And if they do, they may not reconverge once the economy gets better.

The reason startups no longer depend so much on VCs is one that everyone in the startup business knows by now: it has gotten much cheaper to start a startup. There are four main reasons: Moore's law has made hardware cheap; open source has made software free; the web has made marketing and distribution free; and more powerful programming languages mean development teams can be smaller. These changes have pushed the cost of starting a startup down into the noise. In a lot of startups—probably most startups funded by Y Combinator—the biggest expense is simply the founders' living expenses. We've had startups that were profitable on revenues of \$3000

a month.

\$3000 is insignificant as revenues go. Why should anyone care about a startup making \$3000 a month? Because, although insignificant as *revenue*, this amount of money can change a startup's *funding* situation completely.

Someone running a startup is always calculating in the back of their mind how much “runway” they have—how long they have till the money in the bank runs out and they either have to be profitable, raise more money, or go out of business. Once you cross the threshold of profitability, however low, your runway becomes infinite. It's a qualitative change, like the stars turning into lines and disappearing when the Enterprise accelerates to warp speed. Once you're profitable you don't need investors' money. And because Internet startups have become so cheap to run, the threshold of profitability can be trivially low. Which means many Internet startups don't need VC-scale investments anymore. For many startups, VC funding has, in the language of VCs, gone from a must-have to a nice-to-have.

This change happened while no one was looking, and its effects have been largely masked so far. It was during the trough after the Internet Bubble that it became trivially cheap to start a startup, but few realized it because startups were so out of fashion. When startups came back into fashion, around 2005, investors were starting to write checks again. And while founders may not have needed VC money the way they used to, they were willing to take it if offered—partly because there was a tradition of startups taking VC money, and partly because startups, like dogs, tend to eat when given the opportunity. As long as VCs were writing checks, founders were never forced to explore the limits of how little they needed them. There were a few startups who hit these limits accidentally because of their unusual circumstances—most famously 37signals, which hit the limit because they crossed into startup land from the other direction: they started as a consulting firm, so they had revenue before they had a product.

Vcs and founders are like two components that used to be bolted to-

gether. Around 2000 the bolt was removed. Because the components have so far been subjected to the same forces, they still seem to be joined together, but really one is just resting on the other. A sharp impact would make them fly apart. And the present recession could be that impact.

Because of Y Combinator's position at the extreme end of the spectrum, we'd be the first to see signs of a separation between founders and investors, and we are in fact seeing it. For example, though the stock market crash does seem to have made investors more cautious, it doesn't seem to have had any effect on the number of people who want to start startups. We take applications for funding every 6 months. Applications for the current funding cycle closed on October 17, well after the markets tanked, and even so we got a record number, up 40% from the same cycle a year before.

Maybe things will be different a year from now, if the economy continues to get worse, but so far there is zero slackening of interest among potential founders. That's different from the way things felt in 2001. Then there was a widespread feeling among potential founders that startups were over, and that one should just go to grad school. That isn't happening this time, and part of the reason is that even in a bad economy it's not that hard to build something that makes \$3000 a month. If investors stop writing checks, who cares?

We also see signs of a divergence between founders and investors in the attitudes of existing startups we've funded. I was talking to one recently that had a round fall through at the last minute over the sort of trifle that breaks deals when investors feel they have the upper hand—over an uncertainty about whether the founders had correctly filed their 83(b) forms, if you can believe that. And yet this startup is obviously going to succeed: their traffic and revenue graphs look like a jet taking off. So I asked them if they wanted me to introduce them to more investors. To my surprise, they said no—that they'd just spent four months dealing with investors, and they were actually a lot happier now that they didn't have to. There was a friend they wanted

to hire with the investor money, and now they'd have to postpone that. But otherwise they felt they had enough in the bank to make it to profitability. To make sure, they were moving to a cheaper apartment. And in this economy I bet they got a good deal on it.

I've detected this "investors aren't worth the trouble" vibe from several YC founders I've talked to recently. At least one startup from the most recent (summer) cycle may not even raise angel money, let alone VC. Ticketstumbler made it to profitability on Y Combinator's \$15,000 investment and they hope not to need more. This surprised even us. Although YC is based on the idea of it being cheap to start a startup, we never anticipated that founders would grow successful startups on nothing more than YC funding.

If founders decide VCs aren't worth the trouble, that could be bad for VCs. When the economy bounces back in a few years and they're ready to write checks again, they may find that founders have moved on.

There is a founder community just as there's a VC community. They all know one another, and techniques spread rapidly between them. If one tries a new programming language or a new hosting provider and gets good results, 6 months later half of them are using it. And the same is true for funding. The current generation of founders want to raise money from VCs, and Sequoia specifically, because Larry and Sergey took money from VCs, and Sequoia specifically. Imagine what it would do to the VC business if the next hot company didn't take VC at all.

VCs think they're playing a zero sum game. In fact, it's not even that. If you lose a deal to Benchmark, you lose that deal, but VC as an industry still wins. If you lose a deal to None, all VCs lose.

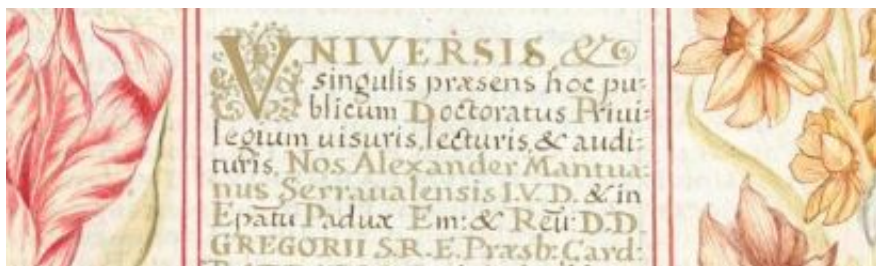
This recession may be different from the one after the Internet Bubble. This time founders may keep starting startups. And if they do, VCs will have to keep writing checks, or they could become irrelevant.

Thanks to Sam Altman, Trevor Blackwell, David Hornik, Jessica Liv-

ington, Robert Morris, and Fred Wilson for reading drafts of this.

Chapter 35

After Credentials



December 2008

A few months ago I read a *New York Times* article on South Korean cram schools that said

Admission to the right university can make or break an ambitious young South Korean.

A parent added:

“In our country, college entrance exams determine 70 to 80 percent of a person’s future.”

It was striking how old fashioned this sounded. And yet when I was in high school it wouldn’t have seemed too far off as a description of the US. Which means things must have been changing here.

The course of people’s lives in the US now seems to be determined less by credentials and more by performance than it was 25 years ago. Where you go to college still matters, but not like it used to.

What happened?

Judging people by their academic credentials was in its time an advance. The practice seems to have begun in China, where starting in 587 candidates for the imperial civil service had to take an exam on classical literature.¹ It was also a test of wealth, because the knowledge it tested was so specialized that passing required years of expensive training. But though wealth was a necessary condition for passing, it was not a sufficient one. By the standards of the rest of the world in 587, the Chinese system was very enlightened. Europeans didn't introduce formal civil service exams till the nineteenth century, and even then they seem to have been influenced by the Chinese example.

Before credentials, government positions were obtained mainly by family influence, if not outright bribery. It was a great step forward to judge people by their performance on a test. But by no means a perfect solution. When you judge people that way, you tend to get cram schools—which they did in Ming China and nineteenth century England just as much as in present day South Korea.

What cram schools are, in effect, is leaks in a seal. The use of credentials was an attempt to seal off the direct transmission of power between generations, and cram schools represent that power finding holes in the seal. Cram schools turn wealth in one generation into credentials in the next.

It's hard to beat this phenomenon, because the schools adjust to suit whatever the tests measure. When the tests are narrow and predictable, you get cram schools on the classic model, like those that prepared candidates for Sandhurst (the British West Point) or the classes American students take now to improve their SAT scores. But as the

¹Miyazaki, Ichisada (Conrad Schirokauer trans.), *China's Examination Hell: The Civil Service Examinations of Imperial China*, Yale University Press, 1981.

Scribes in ancient Egypt took exams, but they were more the type of proficiency test any apprentice might have to pass.

tests get broader, the schools do too. Preparing a candidate for the Chinese imperial civil service exams took years, as prep school does today. But the *raison d'être* of all these institutions has been the same: to beat the system.²

History suggests that, all other things being equal, a society prospers in proportion to its ability to prevent parents from influencing their children's success directly. It's a fine thing for parents to help their children indirectly—for example, by helping them to become smarter or more disciplined, which then makes them more successful. The problem comes when parents use direct methods: when they are able to use their own wealth or power as a substitute for their children's qualities.

Parents will tend to do this when they can. Parents will die for their kids, so it's not surprising to find they'll also push their scruples to the limits for them. Especially if other parents are doing it.

Sealing off this force has a double advantage. Not only does a society get "the best man for the job," but parents' ambitions are diverted from direct methods to indirect ones—to actually trying to raise their kids well.

But we should expect it to be very hard to contain parents' efforts to obtain an unfair advantage for their kids. We're dealing with one of the most powerful forces in human nature. We shouldn't expect naive solutions to work, any more than we'd expect naive solutions for keeping heroin out of a prison to work.

The obvious way to solve the problem is to make credentials better. If

²When I say the *raison d'être* of prep schools is to get kids into better colleges, I mean this in the narrowest sense. I'm not saying that's all prep schools do, just that if they had zero effect on college admissions there would be far less demand for them.

the tests a society uses are currently hackable, we can study the way people beat them and try to plug the holes. You can use the cram schools to show you where most of the holes are. They also tell you when you're succeeding in fixing them: when cram schools become less popular.

A more general solution would be to push for increased transparency, especially at critical social bottlenecks like college admissions. In the US this process still shows many outward signs of corruption. For example, legacy admissions. The official story is that legacy status doesn't carry much weight, because all it does is break ties: applicants are bucketed by ability, and legacy status is only used to decide between the applicants in the bucket that straddles the cutoff. But what this means is that a university can make legacy status have as much or as little weight as they want, by adjusting the size of the bucket that straddles the cutoff.

By gradually chipping away at the abuse of credentials, you could probably make them more airtight. But what a long fight it would be. Especially when the institutions administering the tests don't really want them to be airtight.

Fortunately there's a better way to prevent the direct transmission of power between generations. Instead of trying to make credentials harder to hack, we can also make them matter less.

Let's think about what credentials are for. What they are, functionally, is a way of predicting performance. If you could measure actual performance, you wouldn't need them.

So why did they even evolve? Why haven't we just been measuring actual performance? Think about where credentialism first appeared: in selecting candidates for large organizations. Individual performance is hard to measure in large organizations, and the harder performance is to measure, the more important it is to predict it. If an organization could immediately and cheaply measure the perfor-

mance of recruits, they wouldn't need to examine their credentials. They could take everyone and keep just the good ones.

Large organizations can't do this. But a bunch of small organizations in a market can come close. A market takes every organization and keeps just the good ones. As organizations get smaller, this approaches taking every person and keeping just the good ones. So all other things being equal, a society consisting of more, smaller organizations will care less about credentials.

That's what's been happening in the US. That's why those quotes from Korea sound so old fashioned. They're talking about an economy like America's a few decades ago, dominated by a few big companies. The route for the ambitious in that sort of environment is to join one and climb to the top. Credentials matter a lot then. In the culture of a large organization, an elite pedigree becomes a self-fulfilling prophecy.

This doesn't work in small companies. Even if your colleagues were impressed by your credentials, they'd soon be parted from you if your performance didn't match, because the company would go out of business and the people would be dispersed.

In a world of small companies, performance is all anyone cares about. People hiring for a startup don't care whether you've even graduated from college, let alone which one. All they care about is what you can do. Which is in fact all that should matter, even in a large organization. The reason credentials have such prestige is that for so long the large organizations in a society tended to be the most powerful. But in the US at least they don't have the monopoly on power they once did, precisely because they can't measure (and thus reward) individual performance. Why spend twenty years climbing the corporate ladder when you can get rewarded directly by the market?

I realize I see a more exaggerated version of the change than most other people. As a partner at an early stage venture funding firm, I'm like a jumpmaster shoving people out of the old world of credentials

and into the new one of performance. I'm an agent of the change I'm seeing. But I don't think I'm imagining it. It was not so easy 25 years ago for an ambitious person to choose to be judged directly by the market. You had to go through bosses, and they were influenced by where you'd been to college.

What made it possible for small organizations to succeed in America? I'm still not entirely sure. Startups are certainly a large part of it. Small organizations can develop new ideas faster than large ones, and new ideas are increasingly valuable.

But I don't think startups account for all the shift from credentials to measurement. My friend Julian Weber told me that when he went to work for a New York law firm in the 1950s they paid associates far less than firms do today. Law firms then made no pretense of paying people according to the value of the work they'd done. Pay was based on seniority. The younger employees were paying their dues. They'd be rewarded later.

The same principle prevailed at industrial companies. When my father was working at Westinghouse in the 1970s, he had people working for him who made more than he did, because they'd been there longer.

Now companies increasingly have to pay employees market price for the work they do. One reason is that employees no longer trust companies to deliver deferred rewards: why work to accumulate deferred rewards at a company that might go bankrupt, or be taken over and have all its implicit obligations wiped out? The other is that some companies broke ranks and started to pay young employees large amounts. This was particularly true in consulting, law, and finance, where it led to the phenomenon of yuppies. The word is rarely used today because it's no longer surprising to see a 25 year old with money, but in 1985 the sight of a 25 year old *professional* able to afford a new BMW was so novel that it called forth a new word.

The classic yuppie worked for a small organization. He didn't work for General Widget, but for the law firm that handled General Widget's acquisitions or the investment bank that floated their bond issues.

Startups and yuppies entered the American conceptual vocabulary roughly simultaneously in the late 1970s and early 1980s. I don't think there was a causal connection. Startups happened because technology started to change so fast that big companies could no longer keep a lid on the smaller ones. I don't think the rise of yuppies was inspired by it; it seems more as if there was a change in the social conventions (and perhaps the laws) governing the way big companies worked. But the two phenomena rapidly fused to produce a principle that now seems obvious: paying energetic young people market rates, and getting correspondingly high performance from them.

At about the same time the US economy rocketed out of the doldrums that had afflicted it for most of the 1970s. Was there a connection? I don't know enough to say, but it felt like it at the time. There was a lot of energy released.

Countries worried about their competitiveness are right to be concerned about the number of startups started within them. But they would do even better to examine the underlying principle. Do they let energetic young people get paid market rate for the work they do? The young are the test, because when people aren't rewarded according to performance, they're invariably rewarded according to seniority instead.

All it takes is a few beachheads in your economy that pay for performance. Measurement spreads like heat. If one part of a society is better at measurement than others, it tends to push the others to do better. If people who are young but smart and driven can make more by starting their own companies than by working for existing ones, the existing companies are forced to pay more to keep them. So market

rates gradually permeate every organization, even the government.³

The measurement of performance will tend to push even the organizations issuing credentials into line. When we were kids I used to annoy my sister by ordering her to do things I knew she was about to do anyway. As credentials are superseded by performance, a similar role is the best former gatekeepers can hope for. Once credential granting institutions are no longer in the self-fulfilling prophecy business, they'll have to work harder to predict the future.

Credentials are a step beyond bribery and influence. But they're not the final step. There's an even better way to block the transmission of power between generations: to encourage the trend toward an economy made of more, smaller units. Then you can measure what credentials merely predict.

No one likes the transmission of power between generations—not the left or the right. But the market forces favored by the right turn out to be a better way of preventing it than the credentials the left are forced to fall back on.

The era of credentials began to end when the power of large organizations peaked in the late twentieth century. Now we seem to be entering a new era based on measurement. The reason the new model has advanced so rapidly is that it works so much better. It shows no sign of slowing.

Thanks to Trevor Blackwell, Sarah Harlin, Jessica Livingston, and David Sloo for reading drafts of this.

³Progressive tax rates will tend to damp this effect, however, by decreasing the difference between good and bad measurers.

Chapter 36

Keep Your Identity Small

February 2009

I finally realized today why politics and religion yield such uniquely useless discussions.

As a rule, any mention of religion on an online forum degenerates into a religious argument. Why? Why does this happen with religion and not with Javascript or baking or other topics people talk about on forums?

What's different about religion is that people don't feel they need to have any particular expertise to have opinions about it. All they need is strongly held beliefs, and anyone can have those. No thread about Javascript will grow as fast as one about religion, because people feel they have to be over some threshold of expertise to post comments about that. But on religion everyone's an expert.

Then it struck me: this is the problem with politics too. Politics, like religion, is a topic where there's no threshold of expertise for expressing an opinion. All you need is strong convictions.

Do religion and politics have something in common that explains this similarity? One possible explanation is that they deal with questions that have no definite answers, so there's no back pressure on people's opinions. Since no one can be proven wrong, every opinion is equally valid, and sensing this, everyone lets fly with theirs.

But this isn't true. There are certainly some political questions that have definite answers, like how much a new government policy will cost. But the more precise political questions suffer the same fate as

the vaguer ones.

I think what religion and politics have in common is that they become part of people's identity, and people can never have a fruitful argument about something that's part of their identity. By definition they're partisan.

Which topics engage people's identity depends on the people, not the topic. For example, a discussion about a battle that included citizens of one or more of the countries involved would probably degenerate into a political argument. But a discussion today about a battle that took place in the Bronze Age probably wouldn't. No one would know what side to be on. So it's not politics that's the source of the trouble, but identity. When people say a discussion has degenerated into a religious war, what they really mean is that it has started to be driven mostly by people's identities.¹

Because the point at which this happens depends on the people rather than the topic, it's a mistake to conclude that because a question tends to provoke religious wars, it must have no answer. For example, the question of the relative merits of programming languages often degenerates into a religious war, because so many programmers identify as X programmers or Y programmers. This sometimes leads people to conclude the question must be unanswerable—that all languages are equally good. Obviously that's false: anything else people make can be well or badly designed; why should this be uniquely impossible for programming languages? And indeed, you can have a fruitful discussion about the relative merits of programming languages, so long as you exclude people who respond from identity.

More generally, you can have a fruitful discussion about a topic only if it doesn't engage the identities of any of the participants. What makes politics and religion such minefields is that they engage so many people's identities. But you could in principle have a useful conversation

¹When that happens, it tends to happen fast, like a core going critical. The threshold for participating goes down to zero, which brings in more people. And they tend to say incendiary things, which draw more and angrier counterarguments.

about them with some people. And there are other topics that might seem harmless, like the relative merits of Ford and Chevy pickup trucks, that you couldn't safely talk about with others.

The most intriguing thing about this theory, if it's right, is that it explains not merely which kinds of discussions to avoid, but how to have better ideas. If people can't think clearly about anything that has become part of their identity, then all other things being equal, the best plan is to let as few things into your identity as possible.²

Most people reading this will already be fairly tolerant. But there is a step beyond thinking of yourself as *x* but tolerating *y*: not even to consider yourself an *x*. The more labels you have for yourself, the dumber they make you.

Thanks to Sam Altman, Trevor Blackwell, Paul Buchheit, and Robert Morris for reading drafts of this.

²There may be some things it's a net win to include in your identity. For example, being a scientist. But arguably that is more of a placeholder than an actual label—like putting NMI on a form that asks for your middle initial—because it doesn't commit you to believing anything in particular. A scientist isn't committed to believing in natural selection in the same way a biblical literalist is committed to rejecting it. All he's committed to is following the evidence wherever it leads.

Considering yourself a scientist is equivalent to putting a sign in a cupboard saying "this cupboard must be kept empty." Yes, strictly speaking, you're putting something in the cupboard, but not in the ordinary sense.

Chapter 37

Startups in 13 Sentences

Watch how this essay was written.

February 2009

One of the things I always tell startups is a principle I learned from Paul Buchheit: it's better to make a few people really happy than to make a lot of people semi-happy. I was saying recently to a reporter that if I could only tell startups 10 things, this would be one of them. Then I thought: what would the other 9 be?

When I made the list there turned out to be 13:

1. Pick good cofounders.

Cofounders are for a startup what location is for real estate. You can change anything about a house except where it is. In a startup you can change your idea easily, but changing your cofounders is hard.¹ And the success of a startup is almost always a function of its founders.

2. Launch fast.

The reason to launch fast is not so much that it's critical to get your product to market early, but that you haven't really started working on it till you've launched. Launching teaches you what you should have been building. Till you know that you're wasting your time. So the main value of whatever you launch with is as a pretext for engaging users.

3. Let your idea evolve.

¹Strictly speaking it's impossible without a time machine.

This is the second half of launching fast. Launch fast and iterate. It's a big mistake to treat a startup as if it were merely a matter of implementing some brilliant initial idea. As in an essay, most of the ideas appear in the implementing.

4. Understand your users.

You can envision the wealth created by a startup as a rectangle, where one side is the number of users and the other is how much you improve their lives.² The second dimension is the one you have most control over. And indeed, the growth in the first will be driven by how well you do in the second. As in science, the hard part is not answering questions but asking them: the hard part is seeing something new that users lack. The better you understand them the better the odds of doing that. That's why so many successful startups make something the founders needed.

5. Better to make a few users love you than a lot ambivalent.

Ideally you want to make large numbers of users love you, but you can't expect to hit that right away. Initially you have to choose between satisfying all the needs of a subset of potential users, or satisfying a subset of the needs of all potential users. Take the first. It's easier to expand userwise than satisfactionwise. And perhaps more importantly, it's harder to lie to yourself. If you think you're 85% of the way to a great product, how do you know it's not 70%? Or 10%? Whereas it's easy to know how many users you have.

6. Offer surprisingly good customer service.

Customers are used to being maltreated. Most of the companies they deal with are quasi-monopolies that get away with atrocious customer service. Your own ideas about what's possible have been unconsciously lowered by such experiences. Try making your customer service not merely good, but surprisingly good. Go out of your way to make people happy. They'll be overwhelmed; you'll see. In the

²In practice it's more like a ragged comb.

earliest stages of a startup, it pays to offer customer service on a level that wouldn't scale, because it's a way of learning about your users.

7. You make what you measure.

I learned this one from Joe Kraus.³ Merely measuring something has an uncanny tendency to improve it. If you want to make your user numbers go up, put a big piece of paper on your wall and every day plot the number of users. You'll be delighted when it goes up and disappointed when it goes down. Pretty soon you'll start noticing what makes the number go up, and you'll start to do more of that. Corollary: be careful what you measure.

8. Spend little.

I can't emphasize enough how important it is for a startup to be cheap. Most startups fail before they make something people want, and the most common form of failure is running out of money. So being cheap is (almost) interchangeable with iterating rapidly.⁴ But it's more than that. A culture of cheapness keeps companies young in something like the way exercise keeps people young.

9. Get ramen profitable.

"Ramen profitable" means a startup makes just enough to pay the founders' living expenses. It's not rapid prototyping for business models (though it can be), but more a way of hacking the investment process. Once you cross over into ramen profitable, it completely changes your relationship with investors. It's also great for morale.

10. Avoid distractions.

Nothing kills startups like distractions. The worst type are those that pay money: day jobs, consulting, profitable side-projects. The startup may have more long-term potential, but you'll always interrupt work-

³Joe thinks one of the founders of Hewlett Packard said it first, but he doesn't remember which.

⁴They'd be interchangeable if markets stood still. Since they don't, working twice as fast is better than having twice as much time.

ing on it to answer calls from people paying you now. Paradoxically, fundraising is this type of distraction, so try to minimize that too.

11. Don't get demoralized.

Though the immediate cause of death in a startup tends to be running out of money, the underlying cause is usually lack of focus. Either the company is run by stupid people (which can't be fixed with advice) or the people are smart but got demoralized. Starting a startup is a huge moral weight. Understand this and make a conscious effort not to be ground down by it, just as you'd be careful to bend at the knees when picking up a heavy box.

12. Don't give up.

Even if you get demoralized, don't give up. You can get surprisingly far by just not giving up. This isn't true in all fields. There are a lot of people who couldn't become good mathematicians no matter how long they persisted. But startups aren't like that. Sheer effort is usually enough, so long as you keep morphing your idea.

13. Deals fall through.

One of the most useful skills we learned from Viaweb was not getting our hopes up. We probably had 20 deals of various types fall through. After the first 10 or so we learned to treat deals as background processes that we should ignore till they terminated. It's very dangerous to morale to start to depend on deals closing, not just because they so often don't, but because it makes them less likely to.

Having gotten it down to 13 sentences, I asked myself which I'd choose if I could only keep one.

Understand your users. That's the key. The essential task in a startup is to create wealth; the dimension of wealth you have most control over is how much you improve users' lives; and the hardest part of that is knowing what to make for them. Once you know what to make, it's mere effort to make it, and most decent hackers are capable of that.

Understanding your users is part of half the principles in this list. That's the reason to launch early, to understand your users. Evolving your idea is the embodiment of understanding your users. Understanding your users well will tend to push you toward making something that makes a few people deeply happy. The most important reason for having surprisingly good customer service is that it helps you understand your users. And understanding your users will even ensure your morale, because when everything else is collapsing around you, having just ten users who love you will keep you going.

What I've Learned from Hacker News

February 2009

Hacker News was two years old last week. Initially it was supposed to be a side project—an application to sharpen Arc on, and a place for current and future Y Combinator founders to exchange news. It's grown bigger and taken up more time than I expected, but I don't regret that because I've learned so much from working on it.

Growth

When we launched in February 2007, weekday traffic was around 1600 daily uniques. It's since grown to around 22,000. This growth rate is a bit higher than I'd like. I'd like the site to grow, since a site that isn't growing at least slowly is probably dead. But I wouldn't want it to grow as large as Digg or Reddit—mainly because that would dilute the character of the site, but also because I don't want to spend all my time dealing with scaling.

I already have problems enough with that. Remember, the original motivation for HN was to test a new programming language, and moreover one that's focused on experimenting with language design, not performance. Every time the site gets slow, I fortify myself by recalling McIlroy and Bentley's famous quote

The key to performance is elegance, not battalions of special cases.

and look for the bottleneck I can remove with least code. So far I've

been able to keep up, in the sense that performance has remained consistently mediocre despite 14x growth. I don't know what I'll do next, but I'll probably think of something.

This is my attitude to the site generally. Hacker News is an experiment, and an experiment in a very young field. Sites of this type are only a few years old. Internet conversation generally is only a few decades old. So we've probably only discovered a fraction of what we eventually will.

That's why I'm so optimistic about HN. When a technology is this young, the existing solutions are usually terrible; which means it must be possible to do much better; which means many problems that seem insoluble aren't. Including, I hope, the problem that has afflicted so many previous communities: being ruined by growth.

Dilution

Users have worried about that since the site was a few months old. So far these alarms have been false, but they may not always be. Dilution is a hard problem. But probably soluble; it doesn't mean much that open conversations have "always" been destroyed by growth when "always" equals 20 instances.

But it's important to remember we're trying to solve a new problem, because that means we're going to have to try new things, most of which probably won't work. A couple weeks ago I tried displaying the names of users with the highest average comment scores in orange.

¹ That was a mistake. Suddenly a culture that had been more or less united was divided into haves and have-nots. I didn't realize how united the culture had been till I saw it divided. It was painful to watch.

²

¹I tried ranking users by both average and median comment score, and average (with the high score thrown out) seemed the more accurate predictor of high quality. Median may be the more accurate predictor of low quality though.

²Another thing I learned from this experiment is that if you're going to distinguish between people, you better be sure you do it right. This is one problem where rapid prototyping doesn't work.

So orange usernames won't be back. (Sorry about that.) But there will be other equally broken-seeming ideas in the future, and the ones that turn out to work will probably seem just as broken as those that don't.

Probably the most important thing I've learned about dilution is that it's measured more in behavior than users. It's bad behavior you want to keep out more than bad people. User behavior turns out to be surprisingly malleable. If people are expected to behave well, they tend to; and vice versa.

Though of course forbidding bad behavior does tend to keep away bad people, because they feel uncomfortably constrained in a place where they have to behave well. But this way of keeping them out is gentler and probably also more effective than overt barriers.

It's pretty clear now that the broken windows theory applies to community sites as well. The theory is that minor forms of bad behavior encourage worse ones: that a neighborhood with lots of graffiti and broken windows becomes one where robberies occur. I was living in New York when Giuliani introduced the reforms that made the broken windows theory famous, and the transformation was miraculous. And I was a Reddit user when the opposite happened there, and the transformation was equally dramatic.

I'm not criticizing Steve and Alexis. What happened to Reddit didn't happen out of neglect. From the start they had a policy of censoring nothing except spam. Plus Reddit had different goals from Hacker News. Reddit was a startup, not a side project; its goal was to grow as fast as possible. Combine rapid growth and zero censorship, and the result is a free for all. But I don't think they'd do much differently if they were doing it again. Measured by traffic, Reddit is much more successful than Hacker News.

Indeed, that's the intellectually honest argument for not discriminating between various types of people. The reason not to do it is not that everyone's the same, but that it's bad to do wrong and hard to do right.

But what happened to Reddit won't inevitably happen to HN. There are several local maxima. There can be places that are free for alls and places that are more thoughtful, just as there are in the real world; and people will behave differently depending on which they're in, just as they do in the real world.

I've observed this in the wild. I've seen people cross-posting on Reddit and Hacker News who actually took the trouble to write two versions, a flame for Reddit and a more subdued version for HN.

Submissions

There are two major types of problems a site like Hacker News needs to avoid: bad stories and bad comments. So far the danger of bad stories seems smaller. The stories on the frontpage now are still roughly the ones that would have been there when HN started.

I once thought I'd have to weight votes to keep crap off the frontpage, but I haven't had to yet. I wouldn't have predicted the frontpage would hold up so well, and I'm not sure why it has. Perhaps only the more thoughtful users care enough to submit and upvote links, so the marginal cost of one random new user approaches zero. Or perhaps the frontpage protects itself, by advertising what type of submission is expected.

The most dangerous thing for the frontpage is stuff that's too easy to upvote. If someone proves a new theorem, it takes some work by the reader to decide whether or not to upvote it. An amusing cartoon takes less. A rant with a rallying cry as the title takes zero, because people vote it up without even reading it.

Hence what I call the Fluff Principle: on a user-voted news site, the links that are easiest to judge will take over unless you take specific measures to prevent it.

Hacker News has two kinds of protections against fluff. The most common types of fluff links are banned as off-topic. Pictures of kittens, political diatribes, and so on are explicitly banned. This keeps out

most fluff, but not all of it. Some links are both fluff, in the sense of being very short, and also on topic.

There's no single solution to that. If a link is just an empty rant, editors will sometimes kill it even if it's on topic in the sense of being about hacking, because it's not on topic by the real standard, which is to engage one's intellectual curiosity. If the posts on a site are characteristically of this type I sometimes ban it, which means new stuff at that url is auto-killed. If a post has a linkbait title, editors sometimes rephrase it to be more matter-of-fact. This is especially necessary with links whose titles are rallying cries, because otherwise they become implicit "vote up if you believe such-and-such" posts, which are the most extreme form of fluff.

The techniques for dealing with links have to evolve, because the links do. The existence of aggregators has already affected what they aggregate. Writers now deliberately write things to draw traffic from aggregators—sometimes even specific ones. (No, the irony of this statement is not lost on me.) Then there are the more sinister mutations, like linkjacking—posting a paraphrase of someone else's article and submitting that instead of the original. These can get a lot of up-votes, because a lot of what's good in an article often survives; indeed, the closer the paraphrase is to plagiarism, the more survives.³

I think it's important that a site that kills submissions provide a way for users to see what got killed if they want to. That keeps editors honest, and just as importantly, makes users confident they'd know if the editors stopped being honest. HN users can do this by flipping a switch called showdead in their profile.⁴

³When I catch egregiously linkjacked posts I replace the url with that of whatever they copied. Sites that habitually linkjack get banned.

⁴Digg is notorious for its lack of transparency. The root of the problem is not that the guys running Digg are especially sneaky, but that they use the wrong algorithm for generating their frontpage. Instead of bubbling up from the bottom as they get more votes, as on Reddit, stories start at the top and get pushed down by new arrivals.

The reason for the difference is that Digg is derived from Slashdot, while Reddit

Comments

Bad comments seem to be a harder problem than bad submissions. While the quality of links on the frontpage of HN hasn't changed much, the quality of the median comment may have decreased somewhat.

There are two main kinds of badness in comments: meanness and stupidity. There is a lot of overlap between the two—mean comments are disproportionately likely also to be dumb—but the strategies for dealing with them are different. Meanness is easier to control. You can have rules saying one shouldn't be mean, and if you enforce them it seems possible to keep a lid on meanness.

Keeping a lid on stupidity is harder, perhaps because stupidity is not so easily distinguishable. Mean people are more likely to know they're being mean than stupid people are to know they're being stupid.

The most dangerous form of stupid comment is not the long but mistaken argument, but the dumb joke. Long but mistaken arguments are actually quite rare. There is a strong correlation between comment quality and length; if you wanted to compare the quality of comments on community sites, average length would be a good predictor. Probably the cause is human nature rather than anything specific to comment threads. Probably it's simply that stupidity more often takes the form of having few ideas than wrong ones.

Whatever the cause, stupid comments tend to be short. And since it's hard to write a short comment that's distinguished for the amount of information it conveys, people try to distinguish them instead by

is derived from Delicious/popular. Digg is Slashdot with voting instead of editors, and Reddit is Delicious/popular with voting instead of bookmarking. (You can still see fossils of their origins in their graphic design.)

Digg's algorithm is very vulnerable to gaming, because any story that makes it onto the frontpage is the new top story. Which in turn forces Digg to respond with extreme countermeasures. A lot of startups have some kind of secret about the subterfuges they had to resort to in the early days, and I suspect Digg's is the extent to which the top stories were de facto chosen by human editors.

being funny. The most tempting format for stupid comments is the supposedly witty put-down, probably because put-downs are the easiest form of humor.⁵ So one advantage of forbidding meanness is that it also cuts down on these.

Bad comments are like kudzu: they take over rapidly. Comments have much more effect on new comments than submissions have on new submissions. If someone submits a lame article, the other submissions don't all become lame. But if someone posts a stupid comment on a thread, that sets the tone for the region around it. People reply to dumb jokes with dumb jokes.

Maybe the solution is to add a delay before people can respond to a comment, and make the length of the delay inversely proportional to some prediction of its quality. Then dumb threads would grow slower.⁶

People

I notice most of the techniques I've described are conservative: they're aimed at preserving the character of the site rather than enhancing it. I don't think that's a bias of mine. It's due to the shape of the problem. Hacker News had the good fortune to start out good, so in this case it's literally a matter of preservation. But I think this principle would also apply to sites with different origins.

The good things in a community site come from people more than

⁵The dialog on Beavis and Butthead was composed largely of these, and when I read comments on really bad sites I can hear them in their voices.

⁶I suspect most of the techniques for discouraging stupid comments have yet to be discovered. Xkcd implemented a particularly clever one in its IRC channel: don't allow the same thing twice. Once someone has said "fail," no one can ever say it again. This would penalize short comments especially, because they have less room to avoid collisions in.

Another promising idea is the stupid filter, which is just like a probabilistic spam filter, but trained on corpora of stupid and non-stupid comments instead.

You may not have to kill bad comments to solve the problem. Comments at the bottom of a long thread are rarely seen, so it may be enough to incorporate a prediction of quality in the comment sorting algorithm.

technology; it's mainly in the prevention of bad things that technology comes into play. Technology certainly can enhance discussion. Nested comments do, for example. But I'd rather use a site with primitive features and smart, nice users than a more advanced one whose users were idiots or trolls.

So the most important thing a community site can do is attract the kind of people it wants. A site trying to be as big as possible wants to attract everyone. But a site aiming at a particular subset of users has to attract just those—and just as importantly, repel everyone else. I've made a conscious effort to do this on HN. The graphic design is as plain as possible, and the site rules discourage dramatic link titles. The goal is that the only thing to interest someone arriving at HN for the first time should be the ideas expressed there.

The downside of tuning a site to attract certain people is that, to those people, it can be too attractive. I'm all too aware how addictive Hacker News can be. For me, as for many users, it's a kind of virtual town square. When I want to take a break from working, I walk into the square, just as I might into Harvard Square or University Ave in the physical world.⁷ But an online square is more dangerous than a physical one. If I spent half the day loitering on University Ave, I'd notice. I have to walk a mile to get there, and sitting in a cafe feels different from working. But visiting an online forum takes just a click, and feels superficially very much like working. You may be wasting your time, but you're not idle. Someone is wrong on the Internet, and you're fixing the problem.

Hacker News is definitely useful. I've learned a lot from things I've read on HN. I've written several essays that began as comments there. So I wouldn't want the site to go away. But I would like to be sure it's not a net drag on productivity. What a disaster that would be, to attract thousands of smart people to a site that caused them to waste lots of time. I wish I could be 100% sure that's not a description of HN.

⁷What makes most suburbs so demoralizing is that there's no center to walk to.

I feel like the addictiveness of games and social applications is still a mostly unsolved problem. The situation now is like it was with crack in the 1980s: we've invented terribly addictive new things, and we haven't yet evolved ways to protect ourselves from them. We will eventually, and that's one of the problems I hope to focus on next.

Thanks to Justin Kan, Jessica Livingston, Robert Morris, Alexis Ohanian, Emmet Shear, and Fred Wilson for reading drafts of this.

Can You Buy a Silicon Valley? Maybe.

February 2009

A lot of cities look at Silicon Valley and ask “How could we make something like that happen here?” The organic way to do it is to establish a first-rate university in a place where rich people want to live. That’s how Silicon Valley happened. But could you shortcut the process by funding startups?

Possibly. Let’s consider what it would take.

The first thing to understand is that encouraging startups is a different problem from encouraging startups in a particular city. The latter is much more expensive.

People sometimes think they could improve the startup scene in their town by starting something like Y Combinator there, but in fact it will have near zero effect. I know because Y Combinator itself had near zero effect on Boston when we were based there half the year. The people we funded came from all over the country (indeed, the world) and afterward they went wherever they could get more funding—which generally meant Silicon Valley.

The seed funding business is not a regional business, because at that stage startups are mobile. They’re just a couple founders with laptops.

¹

¹What people who start these supposedly local seed firms always find is that (a) their applicants come from all over, not just the local area, and (b) the local startups also apply to the other seed firms. So what ends up happening is that the applicant

If you want to encourage startups in a particular city, you have to fund startups that won't leave. There are two ways to do that: have rules preventing them from leaving, or fund them at the point in their life when they naturally take root. The first approach is a mistake, because it becomes a filter for selecting bad startups. If your terms force startups to do things they don't want to, only the desperate ones will take your money.

Good startups will move to another city as a condition of funding. What they won't do is agree not to move the next time they need funding. So the only way to get them to stay is to give them enough that they never need to leave.

How much would that take? If you want to keep startups from leaving your town, you have to give them enough that they're not tempted by an offer from Silicon Valley VCs that requires them to move. A startup would be able to refuse such an offer if they had grown to the point where they were (a) rooted in your town and/or (b) so successful that VCs would fund them even if they didn't move.

How much would it cost to grow a startup to that point? A minimum of several hundred thousand dollars. Wufoo seem to have rooted themselves in Tampa on \$118k, but they're an extreme case. On average it would take at least half a million.

So if it seems too good to be true to think you could grow a local silicon valley by giving startups \$15-20k each like Y Combinator, that's because it is. To make them stick around you'd have to give them at least 20 times that much.

However, even that is an interesting prospect. Suppose to be on the safe side it would cost a million dollars per startup. If you could get startups to stick to your town for a million apiece, then for a billion dollars you could bring in a thousand startups. That probably wouldn't

pool gets partitioned by quality rather than geography.

push you past Silicon Valley itself, but it might get you second place.

For the price of a football stadium, any town that was decent to live in could make itself one of the biggest startup hubs in the world.

What's more, it wouldn't take very long. You could probably do it in five years. During the term of one mayor. And it would get easier over time, because the more startups you had in town, the less it would take to get new ones to move there. By the time you had a thousand startups in town, the VCs wouldn't be trying so hard to get them to move to Silicon Valley; instead they'd be opening local offices. Then you'd really be in good shape. You'd have started a self-sustaining chain reaction like the one that drives the Valley.

But now comes the hard part. You have to pick the startups. How do you do that? Picking startups is a rare and valuable skill, and the handful of people who have it are not readily hireable. And this skill is so hard to measure that if a government did try to hire people with it, they'd almost certainly get the wrong ones.

For example, a city could give money to a VC fund to establish a local branch, and let them make the choices. But only a bad VC fund would take that deal. They wouldn't *seem* bad to the city officials. They'd seem very impressive. But they'd be bad at picking startups. That's the characteristic failure mode of VCs. All VCs look impressive to limited partners. The difference between the good ones and the bad ones only becomes visible in the other half of their jobs: choosing and advising startups.²

What you really want is a pool of local angel investors—people investing money they made from their own startups. But unfortunately you run into a chicken and egg problem here. If your city isn't already a

²Interestingly, the bad VCs fail by choosing startups run by people like them—people who are good presenters, but have no real substance. It's a case of the fake leading the fake. And since everyone involved is so plausible, the LPs who invest in these funds have no idea what's happening till they measure their returns.

startup hub, there won't be people there who got rich from startups. And there is no way I can think of that a city could attract angels from outside. By definition they're rich. There's no incentive that would make them move.³

However, a city could select startups by piggybacking on the expertise of investors who weren't local. It would be pretty straightforward to make a list of the most eminent Silicon Valley angels and from that to generate a list of all the startups they'd invested in. If a city offered these companies a million dollars each to move, a lot of the earlier stage ones would probably take it.

Preposterous as this plan sounds, it's probably the most efficient way a city could select good startups.

It would hurt the startups somewhat to be separated from their original investors. On the other hand, the extra million dollars would give them a lot more runway.

Would the transplanted startups survive? Quite possibly. The only way to find out would be to try it. It would be a pretty cheap experiment, as civil expenditures go. Pick 30 startups that eminent angels have recently invested in, give them each a million dollars if they'll relocate to your city, and see what happens after a year. If they seem to be thriving, you can try importing startups on a larger scale.

Don't be too legalistic about the conditions under which they're allowed to leave. Just have a gentlemen's agreement.

Don't try to do it on the cheap and pick only 10 for the initial experiment. If you do this on too small a scale you'll just guarantee failure. Startups need to be around other startups. 30 would be enough to feel like a community.

Don't try to make them all work in some renovated warehouse you've

³Not even being a tax haven, I suspect. That makes some rich people move, but not the type who would make good angel investors in startups.

made into an “incubator.” Real startups prefer to work in their own spaces.

In fact, don’t impose any restrictions on the startups at all. Startup founders are mostly hackers, and hackers are much more constrained by gentlemen’s agreements than regulations. If they shake your hand on a promise, they’ll keep it. But show them a lock and their first thought is how to pick it.

Interestingly, the 30-startup experiment could be done by any sufficiently rich private citizen. And what pressure it would put on the city if it worked.⁴

Should the city take stock in return for the money? In principle they’re entitled to, but how would they choose valuations for the startups? You couldn’t just give them all the same valuation: that would be too low for some (who’d turn you down) and too high for others (because it might make their next round a “down round”). And since we’re assuming we’re doing this without being able to pick startups, we also have to assume we can’t value them, since that’s practically the same thing.

Another reason not to take stock in the startups is that startups are often involved in disreputable things. So are established companies, but they don’t get blamed for it. If someone gets murdered by someone they met on Facebook, the press will treat the story as if it were about Facebook. If someone gets murdered by someone they met at a supermarket, the press will just treat it as a story about a murder. So understand that if you invest in startups, they might build things that get used for pornography, or file-sharing, or the expression of unfashionable opinions. You should probably sponsor this project jointly with your political opponents, so they can’t use whatever the startups do as a club to beat you with.

⁴Thanks to Michael Keenan for pointing this out.

It would be too much of a political liability just to give the startups the money, though. So the best plan would be to make it convertible debt, but which didn't convert except in a really big round, like \$20 million.

How well this scheme worked would depend on the city. There are some towns, like Portland, that would be easy to turn into startup hubs, and others, like Detroit, where it would really be an uphill battle. So be honest with yourself about the sort of town you have before you try this.

It will be easier in proportion to how much your town resembles San Francisco. Do you have good weather? Do people live downtown, or have they abandoned the center for the suburbs? Would the city be described as "hip" and "tolerant," or as reflecting "traditional values?" Are there good universities nearby? Are there walkable neighborhoods? Would nerds feel at home? If you answered yes to all these questions, you might be able not only to pull off this scheme, but to do it for less than a million per startup.

I realize the chance of any city having the political will to carry out this plan is microscopically small. I just wanted to explore what it would take if one did. How hard would it be to jumpstart a silicon valley? It's fascinating to think this prize might be within the reach of so many cities. So even though they'll all still spend the money on the stadium, at least now someone can ask them: why did you choose to do that instead of becoming a serious rival to Silicon Valley?

Thanks to Trevor Blackwell, Jessica Livingston, Robert Morris, and Fred Wilson for reading drafts of this.

Chapter 40

Why TV Lost

March 2009

About twenty years ago people noticed computers and TV were on a collision course and started to speculate about what they'd produce when they converged. We now know the answer: computers. It's clear now that even by using the word "convergence" we were giving TV too much credit. This won't be convergence so much as replacement. People may still watch things they call "TV shows," but they'll watch them mostly on computers.

What decided the contest for computers? Four forces, three of which one could have predicted, and one that would have been harder to.

One predictable cause of victory is that the Internet is an open platform. Anyone can build whatever they want on it, and the market picks the winners. So innovation happens at hacker speeds instead of big company speeds.

The second is Moore's Law, which has worked its usual magic on Internet bandwidth.¹

The third reason computers won is piracy. Users prefer it not just

¹Thanks to Trevor Blackwell for this point. He adds: "I remember the eyes of phone companies gleaming in the early 90s when they talked about convergence. They thought most programming would be on demand, and they would implement it and make a lot of money. It didn't work out. They assumed that their local network infrastructure would be critical to do video on-demand, because you couldn't possibly stream it from a few data centers over the internet. At the time (1992) the entire cross-country Internet bandwidth wasn't enough for one video stream. But wide-area bandwidth increased more than they expected and they were beaten by iTunes and Hulu."

because it's free, but because it's more convenient. Bittorrent and YouTube have already trained a new generation of viewers that the place to watch shows is on a computer screen.²

The somewhat more surprising force was one specific type of innovation: social applications. The average teenage kid has a pretty much infinite capacity for talking to their friends. But they can't physically be with them all the time. When I was in high school the solution was the telephone. Now it's social networks, multiplayer games, and various messaging applications. The way you reach them all is through a computer.³ Which means every teenage kid (a) wants a computer with an Internet connection, (b) has an incentive to figure out how to use it, and (c) spends countless hours in front of it.

This was the most powerful force of all. This was what made everyone want computers. Nerds got computers because they liked them. Then gamers got them to play games on. But it was connecting to other people that got everyone else: that's what made even grandmas and 14 year old girls want computers.

After decades of running an IV drip right into their audience, people in the entertainment business had understandably come to think of them as rather passive. They thought they'd be able to dictate the way shows reached audiences. But they underestimated the force of their desire to connect with one another.

Facebook killed TV. That is wildly oversimplified, of course, but probably as close to the truth as you can get in three words.

²Copyright owners tend to focus on the aspect they see of piracy, which is the lost revenue. They therefore think what drives users to do it is the desire to get something for free. But iTunes shows that people will pay for stuff online, if you make it easy. A significant component of piracy is simply that it offers a better user experience.

³Or a phone that is actually a computer. I'm not making any predictions about the size of the device that will replace TV, just that it will have a browser and get data via the Internet.

The TV networks already seem, grudgingly, to see where things are going, and have responded by putting their stuff, grudgingly, online. But they're still dragging their heels. They still seem to wish people would watch shows on TV instead, just as newspapers that put their stories online still seem to wish people would wait till the next morning and read them printed on paper. They should both just face the fact that the Internet is the primary medium.

They'd be in a better position if they'd done that earlier. When a new medium arises that's powerful enough to make incumbents nervous, then it's probably powerful enough to win, and the best thing they can do is jump in immediately.

Whether they like it or not, big changes are coming, because the Internet dissolves the two cornerstones of broadcast media: synchronicity and locality. On the Internet, you don't have to send everyone the same signal, and you don't have to send it to them from a local source. People will watch what they want when they want it, and group themselves according to whatever shared interest they feel most strongly. Maybe their strongest shared interest will be their physical location, but I'm guessing not. Which means local TV is probably dead. It was an artifact of limitations imposed by old technology. If someone were creating an Internet-based TV company from scratch now, they might have some plan for shows aimed at specific regions, but it wouldn't be a top priority.

Synchronicity and locality are tied together. TV network affiliates care what's on at 10 because that delivers viewers for local news at 11. This connection adds more brittleness than strength, however: people don't watch what's on at 10 because they want to watch the news afterward.

TV networks will fight these trends, because they don't have sufficient flexibility to adapt to them. They're hemmed in by local affiliates in much the same way car companies are hemmed in by dealers and unions. Inevitably, the people running the networks will take the easy route and try to keep the old model running for a couple more years,

just as the record labels have done.

A recent article in the *Wall Street Journal* described how TV networks were trying to add more live shows, partly as a way to make viewers watch TV synchronously instead of watching recorded shows when it suited them. Instead of delivering what viewers want, they're trying to force them to change their habits to suit the networks' obsolete business model. That never works unless you have a monopoly or cartel to enforce it, and even then it only works temporarily.

The other reason networks like live shows is that they're cheaper to produce. There they have the right idea, but they haven't followed it to its conclusion. Live content can be way cheaper than networks realize, and the way to take advantage of dramatic decreases in cost is to increase volume. The networks are prevented from seeing this whole line of reasoning because they still think of themselves as being in the broadcast business—as sending one signal to everyone.⁴

Now would be a good time to start any company that competes with TV networks. That's what a lot of Internet startups are, though they may not have had this as an explicit goal. People only have so many leisure hours a day, and TV is premised on such long sessions (unlike Google, which prides itself on sending users on their way quickly) that anything that takes up their time is competing with it. But in addition to such indirect competitors, I think TV companies will increasingly face direct ones.

Even in cable TV, the long tail was lopped off prematurely by the threshold you had to get over to start a new channel. It will be longer on the Internet, and there will be more mobility within it. In this new world, the existing players will only have the advantages any big

⁴Emmett Shear writes: "I'd argue the long tail for sports may be even larger than the long tail for other kinds of content. Anyone can broadcast a high school football game that will be interesting to 10,000 people or so, even if the quality of production is not so good."

company has in its market.

That will change the balance of power between the networks and the people who produce shows. The networks used to be gatekeepers. They distributed your work, and sold advertising on it. Now the people who produce a show can distribute it themselves. The main value networks supply now is ad sales. Which will tend to put them in the position of service providers rather than publishers.

Shows will change even more. On the Internet there's no reason to keep their current format, or even the fact that they have a single format. Indeed, the more interesting sort of convergence that's coming is between shows and games. But on the question of what sort of entertainment gets distributed on the Internet in 20 years, I wouldn't dare to make any predictions, except that things will change a lot. We'll get whatever the most imaginative people can cook up. That's why the Internet won.

Thanks to Sam Altman, Trevor Blackwell, Nancy Cook, Michael Seibel, Emmett Shear, and Fred Wilson for reading drafts of this.

Chapter 41

How to Be an Angel Investor

March 2009

(This essay is derived from a talk at AngelConf.)

When we sold our startup in 1998 I thought one day I'd do some angel investing. Seven years later I still hadn't started. I put it off because it seemed mysterious and complicated. It turns out to be easier than I expected, and also more interesting.

The part I thought was hard, the mechanics of investing, really isn't. You give a startup money and they give you stock. You'll probably get either preferred stock, which means stock with extra rights like getting your money back first in a sale, or convertible debt, which means (on paper) you're lending the company money, and the debt converts to stock at the next sufficiently big funding round.¹

There are sometimes minor tactical advantages to using one or the other. The paperwork for convertible debt is simpler. But really it doesn't matter much which you use. Don't spend much time worrying about the details of deal terms, especially when you first start angel investing. That's not how you win at this game. When you hear people talking about a successful angel investor, they're not saying "He got a 4x liquidation preference." They're saying "He invested in Google."

¹Convertible debt can be either capped at a particular valuation, or can be done at a discount to whatever the valuation turns out to be when it converts. E.g. convertible debt at a discount of 30% means when it converts you get stock as if you'd invested at a 30% lower valuation. That can be useful in cases where you can't or don't want to figure out what the valuation should be. You leave it to the next investor. On the other hand, a lot of investors want to know exactly what they're getting, so they will only do convertible debt with a cap.

That's how you win: by investing in the right startups. That is so much more important than anything else that I worry I'm misleading you by even talking about other things.

Mechanics

Angel investors often syndicate deals, which means they join together to invest on the same terms. In a syndicate there is usually a "lead" investor who negotiates the terms with the startup. But not always: sometimes the startup cobbles together a syndicate of investors who approach them independently, and the startup's lawyer supplies the paperwork.

The easiest way to get started in angel investing is to find a friend who already does it, and try to get included in his syndicates. Then all you have to do is write checks.

Don't feel like you have to join a syndicate, though. It's not that hard to do it yourself. You can just use the standard series AA documents Wilson Sonsini and Y Combinator published online. You should of course have your lawyer review everything. Both you and the startup should have lawyers. But the lawyers don't have to create the agreement from scratch.²

When you negotiate terms with a startup, there are two numbers you care about: how much money you're putting in, and the valuation of the company. The valuation determines how much stock you get. If you put \$50,000 into a company at a pre-money valuation of \$1

²The expensive part of creating an agreement from scratch is not writing the agreement, but bickering at several hundred dollars an hour over the details. That's why the series AA paperwork aims at a middle ground. You can just start from the compromise you'd have reached after lots of back and forth.

When you fund a startup, both your lawyers should be specialists in startups. Do not use ordinary corporate lawyers for this. Their inexperience makes them overbuild: they'll create huge, overcomplicated agreements, and spend hours arguing over irrelevant things.

In the Valley, the top startup law firms are Wilson Sonsini, Orrick, Fenwick & West, Gunderson Dettmer, and Cooley Godward. In Boston the best are Goodwin Procter, Wilmer Hale, and Foley Hoag.

million, then the post-money valuation is \$1.05 million, and you get $.05/1.05$, or 4.76% of the company's stock.

If the company raises more money later, the new investor will take a chunk of the company away from all the existing shareholders just as you did. If in the next round they sell 10% of the company to a new investor, your 4.76% will be reduced to 4.28%.

That's ok. Dilution is normal. What saves you from being mistreated in future rounds, usually, is that you're in the same boat as the founders. They can't dilute you without diluting themselves just as much. And they won't dilute themselves unless they end up net ahead. So in theory, each further round of investment leaves you with a smaller share of an even more valuable company, till after several more rounds you end up with .5% of the company at the point where it IPOs, and you are very happy because your \$50,000 has become \$5 million.³

The agreement by which you invest should have provisions that let you contribute to future rounds to maintain your percentage. So it's your choice whether you get diluted.⁴ If the company does really

³Your mileage may vary.

⁴These anti-dilution provisions also protect you against tricks like a later investor trying to steal the company by doing another round that values the company at \$1. If you have a competent startup lawyer handle the deal for you, you should be protected against such tricks initially. But it could become a problem later. If a big VC firm wants to invest in the startup after you, they may try to make you take out your anti-dilution protections. And if they do the startup will be pressuring you to agree. They'll tell you that if you don't, you're going to kill their deal with the VC. I recommend you solve this problem by having a gentlemen's agreement with the founders: agree with them in advance that you're not going to give up your anti-dilution protections. Then it's up to them to tell VCs early on.

The reason you don't want to give them up is the following scenario. The VCs recapitalize the company, meaning they give it additional funding at a pre-money valuation of zero. This wipes out the existing shareholders, including both you and the founders. They then grant the founders lots of options, because they need them to stay around, but you get nothing.

Obviously this is not a nice thing to do. It doesn't happen often. Brand-name VCs wouldn't recapitalize a company just to steal a few percent from an angel. But

well, you eventually will, because eventually the valuations will get so high it's not worth it for you.

How much does an angel invest? That varies enormously, from \$10,000 to hundreds of thousands or in rare cases even millions. The upper bound is obviously the total amount the founders want to raise. The lower bound is 5-10% of the total or \$10,000, whichever is greater. A typical angel round these days might be \$150,000 raised from 5 people.

Valuations don't vary as much. For angel rounds it's rare to see a valuation lower than half a million or higher than 4 or 5 million. 4 million is starting to be VC territory.

How do you decide what valuation to offer? If you're part of a round led by someone else, that problem is solved for you. But what if you're investing by yourself? There's no real answer. There is no rational way to value an early stage startup. The valuation reflects nothing more than the strength of the company's bargaining position. If they really want you, either because they desperately need money, or you're someone who can help them a lot, they'll let you invest at a low valuation. If they don't need you, it will be higher. So guess. The startup may not have any more idea what the number should be than you do. ⁵

there's a continuum here. A less upstanding, lower-tier VC might be tempted to do it to steal a big chunk of stock.

I'm not saying you should always absolutely refuse to give up your anti-dilution protections. Everything is a negotiation. If you're part of a powerful syndicate, you might be able to give up legal protections and rely on social ones. If you invest in a deal led by a big angel like Ron Conway, for example, you're pretty well protected against being mistreated, because any VC would think twice before crossing him. This kind of protection is one of the reasons angels like to invest in syndicates.

⁵Don't invest so much, or at such a low valuation, that you end up with an excessively large share of a startup, unless you're sure your money will be the last they ever need. Later stage investors won't invest in a company if the founders don't have enough equity left to motivate them. I talked to a VC recently who said he'd met with a company he really liked, but he turned them down because investors already owned more than half of it. Those investors probably thought they'd been

Ultimately it doesn't matter much. When angels make a lot of money from a deal, it's not because they invested at a valuation of \$1.5 million instead of \$3 million. It's because the company was really successful.

I can't emphasize that too much. Don't get hung up on mechanics or deal terms. What you should spend your time thinking about is whether the company is good.

(Similarly, founders also should not get hung up on deal terms, but should spend their time thinking about how to make the company good.)

There's a second less obvious component of an angel investment: how much you're expected to help the startup. Like the amount you invest, this can vary a lot. You don't have to do anything if you don't want to; you could simply be a source of money. Or you can become a de facto employee of the company. Just make sure that you and the startup agree in advance about roughly how much you'll do for them.

Really hot companies sometimes have high standards for angels. The ones everyone wants to invest in practically audition investors, and only take money from people who are famous and/or will work hard for them. But don't feel like you have to put in a lot of time or you won't get to invest in any good startups. There is a surprising lack of correlation between how hot a deal a startup is and how well it ends up doing. Lots of hot startups will end up failing, and lots of startups no one likes will end up succeeding. And the latter are so desperate for money that they'll take it from anyone at a low valuation.⁶

Picking Winners

It would be nice to be able to pick those out, wouldn't it? The part of

pretty clever by getting such a large chunk of this desirable company, but in fact they were shooting themselves in the foot.

⁶At any given time I know of at least 3 or 4 YC alumni who I believe will be big successes but who are running on vapor, financially, because investors don't yet get what they're doing. (And no, unfortunately, I can't tell you who they are. I can't refer a startup to an investor I don't know.)

angel investing that has most effect on your returns, picking the right companies, is also the hardest. So you should practically ignore (or more precisely, archive, in the Gmail sense) everything I've told you so far. You may need to refer to it at some point, but it is not the central issue.

The central issue is picking the right startups. What "Make something people want" is for startups, "Pick the right startups" is for investors. Combined they yield "Pick the startups that will make something people want."

How do you do that? It's not as simple as picking startups that are already making something wildly popular. By then it's too late for angels. VCs will already be onto them. As an angel, you have to pick startups before they've got a hit—either because they've made something great but users don't realize it yet, like Google early on, or because they're still an iteration or two away from the big hit, like PayPal when they were making software for transferring money between PDAs.

To be a good angel investor, you have to be a good judge of potential. That's what it comes down to. VCs can be fast followers. Most of them don't try to predict what will win. They just try to notice quickly when something already is winning. But angels have to be able to predict.⁷

One interesting consequence of this fact is that there are a lot of people out there who have never even made an angel investment and yet are already better angel investors than they realize. Someone who doesn't know the first thing about the mechanics of venture funding but knows what a successful startup founder looks like is actually far ahead of someone who knows termsheets inside out, but thinks "hacker" means someone who breaks into computers. If you can recognize good startup founders by empathizing with them—if you both resonate at the same frequency—then you may already be a better

⁷There are some VCs who can predict instead of reacting. Not surprisingly, these are the most successful ones.

startup picker than the median professional VC.⁸

Paul Buchheit, for example, started angel investing about a year after me, and he was pretty much immediately as good as me at picking startups. My extra year of experience was rounding error compared to our ability to empathize with founders.

What makes a good founder? If there were a word that meant the opposite of hapless, that would be the one. Bad founders seem hapless. They may be smart, or not, but somehow events overwhelm them and they get discouraged and give up. Good founders make things happen the way they want. Which is not to say they force things to happen in a predefined way. Good founders have a healthy respect for reality. But they are relentlessly resourceful. That's the closest I can get to the opposite of hapless. You want to fund people who are relentlessly resourceful.

Notice we started out talking about things, and now we're talking about people. There is an ongoing debate between investors which is more important, the people, or the idea—or more precisely, the market. Some, like Ron Conway, say it's the people—that the idea will change, but the people are the foundation of the company. Whereas Marc Andreessen says he'd back ok founders in a hot market over great founders in a bad one.⁹

These two positions are not so far apart as they seem, because good people find good markets. Bill Gates would probably have ended up pretty rich even if IBM hadn't happened to drop the PC standard in

⁸It's somewhat sneaky of me to put it this way, because the median VC loses money. That's one of the most surprising things I've learned about VC while working on Y Combinator. Only a fraction of VCs even have positive returns. The rest exist to satisfy demand among fund managers for venture capital as an asset class. Learning this explained a lot about some of the VCs I encountered when we were working on Viaweb.

⁹VCs also generally say they prefer great markets to great people. But what they're really saying is they want both. They're so selective that they only even consider great people. So when they say they care above all about big markets, they mean that's how they choose between great people.

his lap.

I've thought a lot about the disagreement between the investors who prefer to bet on people and those who prefer to bet on markets. It's kind of surprising that it even exists. You'd expect opinions to have converged more.

But I think I've figured out what's going on. The three most prominent people I know who favor markets are Marc, Jawed Karim, and Joe Kraus. And all three of them, in their own startups, basically flew into a thermal: they hit a market growing so fast that it was all they could do to keep up with it. That kind of experience is hard to ignore. Plus I think they underestimate themselves: they think back to how easy it felt to ride that huge thermal upward, and they think "anyone could have done it." But that isn't true; they are not ordinary people.

So as an angel investor I think you want to go with Ron Conway and bet on people. Thermals happen, yes, but no one can predict them—not even the founders, and certainly not you as an investor. And only good people can ride the thermals if they hit them anyway.

Deal Flow

Of course the question of how to choose startups presumes you have startups to choose between. How do you find them? This is yet another problem that gets solved for you by syndicates. If you tag along on a friend's investments, you don't have to find startups.

The problem is not finding startups, exactly, but finding a stream of reasonably high quality ones. The traditional way to do this is through contacts. If you're friends with a lot of investors and founders, they'll send deals your way. The Valley basically runs on referrals. And once you start to become known as reliable, useful investor, people will refer lots of deals to you. I certainly will.

There's also a newer way to find startups, which is to come to events like Y Combinator's Demo Day, where a batch of newly created startups presents to investors all at once. We have two Demo Days a year,

one in March and one in August. These are basically mass referrals.

But events like Demo Day only account for a fraction of matches between startups and investors. The personal referral is still the most common route. So if you want to hear about new startups, the best way to do it is to get lots of referrals.

The best way to get lots of referrals is to invest in startups. No matter how smart and nice you seem, insiders will be reluctant to send you referrals until you've proven yourself by doing a couple investments. Some smart, nice guys turn out to be flaky, high-maintenance investors. But once you prove yourself as a good investor, the deal flow, as they call it, will increase rapidly in both quality and quantity. At the extreme, for someone like Ron Conway, it is basically identical with the deal flow of the whole Valley.

So if you want to invest seriously, the way to get started is to bootstrap yourself off your existing connections, be a good investor in the startups you meet that way, and eventually you'll start a chain reaction. Good investors are rare, even in Silicon Valley. There probably aren't more than a couple hundred serious angels in the whole Valley, and yet they're probably the single most important ingredient in making the Valley what it is. Angels are the limiting reagent in startup formation.

If there are only a couple hundred serious angels in the Valley, then by deciding to become one you could single-handedly make the pipeline for startups in Silicon Valley significantly wider. That is kind of mind-blowing.

Being Good

How do you be a good angel investor? The first thing you need is to be decisive. When we talk to founders about good and bad investors, one of the ways we describe the good ones is to say "he writes checks." That doesn't mean the investor says yes to everyone. Far from it. It means he makes up his mind quickly, and follows through. You may be thinking, how hard could that be? You'll see when you try it. It

follows from the nature of angel investing that the decisions are hard. You have to guess early, at the stage when the most promising ideas still seem counterintuitive, because if they were obviously good, VCs would already have funded them.

Suppose it's 1998. You come across a startup founded by a couple grad students. They say they're going to work on Internet search. There are already a bunch of big public companies doing search. How can these grad students possibly compete with them? And does search even matter anyway? All the search engines are trying to get people to start calling them "portals" instead. Why would you want to invest in a startup run by a couple of nobodies who are trying to compete with large, aggressive companies in an area they themselves have declared *passee*? And yet the grad students seem pretty smart. What do you do?

There's a hack for being decisive when you're inexperienced: ratchet down the size of your investment till it's an amount you wouldn't care too much about losing. For every rich person (you probably shouldn't try angel investing unless you think of yourself as rich) there's some amount that would be painless, though annoying, to lose. Till you feel comfortable investing, don't invest more than that per startup.

For example, if you have \$5 million in investable assets, it would probably be painless (though annoying) to lose \$15,000. That's less than .3% of your net worth. So start by making 3 or 4 \$15,000 investments. Nothing will teach you about angel investing like experience. Treat the first few as an educational expense. \$60,000 is less than a lot of graduate programs. Plus you get equity.

What's really uncool is to be strategically indecisive: to string founders along while trying to gather more information about the startup's trajectory.¹⁰ There's always a temptation to do that, because you just

¹⁰Founders rightly dislike the sort of investor who says he's interested in investing but doesn't want to lead. There are circumstances where this is an acceptable excuse, but more often than not what it means is "No, but if you turn out to be a hot deal, I want to be able to claim retroactively I said yes."

If you like a startup enough to invest in it, then invest in it. Just use the standard

have so little to go on, but you have to consciously resist it. In the long term it's to your advantage to be good.

The other component of being a good angel investor is simply to be a good person. Angel investing is not a business where you make money by screwing people over. Startups create wealth, and creating wealth is not a zero sum game. No one has to lose for you to win. In fact, if you mistreat the founders you invest in, they'll just get demoralized and the company will do worse. Plus your referrals will dry up. So I recommend being good.

The most successful angel investors I know are all basically good people. Once they invest in a company, all they want to do is help it. And they'll help people they haven't invested in too. When they do favors they don't seem to keep track of them. It's too much overhead. They just try to help everyone, and assume good things will flow back to them somehow. Empirically that seems to work.

Thanks to Sam Altman, Paul Buchheit, Jessica Livingston, Robert Morris, and Fred Wilson for reading drafts of this.

Chapter 42

Relentlessly Resourceful

March 2009

A couple days ago I finally got being a good startup founder down to two words: relentlessly resourceful.

Till then the best I'd managed was to get the opposite quality down to one: hapless. Most dictionaries say hapless means unlucky. But the dictionaries are not doing a very good job. A team that outplays its opponents but loses because of a bad decision by the referee could be called unlucky, but not hapless. Hapless implies passivity. To be hapless is to be battered by circumstances — to let the world have its way with you, instead of having your way with the world.¹

Unfortunately there's no antonym of hapless, which makes it difficult to tell founders what to aim for. "Don't be hapless" is not much of a rallying cry.

It's not hard to express the quality we're looking for in metaphors. The best is probably a running back. A good running back is not merely determined, but flexible as well. They want to get downfield, but they adapt their plans on the fly.

¹I think the reason the dictionaries are wrong is that the meaning of the word has shifted. No one writing a dictionary from scratch today would say that hapless meant unlucky. But a couple hundred years ago they might have. People were more at the mercy of circumstances in the past, and as a result a lot of the words we use for good and bad outcomes have origins in words about luck.

When I was living in Italy, I was once trying to tell someone that I hadn't had much success in doing something, but I couldn't think of the Italian word for success. I spent some time trying to describe the word I meant. Finally she said "Ah! Fortuna!"

Unfortunately this is just a metaphor, and not a useful one to most people outside the US. “Be like a running back” is no better than “Don’t be hapless.”

But finally I’ve figured out how to express this quality directly. I was writing a talk for investors, and I had to explain what to look for in founders. What would someone who was the opposite of hapless be like? They’d be relentlessly resourceful. Not merely relentless. That’s not enough to make things go your way except in a few mostly uninteresting domains. In any interesting domain, the difficulties will be novel. Which means you can’t simply plow through them, because you don’t know initially how hard they are; you don’t know whether you’re about to plow through a block of foam or granite. So you have to be resourceful. You have to keep trying new things.

Be relentlessly resourceful.

That sounds right, but is it simply a description of how to be successful in general? I don’t think so. This isn’t the recipe for success in writing or painting, for example. In that kind of work the recipe is more to be actively curious. Resourceful implies the obstacles are external, which they generally are in startups. But in writing and painting they’re mostly internal; the obstacle is your own obtuseness.²

There probably are other fields where “relentlessly resourceful” is the recipe for success. But though other fields may share it, I think this is the best short description we’ll find of what makes a good startup founder. I doubt it could be made more precise.

Now that we know what we’re looking for, that leads to other questions. For example, can this quality be taught? After four years of trying to teach it to people, I’d say that yes, surprisingly often it can. Not to everyone, but to many people.³ Some people are just constitutionally

²There are aspects of startups where the recipe is to be actively curious. There can be times when what you’re doing is almost pure discovery. Unfortunately these times are a small proportion of the whole. On the other hand, they are in research too.

³I’d almost say to most people, but I realize (a) I have no idea what most people

passive, but others have a latent ability to be relentlessly resourceful that only needs to be brought out.

This is particularly true of young people who have till now always been under the thumb of some kind of authority. Being relentlessly resourceful is definitely not the recipe for success in big companies, or in most schools. I don't even want to think what the recipe is in big companies, but it is certainly longer and messier, involving some combination of resourcefulness, obedience, and building alliances.

Identifying this quality also brings us closer to answering a question people often wonder about: how many startups there could be. There is not, as some people seem to think, any economic upper bound on this number. There's no reason to believe there is any limit on the amount of newly created wealth consumers can absorb, any more than there is a limit on the number of theorems that can be proven. So probably the limiting factor on the number of startups is the pool of potential founders. Some people would make good founders, and others wouldn't. And now that we can say what makes a good founder, we know how to put an upper bound on the size of the pool.

This test is also useful to individuals. If you want to know whether you're the right sort of person to start a startup, ask yourself whether you're relentlessly resourceful. And if you want to know whether to recruit someone as a cofounder, ask if they are.

You can even use it tactically. If I were running a startup, this would be the phrase I'd tape to the mirror. "Make something people want" is the destination, but "Be relentlessly resourceful" is how you get there.

Thanks to Trevor Blackwell and Jessica Livingston for reading drafts of this.

are like, and (b) I'm pathologically optimistic about people's ability to change.

Chapter 43

Five Founders

April 2009

Inc recently asked me who I thought were the 5 most interesting startup founders of the last 30 years. How do you decide who's the most interesting? The best test seemed to be influence: who are the 5 who've influenced me most? Who do I use as examples when I'm talking to companies we fund? Who do I find myself quoting?

1. Steve Jobs

I'd guess Steve is the most influential founder not just for me but for most people you could ask. A lot of startup culture is Apple culture. He was the original young founder. And while the concept of "insanely great" already existed in the arts, it was a novel idea to introduce into a company in the 1980s.

More remarkable still, he's stayed interesting for 30 years. People await new Apple products the way they'd await new books by a popular novelist. Steve may not literally design them, but they wouldn't happen if he weren't CEO.

Steve is clever and driven, but so are a lot of people in the Valley. What makes him unique is his sense of design. Before him, most companies treated design as a frivolous extra. Apple's competitors now know better.

2. TJ Rodgers

TJ Rodgers isn't as famous as Steve Jobs, but he may be the best writer among Silicon Valley CEOs. I've probably learned more from him about the startup way of thinking than from anyone else. Not so

much from specific things he's written as by reconstructing the mind that produced them: brutally candid; aggressively garbage-collecting outdated ideas; and yet driven by pragmatism rather than ideology.

The first essay of his that I read was so electrifying that I remember exactly where I was at the time. It was High Technology Innovation: Free Markets or Government Subsidies? and I was downstairs in the Harvard Square T Station. It felt as if someone had flipped on a light switch inside my head.

3. Larry & Sergey

I'm sorry to treat Larry and Sergey as one person. I've always thought that was unfair to them. But it does seem as if Google was a collaboration.

Before Google, companies in Silicon Valley already knew it was important to have the best hackers. So they claimed, at least. But Google pushed this idea further than anyone had before. Their hypothesis seems to have been that, in the initial stages at least, *all* you need is good hackers: if you hire all the smartest people and put them to work on a problem where their success can be measured, you win. All the other stuff—which includes all the stuff that business schools think business consists of—you can figure out along the way. The results won't be perfect, but they'll be optimal. If this was their hypothesis, it's now been verified experimentally.

4. Paul Buchheit

Few know this, but one person, Paul Buchheit, is responsible for three of the best things Google has done. He was the original author of Gmail, which is the most impressive thing Google has after search. He also wrote the first prototype of AdSense, and was the author of Google's mantra "Don't be evil."

PB made a point in a talk once that I now mention to every startup we fund: that it's better, initially, to make a small number of users really love you than a large number kind of like you. If I could tell startups

only ten sentences, this would be one of them.

Now he's cofounder of a startup called Friendfeed. It's only a year old, but already everyone in the Valley is watching them. Someone responsible for three of the biggest ideas at Google is going to come up with more.

5. Sam Altman

I was told I shouldn't mention founders of YC-funded companies in this list. But Sam Altman can't be stopped by such flimsy rules. If he wants to be on this list, he's going to be.

Honestly, Sam is, along with Steve Jobs, the founder I refer to most when I'm advising startups. On questions of design, I ask "What would Steve do?" but on questions of strategy or ambition I ask "What would Sama do?"

What I learned from meeting Sama is that the doctrine of the elect applies to startups. It applies way less than most people think: startup investing does not consist of trying to pick winners the way you might in a horse race. But there are a few people with such force of will that they're going to get whatever they want.

Chapter 44

The Founder Visa

April 2009

I usually avoid politics, but since we now seem to have an administration that's open to suggestions, I'm going to risk making one. The single biggest thing the government could do to increase the number of startups in this country is a policy that would cost nothing: establish a new class of visa for startup founders.

The biggest constraint on the number of new startups that get created in the US is not tax policy or employment law or even Sarbanes-Oxley. It's that we won't let the people who want to start them into the country.

Letting just 10,000 startup founders into the country each year could have a visible effect on the economy. If we assume 4 people per startup, which is probably an overestimate, that's 2500 new companies. *Each year.* They wouldn't all grow as big as Google, but out of 2500 some would come close.

By definition these 10,000 founders wouldn't be taking jobs from Americans: it could be part of the terms of the visa that they couldn't work for existing companies, only new ones they'd founded. In fact they'd cause there to be more jobs for Americans, because the companies they started would hire more employees as they grew.

The tricky part might seem to be how one defined a startup. But that could be solved quite easily: let the market decide. Startup investors work hard to find the best startups. The government could not do better than to piggyback on their expertise, and use investment by recognized startup investors as the test of whether a company was a

real startup.

How would the government decide who's a startup investor? The same way they decide what counts as a university for student visas. We'll establish our own accreditation procedure. We know who one another are.

10,000 people is a drop in the bucket by immigration standards, but would represent a huge increase in the pool of startup founders. I think this would have such a visible effect on the economy that it would make the legislator who introduced the bill famous. The only way to know for sure would be to try it, and that would cost practically nothing.

Thanks to Trevor Blackwell, Paul Buchheit, Jeff Clavier, David Hornik, Jessica Livingston, Greg Mcadoo, Aydin Senkut, and Fred Wilson for reading drafts of this.

Related:

Why Twitter is a Big Deal

April 2009

Om Malik is the most recent of many people to ask why Twitter is such a big deal.

The reason is that it's a new messaging protocol, where you don't specify the recipients. New protocols are rare. Or more precisely, new protocols that take off are. There are only a handful of commonly used ones: TCP/IP (the Internet), SMTP (email), HTTP (the web), and so on. So any new protocol is a big deal. But Twitter is a protocol owned by a private company. That's even rarer.

Curiously, the fact that the founders of Twitter have been slow to monetize it may in the long run prove to be an advantage. Because they haven't tried to control it too much, Twitter feels to everyone like previous protocols. One forgets it's owned by a private company. That must have made it easier for Twitter to spread.

Chapter 46

A Local Revolution?

April 2009

Recently I realized I'd been holding two ideas in my head that would explode if combined.

The first is that startups may represent a new economic phase, on the scale of the Industrial Revolution. I'm not sure of this, but there seems a decent chance it's true. People are dramatically more productive as founders or early employees of startups—imagine how much less Larry and Sergey would have achieved if they'd gone to work for a big company—and that scale of improvement can change social customs.

The second idea is that startups are a type of business that flourishes in certain places that specialize in it—that Silicon Valley specializes in startups in the same way Los Angeles specializes in movies, or New York in finance.¹

¹There are two very different types of startup: one kind that evolves naturally, and one kind that's called into being to “commercialize” a scientific discovery. Most computer/software startups are now the first type, and most pharmaceutical startups the second. When I talk about startups in this essay, I mean type I startups. There is no difficulty making type II startups spread: all you have to do is fund medical research labs; commercializing whatever new discoveries the boffins throw off is as straightforward as building a new airport. Type II startups neither require nor produce startup culture. But that means having type II startups won't get you type I startups. Philadelphia is a case in point: lots of type II startups, but hardly any type I.

Incidentally, Google may appear to be an instance of a type II startup, but it wasn't. Google is not pagerank commercialized. They could have used another algorithm and everything would have turned out the same. What made Google Google is that they cared about doing search well at a critical point in the evolution of the web.

What if both are true? What if startups are both a new economic phase and also a type of business that only flourishes in certain centers?

If so, this revolution is going to be particularly revolutionary. All previous revolutions have spread. Agriculture, cities, and industrialization all spread widely. If startups end up being like the movie business, with just a handful of centers and one dominant one, that's going to have novel consequences.

There are already signs that startups may not spread particularly well. The spread of startups seems to be proceeding slower than the spread of the Industrial Revolution, despite the fact that communication is so much faster now.

Within a few decades of the founding of Boulton & Watt there were steam engines scattered over northern Europe and North America. Industrialization didn't spread much beyond those regions for a while. It only spread to places where there was a strong middle class—countries where a private citizen could make a fortune without having it confiscated. Otherwise it wasn't worth investing in factories. But in a country with a strong middle class it was easy for industrial techniques to take root. An individual mine or factory owner could decide to install a steam engine, and within a few years he could probably find someone local to make him one. So steam engines spread fast. And they spread widely, because the locations of mines and factories were determined by features like rivers, harbors, and sources of raw materials.²

Startups don't seem to spread so well, partly because they're more a social than a technical phenomenon, and partly because they're not tied to geography. An individual European manufacturer could import industrial techniques and they'd work fine. This doesn't seem

²Watt didn't invent the steam engine. His critical invention was a refinement that made steam engines dramatically more efficient: the separate condenser. But that oversimplifies his role. He had such a different attitude to the problem and approached it with such energy that he transformed the field. Perhaps the most accurate way to put it would be to say that Watt reinvented the steam engine.

to work so well with startups: you need a community of expertise, as you do in the movie business.³ Plus there aren't the same forces driving startups to spread. Once railroads or electric power grids were invented, every region had to have them. An area without railroads or power was a rich potential market. But this isn't true with startups. There's no need for a Microsoft of France or Google of Germany.

Governments may decide they want to encourage startups locally, but government policy can't call them into being the way a genuine need could.

How will this all play out? If I had to predict now, I'd say that startups will spread, but very slowly, because their spread will be driven not by government policies (which won't work) or by market need (which doesn't exist) but, to the extent that it happens at all, by the same random factors that have caused startup culture to spread thus far. And such random factors will increasingly be outweighed by the pull of existing startup hubs.

Silicon Valley is where it is because William Shockley wanted to move back to Palo Alto, where he grew up, and the experts he lured west to work with him liked it so much they stayed. Seattle owes much of its position as a tech center to the same cause: Gates and Allen wanted to move home. Otherwise Albuquerque might have Seattle's place in the rankings. Boston is a tech center because it's the intellectual capital of the US and probably the world. And if Battery Ventures hadn't turned down Facebook, Boston would be significantly bigger now on the startup radar screen.

But of course it's not a coincidence that Facebook got funded in the Valley and not Boston. There are more and bolder investors in Silicon Valley than in Boston, and even undergrads know it.

³The biggest counterexample here is Skype. If you're doing something that would get shut down in the US, it becomes an advantage to be located elsewhere. That's why Kazaa took the place of Napster. And the expertise and connections the founders gained from running Kazaa helped ensure the success of Skype.

Boston's case illustrates the difficulty you'd have establishing a new startup hub this late in the game. If you wanted to create a startup hub by reproducing the way existing ones happened, the way to do it would be to establish a first-rate research university in a place so nice that rich people wanted to live there. Then the town would be hospitable to both groups you need: both founders and investors. That's the combination that yielded Silicon Valley. But Silicon Valley didn't have Silicon Valley to compete with. If you tried now to create a startup hub by planting a great university in a nice place, it would have a harder time getting started, because many of the best startups it produced would be sucked away to existing startup hubs.

Recently I suggested a potential shortcut: pay startups to move. Once you had enough good startups in one place, it would create a self-sustaining chain reaction. Founders would start to move there without being paid, because that was where their peers were, and investors would appear too, because that was where the deals were.

In practice I doubt any government would have the balls to try this, or the brains to do it right. I didn't mean it as a practical suggestion, but more as an exploration of the lower bound of what it would take to create a startup hub deliberately.

The most likely scenario is (1) that no government will successfully establish a startup hub, and (2) that the spread of startup culture will thus be driven by the random factors that have driven it so far, but (3) that these factors will be increasingly outweighed by the pull of existing startup hubs. Result: this revolution, if it is one, will be unusually localized.

Thanks to Patrick Collison, Jessica Livingston, and Fred Wilson for reading drafts of this.

Chapter 47

Maker's Schedule, Manager's Schedule

“...the mere consciousness of an engagement will sometimes worry a whole day.”

– Charles Dickens

July 2009

One reason programmers dislike meetings so much is that they're on a different type of schedule from other people. Meetings cost them more.

There are two types of schedule, which I'll call the manager's schedule and the maker's schedule. The manager's schedule is for bosses. It's embodied in the traditional appointment book, with each day cut into one hour intervals. You can block off several hours for a single task if you need to, but by default you change what you're doing every hour.

When you use time that way, it's merely a practical problem to meet with someone. Find an open slot in your schedule, book them, and you're done.

Most powerful people are on the manager's schedule. It's the schedule of command. But there's another way of using time that's common among people who make things, like programmers and writers. They generally prefer to use time in units of half a day at least. You can't write or program well in units of an hour. That's barely enough time to get started.

When you're operating on the maker's schedule, meetings are a disas-

ter. A single meeting can blow a whole afternoon, by breaking it into two pieces each too small to do anything hard in. Plus you have to remember to go to the meeting. That's no problem for someone on the manager's schedule. There's always something coming on the next hour; the only question is what. But when someone on the maker's schedule has a meeting, they have to think about it.

For someone on the maker's schedule, having a meeting is like throwing an exception. It doesn't merely cause you to switch from one task to another; it changes the mode in which you work.

I find one meeting can sometimes affect a whole day. A meeting commonly blows at least half a day, by breaking up a morning or afternoon. But in addition there's sometimes a cascading effect. If I know the afternoon is going to be broken up, I'm slightly less likely to start something ambitious in the morning. I know this may sound oversensitive, but if you're a maker, think of your own case. Don't your spirits rise at the thought of having an entire day free to work, with no appointments at all? Well, that means your spirits are correspondingly depressed when you don't. And ambitious projects are by definition close to the limits of your capacity. A small decrease in morale is enough to kill them off.

Each type of schedule works fine by itself. Problems arise when they meet. Since most powerful people operate on the manager's schedule, they're in a position to make everyone resonate at their frequency if they want to. But the smarter ones restrain themselves, if they know that some of the people working for them need long chunks of time to work in.

Our case is an unusual one. Nearly all investors, including all VCs I know, operate on the manager's schedule. But Y Combinator runs on the maker's schedule. Rtm and Trevor and I do because we always have, and Jessica does too, mostly, because she's gotten into sync with us.

I wouldn't be surprised if there start to be more companies like us. I

suspect founders may increasingly be able to resist, or at least postpone, turning into managers, just as a few decades ago they started to be able to resist switching from jeans to suits.

How do we manage to advise so many startups on the maker's schedule? By using the classic device for simulating the manager's schedule within the maker's: office hours. Several times a week I set aside a chunk of time to meet founders we've funded. These chunks of time are at the end of my working day, and I wrote a signup program that ensures all the appointments within a given set of office hours are clustered at the end. Because they come at the end of my day these meetings are never an interruption. (Unless their working day ends at the same time as mine, the meeting presumably interrupts theirs, but since they made the appointment it must be worth it to them.) During busy periods, office hours sometimes get long enough that they compress the day, but they never interrupt it.

When we were working on our own startup, back in the 90s, I evolved another trick for partitioning the day. I used to program from dinner till about 3 am every day, because at night no one could interrupt me. Then I'd sleep till about 11 am, and come in and work until dinner on what I called "business stuff." I never thought of it in these terms, but in effect I had two workdays each day, one on the manager's schedule and one on the maker's.

When you're operating on the manager's schedule you can do something you'd never want to do on the maker's: you can have speculative meetings. You can meet someone just to get to know one another. If you have an empty slot in your schedule, why not? Maybe it will turn out you can help one another in some way.

Business people in Silicon Valley (and the whole world, for that matter) have speculative meetings all the time. They're effectively free if you're on the manager's schedule. They're so common that there's distinctive language for proposing them: saying that you want to "grab coffee," for example.

Speculative meetings are terribly costly if you're on the maker's schedule, though. Which puts us in something of a bind. Everyone assumes that, like other investors, we run on the manager's schedule. So they introduce us to someone they think we ought to meet, or send us an email proposing we grab coffee. At this point we have two options, neither of them good: we can meet with them, and lose half a day's work; or we can try to avoid meeting them, and probably offend them.

Till recently we weren't clear in our own minds about the source of the problem. We just took it for granted that we had to either blow our schedules or offend people. But now that I've realized what's going on, perhaps there's a third option: to write something explaining the two types of schedule. Maybe eventually, if the conflict between the manager's schedule and the maker's schedule starts to be more widely understood, it will become less of a problem.

Those of us on the maker's schedule are willing to compromise. We know we have to have some number of meetings. All we ask from those on the manager's schedule is that they understand the cost.

Thanks to Sam Altman, Trevor Blackwell, Paul Buchheit, Jessica Livingston, and Robert Morris for reading drafts of this.

Related:

Chapter 48

Ramen Profitable

July 2009

Now that the term “ramen profitable” has become widespread, I ought to explain precisely what the idea entails.

Ramen profitable means a startup makes just enough to pay the founders’ living expenses. This is a different form of profitability than startups have traditionally aimed for. Traditional profitability means a big bet is finally paying off, whereas the main importance of ramen profitability is that it buys you time.¹

¹The “ramen” in “ramen profitable” refers to instant ramen, which is just about the cheapest food available.

Please do not take the term literally. Living on instant ramen would be very unhealthy. Rice and beans are a better source of food. Start by investing in a rice cooker, if you don’t have one.

Rice and Beans for 2n

olive oil or butter n yellow onions other fresh vegetables; experiment 3n cloves garlic n 12-oz cans white, kidney, or black beans n cubes Knorr beef or vegetable bouillon n teaspoons freshly ground black pepper 3n teaspoons ground cumin n cups dry rice, preferably brown

Put rice in rice cooker. Add water as specified on rice package. (Default: 2 cups water per cup of rice.) Turn on rice cooker and forget about it.

Chop onions and other vegetables and fry in oil, over fairly low heat, till onions are glassy. Put in chopped garlic, pepper, cumin, and a little more fat, and stir. Keep heat low. Cook another 2 or 3 minutes, then add beans (don’t drain the beans), and stir. Throw in the bouillon cube(s), cover, and cook on lowish heat for at least 10 minutes more. Stir vigilantly to avoid sticking.

If you want to save money, buy beans in giant cans from discount stores. Spices are also much cheaper when bought in bulk. If there’s an Indian grocery store near you, they’ll have big bags of cumin for the same price as the little jars in supermarkets.

In the past, a startup would usually become profitable only after raising and spending quite a lot of money. A company making computer hardware might not become profitable for 5 years, during which they spent \$50 million. But when they did they might have revenues of \$50 million a year. This kind of profitability means the startup has succeeded.

Ramen profitability is the other extreme: a startup that becomes profitable after 2 months, even though its revenues are only \$3000 a month, because the only employees are a couple 25 year old founders who can live on practically nothing. Revenues of \$3000 a month do not mean the company has succeeded. But it does share something with the one that's profitable in the traditional way: they don't need to raise money to survive.

Ramen profitability is an unfamiliar idea to most people because it only recently became feasible. It's still not feasible for a lot of startups; it would not be for most biotech startups, for example; but it is for many software startups because they're now so cheap. For many, the only real cost is the founders' living expenses.

The main significance of this type of profitability is that you're no longer at the mercy of investors. If you're still losing money, then eventually you'll either have to raise more or shut down. Once you're ramen profitable this painful choice goes away. You can still raise money, but you don't have to do it now.

* * *

The most obvious advantage of not needing money is that you can get better terms. If investors know you need money, they'll sometimes take advantage of you. Some may even deliberately stall, because they know that as you run out of money you'll become increasingly pliable.

But there are also three less obvious advantages of ramen profitability. One is that it makes you more attractive to investors. If you're already profitable, on however small a scale, it shows that (a) you can get at least someone to pay you, (b) you're serious about building things peo-

ple want, and (c) you're disciplined enough to keep expenses low.

This is reassuring to investors, because you've addressed three of their biggest worries. It's common for them to fund companies that have smart founders and a big market, and yet still fail. When these companies fail, it's usually because (a) people wouldn't pay for what they made, e.g. because it was too hard to sell to them, or the market wasn't ready yet, (b) the founders solved the wrong problem, instead of paying attention to what users needed, or (c) the company spent too much and burned through their funding before they started to make money. If you're ramen profitable, you're already avoiding these mistakes.

Another advantage of ramen profitability is that it's good for morale. A company tends to feel rather theoretical when you first start it. It's legally a company, but you feel like you're lying when you call it one. When people start to pay you significant amounts, the company starts to feel real. And your own living expenses are the milestone you feel most, because at that point the future flips state. Now survival is the default, instead of dying.

A morale boost on that scale is very valuable in a startup, because the moral weight of running a startup is what makes it hard. Startups are still very rare. Why don't more people do it? The financial risk? Plenty of 25 year olds save nothing anyway. The long hours? Plenty of people work just as long hours in regular jobs. What keeps people from starting startups is the fear of having so much responsibility. And this is not an irrational fear: it really is hard to bear. Anything that takes some of that weight off you will greatly increase your chances of surviving.

A startup that reaches ramen profitability may be more likely to succeed than not. Which is pretty exciting, considering the bimodal distribution of outcomes in startups: you either fail or make a lot of money.

The fourth advantage of ramen profitability is the least obvious but may be the most important. If you don't need to raise money, you don't have to interrupt working on the company to do it.

Raising money is terribly distracting. You're lucky if your productivity is a third of what it was before. And it can last for months.

I didn't understand (or rather, remember) precisely why raising money was so distracting till earlier this year. I'd noticed that startups we funded would usually grind to a halt when they switched to raising money, but I didn't remember exactly why till YC raised money itself. We had a comparatively easy time of it; the first people I asked said yes; but it took months to work out the details, and during that time I got hardly any real work done. Why? Because I thought about it all the time.

At any given time there tends to be one problem that's the most urgent for a startup. This is what you think about as you fall asleep at night and when you take a shower in the morning. And when you start raising money, that becomes the problem you think about. You only take one shower in the morning, and if you're thinking about investors during it, then you're not thinking about the product.

Whereas if you can choose when you raise money, you can pick a time when you're not in the middle of something else, and you can probably also insist that the round close fast. You may even be able to avoid having the round occupy your thoughts, if you don't care whether it closes.

* * *

Ramen profitable means no more than the definition implies. It does not, for example, imply that you're "bootstrapping" the startup—that you're never going to take money from investors. Empirically that doesn't seem to work very well. Few startups succeed without taking investment. Maybe as startups get cheaper it will become more common. On the other hand, the money is there, waiting to be invested. If startups need it less, they'll be able to get it on better terms, which will make them more inclined to take it. That will tend to produce an equilibrium.²

²There's a good chance that a shift in power from investors to founders would

Another thing ramen profitability doesn't imply is Joe Kraus's idea that you should put your business model in beta when you put your product in beta. He believes you should get people to pay you from the beginning. I think that's too constraining. Facebook didn't, and they've done better than most startups. Making money right away was not only unnecessary for them, but probably would have been harmful. I do think Joe's rule could be useful for many startups, though. When founders seem unfocused, I sometimes suggest they try to get customers to pay them for something, in the hope that this constraint will prod them into action.

The difference between Joe's idea and ramen profitability is that a ramen profitable company doesn't have to be making money the way it ultimately will. It just has to be making money. The most famous example is Google, which initially made money by licensing search to sites like Yahoo.

Is there a downside to ramen profitability? Probably the biggest danger is that it might turn you into a consulting firm. Startups have to be product companies, in the sense of making a single thing that everyone uses. The defining quality of startups is that they grow fast, and consulting just can't scale the way a product can.³ But it's pretty easy to make \$3000 a month consulting; in fact, that would be a low rate for contract programming. So there could be a temptation to slide into consulting, and telling yourselves you're a ramen profitable startup, when in fact you're not a startup at all.

It's ok to do a little consulting-type work at first. Startups usually have

actually increase the size of the venture business. I think investors currently err too far on the side of being harsh to founders. If they were forced to stop, the whole venture business would work better, and you might see something like the increase in trade you always see when restrictive laws are removed.

Investors are one of the biggest sources of pain for founders; if they stopped causing so much pain, it would be better to be a founder; and if it were better to be a founder, more people would do it.

³It's conceivable that a startup could grow big by transforming consulting into a form that would scale. But if they did that they'd really be a product company.

to do something weird at first. But remember that ramen profitability is not the destination. A startup's destination is to grow really big; ramen profitability is a trick for not dying en route.

Thanks to Jessica Livingston for reading drafts of this.

Chapter 49

The Trouble with the Segway

July 2009

The Segway hasn't delivered on its initial promise, to put it mildly. There are several reasons why, but one is that people don't want to be seen riding them. Someone riding a Segway looks like a dork.

My friend Trevor Blackwell built his own Segway, which we called the Segwell. He also built a one-wheeled version, the Eunicycle, which looks exactly like a regular unicycle till you realize the rider isn't pedaling. He has ridden them both to downtown Mountain View to get coffee. When he rides the Eunicycle, people smile at him. But when he rides the Segwell, they shout abuse from their cars: "Too lazy to walk, ya fuckin homo?"

Why do Segways provoke this reaction? The reason you look like a dork riding a Segway is that you look *smug*. You don't seem to be working hard enough.

Someone riding a motorcycle isn't working any harder. But because he's sitting astride it, he seems to be making an effort. When you're riding a Segway you're just standing there. And someone who's being whisked along while seeming to do no work — someone in a sedan chair, for example — can't help but look smug.

Try this thought experiment and it becomes clear: imagine something that worked like the Segway, but that you rode with one foot in front of the other, like a skateboard. That wouldn't seem nearly as uncool.

So there may be a way to capture more of the market Segway hoped to reach: make a version that doesn't look so easy for the rider. It

would also be helpful if the styling was in the tradition of skateboards or bicycles rather than medical devices.

Curiously enough, what got Segway into this problem was that the company was itself a kind of Segway. It was too easy for them; they were too successful raising money. If they'd had to grow the company gradually, by iterating through several versions they sold to real users, they'd have learned pretty quickly that people looked stupid riding them. Instead they had enough to work in secret. They had focus groups aplenty, I'm sure, but they didn't have the people yelling insults out of cars. So they never realized they were zooming confidently down a blind alley.

Chapter 50

What Kate Saw in Silicon Valley

August 2009

Kate Courteau is the architect who designed Y Combinator's office. Recently we managed to recruit her to help us run YC when she's not busy with architectural projects. Though she'd heard a lot about YC since the beginning, the last 9 months have been a total immersion.

I've been around the startup world for so long that it seems normal to me, so I was curious to hear what had surprised her most about it. This was her list:

1. How many startups fail.

Kate knew in principle that startups were very risky, but she was surprised to see how constant the threat of failure was — not just for the minnows, but even for the famous startups whose founders came to speak at YC dinners.

2. How much startups' ideas change.

As usual, by Demo Day about half the startups were doing something significantly different than they started with. We encourage that. Starting a startup is like science in that you have to follow the truth wherever it leads. In the rest of the world, people don't start things till they're sure what they want to do, and once started they tend to continue on their initial path even if it's mistaken.

3. How little money it can take to start a startup.

In Kate's world, everything is still physical and expensive. You can barely renovate a bathroom for the cost of starting a startup.

4. How scrappy founders are.

That was her actual word. I agree with her, but till she mentioned this it never occurred to me how little this quality is appreciated in most of the rest of the world. It wouldn't be a compliment in most organizations to call someone scrappy.

What does it mean, exactly? It's basically the diminutive form of belligerent. Someone who's scrappy manages to be both threatening and undignified at the same time. Which seems to me exactly what one would want to be, in any kind of work. If you're not threatening, you're probably not doing anything new, and dignity is merely a sort of plaque.

5. How tech-saturated Silicon Valley is.

"It seems like everybody here is in the industry." That isn't literally true, but there is a qualitative difference between Silicon Valley and other places. You tend to keep your voice down, because there's a good chance the person at the next table would know some of the people you're talking about. I never felt that in Boston. The good news is, there's also a good chance the person at the next table could help you in some way.

6. That the speakers at YC were so consistent in their advice.

Actually, I've noticed this too. I always worry the speakers will put us in an embarrassing position by contradicting what we tell the startups, but it happens surprisingly rarely.

When I asked her what specific things she remembered speakers always saying, she mentioned: that the way to succeed was to launch something fast, listen to users, and then iterate; that startups required resilience because they were always an emotional rollercoaster; and that most VCs were sheep.

I've been impressed by how consistently the speakers advocate launching fast and iterating. That was contrarian advice 10 years ago, but it's clearly now the established practice.

7. How casual successful startup founders are.

Most of the famous founders in Silicon Valley are people you'd overlook on the street. It's not merely that they don't dress up. They don't project any kind of aura of power either. "They're not trying to impress anyone."

Interestingly, while Kate said that she could never pick out successful founders, she could recognize VCs, both by the way they dressed and the way they carried themselves.

8. How important it is for founders to have people to ask for advice.

(I swear I didn't prompt this one.) Without advice "they'd just be sort of lost." Fortunately, there are a lot of people to help them. There's a strong tradition within YC of helping other YC-funded startups. But we didn't invent that idea: it's just a slightly more concentrated form of existing Valley culture.

9. What a solitary task startups are.

Architects are constantly interacting face to face with other people, whereas doing a technology startup, at least, tends to require long stretches of uninterrupted time to work. "You could do it in a box."

By inverting this list, we can get a portrait of the "normal" world. It's populated by people who talk a lot with one another as they work slowly but harmoniously on conservative, expensive projects whose destinations are decided in advance, and who carefully adjust their manner to reflect their position in the hierarchy.

That's also a fairly accurate description of the past. So startup culture may not merely be different in the way you'd expect any subculture to be, but a leading indicator.

Chapter 51

The Anatomy of Determination

September 2009

Like all investors, we spend a lot of time trying to learn how to predict which startups will succeed. We probably spend more time thinking about it than most, because we invest the earliest. Prediction is usually all we have to rely on.

We learned quickly that the most important predictor of success is determination. At first we thought it might be intelligence. Everyone likes to believe that's what makes startups succeed. It makes a better story that a company won because its founders were so smart. The PR people and reporters who spread such stories probably believe them themselves. But while it certainly helps to be smart, it's not the deciding factor. There are plenty of people as smart as Bill Gates who achieve nothing.

In most domains, talent is overrated compared to determination—partly because it makes a better story, partly because it gives onlookers an excuse for being lazy, and partly because after a while determination starts to look like talent.

I can't think of any field in which determination is overrated, but the relative importance of determination and talent probably do vary somewhat. Talent probably matters more in types of work that are purer, in the sense that one is solving mostly a single type of problem instead of many different types. I suspect determination would not take you as far in math as it would in, say, organized crime.

I don't mean to suggest by this comparison that types of work that depend more on talent are always more admirable. Most people would

agree it's more admirable to be good at math than memorizing long strings of digits, even though the latter depends more on natural ability.

Perhaps one reason people believe startup founders win by being smarter is that intelligence does matter more in technology startups than it used to in earlier types of companies. You probably do need to be a bit smarter to dominate Internet search than you had to be to dominate railroads or hotels or newspapers. And that's probably an ongoing trend. But even in the highest of high tech industries, success still depends more on determination than brains.

If determination is so important, can we isolate its components? Are some more important than others? Are there some you can cultivate?

The simplest form of determination is sheer willfulness. When you want something, you must have it, no matter what.

A good deal of willfulness must be inborn, because it's common to see families where one sibling has much more of it than another. Circumstances can alter it, but at the high end of the scale, nature seems to be more important than nurture. Bad circumstances can break the spirit of a strong-willed person, but I don't think there's much you can do to make a weak-willed person stronger-willed.

Being strong-willed is not enough, however. You also have to be hard on yourself. Someone who was strong-willed but self-indulgent would not be called determined. Determination implies your willfulness is balanced by discipline.

That word balance is a significant one. The more willful you are, the more disciplined you have to be. The stronger your will, the less anyone will be able to argue with you except yourself. And someone has to argue with you, because everyone has base impulses, and if you have more will than discipline you'll just give into them and end up on a local maximum like drug addiction.

We can imagine will and discipline as two fingers squeezing a slippery

melon seed. The harder they squeeze, the further the seed flies, but they must both squeeze equally or the seed spins off sideways.

If this is true it has interesting implications, because discipline can be cultivated, and in fact does tend to vary quite a lot in the course of an individual's life. If determination is effectively the product of will and discipline, then you can become more determined by being more disciplined.¹

Another consequence of the melon seed model is that the more willful you are, the more dangerous it is to be undisciplined. There seem to be plenty of examples to confirm that. In some very energetic people's lives you see something like wing flutter, where they alternate between doing great work and doing absolutely nothing. Externally this would look a lot like bipolar disorder.

The melon seed model is inaccurate in at least one respect, however: it's static. In fact the dangers of indiscipline increase with temptation. Which means, interestingly, that determination tends to erode itself. If you're sufficiently determined to achieve great things, this will probably increase the number of temptations around you. Unless you become proportionally more disciplined, willfulness will then get the upper hand, and your achievement will revert to the mean.

That's why Shakespeare's Caesar thought thin men so dangerous. They weren't tempted by the minor perquisites of power.

The melon seed model implies it's possible to be too disciplined. Is it? I think there probably are people whose willfulness is crushed down by excessive discipline, and who would achieve more if they weren't so hard on themselves. One reason the young sometimes succeed where the old fail is that they don't realize how incompetent they are. This lets them do a kind of deficit spending. When they first start working on something, they overrate their achievements. But that gives

¹Loosely speaking. What I'm claiming with the melon seed model is more like determination is proportionate to $w d^m - k | w - d |^n$, where w is will and d discipline.

them confidence to keep working, and their performance improves. Whereas someone clearer-eyed would see their initial incompetence for what it was, and perhaps be discouraged from continuing.

There's one other major component of determination: ambition. If willfulness and discipline are what get you to your destination, ambition is how you choose it.

I don't know if it's exactly right to say that ambition is a component of determination, but they're not entirely orthogonal. It would seem a misnomer if someone said they were very determined to do something trivially easy.

And fortunately ambition seems to be quite malleable; there's a lot you can do to increase it. Most people don't know how ambitious to be, especially when they're young. They don't know what's hard, or what they're capable of. And this problem is exacerbated by having few peers. Ambitious people are rare, so if everyone is mixed together randomly, as they tend to be early in people's lives, then the ambitious ones won't have many ambitious peers. When you take people like this and put them together with other ambitious people, they bloom like dying plants given water. Probably most ambitious people are starved for the sort of encouragement they'd get from ambitious peers, whatever their age. ²

Achievements also tend to increase your ambition. With each step you gain confidence to stretch further next time.

So here in sum is how determination seems to work: it consists of willfulness balanced with discipline, aimed by ambition. And fortunately at least two of these three qualities can be cultivated. You may be able to increase your strength of will somewhat; you can definitely

²Which means one of the best ways to help a society generally is to create events and institutions that bring ambitious people together. It's like pulling the control rods out of a reactor: the energy they emit encourages other ambitious people, instead of being absorbed by the normal people they're usually surrounded with.

Conversely, it's probably a mistake to do as some European countries have done and try to ensure none of your universities is significantly better than the others.

learn self-discipline; and almost everyone is practically malnourished when it comes to ambition.

I feel like I understand determination a bit better now. But only a bit: willfulness, discipline, and ambition are all concepts almost as complicated as determination.³

Note too that determination and talent are not the whole story. There's a third factor in achievement: how much you like the work. If you really love working on something, you don't need determination to drive you; it's what you'd do anyway. But most types of work have aspects one doesn't like, because most types of work consist of doing things for other people, and it's very unlikely that the tasks imposed by their needs will happen to align exactly with what you want to do.

Indeed, if you want to create the most wealth, the way to do it is to focus more on their needs than your interests, and make up the difference with determination.

Thanks to Sam Altman, Jessica Livingston, and Robert Morris for reading drafts of this.

³For example, willfulness clearly has two subcomponents, stubbornness and energy. The first alone yields someone who's stubbornly inert. The second alone yields someone flighty. As willful people get older or otherwise lose their energy, they tend to become merely stubborn.

Chapter 52

The List of N Things

September 2009

I bet you the current issue of *Cosmopolitan* has an article whose title begins with a number. “7 Things He Won’t Tell You about Sex,” or something like that. Some popular magazines feature articles of this type on the cover of every issue. That can’t be happening by accident. Editors must know they attract readers.

Why do readers like the list of n things so much? Mainly because it’s easier to read than a regular article.¹ Structurally, the list of n things is a degenerate case of essay. An essay can go anywhere the writer wants. In a list of n things the writer agrees to constrain himself to a collection of points of roughly equal importance, and he tells the reader explicitly what they are.

Some of the work of reading an article is understanding its structure—figuring out what in high school we’d have called its “outline.” Not explicitly, of course, but someone who really understands an article probably has something in his brain afterward that corresponds to such an outline. In a list of n things, this work is done for you. Its structure is an exoskeleton.

As well as being explicit, the structure is guaranteed to be of the simplest possible type: a few main points with few to no subordinate ones, and no particular connection between them.

¹Articles of this type are also startlingly popular on Delicious, but I think that’s because delicious/popular is driven by bookmarking, not because Delicious users are stupid. Delicious users are collectors, and a list of n things seems particularly collectible because it’s a collection itself.

Because the main points are unconnected, the list of n things is random access. There's no thread of reasoning you have to follow. You could read the list in any order. And because the points are independent of one another, they work like watertight compartments in an unsinkable ship. If you get bored with, or can't understand, or don't agree with one point, you don't have to give up on the article. You can just abandon that one and skip to the next. A list of n things is parallel and therefore fault tolerant.

There are times when this format is what a writer wants. One, obviously, is when what you have to say actually is a list of n things. I once wrote an essay about the mistakes that kill startups, and a few people made fun of me for writing something whose title began with a number. But in that case I really was trying to make a complete catalog of a number of independent things. In fact, one of the questions I was trying to answer was how many there were.

There are other less legitimate reasons for using this format. For example, I use it when I get close to a deadline. If I have to give a talk and I haven't started it a few days beforehand, I'll sometimes play it safe and make the talk a list of n things.

The list of n things is easier for writers as well as readers. When you're writing a real essay, there's always a chance you'll hit a dead end. A real essay is a train of thought, and some trains of thought just peter out. That's an alarming possibility when you have to give a talk in a few days. What if you run out of ideas? The compartmentalized structure of the list of n things protects the writer from his own stupidity in much the same way it protects the reader. If you run out of ideas on one point, no problem: it won't kill the essay. You can take out the whole point if you need to, and the essay will still survive.

Writing a list of n things is so relaxing. You think of $n/2$ of them in the first 5 minutes. So bang, there's the structure, and you just have to fill it in. As you think of more points, you just add them to the end. Maybe you take out or rearrange or combine a few, but at every stage you have a valid (though initially low-res) list of n things. It's like the

sort of programming where you write a version 1 very quickly and then gradually modify it, but at every point have working code—or the style of painting where you begin with a complete but very blurry sketch done in an hour, then spend a week cranking up the resolution.

Because the list of *n* things is easier for writers too, it's not always a damning sign when readers prefer it. It's not necessarily evidence readers are lazy; it could also mean they don't have much confidence in the writer. The list of *n* things is in that respect the cheeseburger of essay forms. If you're eating at a restaurant you suspect is bad, your best bet is to order the cheeseburger. Even a bad cook can make a decent cheeseburger. And there are pretty strict conventions about what a cheeseburger should look like. You can assume the cook isn't going to try something weird and artistic. The list of *n* things similarly limits the damage that can be done by a bad writer. You know it's going to be about whatever the title says, and the format prevents the writer from indulging in any flights of fancy.

Because the list of *n* things is the easiest essay form, it should be a good one for beginning writers. And in fact it is what most beginning writers are taught. The classic 5 paragraph essay is really a list of *n* things for $n = 3$. But the students writing them don't realize they're using the same structure as the articles they read in *Cosmopolitan*. They're not allowed to include the numbers, and they're expected to spackle over the gaps with gratuitous transitions ("Furthermore...") and cap the thing at either end with introductory and concluding paragraphs so it will look superficially like a real essay.²

It seems a fine plan to start students off with the list of *n* things. It's the easiest form. But if we're going to do that, why not do it openly? Let them write lists of *n* things like the pros, with numbers and no transitions or "conclusion."

²Most "word problems" in school math textbooks are similarly misleading. They look superficially like the application of math to real problems, but they're not. So if anything they reinforce the impression that math is merely a complicated but pointless collection of stuff to be memorized.

There is one case where the list of n things is a dishonest format: when you use it to attract attention by falsely claiming the list is an exhaustive one. I.e. if you write an article that purports to be about *the 7 secrets of success*. That kind of title is the same sort of reflexive challenge as a whodunit. You have to at least look at the article to check whether they're the same 7 you'd list. Are you overlooking one of the secrets of success? Better check.

It's fine to put "The" before the number if you really believe you've made an exhaustive list. But evidence suggests most things with titles like this are linkbait.

The greatest weakness of the list of n things is that there's so little room for new thought. The main point of essay writing, when done right, is the new ideas you have while doing it. A real essay, as the name implies, is dynamic: you don't know what you're going to write when you start. It will be about whatever you discover in the course of writing it.

This can only happen in a very limited way in a list of n things. You make the title first, and that's what it's going to be about. You can't have more new ideas in the writing than will fit in the watertight compartments you set up initially. And your brain seems to know this: because you don't have room for new ideas, you don't have them.

Another advantage of admitting to beginning writers that the 5 paragraph essay is really a list of n things is that we can warn them about this. It only lets you experience the defining characteristic of essay writing on a small scale: in thoughts of a sentence or two. And it's particularly dangerous that the 5 paragraph essay buries the list of n things within something that looks like a more sophisticated type of essay. If you don't know you're using this form, you don't know you need to escape it.

Chapter 53

Post-Medium Publishing

September 2009

Publishers of all types, from news to music, are unhappy that consumers won't pay for content anymore. At least, that's how they see it.

In fact consumers never really were paying for content, and publishers weren't really selling it either. If the content was what they were selling, why has the price of books or music or movies always depended mostly on the format? Why didn't better content cost more? ¹

A copy of *Time* costs \$5 for 58 pages, or 8.6 cents a page. *The Economist* costs \$7 for 86 pages, or 8.1 cents a page. Better journalism is actually slightly cheaper.

Almost every form of publishing has been organized as if the medium was what they were selling, and the content was irrelevant. Book publishers, for example, set prices based on the cost of producing and distributing books. They treat the words printed in the book the same way a textile manufacturer treats the patterns printed on its fabrics.

Economically, the print media are in the business of marking up paper. We can all imagine an old-style editor getting a scoop and saying "this will sell a lot of papers!" Cross out that final S and you're describing

¹I don't like the word "content" and tried for a while to avoid using it, but I have to admit there's no other word that means the right thing. "Information" is too general.

Ironically, the main reason I don't like "content" is the thesis of this essay. The word suggests an undifferentiated slurry, but economically that's how both publishers and audiences treat it. Content is information you don't need.

their business model. The reason they make less money now is that people don't need as much paper.

A few months ago I ran into a friend in a cafe. I had a copy of the *New York Times*, which I still occasionally buy on weekends. As I was leaving I offered it to him, as I've done countless times before in the same situation. But this time something new happened. I felt that sheepish feeling you get when you offer someone something worthless. "Do you, er, want a printout of yesterday's news?" I asked. (He didn't.)

Now that the medium is evaporating, publishers have nothing left to sell. Some seem to think they're going to sell content—that they were always in the content business, really. But they weren't, and it's unclear whether anyone could be.

Selling

There have always been people in the business of selling information, but that has historically been a distinct business from publishing. And the business of selling information to consumers has always been a marginal one. When I was a kid there were people who used to sell newsletters containing stock tips, printed on colored paper that made them hard for the copiers of the day to reproduce. That is a different world, both culturally and economically, from the one publishers currently inhabit.

People will pay for information they think they can make money from. That's why they paid for those stock tip newsletters, and why companies pay now for Bloomberg terminals and Economist Intelligence Unit reports. But will people pay for information otherwise? History offers little encouragement.

If audiences were willing to pay more for better content, why wasn't anyone already selling it to them? There was no reason you couldn't have done that in the era of physical media. So were the print media and the music labels simply overlooking this opportunity? Or is it, rather, nonexistent?

What about iTunes? Doesn't that show people will pay for content? Well, not really. iTunes is more of a tollbooth than a store. Apple controls the default path onto the iPod. They offer a convenient list of songs, and whenever you choose one they ding your credit card for a small amount, just below the threshold of attention. Basically, iTunes makes money by taxing people, not selling them stuff. You can only do that if you own the channel, and even then you don't make much from it, because a toll has to be ignorable to work. Once a toll becomes painful, people start to find ways around it, and that's pretty easy with digital content.

The situation is much the same with digital books. Whoever controls the device sets the terms. It's in their interest for content to be as cheap as possible, and since they own the channel, there's a lot they can do to drive prices down. Prices will fall even further once writers realize they don't need publishers. Getting a book printed and distributed is a daunting prospect for a writer, but most can upload a file.

Is software a counterexample? People pay a lot for desktop software, and that's just information. True, but I don't think publishers can learn much from software. Software companies can charge a lot because (a) many of the customers are businesses, who get in trouble if they use pirated versions, and (b) though in form merely information, software is treated by both maker and purchaser as a different type of thing from a song or an article. A Photoshop user needs Photoshop in a way that no one needs a particular song or article.

That's why there's a separate word, "content," for information that's not software. Software is a different business. Software and content blur together in some of the most lightweight software, like casual games. But those are usually free. To make money the way software companies do, publishers would have to become software companies, and being publishers gives them no particular head start in that domain.²

²Some types of publishers would be at a disadvantage trying to enter the software business. Record labels, for example, would probably find it more natural to expand

The most promising countertrend is the premium cable channel. People still pay for those. But broadcasting isn't publishing: you're not selling a copy of something. That's one reason the movie business hasn't seen their revenues decline the way the news and music businesses have. They only have one foot in publishing.

To the extent the movie business can avoid becoming publishers, they may avoid publishing's problems. But there are limits to how well they'll be able to do that. Once publishing—giving people copies—becomes the most natural way of distributing your content, it probably doesn't work to stick to old forms of distribution just because you make more that way. If free copies of your content are available online, then you're competing with publishing's form of distribution, and that's just as bad as being a publisher.

Apparently some people in the music business hope to retroactively convert it away from publishing, by getting listeners to pay for subscriptions. It seems unlikely that will work if they're just streaming the same files you can get as mp3s.

Next

What happens to publishing if you can't sell content? You have two choices: give it away and make money from it indirectly, or find ways to embody it in things people will pay for.

The first is probably the future of most current media. Give music away and make money from concerts and t-shirts. Publish articles for free and make money from one of a dozen permutations of advertising. Both publishers and investors are down on advertising at the moment, but it has more potential than they realize.

I'm not claiming that potential will be realized by the existing players. The optimal ways to make money from the written word probably require different words written by different people.

into casinos than software, because the kind of people who run them would be more at home at the mafia end of the business spectrum than the don't-be-evil end.

It's harder to say what will happen to movies. They could evolve into ads. Or they could return to their roots and make going to the theater a treat. If they made the experience good enough, audiences might start to prefer it to watching pirated movies at home.³ Or maybe the movie business will dry up, and the people working in it will go to work for game developers.

I don't know how big embodying information in physical form will be. It may be surprisingly large; people overvalue physical stuff. There should remain some market for printed books, at least.

I can see the evolution of book publishing in the books on my shelves. Clearly at some point in the 1960s the big publishing houses started to ask: how cheaply can we make books before people refuse to buy them? The answer turned out to be one step short of phonebooks. As long as it isn't floppy, consumers still perceive it as a book.

That worked as long as buying printed books was the only way to read them. If printed books are optional, publishers will have to work harder to entice people to buy them. There should be some market, but it's hard to foresee how big, because its size will depend not on macro trends like the amount people read, but on the ingenuity of individual publishers.⁴

Some magazines may thrive by focusing on the magazine as a physical object. Fashion magazines could be made lush in a way that would be hard to match digitally, at least for a while. But this is probably not an option for most magazines.

I don't know exactly what the future will look like, but I'm not too worried about it. This sort of change tends to create as many good things as it kills. Indeed, the really interesting question is not what will happen to existing forms, but what new forms will appear.

³I never watch movies in theaters anymore. The tipping point for me was the ads they show first.

⁴Unfortunately, making physically nice books will only be a niche within a niche. Publishers are more likely to resort to expedients like selling autographed copies, or editions with the buyer's picture on the cover.

The reason I've been writing about existing forms is that I don't *know* what new forms will appear. But though I can't predict specific winners, I can offer a recipe for recognizing them. When you see something that's taking advantage of new technology to give people something they want that they couldn't have before, you're probably looking at a winner. And when you see something that's merely reacting to new technology in an attempt to preserve some existing source of revenue, you're probably looking at a loser.

Thanks to Michael Arrington, Trevor Blackwell, Steven Levy, Robert Morris, and Geoff Ralston for reading drafts of this.

Chapter 54

Persuade xor Discover

September 2009

When meeting people you don't know very well, the convention is to seem extra friendly. You smile and say "pleased to meet you," whether you are or not. There's nothing dishonest about this. Everyone knows that these little social lies aren't meant to be taken literally, just as everyone knows that "Can you pass the salt?" is only grammatically a question.

I'm perfectly willing to smile and say "pleased to meet you" when meeting new people. But there is another set of customs for being ingratiating in print that are not so harmless.

The reason there's a convention of being ingratiating in print is that most essays are written to persuade. And as any politician could tell you, the way to persuade people is not just to baldly state the facts. You have to add a spoonful of sugar to make the medicine go down.

For example, a politician announcing the cancellation of a government program will not merely say "The program is canceled." That would seem offensively curt. Instead he'll spend most of his time talking about the noble effort made by the people who worked on it.

The reason these conventions are more dangerous is that they interact with the ideas. Saying "pleased to meet you" is just something you prepend to a conversation, but the sort of spin added by politicians is woven through it. We're starting to move from social lies to real lies.

Here's an example of a paragraph from an essay I wrote about labor unions. As written, it tends to offend people who like unions.

People who think the labor movement was the creation of heroic union organizers have a problem to explain: why are unions shrinking now? The best they can do is fall back on the default explanation of people living in fallen civilizations. Our ancestors were giants. The workers of the early twentieth century must have had a moral courage that's lacking today.

Now here's the same paragraph rewritten to please instead of offending them:

Early union organizers made heroic sacrifices to improve conditions for workers. But though labor unions are shrinking now, it's not because present union leaders are any less courageous. An employer couldn't get away with hiring thugs to beat up union leaders today, but if they did, I see no reason to believe today's union leaders would shrink from the challenge. So I think it would be a mistake to attribute the decline of unions to some kind of decline in the people who run them. Early union leaders were heroic, certainly, but we should not suppose that if unions have declined, it's because present union leaders are somehow inferior. The cause must be external.¹

It makes the same point: that it can't have been the personal qualities of early union organizers that made unions successful, but must have been some external factor, or otherwise present-day union leaders would have to be inferior people. But written this way it seems like a defense of present-day union organizers rather than an attack on early ones. That makes it more persuasive to people who like unions, because it seems sympathetic to their cause.

¹I had a strange feeling of being back in high school writing this. To get a good grade you had to both write the sort of pious crap you were expected to, but also seem to be writing with conviction. The solution was a kind of method acting. It was revoltingly familiar to slip back into it.

I believe everything I wrote in the second version. Early union leaders did make heroic sacrifices. And present union leaders probably would rise to the occasion if necessary. People tend to; I'm skeptical about the idea of "the greatest generation." ²

If I believe everything I said in the second version, why didn't I write it that way? Why offend people needlessly?

Because I'd rather offend people than pander to them, and if you write about controversial topics you have to choose one or the other. The degree of courage of past or present union leaders is beside the point; all that matters for the argument is that they're the same. But if you want to please people who are mistaken, you can't simply tell the truth. You're always going to have to add some sort of padding to protect their misconceptions from bumping against reality.

Most writers do. Most writers write to persuade, if only out of habit or politeness. But I don't write to persuade; I write to figure out. I write to persuade a hypothetical perfectly unbiased reader.

Since the custom is to write to persuade the actual reader, someone who doesn't will seem arrogant. In fact, worse than arrogant: since readers are used to essays that try to please someone, an essay that displeases one side in a dispute reads as an attempt to pander to the other. To a lot of pro-union readers, the first paragraph sounds like the sort of thing a right-wing radio talk show host would say to stir up his followers. But it's not. Something that curtly contradicts one's beliefs can be hard to distinguish from a partisan attack on them, but though they can end up in the same place they come from different sources.

Would it be so bad to add a few extra words, to make people feel better? Maybe not. Maybe I'm excessively attached to conciseness. I write code the same way I write essays, making pass after pass looking for anything I can cut. But I have a legitimate reason for doing this.

²Exercise for the reader: rephrase that thought to please the same people the first version would offend.

You don't know what the ideas are until you get them down to the fewest words.³

The danger of the second paragraph is not merely that it's longer. It's that you start to lie to yourself. The ideas start to get mixed together with the spin you've added to get them past the readers' misconceptions.

I think the goal of an essay should be to discover surprising things. That's my goal, at least. And most surprising means most different from what people currently believe. So writing to persuade and writing to discover are diametrically opposed. The more your conclusions disagree with readers' present beliefs, the more effort you'll have to expend on selling your ideas rather than having them. As you accelerate, this drag increases, till eventually you reach a point where 100% of your energy is devoted to overcoming it and you can't go any faster.

It's hard enough to overcome one's own misconceptions without having to think about how to get the resulting ideas past other people's. I worry that if I wrote to persuade, I'd start to shy away unconsciously from ideas I knew would be hard to sell. When I notice something surprising, it's usually very faint at first. There's nothing more than a slight stirring of discomfort. I don't want anything to get in the way of noticing it consciously.

Thanks to Jessica Livingston and Robert Morris for reading drafts of this.

Note: An earlier version of this essay began by talking about why people dislike Michael Arrington. I now believe that was mistaken, and that most people don't dislike him for the same reason I did when I first met him, but simply because he writes about controversial things.

³Come to think of it, there is one way in which I deliberately pander to readers, because it doesn't change the number of words: I switch person. This flattering distinction seems so natural to the average reader that they probably don't notice even when I switch in mid-sentence, though you tend to notice when it's done as conspicuously as this.